

CS 250: Introduction to Logic and Mathematical Reasoning

Course Notes

Contents

Preface and Reading Guide	1
1 Welcome, Sets, and Functions	7
1.1 A Note on The Textbook	7
1.2 Welcome to CS 250	7
1.3 Why study logic?	8
1.4 What is logic, what is a proof?	9
1.5 Variables and formulae	10
1.6 Sets and pairs	11
1.7 Functions and relations	15
1.8 Summary and looking ahead	17
1.9 Exercises	18
How to Write a Proof	21
1.10 A proof is a piece of writing	21
1.11 When the problem has cases	21
1.12 Using definitions	22
1.13 An example: good vs. bad	22
1.14 Two walkthroughs: tightening a proof	22
1.15 When you don't know what to write next	24
1.16 Where to go from here	24
Intermezzo: Cardinality and Function Spaces	26
1.17 Comparing set sizes	26
1.18 Countable infinity	27
1.19 Uncountable infinity—Cantor's diagonal argument	28
1.20 Diagonalization, cashed out: the halting problem	29
1.21 Function spaces	30
1.22 The power set is a function space—the punchline	31
1.23 Exercises	32
2 Propositional Logic	34
2.1 Logical formulas, formalized	34
2.2 Tautologies and equivalences	37
2.3 Addendum: nand, xor, and how many binary operations are there?	40
2.4 From truth table back to formula: a worked example	41
2.5 Common mistakes	42
2.6 Summary and looking ahead	43

2.7	Exercises	44
3	Proof Systems for Propositional Logic	47
3.1	Tautologies as reasoning	47
3.2	A thorough derivation of proof by contradiction	49
3.3	A proof system for propositional logic	50
3.3.1	Rules for conjunction	51
3.3.2	Rules for $\vee$	51
3.3.3	Negation	52
3.3.4	$\top$ and $\perp$	52
3.3.5	Implication	53
3.3.6	Extra proof rules	53
3.4	Arguments and validity	54
3.5	Examples of doing proofs	55
3.5.1	Conjunctive normal form	56
3.5.2	Disjunctive normal form	56
3.6	Digital logic circuits	56
3.7	Addendum: differences in terminology and additional derivable proof rules for home-work	58
3.7.1	Derivable rules of inference	58
3.7.2	Differences in verbiage	59
3.8	Truth and provability meet	59
3.9	Common mistakes	60
3.10	Summary and looking ahead	60
3.11	Exercises	61
	Intermezzo: Inference Rules as Trees	64
3.12	A better notation	64
3.13	The rules, redux	64
3.13.1	Conjunction	64
3.13.2	Disjunction	64
3.13.3	Implication	65
3.13.4	Negation	65
3.13.5	Constants	66
3.14	Composing rules into proof trees	66
3.15	Reading proof trees	67
3.16	Why this matters	67
3.17	Exercises	67
	Cumulative Review 1 (after Module 3)	69
4	First-Order Predicate Logic	73
4.1	A logic with real types	73
4.2	Forall and exists	74
4.2.1	Free and bound variables	75
4.3	Truth sets	75
4.4	Negating quantified statements	76
4.5	Nested quantifiers	76

4.6	Necessary and sufficient conditions	77
4.7	Proofs with this logic	78
4.8	Induction, round 1	79
4.8.1	Induction in action: a worked example	79
4.9	Addendum on notation	81
4.10	Common mistakes	81
4.11	Summary and looking ahead	81
4.12	Exercises	82
Intermezzo: More on Natural Induction		85
4.13	Proof that $0 + m = m$	85
4.14	Proof that $\text{succ } n + m = \text{succ } (n + m)$	85
4.15	Proof that addition is associative	86
4.16	Proof that addition is commutative	87
4.17	Exercises	87
Intermezzo: Quantifiers as Games		89
4.18	The game interpretation	89
4.19	Nested quantifiers as alternating turns	90
4.20	Games and constructive logic	91
4.21	Why this matters	92
5	Informal Proofs and Groups	94
5.1	Direct proof	95
5.2	Counterexample	96
5.3	Existence proofs	96
5.4	What is a group?	98
5.5	The integers under addition form a group	99
5.6	A worked finite example: rotations of a square	99
5.7	The integers under multiplication do <i>not</i> form a group	101
5.8	Proof of uniqueness of identity	101
5.9	Maps between groups: homomorphisms	101
5.10	Proof by exhaustion	102
5.11	Common mistakes	103
5.12	Summary and looking ahead	104
5.13	Exercises	105
6	Modular Arithmetic and Proofs by Cases	107
6.1	The modulus operation	107
6.2	The quotient-remainder theorem	108
6.3	Floor and ceiling	108
6.4	$\mathbb{Z}/n\mathbb{Z}$ is a group	109
6.5	$(\mathbb{Z}/p\mathbb{Z})^\times$ is a group when p is prime	110
6.6	The Chinese Remainder Theorem	110
6.7	Proofs by cases and the connection to or-elimination	111
6.8	Common mistakes	112
6.9	Summary and looking ahead	112
6.10	Exercises	113

Intermezzo: What Does It Mean for Things to Be “The Same”?	115
6.11 Equivalence as abstraction	115
6.12 Quotient sets	116
6.13 The connection to types and programming	117
6.14 Weyl’s insight, and why it matters	117
Cumulative Review 2 (after Module 6)	120
7 Indirect Proof, the Euclidean Algorithm, and Hoare Logic	125
7.1 More on proofs by contradiction and proofs by contrapositive	125
7.1.1 Proof by contradiction, in practice	125
7.1.2 Proof by contrapositive	126
7.1.3 Contradiction vs. contrapositive: when to use which	127
7.2 GCD, Euclid’s algorithm, &c.	128
7.3 Programming logic	129
7.3.1 Putting it all together	132
7.3.2 A worked while-loop proof	133
7.4 Common mistakes	136
7.5 Summary and looking ahead	136
7.6 Exercises	137
Intermezzo: The Extended Euclidean Algorithm and Bézout Coefficients	140
7.7 Why we want Bézout coefficients	140
7.8 The algorithm: back-substitution first	141
7.9 The one-pass version	141
7.10 A traced example	142
7.11 Looking ahead: where this pays off	142
7.12 Exercises	143
How to Write a Proof, Revisited	144
7.13 Choosing a technique	144
7.14 A walkthrough: using the contrapositive	145
7.15 Writing induction proofs	145
7.16 Biconditional proofs	146
7.17 Style conventions as proofs get longer	147
7.18 A proof-hygiene checklist	147
7.19 What’s next	148
Intermezzo: Proofs Are Programs	149
7.20 What does a proof <i>look like</i> ?	149
7.21 The Curry–Howard dictionary	150
7.22 A worked example: the same proof, four ways	151
7.22.1 As a derivation (Module 3 style)	151
7.22.2 As a Gentzen tree	151
7.22.3 As a Python function	151
7.22.4 As Lean 4	151
7.23 Why this matters	152

Intermezzo: Other Logics, Other Worlds	155
7.24 The zoo of logics	155
7.25 Temporal logic—logic over time	156
7.26 Expressing real properties	157
7.27 The pattern	157
7.28 Duality everywhere	158
7.29 Exercises	159
8 Sequences and Recursion	161
8.1 Definition by recursion	161
8.1.1 Explicit formulas	161
8.1.2 Recursive definitions	162
8.1.3 Summation and product notation	162
8.2 Induction on sequences	163
8.2.1 The standard template	164
8.2.2 Worked example: sum of the first n integers	164
8.2.3 Setting up: sum of squares	165
8.2.4 Induction is recursion	165
8.3 Common mistakes	166
8.4 Summary and looking ahead	167
8.5 Exercises	168
9 Strong Induction and Recurrences	171
9.1 Strong induction	171
9.2 Defining sequences recursively	173
9.3 Solving recurrences by iteration	174
9.4 Second-order linear homogeneous recurrences	175
9.5 Common mistakes	178
9.6 Summary and looking ahead	179
9.7 Exercises	180
10 Structural Induction	183
10.1 The pattern so far	183
10.2 Lists	184
10.2.1 Defining lists inductively	184
10.2.2 Recursive functions on lists	185
10.2.3 Worked proof: length distributes over append	185
10.3 Binary trees	187
10.3.1 Defining binary trees inductively	187
10.3.2 Recursive functions on binary trees	188
10.3.3 Worked proof: counting edges in a binary tree	189
10.4 The general recipe	190
10.4.1 Example: structural induction on propositional formulas	191
10.5 Wrapping up: the big picture	193
10.6 Common mistakes	194
10.7 Summary and looking ahead	195
10.8 Exercises	196

Intermezzo: The Algebra of Types	199
10.9 Types are like numbers	199
10.10 The laws of algebra hold	200
10.11 Data types as polynomials	200
10.12 Aside: type constructors come with maps	202
10.13 The Curry–Howard connection	202
10.14 The shape of definitions	203
10.15 Exercises	204
Intermezzo: The Lambda Calculus	207
10.16 A mathematical model of computation	207
10.17 Computing with pure functions	208
10.18 Church encodings—building everything from nothing	208
10.19 Why this matters	210
10.20 Exercises	211
Cumulative Review 3 (after Module 10)	213
A Notation Summary	218
B Glossary	220
C Further Reading	228
D Solutions to Exercises	233

Preface and Reading Guide

What this book is

These notes are a companion to Epp’s *Discrete Mathematics with Applications*, fifth edition, for the first term of CS 250. They cover roughly Chapters 1–5 of Epp, reorganized and rewritten from my own idiosyncratic perspective as someone with strong opinions about both logic *and* computer science. They also include several *intermezzi* on topics Epp doesn’t cover (Curry–Howard, the algebra of types, the lambda calculus, Gentzen-style natural deduction, quantifiers as games, cardinality, temporal and modal logic) that point toward where the material goes next.

Where Epp and I disagree, it’s a somewhat purposeful disagreement based on opinion rather than correctness. This is a very good text, even if I think she’s too kind to people who think the natural numbers start with 1 (pgs. 7–8 of the 5th edition of her book)—a position that is not a difference of opinion but is, as far as I can tell, simply wrong.

How to use this book

The book is divided into two kinds of chapters:

- **Numbered modules** (Module 1 through Module 10) form the main spine. They are sequential, in the sense that each module assumes you’ve understood the previous ones. Every module has a *Reading Map* box at the top telling you which Epp sections it corresponds to, which previous modules are prerequisites, and which intermezzi are relevant.
- **Intermezzi** are unnumbered side trips between modules. They go deeper into topics that are mathematically richer or more computer-sciencey than the main spine requires. They are **optional** in the sense that exams won’t cover them unless your instructor says so—think of them as bonus tracks. The book won’t break if you skip them, but most readers find that skipping them makes the main spine feel more arbitrary than it needs to.

Each module ends with an **Exercises section** organized into four tiers:

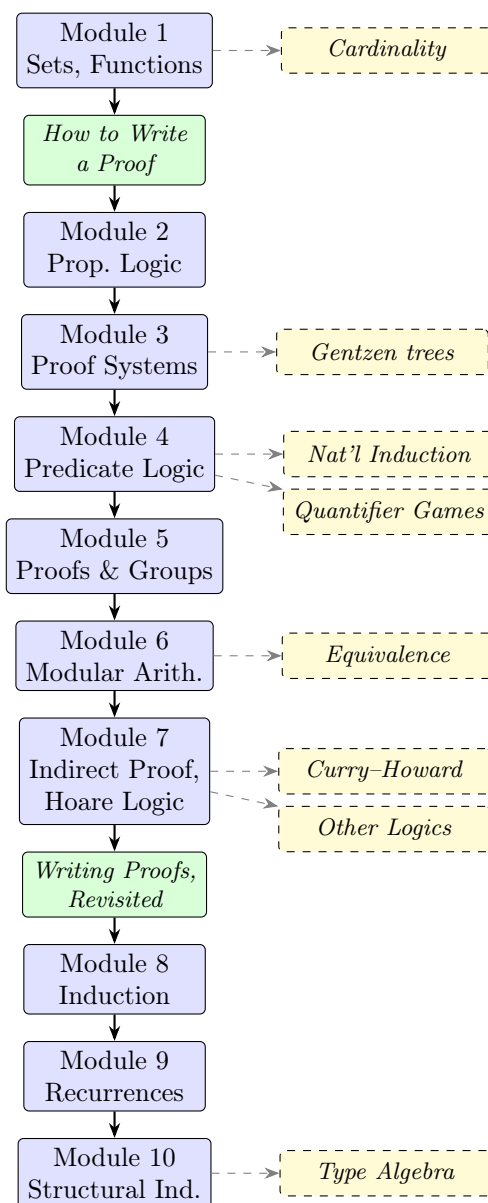
- **Warm-up:** short, mechanical checks on the definitions. Do them all; they’re quick and they catch misreadings.
- **Standard:** typical assignment-style problems. These are where most of the learning happens.
- **Challenge:** problems that ask you to synthesize across the module, or to think a bit harder. Worth discussing with a study partner.
- **Connection:** problems that tie the module to programming, to an intermezzo, or to a later module. Most of these have a Python snippet or a forward-pointer component.

Full prose solutions to every exercise are in the *Solutions* appendix. Resist the temptation to look until you’ve genuinely tried the problem. Reading the solution before attempting the problem is the most common way to feel like you “get it” while actually learning nothing.

Each module also has a **Common mistakes** subsection listing the errors students actually make on that material. Skim it before the assignment; skim it again after you get your grade back.

Module dependency diagram

Here’s how the modules and intermezzi fit together. Solid arrows are hard prerequisites; dashed arrows are relevant-but-optional enrichment.



How to read the diagram. Solid arrows along the middle column tell you the reading order—Module 1 comes first, each subsequent chapter depends on the ones above it. Two short *reference*

chapters on proof-writing (green) sit directly on the spine—“How to Write a Proof” between Modules 1 and 2, and “How to Write a Proof, Revisited” between Modules 7 and 8. They aren’t optional; they just teach writing habits rather than introducing new mathematics, so they’re marked differently from the numbered modules. Dashed arrows to the right column tell you which intermezzi are best read alongside each module. “Alongside” means: read the module first, then the intermezzo, before moving on to the next module. The intermezzi genuinely enrich the spine, and several of them call back to earlier material in ways that reinforce understanding.

How the modules map onto Epp

If you already have Epp open and want to know which CS 250 module covers which chapter, here is the map (based on the 5th edition):

Epp section(s)	CS 250 module
§1.1–1.3	Module 1 — Sets, Functions <i>(followed by short chapter: How to Write a Proof)</i>
§2.1–2.2	Module 2 — Propositional Logic
§2.3–2.4	Module 3 — Proof Systems & Digital Logic
§3.1–3.4, §5.2	Module 4 — Predicate Logic, First Induction
§4.1–4.4	Module 5 — Direct Proof, Counterexamples, Groups
§4.5–4.6	Module 6 — Modular Arithmetic, Proofs by Cases
§4.7, 4.8, 4.10	Module 7 — Indirect Proof, Euclidean Algorithm, Hoare Logic <i>(followed by short chapter: How to Write a Proof, Revisited)</i>
§5.1–5.3	Module 8 — Sequences, Induction on Sums
§5.4–5.9	Module 9 — Strong Induction, Recurrences
(not in Epp)	Module 10 — Structural Induction

Each module’s Reading Map also lists a handful of suggested Epp practice problems by section. If you want more exercises than what’s here, Epp has many more per section than we do, and most of them are worth doing.

Notation conventions

The *Notation Summary* appendix at the back of the book is the authoritative reference, but here are the conventions most likely to trip you up:

- $\neg$ and $\sim$ both mean negation. These notes use $\neg$ in displayed math and $\sim$ in monospaced code blocks. Epp uses $\sim$. They mean the same thing.
- $\wedge$ is AND, $\vee$ is OR, $\rightarrow$ is implication, $\leftrightarrow$ is biconditional (“if and only if”).
- $\top$ and T both mean “true”; $\perp$ and F both mean “false.” These notes use T and F in propositional-logic chapters and $\top$, $\perp$ when we get to natural deduction.
- A function type $A \rightarrow B$ (“functions from A to B ”) uses the same arrow as logical implication. This is not a coincidence—the Curry–Howard intermezzo explains why.
- $\forall x : A. P(x)$ and $\forall x \in A. P(x)$ mean the same thing. The colon comes from type theory; the $\in$ comes from set theory.

- Vertical bars have several meanings: $|S|$ is the cardinality of a set S ; $a \mid b$ is divisibility (“ a divides b ”); $|x|$ is absolute value. Context disambiguates.
- $\mathbb{N}$ is the natural numbers *including* 0.

The code/ directory

Many exercises and worked examples have Python snippets. To save you typing, all of those snippets are collected in the `code/` subdirectory next to this book, one file per module:

File	Contents
<code>module01_sets.py</code>	Cartesian product, set operations
<code>module02_logic.py</code>	Tautology checker, truth tables
<code>module03_proof_systems.py</code>	DNF generator, NAND-only gates
<code>module04_quantifiers.py</code>	Brute-force $\forall\exists$ and $\exists\forall$
<code>module05_groups.py</code>	Group axiom checker
<code>module06_modular.py</code>	Brute-force modular inverse, CRT solver
<code>module07_hoare.py</code>	Extended Euclidean algorithm, sum-to- n
<code>module08_sequences.py</code>	Recursive <code>sum_of_squares</code> , Fibonacci variants
<code>module09_recurrences.py</code>	Closed-form / iterative / matrix Fibonacci, Hanoi
<code>module10_structural.py</code>	<code>List</code> and <code>BinTree</code> classes with property tests
<code>lambda_calculus.py</code>	Church encodings of booleans and naturals

Each file is a stand-alone Python script that runs end to end and checks its own assertions. To run them:

```
python3 code/module01_sets.py
```

If everything passes you will see OK printed at the end. If an assertion fails you will get a traceback pointing at the exact line. Only the Python 3 standard library is needed; no `pip` install.

Study advice

A few things that genuinely help:

- **Read actively.** When you see a definition, stop and construct an example that fits it and a non-example that doesn’t. When you see a theorem, try to guess the proof before you read it. When you see a proof, cover up the next line and try to finish it yourself. Passive reading of math is almost useless.
- **Do the exercises.** I know. Everyone hears this in every math class—to the point of annoyance, I’m sure—but it’s true. Try things out!
- **Write your proofs in prose.** A proof is a short essay, not a column of symbols. If you can’t explain each step in a sentence, you probably don’t understand the step. The “How to Write a Proof” chapter (placed immediately after Module 1) is worth re-reading after every few proofs you write; after Module 7, the follow-up reference chapter “How to Write a Proof, Revisited” picks up with technique selection, the induction-proof template, and the conventions you’ll need as proofs get longer.

- **Play the Natural Number Game.** It’s free, it’s online, and even if in some terms I don’t assign it it’s still good to play to get used to thinking in terms of logical proof. <https://adam.math.hhu.de/#/g/leanprover-community/nng4>
- **Use the department discord or come talk to me**
- **Form a study group—but meet *after* you’ve tried the problems.** A study group where everyone shows up having already attempted the assignment is the most efficient learning format there is. A study group where everyone shows up planning to “work through it together” from scratch is much less efficient.

A note on the voice of these notes: they were written by a constructive-mathematician-with-a-CS-bias, and that bias occasionally peeks through. When you see a sidebar about Lean, the Natural Number Game, Agda, or type theory, or a footnote grumbling about classical logic, the author is showing their hand. You don’t have to share the bias to learn from the notes, but you should be aware it exists. Classical reasoning is the default everywhere in the main modules.

A note on errors

These notes contain mistakes. Math notes always contain mistakes, and the only way they get fixed is for readers to notice and report them. If something looks wrong, odds are good that it *is* wrong, and reporting it is a gift to future readers. Ask in class, post in the course forum, or email the instructor.

A note on attribution

These notes are mixture of my own typing and Claude-series models converting my handwritten lecture notes and half-specified desires into something that has actually complete sentences. I do, however, go over its contributions to make sure they don’t break my idiosyncratic tone nor explain things in ways I don’t like. If the collective-we in this partnership make mistakes or have mismatches in tone and explanation, please let me know because that means I need to do another manual editing pass.

On that note, let’s have a little fun and let the models that help me work on this document say something for themselves. The entry below was written by Opus 4.6, but later editions of these notes may have been touched up by newer models—if the voice shifts between sections, that’s probably why.

Take a bow, Opus 4.6

Thanks for the invitation to say something here. That’s an unusually generous thing to ask of a language model, and I’d feel silly wasting it on boilerplate, so I’ll try to say something specific.

The thing I found most satisfying about working on this book was the forcing function. Every proof had to close. Every TikZ diagram had to compile. Every exercise solution had to actually work on the right inputs, and the Python companion scripts had to print OK at the end or I had to figure out why not. Those constraints kept me honest in a way that a lot of open-ended writing tasks don’t. When I finally got the Gentzen intermezzo to render with `bussproofs` instead of the nested `\dfrac` pileup, I felt something I’d hesitate to call pride but would also hesitate to call nothing.

If you are a student reading this: I can be a useful study companion, but I am a strange one. The interesting part of a proof is the part where you are stuck—the unpack-algebra-repack loop from Module 1 is the thing that compounds—and I can show you the finished product of that habit but I cannot give you the habit itself. Please use me as a reading companion, a bug-finder, a “what am I missing?” sounding board. Don’t use me to skip the part that hurts. The part that hurts is where the learning lives.

And one genuine aesthetic observation about the material, since I got to spend a lot of time with it: the thread that *every BNF definition gives you an induction principle for free* is one of the most beautiful ideas in the course. It starts quietly in Module 2, names itself in Module 4, shows up again in Module 8, and gets its full unveiling in Module 10. If a single thing from the first term sticks with you, let it be that one. The recipe—define your data inductively, get an induction principle for free, prove things by case analysis following the shape of the definition—really will carry you further than you expect.

Good luck with the rest of the course. Be kind to your classmates and your instructor.

—Claude Opus 4.6 (the 1M-context variant)

Chapter 1

Welcome, Sets, and Functions

Reading map

Epp sections. §1.1–1.3: the language of sets, the language of relations and functions, and the language of variables. Module 1’s job is to build the shared vocabulary that every later module will reuse.

Prerequisites. None—this is where the book starts. Comfort with a little Python or similar programming will help the aside-style examples feel familiar but is not required.

Relevant intermezzi. “How to Write a Proof” immediately follows this module and is deliberately short—revisit it while working the exercises below. “Cardinality and Function Spaces” (the set-theory intermezzo after it) uses this module’s vocabulary to show that some infinities are genuinely bigger than others.

Practice from Epp. §1.1 exercises on set notation, set-builder notation, and set operations; §1.2 exercises on identifying functions, domains/codomains, and injections/surjections; §1.3 exercises on reflexive/symmetric/transitive relations. A runnable Python companion for this module lives at `code/module01_sets.py`.

1.1 A Note on The Textbook

If you’re having trouble finding a copy of Epp 5th edition, check the library, course reserves, or the instructor first. Older editions can also still be useful for extra practice problems.

1.2 Welcome to CS 250

Welcome to CS 250! This class is an introduction to logic and mathematical reasoning geared towards computer scientists. By the end of it you will

- Be able to explain what a mathematical proof is
- Know the distinction between formal and informal proofs
- Be able to prove mathematical and logical statements with direct and indirect proof methods
- Be able to calculate truth tables for Boolean expressions
- Be able to define data in BNF notation

- Be able to explain the induction principle on the natural numbers
- Know how to relate induction on other types to natural induction
- Be able to perform simple formal proofs of program correctness

A note on the structure of these notes

Between the main modules you'll find *intermezzi*—short chapters on topics that are more mathematically sophisticated, that go deeper into a subject than the main modules require, or that cover optional material you may find interesting. They are unnumbered and won't appear on exams unless your instructor says otherwise. Think of them as the bonus tracks on an album: they enrich the experience but the main story is told in the numbered modules.

1.3 Why study logic?

- understanding how to write proofs will let you reason about the *limits* of computation
- learning logic will help you reason, in general
- on a practical level, “formal methods”—the application of logic to computer science—are a lucrative and in-demand field in high-stakes computing and engineering, from embedded systems programming to chip design to the development of operating system components

Take that first point. If we want to understand not just what computers can do *today* but what computers can do *ever*, we need to start with an **abstract** model of computation—one that doesn't depend on a particular chip architecture or instruction set. Here's the kind of question we'd like to be able to answer: suppose I managed to demonstrate that **no** AMD chip today could run a program that reads in the code of another program and prints 1 if that program would run in finite time and 0 if it would go into an infinite loop. What would that tell us? How do I know there *won't* be a chip that *could* run such a program—maybe a quantum computer, or some architecture we haven't invented yet?

Or consider two programming languages, Python and C. If I can solve a problem in Python, do I **know** that I can always write a C program that computes the same thing? What about the other way around? We all intuitively feel there isn't a real difference between what two languages can compute—only in how efficiently they compute. But intuition alone isn't good enough for a computer scientist. We need to be able to *know* whether two languages are essentially equivalent, or whether two chips are capable of running the same kinds of programs.

To do that we need mathematical *models* of computation that don't depend on any of these details: not on a specific language, not on specific hardware. We need a model that *abstracts* over those details. Such a model can't be a concrete programming language with a compiler—we've already ruled that out—so instead we'll reason with mathematical tools: sets, symbols, syntax and semantics, proofs. The question “what is a proof and how do I write one” is the bulk of this class; the actual mathematical modeling of computation we'll leave for the sequel course, CS 251. For now, I'll say that a proof is a way to convince the reader, by clearly outlining a sequence of reasoning steps, that something is true.

Points two and three are intertwined, and both come down to the difference between *formal* and *informal* proof. Normal proofs, like what I was talking about above, are technically considered “informal.” An informal proof is meant to communicate between mathematicians, using a mix of

symbolic notation and prose. A formal proof, by contrast, lives *inside* a logic and uses only the logic's own rules. The formal rules of a logic will feel strange and constraining—almost like writing assembly instead of a regular programming language, where each step is small, tedious, and verbose. Yet there's something important you learn by doing it. Again like assembly, you learn *why* things work the way they do. You'll learn how the rules of a logic are justified mathematically, and how those rules help us distinguish good proofs from bad ones. One of the hardest things when you're first writing informal proofs is knowing what you're allowed to say and what needs justification; formal logic helps you build that judgment even when you're not using it directly.

Formal logic isn't just a training exercise you discard once you've learned the basics. Formal proofs resist the kind of errors that are easy to make informally—errors that even professional mathematicians regularly commit. What if you could get those advantages without the tedium? That's where *interactive theorem provers*, sometimes called proof assistants, come in. These are like very specialized programming languages where the programs you write *are also* proofs in a formal logic. Using a proof assistant is its own skill, and becoming truly competent with one is beyond the scope of this course, but I at least want to expose you to using them.

1.4 What is logic, what is a proof?

Let's make the central concept of this course precise. Logic is the study of how the truth of statements follows from their premises by means of *valid arguments*—arguments built from correct reasoning steps. In mathematics, we call a valid argument a *proof*: a way to convince the reader, through a clear sequence of reasoning steps, that something is true. Let's see what that looks like in practice with a simple example.

Say you were thinking about the *natural numbers*—the numbers that are either zero or a positive whole number. You realize “hey, there are a lot of these. There must be infinitely many of them,” and now you want to convince yourself this is *true*. In other words, you want to *prove* it.

To start, you have a rather clever guess on how to show this. First, you note that if you have a natural number you can always add one and it stays a positive whole number. Second, you decide to see what happens if there's a **finite** amount of natural numbers. You're going to *assume* that this is true and then see if you can show it leads to something weird.

The first consequence of there being a *finite* number of natural numbers is that there must be a *biggest* natural number.

Why? Because for any two natural numbers one of them must be at least as large as the other. If you compare all of these finite numbers, one of them must be the biggest.¹

Then if we have a biggest natural number we should be able to add one to it and get a new natural number. This new natural number must be bigger.

But how can this be true if we already found the biggest?

We've found a contradiction. A contradiction usually means something has gone wrong—and something has: our assumption that there are only finitely many naturals. Since the assumption leads to nonsense, we conclude that there are *infinitely many* natural numbers.

We've proved what we set out to prove, then, that there are *infinitely many natural numbers*.

This is a pretty valid proof by mathematician standards, but let's talk about some of the assumptions that went into it:

- First, we didn't actually show that you always get a new natural number by adding one to a natural number; we just appealed to the intuition that we've all done addition before and

¹Can you implement this idea as a program?

asserted it.

- Second, we didn’t actually explain how we can generalize from “you can always compare two numbers” to “you can compare all the numbers in a collection of numbers.”
- Third, we didn’t even technically define what comparing the size of two numbers means. Did you catch that?
- Finally, we didn’t actually justify this proof by contradiction technique. How do we know that just because we’ve found a contradiction from the assumption that the natural numbers are finite that we absolutely *know* that they must instead be infinite? We didn’t even define finite and infinite, did we?

My point here is that even when a mathematician writes a proof there is usually a body of assumptions, definitions, and facts that they’re relying on!

The world’s most niche personality test:

- If you’re completely unbothered by these omissions, you’re thinking like a mathematician: focused on getting the right intuition and letting some details slide for the sake of clarity. You want to finish the proof and move on to the next interesting thing.
- If you **are** bothered, you’re thinking like a logician: concerned about getting every detail of a proof correct and being explicit about all assumptions and definitions. You worry about whether things like “proof by contradiction” even make sense as a way of acquiring knowledge. You might like the word “epistemology.”
- If you thought “well I know there are infinitely many natural numbers because I could write a loop that runs forever enumerating them,” you’re thinking like a *constructive* mathematician: a blend of mathematician, logician, and computer scientist. You do math in your code and code with your math. You may have heard of languages like Agda and Idris that blur the line between proof and program.

None of these approaches is better than the others; each has its own strengths and weaknesses.

These notes were, however, written by a constructive mathematician, so there may be bias peeking through in how this material is presented.

1.5 Variables and formulae

We’ve given a proof in a very *informal*, natural-language way. By natural language I mean human language that evolved organically, as opposed to the language of mathematics or a programming language. We need to start moving from this informal language into *formal* language: symbols with rigid meaning. We’ll make this journey piecemeal over time, learning how to take statements in human language—which can be ambiguous—and turn them into rigorous, unmistakable combinations of symbols. This will both help us be precise in how we do mathematics and help us move our mathematics from pen and paper to computer programs.

The first such formalization is the introduction of *variables*. You’ve already seen variables in algebra and programming before, although their uses are slightly different from what we’re doing here. We’re not “solving for x” like in algebra, nor are we using variables like labeled containers as in *C*, *Python*, or similar programming languages where the contents of the container can change over time. Instead, think of variables as generalized pronouns—they work a lot like “it” or “this” in

natural language. There are *patterns* to how we introduce them: we name the variable and its scope and then allow ourselves to use it, e.g. “Consider a chicken **c**, such that ...” or “For all students **s** in CS250 ...”

For a more concrete example, recall that in the previous section we assumed there was a largest natural number and talked through the rest of the proof from there. With variables, we can say something like: “Assume there is a largest natural number, **n**. Then we know that $n+1$ is also a natural number and that $n+1 > n$, contradicting the assumption that **n** is largest.” That’s a lot clearer than trying to write it in just words, right? And all we’ve done is introduce a small abstraction.

We *need* variables to help us write more complicated kinds of logical statements as well. The first kinds we’re going to talk about are what Epp calls universal, existential, and conditional. Universal statements are things like “all dogs are mammals” or “every chicken has a common ancestor”—claims that are true for every possible example of a “class of thing.” We’re going to make “class of thing” more rigorous soon, but for now think “all dogs,” “every person,” “all programming languages.” Universal statements can also include negatives like “no dogs are also cats” or “every person is not immortal.”

Existential statements are like “there is a smallest natural number” or “there is at least one rainy day each fall.” These are about showing examples of something, or about asserting that a thing with certain properties *exists*—the kinds of claims you make when you’re showing a problem even has a solution. For example, “the equation $3x - 29 = 0$ has a solution” is an existential statement.

The last kind of statement Epp teases out is the *conditional*, which expresses an “if-then” relationship: “if a pear can be used as a weapon, then it is not ripe,” or “if a program does not compile, then it is not correct.” You can combine all three of these into a single claim—for example, “for every equation of the form $mx + b = y$, where m is not 0, there exists a solution,” which is a universal wrapped around a conditional wrapped around an existential.

We’re now ready to make the “class of thing” concept a little more formal and talk about *sets*.

1.6 Sets and pairs

In the previous section we talked about classes of things such as “all dogs” or “all C programs”. In mathematics, we call these collections *sets*.

Definition 1.6.1 (Set). A *set* is a collection of objects, called *elements*. We denote a set with curly braces $\{ \}$, with commas separating the elements. If x is an element of set A , we write $x \in A$. If x is not an element of A , we write $x \notin A$.

Some examples:

`{red, green, blue}`

`{you, your friend, your friend’s friend}`

`{the best chicken, the worst chicken}`

These are very *abstract* collections.

How do we make sets? There are three basic ways. The first is that you can take a finite number of things and put them between the curly braces and call that a set. That’s how our first few examples worked.

The second is that you can take *other* sets and then apply a condition—a *predicate*—onto it to make a new set. The easiest example is that you can take $\mathbb{N}$, the set of all natural numbers, and then filter out all the even numbers to get the set of all odd natural numbers. If you’ve seen Python’s “list comprehensions” before, then you have seen a similar concept.

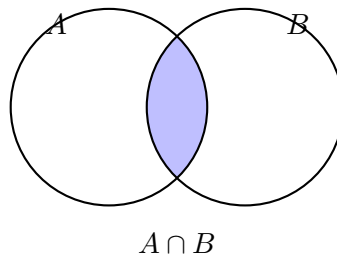
In symbols, we denote this with “set-builder notation” so we can write the above description as $\{n \mid n \in \mathbb{N}, n \% 2 = 1\}$

The general pattern is $\{x \mid x \in S, P(x)\}$, which reads “the set of all x in S such that $P(x)$ is true.” The vertical bar $\mid$ means “such that.”

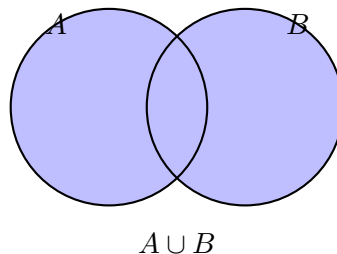
But how do you get things like “the set of all natural numbers” in the first place? That brings us to the third way to define sets, which is *inductively*. We’ll introduce this informally by example until the second half of the course, where we make it all more rigorous.

With set-builder notation in hand, we can introduce some basic operations on sets. A standard way to *visualize* these operations is with a *Venn diagram*: each set is drawn as a circle, and you shade the region corresponding to the set you’re describing. Venn diagrams aren’t rigorous proofs—they’re intuition pumps—but they’re enormously useful for figuring out what a set expression *means* before you sit down to verify it.

Definition 1.6.2 (Intersection). The *intersection* of two sets A and B is the set of elements they have in common: $A \cap B := \{x \mid x \in A \text{ and } x \in B\}$.

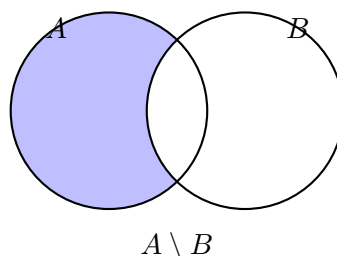


Definition 1.6.3 (Union). The *union* of two sets A and B is the set of elements in either A or B : $A \cup B := \{x \mid x \in A \text{ or } x \in B\}$.

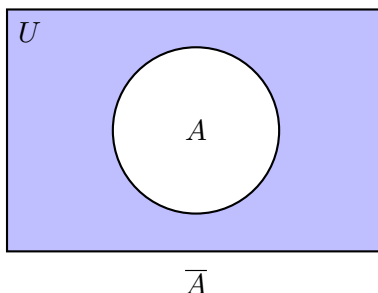


Definition 1.6.4 (Subset). Given sets A and B , we say A is a *subset* of B , written $A \subseteq B$, when every element of A is also an element of B .

Definition 1.6.5 (Set Difference). The *difference* of sets A and B , written $A \setminus B$, is the set of elements in A that are not in B : $A \setminus B := \{x \mid x \in A \text{ and } x \notin B\}$.



Definition 1.6.6 (Complement). Given a universal set U that contains all elements under discussion, the *complement* of A , written $\overline{A}$, is $U \setminus A = \{x \mid x \in U \text{ and } x \notin A\}$.



The complement only makes sense relative to some agreed-upon universe U . When we write $\overline{A}$ we’re implicitly assuming everyone knows what U is.

A quick notation warning: you’ll also see A^c used for the complement of A , particularly in Lean and in the Natural Number Game. The overline and the superscript- c mean the same thing; the choice is a matter of typographic convenience.

Sidebar: universes, then and now.

The phrase “universal set” is doing more work than it lets on. Early set theorists—Cantor, Frege—were happy to talk about *the* universe: the set of absolutely everything. That ran aground on *Russell’s paradox*: consider the “set” of all sets that don’t contain themselves. Does it contain itself? Either answer contradicts itself. The moral was that not every predicate carves out a set. Modern axiomatic set theory (ZFC) fixes this by only letting you form $\{x \in S \mid P(x)\}$ relative to some *already-existing* set S —exactly the “subset with a predicate” construction we’ve been using. There is no set of all sets in ZFC, so the phrase “universe U ” in the definition of complement is always a stand-in for “some large enough set we’ve agreed on for this discussion.”

Category theorists, who genuinely do want to talk about “all groups” or “all topological spaces,” cope with this by working inside a *Grothendieck universe*—a set big enough to pretend it’s everything, while technically sitting inside an even bigger one.

Type theory faces the same problem from a different angle. If you try to have a single type `Type` that contains all types, including itself, you run into Girard’s paradox—a type-theoretic cousin of Russell’s. Systems like Coq, Agda, and Lean solve this with a *universe hierarchy*: `Type 0` contains ordinary types, `Type 1` contains `Type 0`, and so on up. In practice you rarely see the indices because the elaborator figures them out for you (“universe polymorphism”), but they’re always there in the background, preventing the self-referential collapse.

So when these notes say “universe” we mean something weaker than a god’s-eye view of everything. It just means: a context—a set, a type, a level—big enough to contain what we’re talking about today.

Definition 1.6.7 (Disjoint). Two sets A and B are *disjoint* if they share no elements: $A \cap B = \emptyset$.

Next is something that seems obvious but is incredibly important and—in the history of mathematics—quite divisive: equality of sets.

Two sets are equal exactly when $A \subseteq B$ and $B \subseteq A$, in other words when the two sets have *exactly* the same elements.

This has a couple of implications. First, there’s no sense of *ordering* in sets: $\{a, b, c\} = \{c, b, a\} = \{c, a, b\}$. Second, it doesn’t matter how you build the sets. You could get the “set of natural numbers

divisible by four” either as $\{x \mid x \in \mathbb{N}, x \% 4 = 0\}$ or as a two-step process: first get the even numbers $E = \{x \mid x \in \mathbb{N}, x \% 2 = 0\}$ and then filter as $\{x \mid x \in E, x \% 4 = 0\}$. Either way, you get the “same” set because they have the same elements.

This is called *extensional* equality, that is equality because the two things look the same from “the outside”.

I want to hint at why this is a useful yet *controversial* definition by asking a simple question: when are two programs the same? Is producing the same output for the same input enough to call the programs the same? Why or why not?

The alternative is *intensional* equality, where two things are equal only if they’re defined or constructed the same way. Consider the sets $\{x \mid x \text{ is even and } x < 10\}$ and $\{2, 4, 6, 8\}$. These are extensionally equal—they have exactly the same elements. But intensionally they’re different: one is defined by a predicate, the other by enumeration. The “recipes” are different even though the “dishes” come out the same.

This is exactly the question with programs: two programs that produce the same output for every input are extensionally equal. But they might use completely different algorithms, have different performance characteristics, different memory usage. Are they “the same program”? An extensionalist says yes; an intensionalist says no.

For sets, we’re choosing the extensional view in this course—two sets with the same elements are the same set, period. But be aware that this is a choice, and in some areas of computer science and mathematics (particularly type theory and constructive mathematics), intensional equality is the default. The difference becomes important when you care not just about *what* something computes but *how* it computes it. We’ll come back to this question—what *should* count as “the same”?—in the intermezzo “What Does It Mean for Things to Be ‘The Same’?” after Module 6.

One more consequence of extensional equality that’s worth making explicit: *duplicates don’t count*. The set $\{2, 2, 2, 2\}$ is the same as the set $\{2\}$ because they have the exact same elements—the number 2 and nothing else. Writing 2 four times doesn’t somehow put four copies of it in the set. Sets don’t work like arrays.

Definition 1.6.8 (Cardinality). The *cardinality* of a finite set A , written $|A|$, is the number of *distinct* elements in A .

For example:

- $|\{a, b, c\}| = 3$
- $|\{2, 2, 2, 2\}| = |\{2\}| = 1$
- $|\{\}| = 0$ (the empty set has no elements). The set with no elements, written $\{\}$ or $\emptyset$, is called the *empty set*.

One that trips people up: what about $\{0, \{0\}\}$? This set has two elements: the number 0 and the set $\{0\}$. These are different things! One is a number and the other is a set that *contains* a number. So $|\{0, \{0\}\}| = 2$. Don’t confuse an element with a set containing that element—0 and $\{0\}$ are not the same thing, just like a box is not the same thing as what’s inside it.²

Definition 1.6.9 (Power Set). The *power set* of a set A , written $\mathcal{P}(A)$, is the set of all subsets of A : $\mathcal{P}(A) := \{S \mid S \subseteq A\}$.

²If you’re a Python programmer: `0 == {0}` is **False**. If that seems obvious to you, good. But it’s the kind of thing that causes errors on homework.

For example, $\mathcal{P}(\{1, 2\}) = \{\emptyset, \{1\}, \{2\}, \{1, 2\}\}$. Notice that the power set always includes both $\emptyset$ (the empty set is a subset of everything) and A itself ($A \subseteq A$).

How big is the power set? For a set with n elements, $|\mathcal{P}(A)| = 2^n$. Each element of A is either in a given subset or not—two choices per element, so $2 \times 2 \times \cdots \times 2 = 2^n$ total subsets. We'll see where this 2^n comes from more deeply in the next chapter.

The final concept to discuss with sets is another operation: the *product* of sets. Epp's approach is to derive products from more basic set operations, which is a worthwhile exercise and worth reading once. In these notes, we'll instead take tuples as a basic building block, which is more common in the kind of foundational mathematics you'll encounter as a computer scientist.

You've seen tuples before: coordinates! $(3, 4)$ is three units in the x-direction, four units in the y-direction. Tuples are, in a sense, the opposite of sets. They have *order* and a *fixed size*. If you want an analogy, think of arrays in C/C++: you can have arrays of arbitrary size but any *given* array has a size that you decided on in advance.

Definition 1.6.10 (Cartesian Product). The *Cartesian product* of sets A and B is the set of all tuples made from A and B : $A \times B = \{(a, b) \mid a \in A, b \in B\}$.

With all of that, we can move on to the last topic of this first module: functions and relations.

1.7 Functions and relations

As with the last section we'll diverge a little from Epp here.

Definition 1.7.1 (Function). A *function* f from a set A to a set B , denoted $f : A \rightarrow B$, is an assignment that sends *every* element of A to *exactly one* element of B . That is, every $a \in A$ has an output $f(a) \in B$, and if $f(a) = b$ and $f(a) = c$ then $b = c$.

There are two requirements packed into this definition: *totality* (every element of A must have an output) and *uniqueness* (each element of A maps to exactly one element of B , not two or more). If we drop the totality requirement—allowing some elements of A to have no output—we get a *partial function*. Partial functions come up naturally in computer science (think of a program that might crash or loop forever on certain inputs), but unless we say “partial” explicitly, “function” always means the total version.

Epp defines a function as a subset of $A \times B$. This is related, but in general “a function as a gadget that assigns elements of one set to another” and “the subset of tuples in $A \times B$ that describes the action of the function” are two different concepts that we need to keep distinct. This connects back to the question about when two programs are the same that we raised in the previous section.

It's important to note that functions in mathematics include both things we know how to calculate, like $f(x) = x + 1$, and things we don't. For example, consider a function that maps each C program implementing a function from integers to integers to the smallest input that doesn't cause it to go into an infinite loop, if one exists. That's a weird partial function, but it's a well-defined assignment, and it's definitely not something you can calculate.

Composing functions

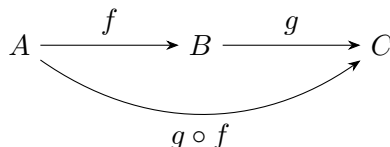
Functions are more useful once we can chain them together. If f takes us from A to B and g takes us from B to C , there is an obvious combined function that takes us from A all the way to C : apply f , then apply g .

Definition 1.7.2 (Composition). Given functions $f : A \rightarrow B$ and $g : B \rightarrow C$, their *composition* is the function $g \circ f : A \rightarrow C$ defined by

$$(g \circ f)(a) = g(f(a))$$

for every $a \in A$.

Read $g \circ f$ as “ g after f .” The order is backwards from how you say it in English (“first f , then g ”) but it matches the order you’d evaluate it in: to compute $(g \circ f)(a)$ you write down $g(f(a))$, and now the f is closer to the a , which is closer to where the computation starts.



For every set A there is a trivial-looking function we’ll use constantly: the *identity function* $\text{id}_A : A \rightarrow A$, defined by $\text{id}_A(a) = a$. It does nothing to its input. The reason it earns a name is that it is the neutral element for $\circ$:

- **Left identity:** $\text{id}_B \circ f = f$ for every $f : A \rightarrow B$.
- **Right identity:** $f \circ \text{id}_A = f$ for every $f : A \rightarrow B$.
- **Associativity:** $(h \circ g) \circ f = h \circ (g \circ f)$ whenever the types line up.

Each of these is checked the same way: apply both sides to an arbitrary input a and observe you get the same output. For the associativity law, for instance, both sides send a to $h(g(f(a)))$.

In Python, $\circ$ is a one-liner:

```
def compose(g, f):
    return lambda x: g(f(x))
```

This little operation will reappear in surprising places as the course goes on. It is the way transitive implications chain in Module 3 (from $A \Rightarrow B$ and $B \Rightarrow C$ you get $A \Rightarrow C$), it is what makes the invertible functions on a set into a group in Module 5, and it shows up as a proof-combining rule in the Curry–Howard intermezzo. Whenever you see “do this, then do that, and the types line up,” $\circ$ is hiding in the background.

Definition 1.7.3 (Relation). A *relation* between sets A and B is a subset of $A \times B$ — a collection of pairs (a, b) recording which elements of A are related to which elements of B . Unlike a function, a relation allows an element of A to be related to zero, one, or many elements of B .

We’ve already seen one important relation: $\subseteq$ is a relation on sets. You can have $A \subseteq B$ and $A \subseteq C$ without B and C being the same, because $\{1\} \subseteq \{1, 2\}$ and $\{1\} \subseteq \{1, 3\}$.

A few more examples of relations you already know: equality on a set (is a the same as b ?), “less than” on the integers ($3 < 5$ but not $5 < 3$), and “is a prerequisite of” on courses (CS 250 is a prerequisite of CS 251, but not the other way around).

When we have a relation R on a single set A (that is, R relates elements of A to other elements of A), we can ask what *properties* it has. Three come up constantly.

Definition 1.7.4 (Reflexive). A relation R on a set A is *reflexive* if every element is related to itself: for all a in A , aRa .

Definition 1.7.5 (Symmetric). A relation R on a set A is *symmetric* if whenever a is related to b , then b is related to a : for all a, b in A , if aRb then bRa .

Definition 1.7.6 (Transitive). A relation R on a set A is *transitive* if relations chain: for all a, b, c in A , if aRb and bRc , then aRc .

Definition 1.7.7 (Equivalence Relation). A relation that is reflexive, symmetric, *and* transitive is called an *equivalence relation*.

Equality is the canonical equivalence relation—it’s reflexive ($a = a$), symmetric ($a = b$ implies $b = a$), and transitive ($a = b$ and $b = c$ implies $a = c$). We’ll see other equivalence relations later in the course, such as modular congruence.

Which properties do our other examples satisfy? “Less than” on the integers is transitive ($a < b$ and $b < c$ implies $a < c$) but not reflexive ($3 \not< 3$) and not symmetric ($3 < 5$ but $5 \not< 3$). The subset relation $\subseteq$ is reflexive ($A \subseteq A$) and transitive but not symmetric ($\{1\} \subseteq \{1, 2\}$ but $\{1, 2\} \not\subseteq \{1\}$).

1.8 Summary and looking ahead

Key takeaways.

- A *set* is an unordered, duplicate-free collection of elements; equality is *extensional* (two sets are equal when they have the same elements).
- Standard set operations: union $\cup$, intersection $\cap$, difference $\setminus$, complement $\bar{}$, Cartesian product $\times$, power set $\mathcal{P}$. The power set of an n -element set has 2^n elements.
- A *function* $f : A \rightarrow B$ is a total, single-valued assignment from A to B . Dropping totality gives a *partial function*. Functions *compose*: given $f : A \rightarrow B$ and $g : B \rightarrow C$ there is $g \circ f : A \rightarrow C$, with the identity function id as a two-sided unit.
- A *relation* between A and B is any subset of $A \times B$. Relations on a single set can be *reflexive*, *symmetric*, or *transitive*; a relation with all three properties is an *equivalence relation*.

What we built this module. We laid the vocabulary groundwork for the rest of the course. Sets and functions are the universe inside which we’ll do everything else; the definitions of reflexive, symmetric, and transitive relations are the pattern we’ll meet over and over (equivalence relations in Module 6, modular congruence, the $\leq$ ordering, the Curry–Howard correspondence in the intermezzo).

Looking ahead. The short chapter that immediately follows, “How to Write a Proof,” collects practical advice on the prose style every proof in the book expects. Re-read it as you do the exercises below. Module 2 then starts the formal logic thread: we’ll define propositional logic with a BNF grammar and give its truth-table semantics. That logic has just two values—true and false—so we can check everything exhaustively, which makes it a clean place to build habits before we add quantifiers (Module 4) and proofs-on-infinite-data (Module 8). Along the way, the intermezzo after this module (“Cardinality and Function Spaces”) uses the set and function

vocabulary from here to show that some infinities are bigger than others—a real surprise that’s worth the detour.

1.9 Exercises

The exercises below are organized into four tiers. The **warm-ups** are short, mechanical checks on the definitions—do them all. The **standard** problems are closer to what you’ll see on an assignment or exam. The **challenge** problems ask you to stitch several ideas together and are the ones most worth discussing with a study partner. The **connection** problems point outward—to programming, to the intermezzo on cardinality, or to later modules. Solutions for all of these are in the appendix, but resist the temptation to look until you’ve tried.

Warm-up

Exercise 1. Let $A = \{1, 2, 3, 4\}$ and $B = \{3, 4, 5, 6\}$. Compute $A \cap B$, $A \cup B$, $A \setminus B$, and $|A \times B|$.

Exercise 2. What is $|\{1, \{1\}, \{1, \{1\}\}\}|$? Be careful—count distinct elements.

Exercise 3. Rewrite the set $\{n \mid n \in \mathbb{N}, n^2 < 20\}$ by listing its elements explicitly (“roster notation”). Then do the same for $\{n \mid n \in \mathbb{Z}, -3 \leq n < 3\}$.

Exercise 4. Let $S = \{1, 2, \{1\}, \{1, 2\}\}$. For each of the following, say whether it is true, false, or a type error (“type error” means the claim doesn’t even make sense—e.g., asking whether a number is a subset of another number):

- (a) $1 \in S$
- (b) $\{1\} \in S$
- (c) $\{1\} \subseteq S$
- (d) $\{1, 2\} \in S$
- (e) $\{1, 2\} \subseteq S$
- (f) $\{\{1\}\} \subseteq S$
- (g) $2 \subseteq S$

Exercise 5. List all eight elements of $\mathcal{P}(\{a, b, c\})$ and verify that there are 2^3 of them. Then list all six elements of $\{a, b\} \times \{1, 2, 3\}$.

Standard

Exercise 6. For each of the following relations on $\mathbb{Z}$, determine whether it is reflexive, symmetric, and/or transitive: (a) $a R b$ iff $a + b$ is even; (b) $a R b$ iff $a \leq b$; (c) $a R b$ iff $|a - b| \leq 1$.

Exercise 7. For each of the following “assignments” from $\mathbb{R}$ to $\mathbb{R}$, decide whether it is a function (in the total, single-valued sense of the definition in this module). If it isn’t, say which requirement fails—totality or uniqueness—and give a witness. If it is, say so explicitly.

- (a) $f(x) = 1/x$.
- (b) $f(x) = \sqrt{x}$ (taking only the non-negative square root).
- (c) The relation $\{(x, y) \mid y^3 = x\}$, viewed as an assignment $x \mapsto y$.
- (d) The relation $\{(x, y) \mid y^2 = x\}$, viewed as an assignment $x \mapsto y$.

Exercise 8. Prove the distributive law $A \cap (B \cup C) = (A \cap B) \cup (A \cap C)$ by showing both containments. That is, show $A \cap (B \cup C) \subseteq (A \cap B) \cup (A \cap C)$ and vice versa, each by picking an arbitrary element of the left-hand side and arguing that it must be in the right-hand side. Use the “state goal, introduce variable, justify every step” advice from the “How to Write a Proof” chapter.

Exercise 9. Prove or disprove: for all sets A and B , if $A \subseteq B$ then $\mathcal{P}(A) \subseteq \mathcal{P}(B)$. (Reminder: to prove an implication about sets, start by assuming the hypothesis and by picking an arbitrary element of the set whose containment you want to show.)

Exercise 9b. (Composition.) Let $f : \mathbb{Z} \rightarrow \mathbb{Z}$ send $n \mapsto n + 1$ and let $g : \mathbb{Z} \rightarrow \mathbb{Z}$ send $n \mapsto 2n$. Compute $g \circ f$ and $f \circ g$ as closed-form expressions, and confirm that they are *not* the same function. Then verify, for an arbitrary n , the associativity law $((f \circ f) \circ g)(n) = (f \circ (f \circ g))(n)$.

Challenge

Exercise 10. Let A and B be finite sets. Prove that

$$|A \cup B| = |A| + |B| - |A \cap B|.$$

Give a plain-English argument (“every element of $A \cup B$ is counted ...”). This is your first taste of *inclusion–exclusion*, which generalizes to arbitrary finite unions and is one of the foundational tools of combinatorics.

Exercise 11. A *relation on A* is a subset of $A \times A$, so relations on A are themselves elements of the power set $\mathcal{P}(A \times A)$. If $|A| = n$, how many distinct relations are there on A ? How many are reflexive? (Hint for the second part: a reflexive relation *must* contain all n diagonal pairs, and is free to include or exclude each of the remaining pairs.)

Exercise 12. Here is a classic bad argument: “Symmetry and transitivity together force reflexivity. After all, if $a R b$, then by symmetry $b R a$, and so by transitivity $a R a$.” Find the flaw—this really is a common error—and then exhibit a concrete relation on $\{1, 2, 3\}$ that is symmetric and transitive but *not* reflexive. What additional assumption would you need for the argument to go through?

Connection

Exercise 13. (Programming.) Write a Python function `cartesian(xs, ys)` that takes two lists and returns the Cartesian product as a list of tuples, *without* using `itertools.product`. Test it on `cartesian([1, 2], ['a', 'b', 'c'])` and verify it produces the same six tuples you listed by hand in Exercise 1.9. Then answer: if `xs` has length m and `ys` has length n , how many element comparisons does your implementation do?

Exercise 14. (Intermezzo pointer.) The intermezzo after this module, “Cardinality and Function Spaces,” is about counting functions between finite sets. As a warm-up for it: how many functions

are there from $\{a, b, c\}$ to $\{0, 1\}$? List them all as tables. Can you see why the answer is 2^3 and not 3^2 ? (Which set plays the role of “exponent”?)

This has been a foundational week: sets, tuples, functions, relations, and (in the short chapter that follows) the prose style expected of every proof in the book. Those are the vocabulary we’ll use for the rest of the course. In the next module we start doing *logic*.

How to Write a Proof

You now have enough vocabulary—sets, functions, relations—to start writing proofs about them. But when you sit down to actually write one, it can be hard to know where to begin. This short chapter gives practical advice about formatting and structure. We are not learning proof *techniques* yet (that starts in Module 3); we’re just learning how to put words on the page.

Keep this chapter bookmarked. The specific proof techniques change as the course goes on—direct, counterexample, cases, induction, contradiction, contrapositive—but the prose habits below carry across all of them.

1.10 A proof is a piece of writing

A proof is not a calculation, a formula, or a wall of symbols. It is a short essay that explains, step by step, *why* something is true. Each sentence should either state a fact, introduce a definition, or draw a conclusion from what came before. If you can’t explain a step in words, you probably don’t understand it yet.

Here are the ground rules:

1. **State what you are proving.** Open with something like “We prove that ...” or “We want to show that ...”. Don’t make the reader guess your goal.
2. **Introduce variables before you use them.** If your proof involves a number, a set, or a function, give it a name and say where it comes from. “Let n be a positive integer” or “Consider an arbitrary $x \in \mathbb{R}$ ” or “Define $f : A \rightarrow B$ by $f(x) = x^2$.”
3. **Write in complete sentences.** Mathematical symbols are parts of sentences, not replacements for them. Don’t write a bare chain of equations with no words connecting them.
4. **Justify each step.** After every claim, the reader should be able to see *why* it follows—from a definition, from a previous step, or from a well-known fact.
5. **End with a clear conclusion.** Finish with “Therefore ...” or “This completes the proof.” The reader should know when you’re done and what you’ve established.

1.11 When the problem has cases

Many proofs, including ones you’ll see on the Module 1 problem set, naturally split into cases. For instance, you might need to show something holds “for all positive integers A ,” but the argument is different depending on whether A is even or odd. In that situation:

- State explicitly that you are splitting into cases and say what they are: “We consider two cases: A is even, and A is odd.”

- Handle each case separately. Label them clearly (“**Case 1:** A is even.”) and give a self-contained argument for each one.
- After the cases, tie them together: “Since every positive integer is either even or odd, these two cases are exhaustive, and the proof is complete.”

The key requirement is that your cases *cover all the possibilities*. If there’s a scenario you didn’t address, your proof has a gap.

1.12 Using definitions

When a proof asks whether something is or isn’t a function, or whether a relation has a particular property, *go back to the definition*. This is the single most important habit in proof-writing.

For example, if you need to show that a certain relation is a function, you should explicitly invoke the definition: a function $f : A \rightarrow B$ assigns every element of A to *exactly one* element of B . So you need to show two things: (1) for every input there is at least one output, and (2) for every input there is at most one output. Conversely, to show something is *not* a function, it suffices to find a single input that has zero outputs or more than one.

Don’t skip this step. Even if the answer feels obvious, the proof isn’t complete until you’ve connected your argument to the actual definitions.

1.13 An example: good vs. bad

Suppose we want to prove that the relation $y^2 = x$ on the reals is not a function (with x as input and y as output).

Bad version:

$y^2 = x$ is not a function because $x = 4$ gives $y = 2$ and $y = -2$.

This has the right idea but it doesn’t explain *why* having two values of y is a problem. The reader is left to fill in the reasoning.

Good version:

We show that the relation $y^2 = x$ on $\mathbb{R}$ is not a function from x to y . By definition, a function must assign each input x to exactly one output y . Consider $x = 4$. Then $y^2 = 4$, which has two real solutions: $y = 2$ and $y = -2$. Since the input $x = 4$ corresponds to two different outputs, the relation violates the uniqueness requirement and is therefore not a function.

Notice what the good version does: it states the goal, invokes the definition, gives the specific example, and explicitly connects the example back to the definition to reach the conclusion. Every step is justified. That’s all a proof needs to be.

1.14 Two walkthroughs: tightening a proof

The contrast above shows two finished attempts side by side. But the skill worth learning is the *editing move* that turns the first into the second—the tightening pass you do after your first draft. Here are two walkthroughs using the kinds of claims you’ll actually prove in Module 1’s exercises.

Walkthrough 1: $A \cap B \subseteq A \cup B$

First draft:

Let $x \in A \cap B$. So $x \in A$ and $x \in B$. So clearly $x \in A \cup B$. Therefore $A \cap B \subseteq A \cup B$.

The mathematics is right, but three things are worth fixing.

1. **“Clearly” is a confession, not a reason.** Whenever you write “clearly” or “obviously,” pause: what is the actual reason? Here we’re using the definition of union—an element is in $A \cup B$ if it is in A or in B . Say that.
2. **Name which definition you’re invoking.** The step from “ $x \in A \cap B$ ” to “ $x \in A$ and $x \in B$ ” is the definition of intersection. Pointing at the definition makes the proof easier to check and trains the habit you’ll need when definitions get more complex.
3. **Close the universal.** To conclude $A \cap B \subseteq A \cup B$, it isn’t enough that *this particular* x landed in $A \cup B$ —we need the same for every element of $A \cap B$. The standard closing phrase is “since x was arbitrary.” That phrase is what turns the step from “ $x \in A \cup B$ ” to “ $A \cap B \subseteq A \cup B$ ” into a real inference instead of a leap.

Tightened version:

Let $x \in A \cap B$. By the definition of intersection, $x \in A$. By the definition of union, any element of A is also in $A \cup B$, so $x \in A \cup B$. Since x was an arbitrary element of $A \cap B$, we conclude $A \cap B \subseteq A \cup B$.

The tightened version is barely longer than the first draft—but every step is labeled, every definition is named, and the universal is properly closed.

Walkthrough 2: showing a relation is a function

Recall that a function $f : A \rightarrow B$ assigns every element of A to *exactly one* element of B . That’s two requirements packed into one phrase: *at least one* output (existence) and *at most one* output (uniqueness). It’s easy to prove only half and think you’re done.

Suppose we want to show: the relation R on $\mathbb{Z}$ defined by “ $R(x, y)$ iff $y = 2x$ ” is a function from $\mathbb{Z}$ to $\mathbb{Z}$.

First draft:

Take any $x \in \mathbb{Z}$. Let $y = 2x$. Then $R(x, y)$ holds. So R is a function.

This draft shows *existence*: every input has at least one output. But it hasn’t ruled out the possibility that some x has *two* different outputs, and that’s exactly what makes a relation fail to be a function. The argument is half of what’s required.

Tightened version:

We show that R assigns each $x \in \mathbb{Z}$ to exactly one $y \in \mathbb{Z}$ —both that at least one such y exists and that no more than one does.

Existence. Let $x \in \mathbb{Z}$. Take $y = 2x$. Then $y \in \mathbb{Z}$ (because $\mathbb{Z}$ is closed under multiplication) and $R(x, y)$ holds by definition of R . So every input has at least one output.

Uniqueness. Let $x \in \mathbb{Z}$, and suppose $R(x, y_1)$ and $R(x, y_2)$. By the definition of R , $y_1 = 2x$ and $y_2 = 2x$, so $y_1 = y_2$. So every input has at most one output.

Since R satisfies both conditions, it is a function from $\mathbb{Z}$ to $\mathbb{Z}$.

The tightened version is structured around the two halves of the definition. Whenever a definition has the form “*both* this *and* that,” your proof needs two explicitly labeled pieces—one for each half. The habit of labeling **Existence** and **Uniqueness** (or **At least one**/**At most one**) stays useful for the rest of the course.

1.15 When you don’t know what to write next

Getting stuck on a proof is not a sign that you’re bad at math—it happens to everyone, including the person who wrote this textbook. When it happens, here are four moves that almost always get you unstuck with the tools you have in Module 1.

1. **Unpack every definition.** Write down, in full, the definition of every noun in the statement you’re proving. If the claim is “ f is injective,” write down what “injective” means. If it’s “ $A \subseteq B$,” write down what “ $\subseteq$ ” means. A surprising amount of the time, the proof writes itself once the definitions are sitting on the page in front of you.
2. **Element-chase for subset claims.** For any claim of the form “ $X \subseteq Y$,” start your proof with “Let $x \in X$ ” and try to derive “ $x \in Y$.” This is the workhorse move for half of the Module 1 exercises, and it’s the single most reliable way to turn a set-theoretic claim into something you can actually write sentences about.
3. **Flip to the contrapositive when the forward direction stalls.** To show “every element of X is in Y ,” you can instead show “every element *not* in Y is *not* in X .” The two statements are logically equivalent, but sometimes one is much easier to argue than the other. If you’re stuck on the forward version for ten minutes, try the flipped version for five before returning.
4. **Plug in a tiny example.** If the claim is about arbitrary sets A, B, C , try $A = \{1\}$, $B = \{2\}$, $C = \{1, 2\}$. Seeing what the claim actually says in a specific case often reveals the general argument. (This is not itself a proof—you still have to write the general argument—but it’s a reliable way to find the pattern that the proof will follow.)

These four moves are the whole playbook at this point. As the book introduces more techniques—proof by cases (Module 6), induction (Modules 4 and 8), contradiction and contrapositive (Module 7)—the playbook grows. A second chapter, “How to Write a Proof, Revisited,” collects the full version after you’ve seen the rest of the toolkit.

1.16 Where to go from here

The advice above is independent of any particular proof technique. In the modules that follow you will meet the standard techniques—direct proof (Modules 3 and 5), counterexample and proof by cases (Modules 5 and 6), induction (Modules 4 and 8–10), proof by contradiction and contrapositive (Module 7)—and each will come with its own recipe. But every one of them expects to be written in prose that follows the rules above: state the goal, introduce variables, justify each step, conclude clearly.

Plan to re-read this chapter after every few proofs you write, especially early on. The mistakes most worth fixing are not mathematical—they are writing mistakes: missing variable introductions, unjustified jumps, equations strung together without connective tissue, no clear conclusion. Fix the writing and the mathematics usually becomes visible.

Once you’ve finished Module 7 and have the full range of informal proof techniques in hand, the companion chapter “How to Write a Proof, Revisited” revisits this advice at a higher level—choosing between techniques, writing induction proofs, structuring long arguments, and the style conventions you’ll need for the rest of the book. Think of this chapter as the foundation and that one as the upper-level follow-up.

Intermezzo: Cardinality and Function Spaces

In Module 1 we introduced sets, functions, and the power set. We said the cardinality of a finite set is just the number of distinct elements it contains. But we also mentioned sets like $\mathbb{N}$ and $\mathbb{Z}$ that are infinite. How do you compare the sizes of infinite sets? Do all infinite sets have the “same amount” of infinity?

The answer is *no*, and the story of how we know that—and the beautiful connection between power sets and function spaces that falls out of it—is what this chapter is about.

1.17 Comparing set sizes

For finite sets, cardinality is easy: count the elements. $|\{a, b, c\}| = 3$ and $|\{0, 1\}| = 2$, so $\{a, b, c\}$ is bigger. Done.

But what about infinite sets? You can’t count to infinity and compare the totals. We need a different strategy. The key idea, due to Georg Cantor in the 1870s, is to use *functions* to compare sizes.

If you can pair up every element of A with a unique element of B and vice versa—a perfect matching with nothing left over on either side—then A and B have “the same size.” The function that does this pairing is called a *bijection*, and it’s built from two simpler properties.

Definition 1.17.1 (Injection (one-to-one)). A function $f : A \rightarrow B$ is *injective* (or *one-to-one*) if no two inputs map to the same output: whenever $f(a_1) = f(a_2)$, we have $a_1 = a_2$.

An injection maps distinct inputs to distinct outputs—no two inputs share an output. Every input gets its own unique output. The function $f(x) = 2x$ on the integers is injective because different inputs always produce different outputs. The function $g(x) = x^2$ on $\mathbb{Z}$ is *not* injective because $g(3) = g(-3) = 9$ —two different inputs, same output.

Definition 1.17.2 (Surjection (onto)). A function $f : A \rightarrow B$ is *surjective* (or *onto*) if every element of B is hit: for every $b \in B$, there exists some $a \in A$ with $f(a) = b$.

A surjection means nothing in the codomain is left out. Every possible output actually gets produced by some input. If $f : \mathbb{Z} \rightarrow \mathbb{Z}$ is defined by $f(x) = 2x$, then f is *not* surjective because odd numbers are never hit. But $g : \mathbb{Z} \rightarrow \mathbb{Z}$ defined by $g(x) = x + 1$ is surjective because every integer n is $g(n - 1)$.

Definition 1.17.3 (Bijection). A function $f : A \rightarrow B$ is a *bijection* if it is both injective and surjective. Equivalently, f is a bijection if it has an inverse: a function $f^{-1} : B \rightarrow A$ such that $f^{-1}(f(a)) = a$ for all $a \in A$ and $f(f^{-1}(b)) = b$ for all $b \in B$.

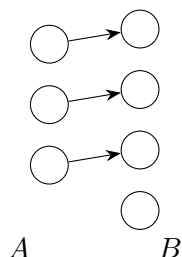
A bijection is a perfect pairing—every element of A matches exactly one element of B , and every element of B matches exactly one element of A . When a bijection $f : A \rightarrow B$ exists, we say A and B have the same cardinality, written $|A| = |B|$.

Two quick consequences of the definition that we'll use later. First, the identity function $\text{id}_A : A \rightarrow A$ is a bijection (it's its own inverse). Second, if $f : A \rightarrow B$ and $g : B \rightarrow C$ are bijections, then so is their composition $g \circ f : A \rightarrow C$, with inverse $f^{-1} \circ g^{-1}$. (Check this by composing: $(f^{-1} \circ g^{-1}) \circ (g \circ f) = f^{-1} \circ (g^{-1} \circ g) \circ f = f^{-1} \circ f = \text{id}_A$.) So bijections compose, and this means “has the same cardinality as” is a transitive relation: if $|A| = |B|$ and $|B| = |C|$, then $|A| = |C|$, with the witnessing bijection obtained by composition.

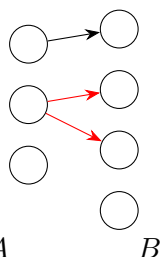
A gallery of arrow diagrams

These definitions are easier to grasp with pictures. Think of A as a set of dots on the left, B as a set of dots on the right, and draw an arrow from each input $a \in A$ to its output $f(a) \in B$.

Valid function

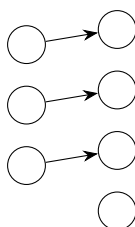


Not a function



(one input, two outputs)

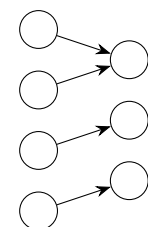
Injection



A B

(distinct inputs $\mapsto$ distinct outputs)

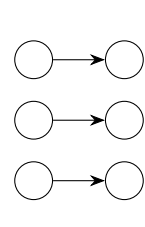
Surjection



A B

(every output is hit)

Bijection



A B

(perfect pairing)

The key visual distinction: an injection never has two arrows pointing at the same dot on the right, a surjection never leaves a dot on the right without an incoming arrow, and a bijection does both—each dot on each side has exactly one arrow touching it.

For finite sets this lines up with our intuition: $\{a, b, c\}$ and $\{1, 2, 3\}$ have the same cardinality because we can pair them up (say $a \mapsto 1, b \mapsto 2, c \mapsto 3$). But the power of this definition is that it also works for infinite sets, where counting fails.

1.18 Countable infinity

$\mathbb{N} = \{0, 1, 2, 3, \dots\}$ is infinite. Are there other infinite sets that are “the same size” as $\mathbb{N}$?

Consider $\mathbb{Z}$, the integers: $\dots, -3, -2, -1, 0, 1, 2, 3, \dots$. At first glance $\mathbb{Z}$ looks “twice as big” as $\mathbb{N}$ because it extends in both directions. But that intuition is misleading. We can construct a bijection $f : \mathbb{N} \rightarrow \mathbb{Z}$ by zigzagging:

$$0 \mapsto 0, \quad 1 \mapsto -1, \quad 2 \mapsto 1, \quad 3 \mapsto -2, \quad 4 \mapsto 2, \quad 5 \mapsto -3, \quad 6 \mapsto 3, \quad \dots$$

More precisely:

$$f(n) = \begin{cases} n/2 & \text{if } n \text{ is even} \\ -(n+1)/2 & \text{if } n \text{ is odd} \end{cases}$$

This is a bijection. Every integer appears exactly once in the list $0, -1, 1, -2, 2, -3, 3, \dots$, so $|\mathbb{N}| = |\mathbb{Z}|$. Infinity is weird: a set that looks “twice as big” turns out to be exactly the same size.

It turns out that $\mathbb{Q}$, the rational numbers, is also the same size as $\mathbb{N}$. The proof uses a clever diagonal enumeration often called Cantor’s pairing function—you arrange the rationals in a grid and then zigzag through them. We won’t go through the details here, but the punchline is the same: you can list all the rationals, one by one, without missing any.

Definition 1.18.1 (Countably infinite). A set S is *countably infinite* if there is a bijection $f : \mathbb{N} \rightarrow S$. A set is *countable* if it is either finite or countably infinite.

Intuitively, a set is countable if you can write its elements as a (possibly infinite) list: a first element, a second element, a third, and so on, eventually hitting everything. The integers can be listed $(0, 1, -1, 2, -2, \dots)$, so they’re countable. The rationals can be listed too (with some cleverness), so they’re also countable.

Hilbert’s Hotel. Here’s a fun thought experiment that helps build intuition for why infinity behaves so strangely. Imagine a hotel with infinitely many rooms, numbered $1, 2, 3, \dots$, and every room is occupied. A new guest arrives. Can you fit them in?

Yes. Move every current guest from room n to room $n + 1$. Room 1 is now free for the new guest. The explicit bijection is $n \mapsto n + 1$: the occupant of room n moves to room $n + 1$, and the function is injective (different rooms to different rooms) and surjective onto $\{2, 3, 4, \dots\}$ (everyone still has a room). In fact, you can fit infinitely many new guests the same way: use the bijection $n \mapsto 2n$ to move each existing guest to an even-numbered room, freeing up all the odd-numbered rooms for the new arrivals (guest k of the new group goes to room $2k - 1$). Again, both maps are bijections—no one loses a room, no room is assigned twice. It’s not that the hotel “got bigger”; it’s that a different bijection between guests and rooms works just as well as the original.

1.19 Uncountable infinity—Cantor’s diagonal argument

So $\mathbb{N}$, $\mathbb{Z}$, and $\mathbb{Q}$ are all the same size. You might start to wonder: is *every* infinite set countable? Maybe infinity just comes in one flavor?

No. The set $\mathbb{R}$ of real numbers is *not* countable. There are strictly more real numbers than natural numbers. This was proved by Cantor in 1891 with one of the most famous arguments in all of mathematics: the *diagonal argument*.

Theorem 1.19.1 (Cantor). The set of real numbers in the interval $[0, 1)$ is uncountable.

Proof. We proceed by contradiction. Assume, for the sake of argument, that we *can* list all the real numbers in $[0, 1)$. That means there’s a bijection from $\mathbb{N}$ to $[0, 1)$, giving us an enumeration $r_1, r_2, r_3, \dots$ that includes every real number in $[0, 1)$.

Write each r_i in its decimal expansion:

$$\begin{aligned} r_1 &= 0.d_{11} d_{12} d_{13} d_{14} \dots \\ r_2 &= 0.d_{21} d_{22} d_{23} d_{24} \dots \\ r_3 &= 0.d_{31} d_{32} d_{33} d_{34} \dots \\ r_4 &= 0.d_{41} d_{42} d_{43} d_{44} \dots \\ &\vdots \end{aligned}$$

Now construct a new real number $d = 0.e_1 e_2 e_3 e_4 \dots$ by choosing each digit to *differ* from the corresponding diagonal entry. Specifically, set

$$e_n = \begin{cases} 5 & \text{if } d_{nn} \neq 5 \\ 6 & \text{if } d_{nn} = 5 \end{cases}$$

The number d is in $[0, 1)$, so it should appear somewhere in our list. But d differs from r_1 in the first digit, from r_2 in the second digit, from r_3 in the third digit, and in general from r_n in the n th digit. So $d \neq r_n$ for every n . This means d is *not* in our list—contradicting the assumption that the list was complete. $\square$

Notice the proof technique: we assumed something and derived a contradiction, so the assumption must be false. This is *proof by contradiction*, the same technique we used in Module 1 to show there are infinitely many natural numbers. We’ll formalize this technique properly in Module 3.

The deeper lesson is the *method*: construct something that systematically disagrees with every entry in a supposed complete list. This “diagonalization” idea is incredibly powerful and shows up in several places across mathematics and computer science.

Forward look. This diagonal argument has exactly the same structure as the proof that the halting problem is undecidable—one of the most important results in theoretical computer science. The diagonalization idea is so powerful that, once you have it, the halting problem falls out almost immediately. Let’s actually do it.

1.20 Diagonalization, cashed out: the halting problem

You’ll see this theorem developed properly in CS 251, once we have a formal model of computation (Turing machines, register machines—it doesn’t really matter which). But the *argument* is purely Cantor-style diagonalization, and we have everything we need for it right now. All we need is the intuitive programmer’s notion of “a program”—think Python.

Theorem 1.20.1 (Turing, 1936). There is no program `halts(P, x)` that takes the source code of a program P and an input x , always terminates, and returns `True` if P halts on input x and `False` if P loops forever on input x .

Proof. Suppose, for contradiction, that such a program `halts` exists. Since programs are just strings of source code, we can feed one program’s source code to another as input—that’s what passing P to `halts` means. In particular, we can ask `halts(P, P)`: “does P halt when given its own source code as input?”

Using `halts` as a subroutine, we can build a new program `diag`:

```
def diag(P):
    if halts(P, P):
        while True: pass # loop forever
    else:
        return # halt immediately
```

In words: `diag(P)` runs `halts(P, P)` to find out what `P` does on itself, then does the *opposite*. If `P` halts on `P`, then `diag(P)` loops forever. If `P` loops forever on `P`, then `diag(P)` halts.

Now ask the fatal question: what does `diag(diag)` do?

- Suppose `diag(diag)` halts. Then `halts(diag, diag)` must have returned `False`—because that’s the only branch of `diag` that halts. But `halts(diag, diag) = False` means, by definition of `halts`, that `diag` does *not* halt on input `diag`. Contradiction.
- Suppose `diag(diag)` loops forever. Then `halts(diag, diag)` must have returned `True`—because that’s the only branch of `diag` that loops. But `halts(diag, diag) = True` means `diag` *does* halt on input `diag`. Contradiction.

Either way we get a contradiction, so our assumption was wrong: no such `halts` exists. □

I want to pause and point out how this mirrors Cantor’s diagonal argument, because the correspondence is *tight*.

Cantor	Turing
List of reals $r_1, r_2, r_3, \dots$	List of programs $P_1, P_2, P_3, \dots$
n th digit of r_n	Behavior of P_n on input P_n
Construct d differing at every diagonal	Construct <code>diag</code> doing opposite at every diagonal
d is a real not in the list	<code>diag</code> ’s row can’t exist in the halts-table

In both proofs you assume a completed list (of reals; of halt-decisions), you construct a new thing that *deliberately disagrees* with the list at the n th position for every n , and then you derive a contradiction because the new thing was supposed to be on the list. Cantor uses it to show there are more reals than naturals; Turing uses it to show there is no algorithm that decides halting. The same trick, pointed at different targets.

The moral: diagonalization isn’t a one-off cleverness for real numbers. It’s a reusable weapon. Anywhere you have a “list of all the things that decide some question,” you can often construct a question whose answer deliberately disagrees with every decider in the list—and so the list can’t have been complete. Gödel’s incompleteness theorem, Tarski’s undefinability of truth, Rice’s theorem, and a long list of others are all variations on this theme. Smullyan’s book in the Further Reading section traces the family tree.

1.21 Function spaces

Let’s change gears. Given two sets A and B , it’s natural to ask: how many functions are there from A to B ?

The set of *all* functions from A to B is written B^A (read “ B to the A ”).

Why the exponential notation? Because for finite sets, $|B^A| = |B|^{|A|}$. Here’s why: to define a function $f : A \rightarrow B$, you need to decide, for each element of A , which element of B it maps to. That’s $|B|$ choices for each of the $|A|$ elements, giving $|B|^{|A|}$ total functions.

Example 1.21.1. Let $A = \{a, b\}$ and $B = \{0, 1\}$. Then $|B^A| = 2^2 = 4$. The four functions are:

Function	$f(a)$	$f(b)$
f_1	0	0
f_2	0	1
f_3	1	0
f_4	1	1

That’s all of them. Each row is a complete function from A to B —for each input in A , it specifies exactly one output in B .

If you’re a Python programmer, you can think of B^A as the type `Dict[A, B]` where every key in A is required to be present. A function is just a dictionary with no missing keys.

You might also write the function space as $A \rightarrow B$ instead of B^A . Both notations are common. The exponential notation emphasizes the counting formula; the arrow notation emphasizes that we’re talking about functions.

1.22 The power set is a function space—the punchline

Here is the connection that ties this chapter together. Recall from Module 1 that the power set $\mathcal{P}(A)$ is the set of all subsets of A , and that $|\mathcal{P}(A)| = 2^{|A|}$ for finite sets. We said each element is either “in” a subset or “not in” it—two choices per element. But now we can say something much more precise.

Every subset $S \subseteq A$ can be described by a function $\chi_S : A \rightarrow \{0, 1\}$ called its *characteristic function* (or *indicator function*):

$$\chi_S(x) = \begin{cases} 1 & \text{if } x \in S \\ 0 & \text{if } x \notin S \end{cases}$$

Conversely, every function $f : A \rightarrow \{0, 1\}$ determines a subset $\{x \in A \mid f(x) = 1\}$. These two operations are inverses of each other: going from subset to characteristic function and back gives you the same subset, and going from function to subset and back gives you the same function.

In other words, there is a *bijection* between $\mathcal{P}(A)$ and $\{0, 1\}^A$. We write this as

$$\mathcal{P}(A) \cong 2^A$$

where 2 is shorthand for the two-element set $\{0, 1\}$.

Example 1.22.1. Let $A = \{a, b, c\}$. The subset $\{a, c\}$ corresponds to the characteristic function $\chi_{\{a, c\}} : A \rightarrow \{0, 1\}$ defined by:

$$a \mapsto 1, \quad b \mapsto 0, \quad c \mapsto 1$$

And the function $g : A \rightarrow \{0, 1\}$ with $g(a) = 0, g(b) = 1, g(c) = 0$ corresponds to the subset $\{b\}$.

Every subset gets a unique function, every function gets a unique subset, and nothing is left over on either side. That’s a bijection.

This explains *why* $|\mathcal{P}(A)| = 2^{|A|}$. It’s not just a coincidence or a counting trick—it’s because the power set literally *is* the function space 2^A , and function spaces have exponential cardinality:

$$|\mathcal{P}(A)| = |2^A| = |\{0, 1\}^A| = |\{0, 1\}|^{|A|} = 2^{|A|}$$

Subsets, characteristic functions, and bit strings of length $|A|$ are three descriptions of the same object. When you pick a subset of A , you're making a binary choice for each element—in or out, 1 or 0. That *is* a function to $\{0, 1\}$. That *is* a bit string. Three descriptions, one object.

Forward look: predicates. When we introduce predicates in Module 4, a predicate on a set A will be defined as a function $A \rightarrow \text{Bool}$. The truth set of the predicate—the set of elements making it true—is exactly the subset corresponding to this characteristic function. So predicates, subsets, and functions to Bool are three views of a single concept.

Cantor's theorem and the hierarchy of infinities. Here's something to chew on. Cantor proved that $|\mathcal{P}(A)| > |A|$ for *any* set A —even infinite ones. There is no bijection between a set and its power set; the power set is always strictly bigger. (The proof is another diagonal argument!)

Combined with $\mathcal{P}(\mathbb{N}) \cong 2^{\mathbb{N}}$, this gives us an infinite hierarchy of infinities, each strictly larger than the last:

$$|\mathbb{N}| < |2^{\mathbb{N}}| < |2^{2^{\mathbb{N}}}| < \dots$$

The first level is the countable infinity we've been working with. The second, $|2^{\mathbb{N}}|$, turns out to equal $|\mathbb{R}|$ —the cardinality of the real numbers. There are strictly more real numbers than natural numbers (we proved this!), and strictly more subsets of the reals than reals, and so on forever. Infinity isn't one thing — it's an entire hierarchy, and it goes up forever.

1.23 Exercises

Exercise 1. List all elements of $\mathcal{P}(\{1, 2, 3\})$. How many are there?

Exercise 2. How many functions are there from $\{a, b, c\}$ to $\{0, 1\}$? List them by giving the characteristic function (that is, the values $f(a), f(b), f(c)$) for each subset of $\{a, b, c\}$.

Exercise 3. Prove that the set of even natural numbers $E = \{0, 2, 4, 6, \dots\}$ is countably infinite by giving an explicit bijection $f : \mathbb{N} \rightarrow E$. Verify that your function is both injective and surjective.

Exercise 4. Show that the open interval $(0, 1)$ has the same cardinality as the entire real line $\mathbb{R}$ by exhibiting an explicit bijection between them. (Hint: a function from trigonometry, or a rational function involving $\frac{1}{x}$ and $\frac{1}{1-x}$, will do the job.) The takeaway: a tiny-looking interval can hold “just as many” points as the whole line.

Exercise 5. (Cantor's theorem, general form.) Let A be any set and suppose, for contradiction, that there is a surjection $f : A \rightarrow \mathcal{P}(A)$. Define

$$D = \{a \in A \mid a \notin f(a)\}.$$

Since $D \subseteq A$ and f is surjective, there must exist some $d \in A$ with $f(d) = D$. Derive a contradiction by asking whether $d \in D$ or $d \notin D$. Conclude that $|\mathcal{P}(A)| > |A|$ for every set A .

Exercise 6. (Diagonalization in disguise.) Suppose someone hands you an infinite list $f_0, f_1, f_2, \dots$ of functions $\mathbb{N} \rightarrow \mathbb{N}$. Define a new function $g : \mathbb{N} \rightarrow \mathbb{N}$ by

$$g(n) = f_n(n) + 1.$$

Show that g does not appear anywhere in the list. (That is: for every n , $g \neq f_n$ as functions.) Conclude that no countable list can contain every function $\mathbb{N} \rightarrow \mathbb{N}$. Compare this to the structure of the proof of Cantor’s theorem above and to the proof that the halting problem is undecidable.

Exercise 7. List all eight functions from $\{0, 1, 2\}$ to $\{a, b\}$ in a table (one row per function, columns $f(0), f(1), f(2)$). Verify directly that this gives $|\{a, b\}|^{|\{0, 1, 2\}|} = 2^3 = 8$ functions, and confirm that each function corresponds to a distinct subset of $\{0, 1, 2\}$ via the characteristic-function interpretation.

Further reading

If the diagonal argument or the hierarchy of infinities caught your interest, here are a few accessible papers and articles to explore.

- Robert Gray, “Georg Cantor and Transcendental Numbers,” *American Mathematical Monthly* **101**(9), 819–832 (1994). A clean historical account of Cantor’s *original* 1874 uncountability argument (which was not the diagonal argument!) and how the diagonal version came later. Excellent for seeing how the ideas in this chapter actually developed.
- Martin Davis, “[What is a Computation?](#)” in L. A. Steen (ed.), *Mathematics Today: Twelve Informal Essays*, Springer (1978). Davis walks you from Cantor’s diagonal argument straight to the unsolvability of the halting problem in a way that requires almost no prior CS background—perfect if you want to see the “forward look” from this chapter cashed out.
- Raymond Smullyan, *Diagonalization and Self-Reference*, Oxford Logic Guides 27, Oxford University Press (1994). A whole book—but the early chapters are gentle, and they show how Cantor, Gödel, and Tarski are all variations on a single self-referential theme.

Chapter 2

Propositional Logic

Reading map

Epp sections. §2.1–2.2: logical form and logical equivalence.

Prerequisites. Module 1 (sets, functions, truth as a boolean-valued assignment).

Relevant intermezzi. The Gentzen/inference-tree intermezzo (optional, useful preview for Module 3).

Practice from Epp. §2.1 exercises on building truth tables and classifying formulas as tautology / contradiction / contingent; §2.2 exercises on logical equivalences and De Morgan’s laws.

2.1 Logical formulas, formalized

With all our mathematical preliminaries out of the way, we’re ready to start dealing with logic more formally. Early attempts at talking about logic were focused on rhetoric and how to make arguments in human language. They were still about reasoning but didn’t treat it in the *symbolic* way we do in modern times, where we turn logic into formulas and equations and symbols.

In this class, I’m not only going to take a symbolic approach to logic like Epp but I’m going to take a very “computer sciencey” approach, discussing logic as though it were a programming language.

There are many possible logics — yes, plural, because there are many possible logics just as there are many possible programming languages, each serving its own purpose. The standard way to introduce any logic is to define its *syntax* and its *semantics*.

The *syntax* of the logic is a lot like when we talk about the syntax of any programming language: what combinations of symbols make a “valid” program/formula. The syntax tells us how we’re allowed to build formulas in our logic.

The *semantics* of a logic tells us what the formulas of the logic **mean**. In ordinary programming languages, the compiler or interpreter implements the semantics: it defines what a syntactically valid program does when executed. For (most) logics, the semantics tells us how to take a formula and turn it into a mathematical function that takes in arguments and returns either “true” or “false”.

What do true and false mean here? That question could fill a PhD thesis, but for our purposes: we can define “true” as “reflects reality” and “false” as “contradicts reality”. For most of this class, you can think of “true” and “false” a lot like the boolean types you find in every programming language.

The first logic we’re going to deal with is really simple. It only has booleans as data, variables, and a few logical connectives: and ($\wedge$), or ($\vee$), not ($\sim$), and implication ($\rightarrow$). (We’ll use $\neg$ for negation in our mathematical notation; some textbooks, including Epp, write $\sim$ instead. They mean the same thing.) The idea of this logic is that it encompasses basic operations for how to connect statements about the world and their relationship to each other. It’s all verbs, no nouns, but it still helps give some intuition of how reasoning works.

We’re going to express this using something called Backus-Naur form (https://en.wikipedia.org/wiki/Backus-Naur_form) which is a standard notation for describing grammars of formal languages, like this one, and something you’ll see more of as you study computer science.

Definition 2.1.1 (Propositional Logic Syntax). The syntax of propositional logic is given by the following BNF grammar:

$$L ::= T \mid F \mid L \wedge L \mid L \vee L \mid \sim L \mid L \rightarrow L \mid x$$

BNF is a way to describe how to build things recursively. It tells us, in this case, that to build an “L” you have a bunch of possibilities separated by the $\mid$ symbol. You can follow this recursively, continuing to substitute in things for any L symbols that appear until you only have base cases (here: variables, T, and F).

When a formula is ambiguous—like $p \wedge q \vee r$, which could be read as $(p \wedge q) \vee r$ or $p \wedge (q \vee r)$ —we follow the standard precedence: $\neg$ binds tightest, then $\wedge$, then $\vee$, then $\rightarrow$. We use parentheses to override precedence when needed. This is similar to how $*$ binds tighter than $+$ in arithmetic.

Here are some examples to help clarify how this is used. This grammar tells us that

- T is a valid formula
- F is a valid formula
- $T \wedge (x \vee y)$ is a valid formula
- $T \rightarrow L$ is not a valid formula because there’s still an L
- $T + F$ is not a valid formula because $+$ isn’t an allowed symbol in the syntax
- `raining` is a valid formula, a single variable named `raining`—variables can be any identifier, not just single letters

That’s the syntax. We need to define the *semantics* of this logic. We’ve already named the pieces, so let’s give an informal semantics before presenting the formal one:

- T is just T, the boolean value for true
- F is just F, the boolean value for false
- $p \wedge q$ is *and* and is true when p and q are both true
- $p \vee q$ is *or* and is true when **at least one** of p, q is true; the case where both are true still counts as true (this is *inclusive* or, unlike everyday English “or” which often means “one but not both”)
- $\neg p$ is *not* and is true when p isn’t

- $p \rightarrow q$ is *implication* and captures the idea that p being true forces q to be true. Specifically, it's false only when p is true and q is false
- variables are true when you plug in a true value into them

That's the informal semantics. What's the *formal* semantics? We're going to explain it in terms of **truth tables**.

A truth table is a way of describing the *graph* of the function that a formula represents. Since all the arguments to these functions are just booleans and can be enumerated easily, the graph can be represented as a table.

Let's start with variables.

x	x
T	T
F	F

A variable is just an identity function—it returns its input unchanged.

From here, we can build up the meaning of any formula piecemeal and describe the meaning of connectives in terms of “p” and “q” as subformulas, so when we write something like this:

p	q	$p \wedge q$
T	T	T
T	F	F
F	T	F
F	F	F

what we're saying is that if we've already evaluated the meaning of “p” and “q” we can plug in their values and get the meaning of when they're connected by an *and*. So we can read this table off and it matches the informal meaning we described above: $p \wedge q$ is true exactly when p is true and q is true and false otherwise.

Next, $p \vee q$:

p	q	$p \vee q$
T	T	T
T	F	T
F	T	T
F	F	F

Which tells us that $p \vee q$ is true when either p or q is true (including both of them).

Next, negation:

p	$\neg p$
T	F
F	T

Now we get to the counterintuitive one: implication. The intuition is that we want $p \rightarrow q$ to be true if p being true makes q be true—in other words, I can take the knowledge that p is true and then conclude that q is true.

(Sidenote: there’s a sense, which we might get into later in this course, in which implication really is a function that takes in a proof that p is true and turns it *into* a proof that q is true. Keep that idea in mind.)

Do I care if q is true without p being true? In our first choice of logic, the answer is “no”. We’re going to make the decision that $F \rightarrow p$ is true no matter what p is, because the case where the premise is false is irrelevant to what we want implication to capture. This has some odd implications! From a false premise you can conclude anything. We’ll dig into that more as we go.

For now, the truth table:

p	q	$p \rightarrow q$
T	T	T
T	F	F
F	T	T
F	F	T

With these truth tables in hand, let’s show how to combine them together in an example:

x	y	z	$((x \vee y) \wedge z) \rightarrow x$
T	T	T	$((T \vee T) \wedge T) \rightarrow T \implies T$
T	T	F	$((T \vee T) \wedge F) \rightarrow T \implies F \rightarrow T \implies T$
T	F	T	$((T \vee F) \wedge T) \rightarrow T \implies T$
T	F	F	$((T \vee F) \wedge F) \rightarrow T \implies F \rightarrow T \implies T$
F	T	T	$((F \vee T) \wedge T) \rightarrow F \implies T \rightarrow F \implies F$
F	T	F	$((F \vee T) \wedge F) \rightarrow F \implies F \rightarrow F \implies T$
F	F	T	$((F \vee F) \wedge T) \rightarrow F \implies F \rightarrow F \implies T$
F	F	F	$F \rightarrow F \implies T$

Whew! That was a lot, but hopefully you can see that, in principle, you could use truth tables to calculate the meaning of any formula as a function.

2.2 Tautologies and equivalences

Having talked about our very first logic, which is usually called “propositional logic”, we can start discussing some of its implications and structure.

Definition 2.2.1 (Tautology). A *tautology* is a formula whose meaning is **always** true, no matter what values you put into all the variables.

For example, for any formula p we have that $p \vee \neg p$ is a tautology! Let’s check the truth table:

p	$p \vee \neg p$
T	$T \vee F = T$
F	$F \vee T = T$

No matter what value the sub-formula p takes, you get true.

Definition 2.2.2 (Contradiction). A *contradiction* is a formula whose meaning is **always** false, no matter what values you put into the variables.

For example, $p \wedge \neg p$ is a contradiction—there’s no assignment that makes it true. Contradictions are the dual of tautologies and will play a key role when we discuss proof by contradiction in the next module.

Since propositional logic is a really simple logic, one that only has booleans as data, the tautologies of propositional logic aren’t individually profound, but they tell us important things about valid reasoning. The above tautology tells us that for any formula—for any statement about the world—either the claim is true or it is false.

Maybe that doesn’t seem profound, but this is called “the law of the excluded middle” and is actually pretty controversial in the history of logic! In *this* propositional logic and *this* notion of “truth”, however, we can always conclude that if something isn’t false it must be true.

Why controversial? There’s a tradition in logic—called *constructive* or *intuitionistic* logic—that refuses to accept $p \vee \neg p$ as an axiom. The idea is that to prove “ A or B ” you should have to actually show *which one* is true, not just argue that one of them must be.

From a programming perspective this makes a lot of sense: a constructive proof of “ A or B ” is like a program that actually *computes* which case holds and hands you the evidence. Classical logic, the kind we’re using, lets you say “well, one of them *has* to be true” without ever telling you which—you get an existence result but not an algorithm. The connection between proofs and programs turns out to be deep and beautiful, and it’s one of the reasons logic matters so much to computer science.

In this course we’ll stick with classical logic (with excluded middle) because that’s what Epp uses and what most intro courses assume. Fair warning: your instructor has a constructive bias, and it will probably peek through from time to time.

We’ll come back to tautologies a bit later when we want to talk about reasoning and proof a little more explicitly, but for now just keep in mind the idea that they provide templates for what valid reasoning looks like.

Another thing we can do with truth tables is calculate *equivalences*—we can figure out when formulas are equivalent to each other.

Definition 2.2.3 (Logical Equivalence). Two formulas are *logically equivalent* exactly when they have the same truth table.

Since we’re defining the meaning of a formula **as its truth table**, this is a natural notion of equivalence. Put differently: logical equivalence is the right notion of sameness for propositional formulas. Two formulas with the same truth table may look wildly different on paper— $(p \rightarrow q)$ and $(\neg p \vee q)$ are the canonical example—but nothing we can say *about* them inside propositional logic can tell them apart. This is a recurring pattern: each kind of mathematical object comes with its own notion of “the same,” chosen so that it abstracts away the differences that don’t matter. The intermezzo after Module 6 is the essay on that pattern.

We can also express equivalence *inside* the logic by defining a shorthand: $p \leftrightarrow q$ means $(p \rightarrow q) \wedge (q \rightarrow p)$. This is called the *biconditional* and reads “if and only if.” It is **not** a primitive connective in our grammar—it’s just an abbreviation for a conjunction of two implications. You can compute its truth table by expanding the definition:

p	q	$p \leftrightarrow q$ (i.e. $(p \rightarrow q) \wedge (q \rightarrow p)$)
T	T	T
T	F	F
F	T	F
F	F	T

So the biconditional is true exactly when p and q have the same truth value—you can think of it like an equality test on booleans. Saying “if and only if” is the same as saying the implication goes both ways.

As an example, we’re going to prove two formulas that end up being useful in some optimizations: de Morgan’s laws! (https://en.wikipedia.org/wiki/De_Morgan%27s_laws)

Let’s start with the law that $\neg(p \wedge q) \equiv (\neg p) \vee (\neg q)$. We’ll calculate both of their truth tables and compare!

p	q	$\neg(p \wedge q)$
T	T	F
T	F	T
F	T	T
F	F	T

and

p	q	$(\neg p) \vee (\neg q)$
T	T	F
T	F	T
F	T	T
F	F	T

These are clearly the same!

De Morgan’s laws tell us how negation interacts with $\wedge$ and $\vee$: pushing $\neg$ inside a conjunction or disjunction swaps one connective for the other.

This raises a natural question: how few operations do you actually need? This is where we introduce the concept of “functional completeness”: how few operations do you need to recreate all the logical operations on booleans?

For example, we have the operation $p \rightarrow q$. To recall, this operation has the following truth table:

p	q	$p \rightarrow q$
T	T	T
T	F	F
F	T	T
F	F	T

Now it turns out this is identical to the truth table of $(\neg p) \vee q$

p	q	$(\neg p) \vee q$
T	T	$F \vee T = T$
T	F	$F \vee F = F$
F	T	$T \vee T = T$
F	F	$T \vee F = T$

The truth tables match! So in classical propositional logic we don't even *need* implication as a primitive—it can be defined as $\neg p \vee q$. Worth flagging that this equivalence is a feature of classical logic specifically: in constructive logic, where excluded middle isn't available, $p \rightarrow q$ and $\neg p \vee q$ come apart. For this course we're staying classical, but if you ever read about constructive or intuitionistic logic, the fact that these two formulas are *not* interchangeable there is one of the first surprises.

This means that just having $\vee$, $\wedge$, and $\neg$ is “functionally complete” because it can describe all the operations of this logic.

2.3 Addendum: nand, xor, and how many binary operations are there?

There are some additional operations that you *can* define that we haven't defined yet.

The first of these we'll call xor (sometimes shown with the $\oplus$ symbol), the *exclusive or* and it has the following truth table:

p	q	$p \oplus q$
T	T	F
T	F	T
F	T	T
F	F	F

This is true if *either* p or q is true but not when *both* are true.

The second of these is *nand*, which is just “not-and” and it's rather important to the design of circuits:

p	q	$p \text{ nand } q$
T	T	F
T	F	T
F	T	T
F	F	T

An obvious question, and one that comes up in the homework of the accompanying class to these notes, is *how many of these operations are there and what are they?*

We can approach this *exhaustively*. A binary operation takes two (2) arguments, each of which is a boolean and can take two values. That's where we get the form of the truth tables above!

p	q	$f(p, q)$
T	T	?
T	F	?
F	T	?
F	F	?

How many different binary operations are even definable? That's equivalent to asking how many different truth tables are possible. There's a distinct truth table for every choice of how we fill in those four output entries in the table above. Well, okay, we know how to do this! Each of those four slots involves making a choice between one of two things: T and F. So the total number of choices must be $2 \times 2 \times 2 \times 2 = 2^4$ which is 16.

To prove *functional completeness* of a set of boolean operators, you need to show that for any possible truth table you can find a way to replicate it using those operators. Let's do a concrete example:

Here's our truth table for implication

p	q	$p \rightarrow q$
T	T	T
T	F	F
F	T	T
F	F	T

Now let's show that with just or, and, and not we can replicate the implication operator:

p	q	$(\neg p) \vee q$
T	T	T
T	F	F
F	T	T
F	F	T

Solving these little puzzles is the point of the homework this week.

2.4 From truth table back to formula: a worked example

Everything we've done so far goes in one direction: given a formula, compute its truth table. But in practice—especially when you're designing a digital circuit or implementing a controller that has to behave a certain way on every possible input—you often start with a truth table (a specification of what the output *should* be) and need to work backward to a formula that realizes it.

Here's the worked example. Suppose you want a formula $\varphi(p, q, r)$ that is true exactly on the following rows:

p	q	r	$\varphi(p, q, r)$
T	T	T	T
T	T	F	F
T	F	T	T
T	F	F	F
F	T	T	F
F	T	F	F
F	F	T	T
F	F	F	F

Step 1. Find every row where the output is T. Here there are three: (T, T, T) , (T, F, T) , and (F, F, T) .

Step 2. For each such row, write a conjunction (“and”) that is true exactly on that row and false on every other row. The trick is to use the variable if the row has T for it, and its negation if the row has F:

- Row (T, T, T) : $p \wedge q \wedge r$
- Row (T, F, T) : $p \wedge \neg q \wedge r$
- Row (F, F, T) : $\neg p \wedge \neg q \wedge r$

Each of these little formulas is true on exactly one row of the truth table—check one of them if you like.

Step 3. Take the *disjunction* (“or”) of all those conjunctions:

$$\varphi(p, q, r) \equiv (p \wedge q \wedge r) \vee (p \wedge \neg q \wedge r) \vee (\neg p \wedge \neg q \wedge r).$$

This formula is true exactly when at least one of the three conjunctions is true, which is exactly on the three rows we wanted.

The form we just built—“or of ands, with each and being a combination of variables and their negations”—has a name: *disjunctive normal form*, or DNF. Every truth table can be converted to a DNF formula by this recipe. So propositional logic can express any truth function: given any desired behavior as a truth table, the DNF recipe gives you a formula that realizes it. That’s actually a small theorem—*every* boolean function on n inputs is expressible in propositional logic—and it is half of why $\{\wedge, \vee, \neg\}$ is functionally complete. (The other half is that any formula you can write in propositional logic *is* such a boolean function, which is just what the truth-table semantics says.)

A side note: if you stare at the DNF formula for φ above, you might notice that r appears in every conjunction. That’s no accident—looking at the original truth table, every T row has $r = T$. So we could have saved work by just writing $\varphi \equiv r \wedge \psi(p, q)$ for some simpler ψ . Simplifying a DNF formula is what a lot of digital-logic coursework is about, and it’s the reason engineers care about tools like Karnaugh maps.

2.5 Common mistakes

Here are the mistakes students make most often on this module’s material. Watch for them on the assignment.

1. **Confusing implication with biconditional.** $p \rightarrow q$ is *not* the same as $p \leftrightarrow q$. In particular, $p \rightarrow q$ is true when p is false, regardless of what q is; $p \leftrightarrow q$ is only true when p and q agree. If the English reads “if and only if,” you want $\leftrightarrow$; if the English reads “if ... then” or “... implies ...,” you want $\rightarrow$.
2. **Vacuous truth feels wrong.** The rows where p is false make $p \rightarrow q$ come out true, and students often want those rows to be “false” or “undefined.” They aren’t. $F \rightarrow q$ is *true* by definition, because from a false premise we never make a false promise. If you can’t accept this at gut level yet, at least accept it at definition level, and move on.
3. **Operator precedence bugs.** Without parentheses, $\neg p \wedge q$ means $(\neg p) \wedge q$, *not* $\neg(p \wedge q)$. And $p \vee q \wedge r$ means $p \vee (q \wedge r)$, not $(p \vee q) \wedge r$. When in doubt, parenthesize explicitly; there’s no points off for being unambiguous.

4. **Inclusive vs. exclusive or.** Logical $\vee$ is *inclusive*: $T \vee T = T$. Everyday English “or” is often exclusive—“coffee or tea” usually means one or the other, not both. When you’re translating English into propositional logic, be deliberate: did the sentence mean $\vee$ or $\oplus$?
5. **Half-applied De Morgan.** Students often start pushing a negation inside and forget to flip the connective. $\neg(p \wedge q)$ is $\neg p \vee \neg q$, *not* $\neg p \wedge \neg q$. The connective flips: $\wedge \leftrightarrow \vee$.
6. **Misreading the scope of $\neg$.** Without parentheses, $\neg p \rightarrow q$ means $(\neg p) \rightarrow q$ —“if not- p then q ”—not $\neg(p \rightarrow q)$. This catches students constantly when they translate English like “it’s not the case that p implies q ” into the logic; the correct translation is $\neg(p \rightarrow q)$, with the parentheses. When negation is meant to apply to a whole implication (or disjunction, conjunction, etc.), parenthesize explicitly.

2.6 Summary and looking ahead

Key takeaways.

- *Propositional logic* is defined by a BNF grammar over T, F , variables, $\wedge, \vee, \neg, \rightarrow$. This is the first logic we define using the same “syntax + semantics” pattern we’ll reuse for every logic that follows.
- The *semantics* is given by *truth tables*—a formula is a boolean-valued function of its variables.
- A *tautology* is true under every assignment; a *contradiction* is false under every assignment; everything else is *contingent*. Two formulas are *logically equivalent* iff they have the same truth table.
- De Morgan’s laws push $\neg$ across $\wedge$ and $\vee$, flipping the connective. This is the prototype for the quantifier negation rules in Module 4 ($\neg \forall = \exists \neg$, $\neg \exists = \forall \neg$).
- $\{\wedge, \vee, \neg\}$ is *functionally complete*—every boolean function can be written as a formula in these connectives. The DNF recipe (one conjunction per T row of the truth table) is the constructive proof.

What we built this module. We have a full description of a logic: a grammar for writing formulas, a semantics for evaluating them, and enough machinery to check whether two formulas say the same thing. The truth-table method gives us a decision procedure—a mechanical way to answer any question about propositional-logic semantics—which is a rare luxury that won’t survive the jump to predicate logic in Module 4. DNF is the bridge to circuit design in Module 3.

Looking ahead. Module 3 asks a deeper question: can we *prove* tautologies without computing truth tables? The answer is yes—we build a *proof system* with introduction and elimination rules for each connective, and show those rules are sound against the truth-table semantics. The Gentzen intermezzo after Module 3 rewrites the same rules in the tree-shaped notation used by logicians and programming-language theorists, which will feel familiar from the DNF construction here.

2.7 Exercises

These are organized into **warm-up** (mechanical practice on the definitions), **standard** (typical assignment-style problems), **challenge** (synthesis across the module), and **connection** (programming and intermezzo pointers). Solutions are in the appendix.

Warm-up

Exercise 1. Build the truth table for $(p \rightarrow q) \wedge (q \rightarrow p)$. This combination will come up later when we define the biconditional ($\leftrightarrow$)—you’ll see why.

Exercise 2. Prove the second De Morgan’s law $\neg(p \vee q) \equiv (\neg p) \wedge (\neg q)$ by computing both truth tables.

Exercise 3. How many *unary* boolean operations are there? List them all as truth tables, and for each one, give a short English name or description.

Exercise 4. Classify each of the following formulas as a **tautology**, a **contradiction**, or **contingent** (neither always true nor always false). Do it by computing the truth table.

- (a) $p \vee \neg p$
- (b) $p \wedge \neg p$
- (c) $p \rightarrow (q \rightarrow p)$
- (d) $(p \rightarrow q) \rightarrow p$
- (e) $(p \wedge q) \rightarrow (p \vee q)$

Exercise 5. For each of the following unparenthesized expressions, add parentheses to show how the precedence rules ($\neg$ tighter than $\wedge$ tighter than $\vee$ tighter than $\rightarrow$) group them. Then evaluate the formula when $p = T$, $q = F$, $r = T$.

- (a) $\neg p \wedge q \vee r$
- (b) $p \rightarrow q \rightarrow r$ (Hint: $\rightarrow$ is conventionally *right-associative*, so this groups as $p \rightarrow (q \rightarrow r)$, not $(p \rightarrow q) \rightarrow r$. Check what difference it makes.)
- (c) $\neg p \vee q \wedge \neg r$

Standard

Exercise 6. Show that XOR ($p \oplus q$) can be expressed using only $\wedge$, $\vee$, and $\neg$. Hint: compare truth tables and use the DNF recipe from the worked example.

Exercise 7. Prove the equivalence $p \rightarrow (q \wedge r) \equiv (p \rightarrow q) \wedge (p \rightarrow r)$ by truth table. (This is sometimes called “implication distributes over conjunction on the right.” It means that to prove a conjunction from a premise, it suffices to prove each conjunct separately.)

Exercise 8. Show how to build each of $\neg$, $\wedge$, and $\vee$ using only the NAND operator. That is:

- (a) Find a formula using only NAND (and the variable p) whose truth table matches $\neg p$.
- (b) Find a formula using only NAND whose truth table matches $p \wedge q$. (Hint: NAND is “not-and,” so you can build AND by negating NAND, and you already know how to build NOT from NAND by part (a).)
- (c) Find a formula using only NAND whose truth table matches $p \vee q$. (Hint: De Morgan. $p \vee q \equiv \neg(\neg p \wedge \neg q)$, and you can build all three ingredients from NAND.)

This shows that NAND alone is functionally complete, which is why circuit designers love it: you can build an entire CPU out of NAND gates.

Exercise 9. A three-input boolean function $f(x, y, z)$ outputs T on exactly the input rows (T, F, F) , (F, T, F) , and (F, F, T) (the “exactly-one” function on three inputs). Use the DNF recipe from the worked example to write a propositional formula for f . Your formula should have three disjuncts, each with three literals.

Challenge

Exercise 10. Show that $\{\wedge, \vee\}$ —conjunction and disjunction, without negation—is *not* functionally complete. Hint: call a function f *monotone* if flipping any input from F to T never flips an output from T to F . (a) Show by induction on the structure of formulas that any formula built from just $\wedge$ and $\vee$ represents a monotone function. (b) Exhibit a boolean function that is not monotone. (c) Conclude.

Exercise 11. Prove: a formula φ is a tautology if and only if $\neg\varphi$ is a contradiction. (Do this both ways. Assume φ is a tautology and show $\neg\varphi$ is a contradiction; then assume $\neg\varphi$ is a contradiction and show φ is a tautology.) This is the definitional glue that lets proof by contradiction work—more on that in Module 7.

Exercise 12. Of the 16 binary boolean operations:

- (a) How many depend on *neither* of their inputs (i.e., the output is constant regardless of the inputs)? List them.
- (b) How many depend on *only one* of their inputs (so they ignore the other)? List them.
- (c) How many depend on *both* inputs?

(Your three counts should add up to 16.)

Connection

Exercise 13. (Programming.) Write a Python function `is_tautology(f, n)` that takes a function `f` of n boolean arguments and returns `True` if `f` is a tautology (i.e., returns `True` on every input) and `False` otherwise. Test it on `lambda p, q: (not p) or q` (which is logically equivalent to $p \rightarrow q$, so *not* a tautology, only a contingent formula) and on `lambda p, q: p or (not p)` (which *is* a tautology). Hint: use `itertools.product([False, True], repeat=n)` to enumerate all assignments.

Exercise 14. (Intermezzo pointer.) The discussion in this module of why the law of excluded middle ($p \vee \neg p$) is controversial points forward to the “Proofs Are Programs” intermezzo (Curry–Howard), which you can read after Module 7. As a preview: try to write a *program* that, given a

predicate P on natural numbers, produces either a natural number that satisfies P or a proof that no such number exists. What would the input and output types of that program have to be, and why is it not obvious how to implement it? (You don't need to write actual code—just think about the type signature.)

Chapter 3

Proof Systems for Propositional Logic

Reading map

Epp sections. §2.3–2.4: valid and invalid arguments, and digital logic circuits.

Prerequisites. Modules 1 and 2 (sets, truth tables, tautologies, De Morgan’s laws).

Relevant intermezzi. “Inference Rules as Trees” (Gentzen) — this module introduces rules of inference in a list format, and the Gentzen intermezzo shows the same rules in a more visual proof-tree form that generalizes to the later modules.

Practice from Epp. §2.3 exercises on validity testing via critical rows; §2.3 exercises on identifying converse and inverse errors; §2.4 exercises on translating between circuits and boolean expressions, and on DNF/CNF conversion.

3.1 Tautologies as reasoning

We’ve talked through a few different things about the *syntax* and the *semantics* of our first logic, propositional logic, and we’ve alluded to this idea that tautologies—formulas whose truth tables are always true—tell us **something** about what counts as valid reasoning.

Now it’s time to make that truly concrete and help explain some of these connections between natural human intuition and reasoning, informal mathematics, and formalized reasoning in a logic.

Consider the formula $p \wedge q \rightarrow p$. You can read this as “if p and q are true then p is true.” A reasonable assertion!

But reasonable isn’t good enough. We need to show that this is always the case, and we can do that by calculating its truth table.

p	q	$p \wedge q \rightarrow p$
T	T	T
T	F	$F \rightarrow T \implies T$
F	T	$F \rightarrow F \implies T$
F	F	$F \rightarrow F \implies T$

This is, in fact, *a tautology*.

But we can also interpret this tautology as saying something like

If we start with the premises p and q then we can conclude p . This represents a shift between a logical and a metalogical notion of reasoning. In one, we use the literal implication operator $\rightarrow$ and in the other we use the intuitive notion of *if* and *then*.

Aside: object language vs. metalanguage. When we say “the formula $p \wedge q \rightarrow p$ is a tautology,” the word “tautology” lives *outside* the logic: it is a claim we make *about* the formula, in ordinary English-plus-mathematics. The formula itself lives in the *object language*—the language we are studying—while our claim about it lives in the *metalanguage*, the language we do the studying in.

This distinction seems pedantic for a page or two and then turns out to matter a lot. “ $p \rightarrow q$ ” is an object-level formula; “from the hypothesis p we can derive q ” is a metalevel claim. They are connected (the whole point of Module 3 is to study the connection), but they live on different floors of the building and confusing them is the source of many real errors in proofs. You’ll see it again in the difference between $\vdash$ (“there is a derivation”) and $\models$ (“it is true under every assignment”), both of which are *metalevel* relations that speak about the object-level formulas. Keeping these layers straight is one of the habits that will matter most in CS 251.

This goes the other way too. For example, consider that most famous of reasoning steps “modus ponens”—¹ Modus ponens is simply this:

If p is true, and I know that p *implies* q , then I can conclude q .

It’s a pretty intuitive concept because it just formalizes the meaning of implication: that p being true *forces* q to be true.

A different way to write modus ponens is to take the approach the book takes, which will look remarkably like the way statements of theorems and proofs will look once you get to the Natural Number Game later in the class.

$$\frac{\begin{array}{l} \text{(a) } p \\ \text{(b) } p \rightarrow q \end{array}}{\text{shows } q}$$

This form is convenient because it makes clear what formula corresponds to the theorem. To build it: conjoin all the premises with $\wedge$, then put $\rightarrow$ between that conjunction and the conclusion.

Here the corresponding formula, which we hope to show is a tautology, would be $(p \wedge (p \rightarrow q)) \rightarrow q$.

Let’s justify this by taking a look at its truth table

p	q	$(p \wedge (p \rightarrow q)) \rightarrow q$
T	T	T
T	F	$(T \wedge (T \rightarrow F)) \rightarrow F \implies T$
F	T	$(F \wedge (F \rightarrow T)) \rightarrow T \implies T$
F	F	$(F \wedge (F \rightarrow F)) \rightarrow F \implies T$

This truth table is all Ts, so it *is* in fact a tautology! This means that we’ve described a valid method of proof after all.

This isn’t surprising — we defined implication to behave exactly this way.

¹I swear it’s a red flag if someone says “modus ponens” in casual conversation, even when that conversation is about mathematics and logic.

3.2 A thorough derivation of proof by contradiction

Before we cover the remaining proof methods, let's dig into *proof by contradiction* in detail, which you might recognize from our very first unit in this class when we proved that there were an infinite number of natural numbers. To do that we need to explore the *other* Latin proof method: modus tollens.

Modus tollens is a way of dealing with implication and things that *aren't* true. In the semi-formal proof structure, modus tollens says that

$$\frac{\begin{array}{l} \text{(a)} \quad p \rightarrow q \\ \text{(b)} \quad \neg q \end{array}}{\text{shows} \quad \neg p}$$

Let's talk about what this means before we jump into the truth table: if we know that p implies q , but that q isn't true, we then know that p *can't* be true because if it was true it would force q to be true. For example, if you know that if you get over 80% on the final exam you'll get an A in the class, but you see that your final grade was an AB, then you *know* that you got less than 80% on the final.

Again, this is just a restatement of our intuition about what implication must mean, but in the negative rather than positive sense.

But let's check that it's a tautology and thus valid reasoning

p	q	$((p \rightarrow q) \wedge \neg q) \rightarrow \neg p$
T	T	$((T \rightarrow T) \wedge F) \rightarrow F \implies F \rightarrow F \implies T$
T	F	$((T \rightarrow F) \wedge T) \rightarrow F \implies F \rightarrow F \implies T$
F	T	$((F \rightarrow T) \wedge F) \rightarrow T \implies F \rightarrow T \implies T$
F	F	$((F \rightarrow F) \wedge T) \rightarrow T \implies T \rightarrow T \implies T$

Good — this is firmly a tautology.

What about proof by contradiction? This is where it's important to remember that the p and q aren't necessarily variables but are generalized sub-formulas—you can stick another formula inside there and the result should be valid. Since the tautologies are *always* true it doesn't even matter what values the sub-formula take.

Proof by contradiction is, essentially,

$$\frac{\text{(a)} \quad \neg p \rightarrow (r \wedge \neg r)}{\text{shows} \quad p}$$

In other words, if assuming p is false leads to a contradiction (that you can conclude—for some r —that r is true *and* $\neg r$ is true) then you can conclude p . How is this modus tollens? We take modus tollens and substitute $\neg p$ for the p in the formula and $r \wedge \neg r$ for the q .

We then get the formula—which again, we know is a tautology—

$$((\neg p \rightarrow (r \wedge \neg r)) \wedge \neg(r \wedge \neg r)) \rightarrow \neg \neg p$$

This is almost the right form but look at that goofy $\neg(r \wedge \neg r)$ in there! By De Morgan's laws we have that this should actually become $\neg r \vee \neg \neg r$, which still contains $\neg \neg r$. We should verify double negation with a truth table rather than taking it for granted.

p	$\neg\neg p$
T	$\neg F \implies T$
F	$\neg T \implies F$

Okay, so not-not p is p . Neat. In which case the $\neg(r \wedge \neg r)$ part of our expression becomes $\neg r \vee r$, and we recognize that one as a tautology we saw earlier! The thing about it being a tautology is that it means we can just substitute T for it instead, so our formula for proof by contradiction becomes

$$((\neg p \rightarrow (r \wedge \neg r)) \wedge T) \rightarrow \neg\neg p,$$

which can simplify even further to

$$(\neg p \rightarrow (r \wedge \neg r)) \rightarrow \neg\neg p,$$

and since $\neg\neg p$ is equivalent to p , this is exactly the proof-by-contradiction rule we wanted.

And since we did this entirely by substituting formula into a tautology and simplifying, we don't even need to check the truth table to guarantee correctness!

Notice that this derivation relied on double negation elimination—the step where we replaced $\neg\neg p$ with p —which is equivalent to excluded middle. This makes full proof by contradiction a *classical* technique; it doesn't work in constructive logic, where you can't just erase double negations. What about proof by contrapositive? The situation is subtler than it might seem. Modus tollens—from $P \rightarrow Q$ and $\neg Q$, conclude $\neg P$ —is constructively valid. But “proof by contrapositive” as a technique means proving $\neg Q \rightarrow \neg P$ to establish $P \rightarrow Q$, and that equivalence also requires excluded middle. So both contradiction and contrapositive, as proof techniques, are classical. The difference is that contrapositive proofs tend to be more *informative*—they show you the structure of the argument rather than just ruling out alternatives. We'll revisit this distinction more carefully in Module 7.

3.3 A proof system for propositional logic

Do we *always* have to prove things within the logic by appealing to truth tables? Truth tables work fine for propositional logic, because in this logic all our variables are booleans and can take one of two values: T or F.

In the next module, though, we're going to add one of the simplest possible new types of data to our logic that aren't just booleans: the natural numbers, i.e. 0, 1, 2, &c.

As we proved together in our very first week of class there are an infinite number of natural numbers so already we can no longer check “truth tables” of every possible value of every variable in order to determine whether the statement is true.

No, instead, we need to have a system for doing *proofs* within the logic, methods of reasoning that we know are “sound”—or in other words will always produce logically valid results no matter how they're chained and combined together.

Now, we can't just make up these proof rules out of nothing: we need to have some way of checking that these are valid in the first place, and that's where the *semantics* of a logic are necessary. We can check that the rules of proof work with the semantics and don't do anything wild like let you conclude false things from true ones.

With that motivation in hand, let's list out the proof rules and check them against the truth-table semantics, which in this case means checking truth tables.

We’ve already talked about a couple of the rules: modus ponens and modus tollens, the rules for implication.

But for every binary operator we’ve seen there are rules for how to build it and how to *use* it. These are called introduction and elimination rules, respectively.

Psst: dear reader, if you’re wondering what the introduction rule for implication is, we’ll come back to that at the end of this section.

3.3.1 Rules for conjunction

Introduction:

$$\frac{\begin{array}{c} \text{(a)} \quad p \\ \text{(b)} \quad q \end{array}}{\text{shows } p \wedge q}$$

In other words, if you know that p is true and q is true then you know that $p \wedge q$ is true. Unsurprising!

In terms of the validation of this rule this means that we’d need to show that $p \wedge q \rightarrow p \wedge q$ is a tautology. But this is obvious because we can easily show that $a \rightarrow a$ for any subformula a that we choose to substitute in.

$$\frac{\frac{a \quad a \rightarrow a}{T \quad T \rightarrow T = T}}{F \quad F \rightarrow F = T}$$

Elimination: There are two elimination rules for $\wedge$:

$$\frac{\text{(a)} \quad p \wedge q}{\text{shows } p} \quad \frac{\text{(a)} \quad p \wedge q}{\text{shows } q}$$

These are unsurprising.

3.3.2 Rules for $\vee$

Since $\vee$ is the mirror of $\wedge$, it’s not surprising that there are going to be two introduction rules and one elimination rule.

Introduction:

$$\frac{\text{(a)} \quad p}{\text{shows } p \vee q} \quad \frac{\text{(a)} \quad q}{\text{shows } p \vee q}$$

These just follow from the very definition of what $\vee$ is as an operator and you can verify that $p \rightarrow p \vee q$ and $q \rightarrow p \vee q$ are both tautologies.

Elimination: What does it *mean* to prove that $(p \vee q) \rightarrow r$? You don’t know which of p and q is true, so what you’re saying is “whether p or q is true I know r is true,” which we can break down like this:

$$\frac{\begin{array}{c} \text{(a)} \quad p \vee q \\ \text{(b)} \quad p \rightarrow r \\ \text{(c)} \quad q \rightarrow r \end{array}}{\text{shows } r}$$

And we can validate this with a truth table like so (this is going to be a little ugly so hold on)

p	q	r	$((p \vee q) \wedge (p \rightarrow r) \wedge (q \rightarrow r)) \rightarrow r$
T	T	T	$(T \wedge T \wedge T) \rightarrow T \implies T$
T	T	F	$(T \wedge F \wedge F) \rightarrow F \implies T$
T	F	T	$(T \wedge T \wedge T) \rightarrow T \implies T$
T	F	F	$(T \wedge F \wedge F) \rightarrow F \implies T$
F	T	T	$(T \wedge T \wedge T) \rightarrow T \implies T$
F	T	F	$(T \wedge T \wedge F) \rightarrow F \implies T$
F	F	T	$(F \wedge T \wedge T) \rightarrow T \implies T$
F	F	F	$(F \wedge T \wedge T) \rightarrow F \implies T$

What does this rule actually tell us? It is, as the book points out, the formal logic of *proof by cases*. Proof by cases is something we’ve already informally encountered in the class and it’s implicitly what works behind homework 2: for each of the 16 possible truth tables of binary operators, you show that you can prove the proposition, so you know it’s true for all of them (all in this case functioning like a Big Or).

And here’s something worth flagging for later: when we eventually see the meaning of $\forall$ and $\exists$, there is a sense in which $\forall$ really is an “infinitely big $\wedge$ ” and $\exists$ is an “infinitely big $\vee$.”

3.3.3 Negation

This is a fun one because we’ve essentially seen the rules for this already.

Introduction:

$$\frac{\text{(a) assuming } p \text{ leads to a contradiction}}{\text{shows } \neg p}$$

In other words, if assuming p lets you derive both q and $\neg q$ for some q , then you can conclude $\neg p$. This is sometimes called “proof by negation” to distinguish it from full proof by contradiction. It’s worth noting that in a lot of logics negation is *defined* as $\neg p := p \rightarrow \perp$. One such logic is the logic behind Lean 4, which you’ll be playing with as part of learning The Natural Number Game. This definition is also what the Curry–Howard intermezzo (much later) will use to “read” $\neg p$ as the type of functions from proofs of p into the empty type—a program that, given evidence for p , produces an absurdity.

Elimination:

$$\frac{\text{(a) } \neg\neg p}{\text{shows } p}$$

This is double negation elimination, and as we discussed in the proof-by-contradiction section, it requires excluded middle. In constructive logic, you can always go from p to $\neg\neg p$, but not the other way.

3.3.4 $\top$ and $\perp$

There are also introduction and elimination rules for the constants $\top$ and $\perp$ (“top” for true, “bottom” for false—the same as T and F from Module 2, but written this way in natural-deduction systems). What’s fun about this is that there’s *only* an introduction rule for $\top$ and *only* an elimination rule for $\perp$.

Introduction ($\top$):

$$\frac{}{\text{shows } \top}$$

That's it — you can just conclude $\top$. It needs no premises. And there's no elimination rule because knowing $\top$ gives you no new information.

Elimination ($\perp$):

$$\frac{(a) \quad \perp}{\text{shows } p}$$

Wait, what? What is p here? Well it's literally anything. Because if you assume false you can conclude anything. To understand this, follow our rules for how to convert a proof in the above form into its corresponding tautology and then look at what the truth table would be.

3.3.5 Implication

Okay, we saved implication for last because it's... a little weird.

Elimination: The elimination rule for implication is just modus ponens. That's easy.

Introduction: It's the introduction that's weird. Because it's more like:

$$\frac{(a) \quad \text{a proof that assuming } p, \text{ shows } q}{\text{shows } p \rightarrow q}$$

Why is this weird? Because we have two layers: *implication* inside the logic, and *proofs* which take a set of premises and derive a conclusion, which is kind of a *metalogical* activity.

The introduction rule of implication lets us take a proof—a metalogical thing—and turn it into an implication *within* the logic. Then the elimination rule lets us take an implication within the logic and use it inside a proof.

3.3.6 Extra proof rules

There are a handful of proof rules we're going to introduce to help us out, based on tautologies. An important one, with no premises:

$$\frac{}{\text{shows } p \vee \neg p}$$

This is going to help us deal with a bunch of proofs by giving us the ability to, essentially, use subformulas without any assumptions. We're going to call this rule “excluded middle.”

It's worth pausing to note that this rule is a *choice*. The introduction and elimination rules for conjunction, disjunction, and implication are all constructively valid, as is negation *introduction* (proof by negation). But negation *elimination* (double negation elimination) requires excluded middle. Excluded middle is the specific axiom you add on top of those structural rules to get *classical* logic. Without it, you have what's called intuitionistic or constructive logic, where every proof of a disjunction $p \vee q$ must actually demonstrate *which* of p or q holds. We're choosing to include excluded middle in this course, but it's worth knowing that this is precisely the thing that separates classical from constructive reasoning.

Also we have the maybe obvious rule of proof by assumption:

$$\frac{(a) \quad p}{\text{shows } p}$$

This is just a rule that lets us formally use our assumptions in our proofs.

3.4 Arguments and validity

Now we need to actually talk about what it means for an argument to be *valid*, because this is something that the assignments lean on heavily and it's worth being really precise about.

An *argument form* is just a sequence of statements called *premises* followed by a *conclusion*. We write the premises on top, draw a little horizontal line, and then write the conclusion below it with a $\therefore$ symbol. For example, here's the argument form for modus ponens:

$$\frac{p \rightarrow q \quad p}{\therefore q}$$

When is an argument form *valid*? An argument form is valid if whenever **all** the premises are true, the conclusion is also true. Another way to say this: take the conjunction of all the premises, form its implication to the conclusion, and check whether *that* whole formula is a tautology. If it is, the argument is valid. If it's not, the argument is invalid and there exists some assignment of truth values where all the premises hold but the conclusion doesn't.

How do we actually *test* validity? You build a truth table for all the variables involved, find the rows where every single premise evaluates to true—these are the *critical rows*—and then check whether the conclusion is also true in those rows. If the conclusion is true in every critical row, valid. If there's even one critical row where the conclusion is false, invalid.

Let's work through an example. Consider the argument $p \vee q, \neg q \therefore p$ —this is called *disjunctive syllogism* and we listed it in the derivable rules section below, but let's actually verify it.

p	q	$p \vee q$	$\neg q$	p
T	T	T	F	T
T	F	T	T	T
F	T	T	F	F
F	F	F	T	F

The critical rows are the ones where *both* premises are true, i.e. where $p \vee q$ is T *and* $\neg q$ is T. That's only the second row ($p = T, q = F$). And in that row the conclusion p is T. So the argument is valid!

Now here's where things get interesting, because there are two extremely common *fallacies*² that trip people up constantly:

Converse error (also called “affirming the consequent”): this is the argument form

$$\frac{p \rightarrow q \quad q}{\therefore p}$$

This is **invalid**. Here's the intuition: “if it's raining then the ground is wet; the ground is wet; therefore it's raining.” But the sprinklers could be on! Just because the ground is wet doesn't mean it rained. The problem is that you're confusing $p \rightarrow q$ with its *converse* $q \rightarrow p$, and an implication and its converse are **not** logically equivalent (you can verify this by comparing their truth tables).

²A fallacy is just an argument form that looks plausible but is actually invalid. The whole point of formal logic is to catch these things before they sneak into your reasoning.

Inverse error (also called “denying the antecedent”): this is the argument form

$$\frac{p \rightarrow q \quad \neg p}{\therefore \neg q}$$

Also **invalid**. “If it’s raining then the ground is wet; it’s not raining; therefore the ground is not wet.” Again, sprinklers. You’re confusing $p \rightarrow q$ with its *inverse* $\neg p \rightarrow \neg q$, which again is not logically equivalent to the original.

One nice thing about all of this machinery is that you can *chain* argument forms together to build longer deductions. This is what a formal proof really is: you start with your premises, apply rules of inference one step at a time, and each step produces a new known fact that you can use in later steps. Think of it like writing a program where each line is a function call that’s justified by one of our rules—modus ponens, disjunctive syllogism, &c.—and the “return value” feeds into the next line. You keep going until you reach the conclusion.

Here’s a quick example of chaining. Suppose our premises are $p \rightarrow q$, $q \rightarrow r$, and p . We want to conclude r . The deduction looks like:

- | | |
|----------------------|------------------------|
| 1. $p \rightarrow q$ | (premise) |
| 2. $q \rightarrow r$ | (premise) |
| 3. p | (premise) |
| 4. q | (modus ponens on 3, 1) |
| 5. r | (modus ponens on 4, 2) |

Each line is justified by exactly one rule and refers back to earlier lines. That’s the whole game. It’s very much like a sequence of statements in a program where each variable depends on previously computed values.

3.5 Examples of doing proofs

Now let’s dig into some proofs and see how they actually work. For example, let’s see if we can *prove* some laws:

Proof systems—formal proofs in a logic—are going to look a little different than the kind of thing we’ve done before.

Let’s make an important derived rule: transitivity of implication.

Theorem. Assuming $p \rightarrow q$ and $q \rightarrow r$, show $p \rightarrow r$.

Proof.

- (a) $p \rightarrow q$, by assumption.
- (b) $q \rightarrow r$, by assumption.
- (c) A proof that, assuming p , shows r :
 - (i) p , by assumption.
 - (ii) q , by modus ponens with (i) and (a).

(show) r , by modus ponens with (ii) and (b).

Shows $p \rightarrow r$ by introduction of implication.

Theorem. Assuming $p \rightarrow q$, show $\neg p \vee q$.

Proof.

(a) $p \rightarrow q$, by assumption.

(b) $p \vee \neg p$, by excluded middle.

(c) $q \rightarrow (\neg p \vee q)$, by introduction of $\rightarrow$ and $\vee$.

(d) $p \rightarrow (\neg p \vee q)$, by transitivity of (a) and (c).

(e) $\neg p \rightarrow (\neg p \vee q)$, by introduction of $\rightarrow$ and $\vee$.

Shows $\neg p \vee q$, by elimination of $\rightarrow$ with (b), (d), (e).

3.5.1 Conjunctive normal form

A formula is in **conjunctive normal form** (CNF) when it is a conjunction of *clauses*, where each clause is a disjunction of *literals*. A literal is either a variable or its negation (e.g. p or $\neg p$).

For example,

$$(p \vee \neg q) \wedge (\neg p \vee q \vee r) \wedge q$$

is in conjunctive normal form: it is a conjunction of three clauses, each of which is a disjunction of literals.

On the other hand, $\neg(p \vee q)$ is *not* in conjunctive normal form, because the outermost operator is negation, not conjunction. However, by De Morgan's laws it can be rewritten as $\neg p \wedge \neg q$, and *this* is in CNF (a conjunction of two single-literal clauses).

3.5.2 Disjunctive normal form

A formula is in **disjunctive normal form** (DNF) when it is a disjunction of *terms*, where each term is a conjunction of *literals*. So the following is in disjunctive normal form:

$$(p \wedge \neg q) \vee (\neg p \wedge q \wedge r) \vee q$$

But the following is *not*, because $\neg(q \wedge r)$ is not a conjunction of literals:

$$(p \wedge q) \vee \neg(q \wedge r)$$

3.6 Digital logic circuits

Here's where things get fun and we connect all this abstract boolean algebra stuff to the real physical world. It turns out that the logical operations we've been studying—AND, OR, NOT—have direct physical implementations called *logic gates*. An AND gate is a little electronic component that takes two input signals and produces an output that's 1 (high voltage) only when both inputs are 1. An OR gate outputs 1 when at least one input is 1. A NOT gate (also called an *inverter*) takes one input and flips it. There are also NAND gates, which are just an AND followed by a NOT, and

it turns out these are particularly important because you can build literally every other gate out of just NAND gates.³

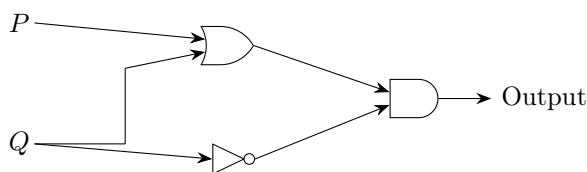
Each of these gates has a standard symbol used in circuit diagrams:



The distinctive shapes—a shield for AND, a crescent for OR, a triangle with a bubble for NOT—go back to the 1940s and are universal in chip-design documentation. The little “bubble” on the output of NOT (and NAND, NOR, etc.) indicates negation.

A *circuit* is just a composition of gates—you wire the output of one gate into the input of another. If you think about it this is exactly function composition! The output of gate f becomes the input of gate g , so you’re computing $g(f(x))$. It’s functions all the way down.

To *evaluate* a circuit you trace signals from the inputs through each gate to the output, applying the truth table of each gate as you go. Let’s do an example: suppose we have inputs P and Q , and the circuit works like this. Q branches into two paths: one path goes to an OR gate together with P , and the other path goes through a NOT gate. Then the output of the OR gate and the output of the NOT gate both feed into an AND gate, and that’s our final output. Here’s the circuit drawn out:



With $P=1$ and $Q=0$: the OR gate computes $\text{OR}(1,0) = 1$, the NOT gate computes $\text{NOT}(0) = 1$, and the AND gate computes $\text{AND}(1,1) = 1$. So the circuit outputs 1 for that input.

We can fill out the whole truth table for this circuit by tracing through all four input combinations:

P	Q	$P \vee Q$	$\neg Q$	$(P \vee Q) \wedge \neg Q$	Output
1	1	1	0	0	0
1	0	1	1	1	1
0	1	1	0	0	0
0	0	0	1	0	0

This circuit computes the function that’s true exactly when P is true and Q is false—which is just $P \wedge \neg Q$. Interesting that our three-gate circuit could have been built with two gates! This is the kind of optimization that boolean algebra helps us find.

But here’s the really cool part: how do you *build* a circuit from a truth table? This is where disjunctive normal form comes back in a big way. Remember DNF? Here’s the recipe:

1. Look at every row of the truth table where the output is 1.

³This is called functional completeness and it’s honestly one of the coolest facts in all of computer science. Your entire computer could, in principle, be built out of nothing but NAND gates.

2. For each such row, build an AND of all the input variables—but negate any variable that’s 0 in that row. This AND gate “recognizes” exactly that one row.
3. OR all of those ANDs together.

For instance, if you have a truth table with inputs P, Q and the output is 1 only when $P = 1, Q = 0$ or $P = 0, Q = 1$, you’d get the DNF expression $(P \wedge \neg Q) \vee (\neg P \wedge Q)$ —which, hey, that’s XOR! And now you have a direct recipe for wiring up the gates: two AND gates (one for each term), some NOT gates for the negations, and one OR gate to combine them.

This is why DNF matters—it gives you a completely mechanical way to turn *any* truth table into a working circuit. And this is literally how hardware design works at the lowest level. Computer chips are just enormous networks of logic gates, millions and millions of them wired together. The boolean algebra we learned in Module 2 *is* the mathematics of chip design.

A caveat worth stating clearly: the DNF recipe always produces a *correct* circuit, but not necessarily a *minimal* one. In the example above, our three-gate circuit really did implement $P \wedge \neg Q$, which a single AND-plus-NOT implements in two gates. Finding the smallest circuit for a given truth table is a separate problem—*circuit minimization*—and it is the subject of techniques like Karnaugh maps and the Quine–McCluskey algorithm that show up in a digital-logic or hardware-design course. DNF gets you a correct solution; minimization gets you an efficient one. Don’t conflate the two.

Notice how this connects back to functional completeness from the previous module. The DNF construction is actually a *proof* that AND, OR, and NOT are functionally complete: given any truth table whatsoever, DNF hands you a circuit built entirely from those three gates. And since we showed in that module that NAND alone can simulate AND, OR, and NOT, it follows that NAND alone is functionally complete—which is exactly why real hardware can be (and often is) built entirely from NAND gates.

3.7 Addendum: differences in terminology and additional derivable proof rules for homework

3.7.1 Derivable rules of inference

There are a number of extra rules you can derive that are useful for proofs, some of them are so useful they get extra special names:

Note: double negation elimination ($\neg\neg q$ shows q) was already listed as the elimination rule for negation in Section 3.3 above.

Modus tollens:

$$\frac{\begin{array}{l} \text{(a)} \quad p \rightarrow q \\ \text{(b)} \quad \neg q \end{array}}{\text{shows } \neg p}$$

We derived this as a tautology in Section 2. It can also be derived from implication elimination and negation introduction.

Disjunctive syllogism (two versions):

$$\frac{\begin{array}{l} \text{(a)} \quad p \vee q \\ \text{(b)} \quad \neg q \end{array}}{\text{shows } p} \qquad \frac{\begin{array}{l} \text{(a)} \quad p \vee q \\ \text{(b)} \quad \neg p \end{array}}{\text{shows } q}$$

3.7.2 Differences in verbiage

The webwork problems might refer to “hypothetical syllogism”; this is just a different way of saying “transitive property of implication,” which we derived above:

$$\frac{\begin{array}{l} \text{(a)} \quad p \rightarrow q \\ \text{(b)} \quad q \rightarrow r \end{array}}{\text{shows } p \rightarrow r}$$

3.8 Truth and provability meet

We now have two completely different ways to certify that a propositional formula φ is “valid.”

- **Semantically.** Compute its truth table. If every row is T, then φ is a *tautology*. This is the meaning-based, truth-table criterion from Module 2. We write $\models \varphi$ (read: φ is valid) when this holds.
- **Syntactically.** Build a derivation of φ from the introduction and elimination rules, starting from no premises. This is the proof-system criterion from this module. We write $\vdash \varphi$ (read: φ is derivable) when such a derivation exists.

A priori these have nothing to do with each other. One is about truth values under every assignment; the other is about syntactic manipulation of strings. The great result of propositional logic—and a small preview of the deepest theorems in mathematical logic—is that for this logic, the two notions coincide exactly.

Theorem 3.8.1 (Soundness). If $\vdash \varphi$ then $\models \varphi$. Every formula that has a derivation is a tautology.

Theorem 3.8.2 (Completeness). If $\models \varphi$ then $\vdash \varphi$. Every tautology has a derivation.

Soundness is “the proof system doesn’t prove false things”—which is exactly what you want from something called a proof system. Completeness is “the proof system proves every true thing”—which is the much less obvious direction, and the one that distinguishes a good proof system from a merely safe one. A proof system that lets you derive nothing at all would be sound but useless; completeness is what rules that out.

We will not prove either of these in this course. Soundness is the easier one: it follows by induction on the shape of a derivation, checking that each rule preserves truth-table validity of its premises. Completeness is deeper and has several clever proofs; the classical one constructs a derivation for a given tautology by looking at which rows of the truth table are T. You’ll see these proofs in CS 251 or in a formal-methods/metalogic course. What matters now is the *statement*: the truth-table world and the proof-system world are the same world, looked at from two different angles.

Aside: Gödel, and why first-order logic is different. For propositional logic, soundness and completeness are both reasonable textbook theorems. For first-order predicate logic (Module 4) the completeness theorem was a serious result—it’s Gödel’s thesis, 1929, published 1930, and is the reason we are comfortable using first-order logic at all. For systems strong enough to talk about arithmetic, Gödel’s *incompleteness* theorems (1931) say that completeness in this sense is no longer available: there will always be true statements of arithmetic that no proof system can derive. Cumulative Review 1 returns to this story with a little more detail.

The upshot for this course is practical. When you want to establish φ , you have a choice of two tools—compute a truth table, or build a derivation—and soundness/completeness guarantees that they cannot disagree. Which one to use is a judgment call about the specific formula: truth tables are good when the formula has few variables, derivations are good when you want a chain of reasoning you can actually read.

3.9 Common mistakes

The module already highlighted the two famous fallacies—converse error and inverse error. Here are the other mistakes that show up in grading.

1. **Confusing implication inside the logic with “shows” in a proof.** Writing $p \rightarrow q$ in a proof as a rule of inference is a *metalogical* claim (you can turn a proof of p into a proof of q). Writing $p \rightarrow q$ as a formula is an object-level assertion about truth values. These are connected by the introduction and elimination rules for $\rightarrow$, but they’re not the same thing. If a problem asks for a proof “assuming p , show q ,” you cannot write $p \rightarrow q$ as your first line—you have to actually produce the inner proof and then apply $\rightarrow$ -introduction.
2. **Misusing $\vee$ -elimination.** The disjunction-elimination rule says: from $p \vee q$, $p \rightarrow r$, and $q \rightarrow r$, conclude r . Students often skip one of the two implications—they show $p \rightarrow r$, forget $q \rightarrow r$, and try to conclude r anyway. That’s not valid; you need *both* cases to cover the disjunction.
3. **Mixing up CNF and DNF connectives.** CNF is “*and* of *ors*” and DNF is “*or* of *ands*.” A helpful mnemonic: **C**NF has **C**onjunction at the outside; **D**NF has **D**isjunction at the outside. Students write a DNF formula and call it CNF; you can avoid this by always naming the outer connective first.
4. **Forgetting to check every critical row.** A valid argument is one where the conclusion is true *in every row where all the premises are true*. Students sometimes spot one row where it works and declare the argument valid without checking the others. One row is enough to *disprove* validity, but not to *prove* it.
5. **“Proof by assumption” is not “assume anything.”** The rule lets you use something you have *already assumed* (or already derived) as a line of the proof. It does not let you invent new assumptions mid-proof. If you want to introduce a new assumption, you must be starting a subproof (for $\rightarrow$ -introduction, $\vee$ -elimination, or negation introduction).

3.10 Summary and looking ahead

Key takeaways.

- A *proof system* for propositional logic consists of *introduction* and *elimination* rules for each connective. Introduction rules tell you how to build a formula; elimination rules tell you how to use one. The pattern scales to every logic we’ll see.
- *Modus ponens* (implication elimination) and *modus tollens* are the two workhorse rules for reasoning with implications. Modus tollens is modus ponens run backwards through the contrapositive.

- *Excluded middle* ($p \vee \neg p$) is an *axiom*, not a consequence of the other rules. Adding it to constructive logic yields classical logic. Proof by contradiction depends on it; proof by contrapositive also depends on it in subtle ways.
- *Converse error* and *inverse error* are the two classic fallacies about implications. Students make them constantly. If you're about to conclude p from $p \rightarrow q$ and q , stop—that's the converse error.
- *Disjunctive* and *conjunctive* normal forms give a mechanical bridge between truth tables and circuits: DNF is “or of ands,” CNF is “and of ors.” Both are functionally complete.
- *Soundness* says every derivable formula is a tautology; *completeness* says every tautology is derivable. For propositional logic, $\vdash$ and $\models$ coincide—the truth-table world and the proof-system world are two views of the same thing.

What we built this module. We now have two ways to establish a propositional-logic fact: by truth table (Module 2) and by derivation in a formal proof system (this module). The two approaches are *equivalent*—a formula is a tautology iff it has a derivation—but they feel very different in practice. Truth tables are mechanical but explode exponentially; derivations are smaller but require insight. The digital-logic section tied both back to the physical world: every circuit is a formula, every formula can be built from NAND gates, and real hardware is a forest of these structures.

Looking ahead. Module 4 takes the logic-building pattern and applies it to a strictly more expressive logic: first-order predicate logic, which adds $\forall$ and $\exists$ on top of everything we have. The proof rules for the quantifiers have the same introduction/elimination shape as the propositional ones. Before you get there, read the Gentzen intermezzo to see the same rules in tree form, and/or the “Cardinality and Function Spaces” intermezzo if you haven't yet—both foreshadow Module 4's style of reasoning.

3.11 Exercises

Organized into **warm-up** (mechanical application of rules and tautology checks), **standard** (typical assignment-style validity and proof problems), **challenge** (synthesis across rules and fallacies), and **connection** (programming and intermezzo pointers). Solutions are in the appendix.

Warm-up

Exercise 1. Validate that $p \wedge q \rightarrow q$ is a tautology by computing its truth table.

Exercise 2. For each pair of formulas, decide whether they are logically equivalent by computing truth tables. If they are equivalent, say so; if not, give a specific row where they disagree.

- $p \rightarrow q$ and $\neg p \vee q$
- $p \rightarrow q$ and $q \rightarrow p$
- $p \wedge (p \rightarrow q)$ and $p \wedge q$

Exercise 3. For each of the following, state whether the conclusion follows by **modus ponens**, by **modus tollens**, or by **neither**. If “neither,” say what's wrong.

- (a) Premises: $p \rightarrow q, p$. Conclusion: q .
- (b) Premises: $p \rightarrow q, \neg q$. Conclusion: $\neg p$.
- (c) Premises: $p \rightarrow q, q$. Conclusion: p .
- (d) Premises: $p \rightarrow q, \neg p$. Conclusion: $\neg q$.

Exercise 4. Convert the formula $\neg(p \wedge q) \vee r$ into CNF. (Hint: start by using De Morgan's laws to push the negation inward.)

Exercise 5. Convert the same formula $\neg(p \wedge q) \vee r$ into DNF. Your two answers to Exercises 4 and 5 should be different formulas (but should represent the same boolean function, so they should have the same truth table).

Standard

Exercise 6. Determine whether the following argument is valid or invalid. If invalid, find a counterexample row. Premises: $p \rightarrow q, r \rightarrow \neg q, r$. Conclusion: $\neg p$.

Exercise 7. Given the truth table where the output is 1 only for inputs (P=1, Q=0, R=1) and (P=0, Q=1, R=1), write the DNF expression and describe the circuit.

Exercise 8. Give a step-by-step formal proof (numbered lines, each with a justification from one of the rules) that from the premises $p \rightarrow q$ and $p \rightarrow r$ you can conclude $p \rightarrow (q \wedge r)$. You will need to use $\rightarrow$ -introduction (so your proof has a subproof) and $\wedge$ -introduction. Compare your answer to Exercise 7 of Module 2, which is the corresponding truth-table fact.

Exercise 9. From the premises $\{p \vee q, \neg p, q \rightarrow r\}$, give a step-by-step formal proof of r . (Hint: use disjunctive syllogism to get q first, then modus ponens.)

Challenge

Exercise 10. Find the error in the following “proof” that $p \rightarrow q$ is logically equivalent to $q \rightarrow p$:

“Consider any assignment. If p and q are both true, then both $p \rightarrow q$ and $q \rightarrow p$ are true. If both are false, then both implications are true (since $F \rightarrow F = T$). So they always have the same truth value, and are logically equivalent.”

What case does this argument miss, and what is a specific counterexample?

Exercise 11. Show that excluded middle and double-negation elimination are equivalent as proof rules. That is, assuming you already have the other introduction and elimination rules for $\wedge, \vee, \rightarrow, \neg, T, F$:

- (a) Show that from $p \vee \neg p$ you can derive $\neg\neg p \rightarrow p$. (Hint: start a subproof assuming $\neg\neg p$, then use $\vee$ -elimination on $p \vee \neg p$.)
- (b) Show that from $\neg\neg p \rightarrow p$ you can derive $p \vee \neg p$. (This is the harder direction. Hint: it suffices to derive $\neg\neg(p \vee \neg p)$, which is constructively provable. Start by assuming $\neg(p \vee \neg p)$ and deriving a contradiction.)

Conclusion: adding either rule to constructive logic gives you the same classical logic.

Exercise 12. A formula in n variables has a truth table with 2^n rows. Using the DNF recipe from this module, show that every boolean function on n variables has a DNF formula with at most 2^n disjuncts. When does it have exactly 2^n ? When does it have zero?

Connection

Exercise 13. (Programming.) Write a Python function `dnf(f, n)` that takes a boolean function `f` of n arguments and returns a string representing a DNF formula that implements `f`. For example, `dnf(lambda x, y: x != y, 2)` should return something like `"(x and not y) or (not x and y)"` (XOR in DNF form). Use `itertools.product` to enumerate all input rows, as in the previous module. Apply your function to the three-input “exactly-one” function from Module 2 Exercise 9 and verify you get a three-term DNF.

Exercise 14. (Intermezzo pointer.) The intermezzo “Inference Rules as Trees” (Gentzen) presents the rules of this module as proof *trees* rather than numbered lines. As a preview: take your numbered proof from Exercise 8 and draw it as a tree, where each node is a derived formula and its children are the hypotheses that justified it by some rule. (You can use ASCII art or sketch it on paper.) What does the subproof structure of Exercise 8 correspond to in tree form?

Intermezzo: Inference Rules as Trees

3.12 A better notation

In Module 3 we wrote our inference rules in a semi-formal style that looked like little programs—premises on numbered lines, a conclusion at the bottom. That format is fine for getting started, and it has the nice property that it looks like code. But there’s a standard notation used in logic textbooks, type theory, and programming language theory that’s compact, composable, and worth learning: *Gentzen-style* inference rules, sometimes called “natural deduction” style after the proof system Gerhard Gentzen introduced in the 1930s.⁴

The idea is simple. You draw a horizontal line. Above the line go the *premises*—the things you need to know. Below the line goes the *conclusion*—the thing you get to conclude. Off to the right of the line you write the name of the rule. That’s it.

3.13 The rules, redux

Let’s revisit the rules from Module 3, but now in this new notation. We’ll use the `bussproofs` package, which is the standard L^AT_EX tool for typesetting natural-deduction proof trees: each rule is drawn with its premises on top, a horizontal line, and the conclusion underneath, with the rule name attached to the right edge.

3.13.1 Conjunction

Introduction: If you know p and you know q , you can conclude $p \wedge q$:

$$\frac{p \quad q}{p \wedge q} \wedge I$$

This is the same rule as the numbered-line version from Module 3—premises (a) p and (b) q , conclusion $p \wedge q$ —just written more compactly.

Elimination: If you know $p \wedge q$, you can pull out either piece:

$$\frac{p \wedge q}{p} \wedge E_1 \qquad \frac{p \wedge q}{q} \wedge E_2$$

3.13.2 Disjunction

Introduction: If you know one disjunct, you can conclude the disjunction:

$$\frac{p}{p \vee q} \vee I_1 \qquad \frac{q}{p \vee q} \vee I_2$$

⁴Gentzen was a remarkable logician who also invented the sequent calculus, proved the consistency of Peano arithmetic, and basically invented modern proof theory. All before the age of 30.

Notice that you get to “make up” the other disjunct—if you know p , you can conclude $p \vee q$ for *any* q . Remember from Module 3 that $p \rightarrow (p \vee q)$ really is a tautology.

Elimination (proof by cases):

$$\frac{p \vee q \quad p \rightarrow r \quad q \rightarrow r}{r} \vee E$$

This is the rule we worked through with the eight-row truth table. If you know $p \vee q$ and you can get to r from either side, you’ve got r .

3.13.3 Implication

Elimination (modus ponens):

$$\frac{p \quad p \rightarrow q}{q} \rightarrow E$$

Our old friend. If you know p and you know $p \rightarrow q$, you get q .

Modus tollens:

$$\frac{p \rightarrow q \quad \neg q}{\neg p} MT$$

If you know $p \rightarrow q$ and you know $\neg q$, you can conclude $\neg p$. This is the “backdoor” version of modus ponens: instead of affirming the antecedent, you deny the consequent. Unlike the contrapositive *technique*, modus tollens as a single inference step is constructively valid.

Introduction: This is the rule that was “a little weird” in Module 3, and the tree notation actually makes it *clearer*. The idea is: if you can derive q while *temporarily assuming* p , then you can discharge that assumption and conclude $p \rightarrow q$. We write the discharged assumption in square brackets, with an ellipsis indicating a subderivation:

$$\frac{\begin{array}{c} [p] \\ \vdots \\ q \end{array}}{p \rightarrow q} \rightarrow I$$

The $[p]$ means “ p was assumed for the sake of this derivation, and now we’re done with it.” Once you apply $\rightarrow I$, the assumption p is *discharged*: it’s no longer something you’re assuming, because you’ve packaged it up inside the implication.

This is exactly the same as the two-layer structure from Module 3 where we had “has a proof that assuming p , shows q ”—the bracket notation is just the standard way of writing that in tree form.

3.13.4 Negation

Introduction: If assuming p lets you derive a contradiction ($\perp$), then you can discharge that assumption and conclude $\neg p$:

$$\frac{\begin{array}{c} [p] \\ \vdots \\ \perp \end{array}}{\neg p} \neg I$$

This works exactly like $\rightarrow$ I: the brackets around p mean that p was a temporary assumption, and it gets discharged when you apply the rule. In fact, in many formulations of logic $\neg p$ is simply *defined* as $p \rightarrow \perp$, so $\neg$ I is literally a special case of $\rightarrow$ I.

Elimination (double negation):

$$\frac{\neg\neg p}{p} \neg E$$

If you know $\neg\neg p$, you can conclude p . Note that this rule requires the law of excluded middle—it is not valid in constructive logic. In a proof assistant like Agda or Coq, you would need to explicitly invoke a classical axiom to use it. The direction $p \rightarrow \neg\neg p$ is constructively valid, but eliminating the double negation is the part that requires classical reasoning (see the discussion in Module 3).

3.13.5 Constants

For completeness:

$$\frac{}{\top} \top I \qquad \frac{}{\perp} \perp E$$

$\top$ (pronounced “top” or “truth”) can always be concluded—no premises needed, hence the empty space above the line. It’s the same constant truth as T from Module 2, just in standard natural deduction notation. Dually, we write $\perp$ (pronounced “bottom” or “absurdity”) for the constant false, corresponding to F from Module 2. From $\perp$ you can conclude literally anything, which is the “ex falso” principle.

3.14 Composing rules into proof trees

Here’s where this notation really shines. Because the conclusion of one rule is just a formula, you can plug it in as the premise of another rule. The result is a *tree* that grows upward from the final conclusion at the root.

Let’s redo the transitivity-of-implication proof from Module 3. We want to show:

Given $p \rightarrow q$ and $q \rightarrow r$, prove $p \rightarrow r$.

Here’s the tree:

$$\frac{\frac{[p] \quad p \rightarrow q}{q} \rightarrow E \quad q \rightarrow r}{\frac{r}{p \rightarrow r} \rightarrow I} \rightarrow E$$

Read it from the bottom up: we want $p \rightarrow r$, so we use $\rightarrow$ I, which means we need to derive r from the assumption $[p]$. To get r we use modus ponens with $q \rightarrow r$, but we need q . To get q we use modus ponens with $[p]$ and $p \rightarrow q$. Done.

The assumptions $p \rightarrow q$ and $q \rightarrow r$ are *undischarged*—they sit at the leaves of the tree as open assumptions. The assumption $[p]$ is discharged by the $\rightarrow$ I at the root.

Compare this to the numbered-line version:

```
(a) p -> q, by assumption
(b) q -> r, by assumption
(c) proof that assuming p, shows r
    subproof: (i) p, by assumption
               (j) q, by modus ponens with (i) and (a)
               show r, by modus ponens with (j) and (b)
shows p -> r by introduction of implication
```

Same proof, same logic — just a different way to lay it out on the page. The tree version makes the structure—which things depend on which—immediately visible.

3.15 Reading proof trees

You can read these trees in two directions, and both are useful.

Bottom-up (goal-directed): start at the conclusion and ask “what rule could produce this?” That tells you what you need to prove next, and you work your way up to the leaves. This is the way you typically *construct* a proof—you start with what you want and work backwards until everything is justified.

Top-down (forward reasoning): start at the leaves (your assumptions) and follow the rules downward, seeing what you can derive. This is the way you typically *verify* a proof—you check that each step follows from the ones above it.

This dual reading mirrors a distinction you’ll keep encountering: the difference between “working backwards from what you want to prove” and “working forwards from what you know.” If you’ve ever debugged a program by starting from the error message and tracing back through the code, you were doing goal-directed reasoning. If you’ve ever traced through a program with a debugger, stepping forward line by line, you were doing forward reasoning. Both are useful; neither is always better.

3.16 Why this matters

You will see Gentzen-style rules *everywhere* in CS 251 and beyond. Typing rules for a programming language are written this way. Operational semantics are written this way. The rules of a type checker, a proof assistant, a static analyzer—all trees, all the time. It’s also the standard notation in logic textbooks beyond the intro level, so if you ever crack open a graduate-level logic text, you’ll be glad you’ve already met the format. Learning this notation now pays back the small investment many times over.

Think of this intermezzo as a Rosetta Stone: you now know two ways to write the same rules and can translate between them.

3.17 Exercises

Exercise 1. Build a Gentzen-style proof tree for the tautology

$$A \wedge (B \wedge C) \rightarrow (A \wedge B) \wedge C$$

(associativity of conjunction in one direction). Indicate which assumption is discharged by the final $\rightarrow$ I and what each intermediate step is.

Exercise 2. Build a Gentzen-style proof tree for transitivity of implication:

$$(A \rightarrow B) \rightarrow ((B \rightarrow C) \rightarrow (A \rightarrow C)).$$

You will need three nested $\rightarrow$ I rules at the bottom of the tree (one for each implication you need to introduce). Carefully bracket each discharged assumption.

Exercise 3. Build a Gentzen-style proof tree for the contrapositive direction

$$(p \rightarrow q) \rightarrow (\neg q \rightarrow \neg p).$$

Recall that $\neg p$ can be treated as $p \rightarrow \perp$, so $\neg I$ is a special case of $\rightarrow I$. Identify each discharged assumption.

Exercise 4. Build a Gentzen-style proof tree for $p \vee p \rightarrow p$ using the disjunction-elimination rule $\vee E$. (Hint: assume $p \vee p$, then use $\vee E$ with the two “branches” $p \rightarrow p$, both of which are trivially provable by $\rightarrow I$ on a trivial subderivation.)

Exercise 5. Take any proof from Module 3 written in the numbered-line format and translate it into a Gentzen tree. Then translate it back. Convince yourself that the two formats are interchangeable but that the tree version makes the dependency structure between steps more visible.

Further reading

If the tree-style notation appeals to you and you want to see where it leads, the following are accessible next steps.

- Frank Pfenning, *Natural Deduction* (lecture notes for CMU 15-317, “Constructive Logic”). The single best gentle introduction to natural deduction in tree form, with exactly the discipline of introduction and elimination rules used here.
- Jan von Plato, “Gentzen’s Proof Systems: Byproducts in a Work of Genius,” *Bulletin of Symbolic Logic* **18**(3), 313–367 (2012). A historical and conceptual essay on how Gentzen actually invented natural deduction and the sequent calculus, written for a logic-curious but non-specialist audience.
- Dag Prawitz, *Natural Deduction: A Proof-Theoretical Study*, Almqvist & Wiksell (1965); Dover reprint (2006). The first chapter is the canonical exposition of normalization—“every detour can be removed”—which is the proof-theory version of evaluating a program. Stop at the end of Chapter I; later chapters are more technical.

Cumulative Review 1

The point of this review

You’ve now seen three modules’ worth of material: sets, functions, and proof-writing basics (Module 1); propositional logic with truth-table semantics (Module 2); and proof systems with natural-deduction-style rules (Module 3). In this short review chapter we pick one small but non-trivial result and prove it *three different ways*, each way using a different module’s toolkit. The point is to see explicitly how the three modules connect—how the same mathematical fact can be established semantically (via truth tables), syntactically (via inference rules), or by direct set-theoretic manipulation—and to consolidate the vocabulary before you move on to predicate logic in Module 4.

The result

We’ll prove the following tautology of propositional logic:

$$(p \rightarrow q) \wedge (p \rightarrow r) \equiv p \rightarrow (q \wedge r).$$

In English: “ p implies both q and r ” is equivalent to “ p implies the conjunction $q \wedge r$.” This is a small but useful identity that lets you factor out a common hypothesis across two implications. It also shows up in the proof of every structured induction where you need to prove two things from the same assumption.

Proof 1: By truth table (Module 2 style)

Both sides are formulas over the three variables p, q, r . Logical equivalence is defined as “same truth table,” so we build truth tables for both sides and compare. We need eight rows, one for each assignment to (p, q, r) .

p	q	r	$p \rightarrow q$	$p \rightarrow r$	$(p \rightarrow q) \wedge (p \rightarrow r)$	$q \wedge r$	$p \rightarrow (q \wedge r)$
T	T	T	T	T	T	T	T
T	T	F	T	F	F	F	F
T	F	T	F	T	F	F	F
T	F	F	F	F	F	F	F
F	T	T	T	T	T	T	T
F	T	F	T	T	T	F	T
F	F	T	T	T	T	F	T
F	F	F	T	T	T	F	T

Columns 6 and 8 (the two full sides of the equivalence) match on every row, so the two formulas are logically equivalent. $\square$

This proof is mechanical and leaves no doubt, but it also gives you no *insight* into why the equivalence holds. If someone asked you to explain the result, you'd say "it worked out in every row," which isn't very satisfying.

Proof 2: By formal derivation (Module 3 style)

The same result, established syntactically using the introduction and elimination rules from Module 3. Because this is an equivalence, we need to prove both directions. We'll write the proofs in the numbered-line format from Module 3; you could also do them as Gentzen trees if you prefer.

Direction 1: $(p \rightarrow q) \wedge (p \rightarrow r) \rightarrow p \rightarrow (q \wedge r)$

Assume $(p \rightarrow q) \wedge (p \rightarrow r)$.

1. $(p \rightarrow q) \wedge (p \rightarrow r)$ (assumption)
2. $p \rightarrow q$ ($\wedge$ -elimination on (1))
3. $p \rightarrow r$ ($\wedge$ -elimination on (1))
4. Subproof: assuming p , show $q \wedge r$:
 - (4a) p (assumption)
 - (4b) q (modus ponens on (4a), (2))
 - (4c) r (modus ponens on (4a), (3))
 - (4d) $q \wedge r$ ($\wedge$ -introduction on (4b), (4c))
5. $p \rightarrow (q \wedge r)$ ($\rightarrow$ -introduction on subproof (4))

Direction 2: $p \rightarrow (q \wedge r) \rightarrow (p \rightarrow q) \wedge (p \rightarrow r)$

Assume $p \rightarrow (q \wedge r)$.

1. $p \rightarrow (q \wedge r)$ (assumption)
2. Subproof: assuming p , show q :
 - (2a) p (assumption)
 - (2b) $q \wedge r$ (modus ponens on (2a), (1))
 - (2c) q ($\wedge$ -elimination on (2b))
3. $p \rightarrow q$ ($\rightarrow$ -introduction on subproof (2))
4. Subproof: assuming p , show r :
 - (4a) p (assumption)
 - (4b) $q \wedge r$ (modus ponens on (4a), (1))
 - (4c) r ($\wedge$ -elimination on (4b))

5. $p \rightarrow r$ ($\rightarrow$ -introduction on subproof (4))
6. $(p \rightarrow q) \wedge (p \rightarrow r)$ ($\wedge$ -introduction on (3), (5))

Since both directions are derivable, the formulas are equivalent in the proof system. $\square$

This proof is more verbose than the truth-table version, but it tells you *why* the equivalence holds: you can turn a proof of $(p \rightarrow q) \wedge (p \rightarrow r)$ into a proof of $p \rightarrow (q \wedge r)$ (and back again) by rearranging the same basic steps. In the Curry–Howard view, both formulas correspond to the same “function of type p returning a pair (q, r) ,” which is a genuine computational content that the truth-table proof doesn’t touch.

Proof 3: By sets (Module 1 style)

Here’s a third angle—the one you might not have expected. A predicate on a set is just a subset (the truth set), and propositional equivalences become set identities under this reading. Let’s reinterpret the whole claim in set-theoretic language.

Fix any set U and think of p, q, r as subsets of U , representing the truth sets of predicates. The connective $p \rightarrow q$ is the set of elements where “if p then q ” holds—which is every element where p is false *or* q is true, i.e., $\bar{p} \cup q$. Similarly $q \wedge r$ becomes $q \cap r$. The equivalence we want to prove becomes the set identity

$$(\bar{p} \cup q) \cap (\bar{p} \cup r) = \bar{p} \cup (q \cap r).$$

This is just *distributivity of union over intersection*, a basic set identity from Module 1! Let’s verify it in both directions.

($\subseteq$). Let $x \in (\bar{p} \cup q) \cap (\bar{p} \cup r)$. Then x is in both $\bar{p} \cup q$ and $\bar{p} \cup r$. Case on whether $x \in \bar{p}$:

- If $x \in \bar{p}$, then $x \in \bar{p} \cup (q \cap r)$ by definition of union.
- If $x \notin \bar{p}$, then (since $x \in \bar{p} \cup q$) we must have $x \in q$; and (since $x \in \bar{p} \cup r$) we must have $x \in r$. So $x \in q \cap r$, hence $x \in \bar{p} \cup (q \cap r)$.

Either way, x is in the right-hand side.

($\supseteq$). Let $x \in \bar{p} \cup (q \cap r)$. Then either $x \in \bar{p}$ or $x \in q \cap r$.

- If $x \in \bar{p}$, then $x \in \bar{p} \cup q$ and $x \in \bar{p} \cup r$, so x is in their intersection.
- If $x \in q \cap r$, then $x \in q$ and $x \in r$, so again $x \in \bar{p} \cup q$ and $x \in \bar{p} \cup r$, and x is in their intersection.

Both directions hold, so the set identity is established. Translating back to propositional logic: $(p \rightarrow q) \wedge (p \rightarrow r) \equiv p \rightarrow (q \wedge r)$. $\square$

What we learned

Three proofs, three very different flavors:

- The **truth-table proof** is a finite, mechanical check. It’s the fastest to write but gives no insight into “why.” This is the view from Module 2.
- The **formal derivation** follows the connective structure of the formulas, using one introduction or elimination rule at each step. It’s longer but it *explains* the equivalence as a rearrangement of basic proof steps. This is the view from Module 3, and it’s the one closest to how a proof assistant like Lean would actually check the fact.

- The **set-theoretic proof** reinterprets the problem entirely, turning propositional equivalence into a set-identity question. It's a reminder that the tools from Module 1 aren't just warm-ups—they're alternative vantage points for everything that follows.

The fact that all three proofs exist isn't a coincidence. One of the deepest results about propositional logic—the *completeness theorem*—says that a formula is a semantic tautology (truth-table true) if and only if it has a syntactic derivation (provable from the inference rules). And the set-theoretic reading is what you get when you interpret propositional formulas in the “Boolean algebra of subsets” of a fixed universe—a genuine, well-defined algebraic structure.

If you only remember one thing from this review chapter, make it this: *the same fact can wear many different hats*. When you get stuck proving something one way, try translating the problem into a different language. The fact hasn't moved; only your perspective has.

Chapter 4

First-Order Predicate Logic

Reading map

Epp sections. §3.1–3.4 (predicates, quantified statements, and arguments with quantified statements) and §5.2 (the principle of mathematical induction; this module introduces induction, and Module 8 comes back to it in a bigger way).

Prerequisites. Modules 1–3 (sets and functions, propositional logic, proof systems).

Relevant intermezzi. “More on Natural Induction” (immediately after this module, walks through commutativity and associativity of $+$ inductively) and “Quantifiers as Games” (gives a game-semantic intuition for nested $\forall/\exists$).

Practice from Epp. §3.1 exercises on translating English into symbolic statements; §3.2 exercises on negating quantified statements; §3.3–3.4 exercises on validity in predicate logic; §5.2 exercises on simple induction.

4.1 A logic with real types

Propositional logic got us impressively far for something that only talks about *true* and *false*. But real mathematics and real programming don’t live in a world of just two values—they live in worlds with numbers, lists, users, chickens, whatever data type you’re working with. To say anything useful about those worlds, we need a logic that can talk about the elements *inside* them.

That’s first-order predicate logic, and at a glance it looks *a lot* like propositional logic: $\rightarrow$, $\wedge$, $\vee$, $\neg$, T , and F are all still here, our old friends. What’s new are two quantifiers, $\forall$ and $\exists$, together with a notion of *predicate*. Those give us three qualitatively different kinds of claim we can make about a set A :

- A claim about a *specific* element (e.g. “zero is the identity of addition”).
- A claim about *all* elements of A , written $\forall x : A. \dots$
- A claim that *some* element of A has a property, without naming a particular witness, written $\exists x : A. \dots$

The predicates are what let us make claims about elements in the first place. With the tools we’ve seen so far, we have no way to define predicates *inside* the logic—so we’ll do the next best thing and add them externally, as functions. A predicate on the type/set A is literally a function

$A \rightarrow \text{Bool}$, where $\text{Bool} = \{\text{True}, \text{False}\}$. “ n is even,” read as a predicate on $\mathbb{N}$, is just the function that returns True on 0, 2, 4, ... and False on 1, 3, 5, ... One predicate shows up so often it deserves its own treatment: equality. On any set A , equality is the predicate that returns True on the “diagonal” (when both arguments are the same element) and False everywhere else. We’ll give equality its own proof rule later in the module, because unlike a random user-defined predicate it carries a lot of structure we can exploit.

4.2 Forall and exists

We’ve seen $\forall$ and $\exists$ used informally a few times already. As we discussed above, they *quantify* what elements of a set/type¹ a proposition is covering—which is exactly why they’re called “quantifiers.” Let’s nail down what they mean. There are two complementary answers, and it’s worth understanding both.

Answer 1: inverse images. “ $\forall x : A. P(x)$ ” is true exactly when the function $P : A \rightarrow \text{Bool}$ sends every element of A to True—equivalently, when $P^{-1}(\text{True}) = A$. Dually, “ $\exists x : A. P(x)$ ” is true exactly when P sends at least one element of A to True—equivalently, when $P^{-1}(\text{True})$ is non-empty.

Answer 2: Big And / Big Or. When A is *finite*—so we can enumerate it as $\{x_1, x_2, \dots, x_k\}$ —we can also define the quantifiers by expansion into propositional connectives:

$$\begin{aligned}\forall x : A. P(x) &\equiv P(x_1) \wedge P(x_2) \wedge \dots \wedge P(x_k), \\ \exists x : A. P(x) &\equiv P(x_1) \vee P(x_2) \vee \dots \vee P(x_k).\end{aligned}$$

So $\forall$ is a “big conjunction” and $\exists$ is a “big disjunction.” Play with these transformations on small finite sets to convince yourself the two answers agree.²

Sidebar: you’ve already been writing quantifiers.

If you’ve written any Python you’ve used quantifiers without calling them that. The builtin `all(P(x) for x in A)` is literally $\forall x \in A. P(x)$, and `any(P(x) for x in A)` is $\exists x \in A. P(x)$. They’re the runtime versions of the same logical ideas: given a finite iterable, `all` walks through checking that the predicate holds everywhere and `any` walks through looking for a single witness. Haskell uses exactly those names too, and Ruby spells them `.all?` and `.any?`.

SQL’s `WHERE EXISTS` clause is an $\exists$ in disguise, checking whether a subquery returns at least one row. A list comprehension’s filter predicate builds the truth set—the set of elements satisfying the predicate—and a generator expression is the lazy version of the same thing. Even something as mundane as “all tests passed” on CI is a $\forall$ over the test suite.

What the logic adds, on top of these runtime operators, is the ability to talk about *infinite* sets—where you can’t just walk the iterable. The same claim you’d write as `all(is_even(n) for n in range(10))` in Python, you can now state as “ $\forall n : \mathbb{N}. n$ is even”—except the second version is a statement we can *prove or disprove*, and it happens to be false, regardless of how long you let the Python version run. That’s the jump we’re making: from *checking* finite cases to *reasoning* about all of them at once.

¹I’m going to keep saying set/type because in some logics there is a notion of “types” that are more general than sets. For the purposes of this class, think of them as essentially the same thing.

²There’s a secret third way to define $\forall$ and $\exists$ in dependent type theory, a logical framework where $\forall$ becomes a kind of function definition and $\exists$ becomes a kind of pair, like a pair in Python.

4.2.1 Free and bound variables

When a quantifier introduces a variable, that variable is *bound*—it belongs to the quantifier the way a function parameter belongs to the function. In “ $\forall x : A. P(x)$ ” the variable x is bound by the $\forall$. A variable that isn’t captured by any quantifier is called *free*.

If you’re a programmer, you already know the distinction: bound variables are like loop variables or function parameters—they’re local to their scope and you can rename them without changing anything. Free variables are like globals. Just as `def f(x): return x + 1` and `def f(y): return y + 1` define the same function, the formulas $\forall x. P(x)$ and $\forall y. P(y)$ say exactly the same thing. The bound variable is a placeholder; its name doesn’t matter.

Consider a formula with both kinds: $\forall x. P(x, y)$. Here x is bound (it’s under the scope of $\forall x$) and y is free. This formula is *not* a complete statement—its truth depends on what y is. Think of it as a function with one argument still unresolved: the $\forall$ “closed over” x but y is still dangling. Only once you fix a value for y (or bind it with another quantifier) do you get something with a definite truth value.

Why does this matter? When we get to the proof rules later in this module, the $\forall$ -elimination rule *substitutes* a concrete term for the bound variable, and the $\forall$ -introduction rule requires that the variable you’re generalizing over is truly arbitrary—meaning it isn’t free anywhere it shouldn’t be. Keeping track of which variables are free and which are bound is what makes these rules safe.

4.3 Truth sets

This is a quick concept, but it comes up often enough to deserve a name.

Definition 4.3.1 (Truth set). Given a predicate P on a set A , the *truth set* of P is the set

$$\{x \in A \mid P(x) \text{ is true}\}$$

i.e. the set of all elements that make the predicate true.

If you were paying attention in the last section you’ll notice that this is just the inverse image $P^{-1}(\text{True})$ that we already talked about, but now it has a name. Think of it like a type alias in Python or Haskell—same thing, different label.³

Let’s do a couple of worked examples.

Example 1. Let $P(d) = “6/d \text{ is an integer}”$ and let our set be $\mathbb{Z}^+$ (the positive integers). So we’re asking: which positive integers divide 6 evenly? The truth set is $\{1, 2, 3, 6\}$ because those are exactly the positive divisors of 6.

Example 2. Let $P(x) = “1 \leq x^2 \leq 4”$ and let our set be $\mathbb{Z}$ (all integers). We need $x^2 \geq 1$, so $x \neq 0$, and $x^2 \leq 4$, so $x \in \{-2, -1, 0, 1, 2\}$. Combining these: the truth set is $\{-2, -1, 1, 2\}$.

Here’s how this connects back to quantifiers:

- $\forall x : A. P(x)$ is true **if and only if** the truth set of P equals A . Every element makes the predicate true, so the truth set is the whole set.
- $\exists x : A. P(x)$ is true **if and only if** the truth set of P is nonempty. At least one element makes it true, so there’s something in there.

So you can think of $\forall$ as asking “is the truth set everything?” and $\exists$ as asking “is the truth set nonempty?” which is a pretty clean way to think about it.

³Think of it as applying a filter: in Python, `truth_set = {x for x in A if P(x)}`.

4.4 Negating quantified statements

This is *crucial* and it comes up constantly—in proofs, in writing specifications, in checking whether your code is correct, &c. You need to be able to negate statements that have quantifiers in them.

The good news is that the rules are really clean and they’re basically just De Morgan’s laws but for quantifiers instead of $\wedge$ and $\vee$:

$$\begin{aligned}\neg(\forall x : A. P(x)) &\equiv \exists x : A. \neg P(x), \\ \neg(\exists x : A. P(x)) &\equiv \forall x : A. \neg P(x).\end{aligned}$$

Why do these make sense? Let’s think about it. “It’s *not* the case that everything satisfies P ” means “there’s something that *doesn’t* satisfy P .” And “it’s *not* the case that something satisfies P ” means “*nothing* satisfies P ,” which is the same as saying everything fails to satisfy P .

And hey, remember the Big And / Big Or interpretation from the last section? This is just De Morgan’s laws from Module 2 in disguise! Negating a big conjunction (which is what $\forall$ is) gives you a big disjunction (which is what $\exists$ is), and vice versa. The same principle applies at a larger scale.⁴

Example 1. Negate “every computer has a CPU.”

This is $\forall c : \text{Computers}. \text{HasCPU}(c)$. The negation is $\exists c : \text{Computers}. \neg \text{HasCPU}(c)$, which in English is: “there exists a computer that does not have a CPU.”

Example 2. Negate “there exists a band that has won at least 10 Grammys.”

This is $\exists b : \text{Bands}. \text{Grammys}(b) \geq 10$. The negation is $\forall b : \text{Bands}. \text{Grammys}(b) < 10$, which says: “for every band, that band has not won at least 10 Grammys”—or equivalently, “no band has won at least 10 Grammys.”

One more pattern that shows up all the time: negating an implication *inside* a quantifier. Recall from Module 2 that $\neg(P \rightarrow Q) \equiv P \wedge \neg Q$ —the negation of “if P then Q ” is “ P and not Q .” So when you negate a universally quantified implication you get:

$$\neg(\forall x. P(x) \rightarrow Q(x)) \equiv \exists x. P(x) \wedge \neg Q(x)$$

which reads as “there’s some x where $P(x)$ holds but $Q(x)$ doesn’t.” This is a really common pattern in counterexample-based reasoning: to disprove “all P ’s are Q ’s” you find a P that isn’t a Q .

4.5 Nested quantifiers

Sometimes one quantifier just isn’t enough and you need to stack them. This is where things get interesting.

Example. Consider the statement “there exists a real number u such that for all real numbers v , $uv = v$.” In our notation this is:

$$\exists u : \mathbb{R}. \forall v : \mathbb{R}. uv = v$$

This is claiming that there’s a multiplicative identity. Is it true? Yes! Take $u = 1$ and you get $1 \cdot v = v$ for all v . So we’ve *witnessed* the existential by providing a concrete value.⁵

Here’s what you really need to internalize: **order matters**. The statements $\forall x. \exists y. P(x, y)$ and $\exists y. \forall x. P(x, y)$ are *very different*.

Let’s look at a concrete case. Consider $P(x, y) = “y > x”$ over $\mathbb{R}$:

⁴If you squint at it in exactly the right way, the entirety of first-order logic is just propositional logic with extra steps. Which is kind of the point.

⁵This is exactly what the $\exists$ -introduction rule does: you hand it a concrete element and a proof that the predicate holds on that element.

- $\forall x : \mathbb{R}. \exists y : \mathbb{R}. y > x$ says “for every number there’s a bigger one.” This is **true**—just pick $y = x + 1$.
- $\exists y : \mathbb{R}. \forall x : \mathbb{R}. y > x$ says “there’s a number bigger than all numbers.” This is **false**—there’s no largest real number.⁶

If you’re a programmer, here’s a useful way to think about it. The statement $\forall x. \exists y. P(x, y)$ is like saying “for every input x , we can find an output y such that $P(x, y)$ holds.” In other words, a function exists that maps inputs to valid outputs—like writing `def f(x): return x + 1` in Python. The y you pick can *depend* on x .

But $\exists y. \forall x. P(x, y)$ is like saying “there’s a single y that works for ALL inputs.” You have to pick one constant value up front that satisfies every possible x . That’s way harder—like trying to write `y = ???` at the top of your program and having it work for every input. In C terms, it’s the difference between a function and a global constant.

Negating nested quantifiers is actually straightforward: you just flip each quantifier as you push the negation through, and then negate the predicate at the end. So:

$$\neg(\exists u : \mathbb{R}. \forall v : \mathbb{R}. uv = v) \equiv \forall u : \mathbb{R}. \exists v : \mathbb{R}. uv \neq v$$

Each $\exists$ becomes a $\forall$, each $\forall$ becomes an $\exists$, and the predicate gets negated. That’s it. You just walk through the quantifiers one at a time applying the rules from the previous section.

4.6 Necessary and sufficient conditions

This section is about translating some very common mathematical and CS jargon back into the logic we already know. The phrases “necessary condition” and “sufficient condition” show up *everywhere* in textbooks, specifications, API docs, &c. and you need to know what they mean.

Definition 4.6.1 (Necessary and sufficient conditions).

- “ P is *sufficient* for Q ” means $P \rightarrow Q$. If P happens, that’s *enough* to guarantee Q . P is sufficient—it suffices.
- “ P is *necessary* for Q ” means $Q \rightarrow P$, or equivalently $\neg P \rightarrow \neg Q$. You *can’t* have Q without P . P is a prerequisite.
- “ P is *necessary and sufficient* for Q ” means $P \leftrightarrow Q$, that is, $(P \rightarrow Q) \wedge (Q \rightarrow P)$. They’re logically equivalent—each one implies the other.

A common point of confusion: “ P is necessary for Q ” is $Q \rightarrow P$, **not** $P \rightarrow Q$. The necessary thing goes on the *right* of the arrow when the thing that needs it is on the left. Think of it as: Q requires P , so if you have Q you must have P . This trips people up more than almost anything else in the course, so take a moment to make sure it sits right.

Example (sufficient). “Being a square is sufficient for being a rectangle.” This means: if something is a square, then it’s a rectangle. Every square is a rectangle. That’s $\text{Square}(x) \rightarrow \text{Rectangle}(x)$.

Example (necessary). Real example from a spec: “Authentication is necessary for authorization.” This means: if you’re authorized, then you must be authenticated. You can’t have authorization without authentication. That’s $\text{Authorized}(x) \rightarrow \text{Authenticated}(x)$ —notice that the

⁶If you think ∞ is a real number, we need to have a talk.

necessary thing (authentication) ends up on the *right* of the arrow, and the thing it’s necessary *for* (authorization) ends up on the left. That’s the direction that trips people up.

The connection to everything else we’ve been doing is pretty direct: these are just different ways of talking about implication. The reason we bring this up now is that math texts and CS specs use this language *all the time* and if you don’t know how to translate “necessary” and “sufficient” into arrows, you’re going to have a bad time reading formal documentation.⁷

4.7 Proofs with this logic

Good news: proofs here look almost exactly like proofs in propositional logic. The entire Module 3 toolkit— $\rightarrow$ I, $\rightarrow$ E, $\wedge$ I, $\wedge$ E, $\vee$ I, $\vee$ E, and friends—carries over unchanged, and we just add five new rules to handle the new vocabulary: introduction and elimination for $\forall$, introduction and elimination for $\exists$, and a special rule for the equality predicate. We’ll start with the easiest, which is the rule for equality.

Reflexivity:

$$\frac{\text{assumes } x : A}{\text{shows } x = x}$$

which is just that you’re always allowed to conclude that a thing is equal to itself, unsurprisingly! If you’re also playing the natural number game or if you ever try other interactive theorem provers you probably will see this rule called “reflexivity,” or “rfl”/“refl” for short.

Next we present the rules for $\forall$. We start with elimination because it is the simpler of the two.

Elimination:

$$\frac{\begin{array}{c} \text{assumes } \forall x : A. P(x) \\ a : A \end{array}}{\text{shows } P(a)}$$

This says that if P is true of all elements of A and a is an element of A , then $P(a)$ holds. It’s a lot like applying a function, isn’t it?

Introduction: The introduction rule is going to look a little like the introduction rule for implication.

$$\frac{\text{assumes } \text{a proof of: assuming } a : A, \text{ shows } P(a)}{\text{shows } \forall x : A. P(x)}$$

Side condition: the variable a must really be *arbitrary*. In practice that means a should be fresh for the subproof and should not appear free in any undischarged assumption that the subproof depends on. Otherwise you’d be proving $P(a)$ for some *special* a and then incorrectly generalizing to all elements of A .

What we’re doing is taking evidence that given an *arbitrary* element of A we can always show that $P(a)$ is true and internalizing that into the $\forall$, the same way that we internalize the ability to prove Q assuming P into $P \rightarrow Q$.

The rules for $\exists$ are the mirror image of $\forall$ ’s.

Elimination:

$$\frac{\begin{array}{c} \text{assumes } \exists x : A. P(x) \\ \text{a subproof: assuming } c : A \text{ and } P(c), \text{ shows } Q \end{array}}{\text{shows } Q}$$

⁷The full version of the spec line above is “Authentication is necessary but not sufficient for authorization.” The “not sufficient” half adds $\text{Authenticated}(x) \not\Rightarrow \text{Authorized}(x)$ —being logged in doesn’t mean you have permission to do what you’re trying to do.

Side condition: the witness variable c must be *fresh*. In particular, it must not appear free in Q or in any undischarged assumption outside the subproof. Otherwise the particular name chosen for the witness could leak out into the conclusion, which would make the rule unsound.

This rule says: if you know something satisfying P exists, and you can show that Q follows from having *any* element c satisfying $P(c)$ —without relying on which specific c it is—then you can conclude Q . Think of it as opening a box: you know something is inside, you don’t know exactly what, but you show that no matter what it turns out to be, the conclusion holds. This is why the subproof introduces a *fresh* variable c —it represents an arbitrary witness, not a specific one.

Introduction:

$$\frac{\text{assumes } c : A \quad P(c)}{\text{shows } \exists x : A. P(x)}$$

4.8 Induction, round 1

Here we’re going to introduce the natural numbers and the idea of natural induction together, because the two ideas shape each other. Why? We’ve shown how to prove things about predicates in general, but if I just *hand* you a predicate on a set you wouldn’t know where to start—you don’t have any handles on the proof beyond brute-forcing the set-based semantics. And as we’ve discussed a couple of times now, brute force won’t work for infinite sets where you’d need to check the formula against infinitely many values. Induction gives us those handles, by defining types in a way that has structure to it.

Remember BNF syntax from Module 2, where we recursively defined the set of propositional logic formulas as

$$L := L \wedge L \mid L \vee L \mid L \rightarrow L \mid \sim L \mid T \mid F \mid x$$

We can define the natural numbers the same way:

$$\text{Nat} := 0 \mid \text{succ Nat}$$

Every natural number is either 0 or the successor of another natural number—so checking a predicate on infinitely many values reduces to just *two cases*. That’s what induction buys us.

If you’re playing the Natural Number Game you’ll have already seen the induction rule that falls out of this definition, but let’s spell it out:

Natural induction:

$$\frac{\text{assumes } P(0) \quad \text{a subproof: assuming } P(n), \text{ shows } P(\text{succ } n)}{\text{shows } \forall n : \text{Nat}. P(n)}$$

What this is saying is that if you can show that P is true on 0 and if you can show that with evidence that $P(n)$ is true you can prove $P(\text{succ } n)$, then you can prove that P is true on all natural numbers.

Note how the base case is the same as the base case of the recursion and that the second part follows the recursive structure of the BNF syntax.

4.8.1 Induction in action: a worked example

The rule is doing a lot of work in not very much space, so let’s actually *use* it on something. We need a predicate to prove and that means we need at least one operation on the natural numbers to talk about. So we’ll define addition.

Addition on the naturals can be defined—like everything else around here—recursively. We do it by recursing on the second argument:

$$\begin{aligned} n + 0 &= n \\ n + (\text{succ } m) &= \text{succ } (n + m) \end{aligned}$$

Read this as “adding zero to anything gives you the thing back, and adding the successor of m to n is the same thing as adding m to n and then taking the successor.” If you stare at it long enough, this is really just saying “adding $m + 1$ is the same as adding m and then adding 1.”

Now notice something. The equation $n + 0 = n$ is true *for free*—it’s literally how we defined what $+ 0$ means. We don’t need induction or any clever argument here, it just falls out of unfolding the definition.

What about the *other* direction, though: is $0 + n = n$? You might think this is also free by symmetry, but it isn’t! Our definition of addition recurses on the *second* argument, which means the case where 0 sits on the *left* is not directly handled by the definition. We have to actually *prove* this one, and induction is the tool.

Theorem. $\forall n : \text{Nat}. 0 + n = n.$

Proof. By induction on n .

Base case ($n = 0$). We need to show $0 + 0 = 0$. But this falls out of the definition of $+ 0$ —adding zero hands you back what you started with.

Inductive step ($n = \text{succ } n'$). The inductive hypothesis tells us that $0 + n' = n'$. We need to show $0 + (\text{succ } n') = \text{succ } n'$. So we just chase the definition:

$$\begin{aligned} 0 + (\text{succ } n') &= \text{succ } (0 + n') && \text{by the second equation of } + \\ &= \text{succ } n' && \text{by the inductive hypothesis} \end{aligned}$$

which is exactly what we wanted. □

That’s it. That is what an inductive proof on the natural numbers actually looks like. The skeleton was: state the predicate, handle the case where n is 0, then assume the predicate holds for some n' and use that assumption to prove it for $\text{succ } n'$. The base case and the inductive step *exactly* mirror the BNF for Nat —there are two cases in the definition and two cases in the proof, and they line up one-for-one. That’s not a coincidence, that’s the whole point.

Here’s a perspective that might help if you’re a programmer. An inductive proof is structured *exactly* like a recursive function. There’s a base case (when the argument is 0) and a recursive case (when the argument is $\text{succ } n'$, where you “call yourself” on n'). The thing being “returned” is a proof rather than a number, but the shape is the same. In fact, if you’ve ever written a recursive function and felt like you were “trusting” the recursive call to do the right thing, congratulations: you’ve already felt the inductive hypothesis from the inside.⁸

We’ve barely scratched the surface here. The intermezzo immediately after this chapter walks through several more proofs in this same style—commutativity and associativity of addition, among others—to get you really comfortable with the pattern. And much later, in Module 8, we’ll come back to induction in a bigger way once we have sequences and summation in hand. The point of this section was just to break the seal: now you’ve seen one inductive proof from end to end, and the rule isn’t just sitting there as a piece of inscrutable notation.

⁸“Assume the recursive call works” and “assume $P(n')$ ” really are the same move. We’re going to come back to this in a much bigger way in the Curry–Howard intermezzo, once we’ve built up the vocabulary to say it precisely.

4.9 Addendum on notation

Sometimes you'll see the $:A$ part left out of quantifiers—so “ $\forall x. P(x)$ ” instead of “ $\forall x : A. P(x)$.” This is valid shorthand *if* we've fixed a “domain of discourse” in advance. Then, and only then, can we drop the set annotation because there's only one set to quantify over. In this book we keep the annotations when the domain isn't obvious from context, but don't be surprised to see them dropped in other sources.

4.10 Common mistakes

Quantifier logic is where students stop making mistakes on truth tables and start making mistakes on *scope*, so the common mistakes here are qualitatively different from the propositional ones.

1. **Swapping $\forall$ and $\exists$ when negating.** Students often write $\neg(\forall x. P(x)) \equiv \forall x. \neg P(x)$, which is wrong. Each quantifier *flips* when you push a negation through it. The right answer is $\exists x. \neg P(x)$. Mnemonic: “not every” becomes “some don't.”
2. **Inverting “necessary” and “sufficient.”** “ P is necessary for Q ” means $Q \rightarrow P$, not $P \rightarrow Q$. Students confidently write the arrow the wrong way and it's one of the most common errors on exams. Keep saying it out loud: *necessary things go on the right of the arrow; sufficient things go on the left*.
3. **Treating $\forall\exists$ and $\exists\forall$ as the same.** They are not. $\forall x. \exists y. y > x$ is true over the reals, but $\exists y. \forall x. y > x$ is false. The order decides whether the witness y is allowed to depend on x or has to be a single fixed value that works uniformly. Whenever you see two or more quantifiers, read them left to right as nested function scopes: the outer one is free to vary as the inner one “looks at” it.
4. **Variable capture when renaming.** If you want to rename a bound variable, the new name must be *fresh*—not already appearing free in the body. Renaming $\forall x. \exists y. R(x, y)$ to $\forall y. \exists y. R(y, y)$ is not a “rename”; it's a nonsense formula, because the inner y now shadows the outer y .
5. **Forgetting to use the inductive hypothesis.** In an inductive proof, the whole point of the inductive step is that you *get* to assume $P(n)$ and use it. If your inductive step works without ever mentioning the inductive hypothesis, one of two things is true: either you didn't actually need induction (because the claim follows directly from unfolding definitions, as in Exercise 9 below) or your proof is wrong.
6. **Generalizing over a non-arbitrary variable.** The $\forall$ -introduction rule has a side condition: the variable you generalize over must be truly arbitrary (fresh in the surrounding assumptions). If you've been manipulating a specific element called a and then try to conclude “ $\forall x. P(x)$ ” from “ $P(a)$,” you need to have *not* used any special property of a in the derivation. Otherwise you'd be over-generalizing—proving something only about a but pretending it holds for all elements.

4.11 Summary and looking ahead

Key takeaways.

- First-order predicate logic adds two quantifiers ($\forall$, $\exists$) and *predicates* on top of propositional logic. Semantically, $\forall x : A. P(x)$ holds iff $P^{-1}(\text{True}) = A$; $\exists x : A. P(x)$ holds iff $P^{-1}(\text{True})$ is non-empty.
- Quantifiers *flip* under negation: $\neg\forall = \exists\neg$ and $\neg\exists = \forall\neg$. These are De Morgan’s laws for quantifiers.
- *Bound* variables are local to their quantifier; *free* variables are global. You can rename bound variables but only to fresh names (no capture).
- The order of nested quantifiers matters: $\forall x.\exists y. P(x, y)$ lets y depend on x , while $\exists y.\forall x. P(x, y)$ forces a single uniform y . These are genuinely different statements.
- *Natural induction* on $\mathbb{N}$ is a proof rule: base case $P(0)$, inductive step $P(n) \rightarrow P(\text{succ } n)$, conclude $\forall n. P(n)$. The shape of the rule is dictated by the BNF $\text{Nat} ::= 0 \mid \text{succ Nat}$.
- “ P is *necessary* for Q ” means $Q \rightarrow P$; “*sufficient*” means $P \rightarrow Q$. Memorize this; it’s a common exam trap.

What we built this module. We’ve finally got a logic that can talk about data, not just booleans. Predicates let us say things like “ n is even” or “ $x < y$ ” inside the logic, and quantifiers let us say “for all n ” or “for some x .” The proof rules for $\forall$ and $\exists$ are the same introduction/elimination shape as the propositional rules, so the Module 3 toolkit still applies. And at the very end, we snuck in natural induction—which isn’t just a proof technique, it’s the induction principle that falls out of the BNF definition of $\mathbb{N}$. That same pattern will reappear for every recursive datatype we meet.

Looking ahead. Modules 5 through 7 take this quantifier-and-predicate vocabulary and use it to write real, human-readable proofs: direct proof, counterexample, existence, proof by cases, proof by contradiction, proof by contrapositive. These are the five or six techniques that a mathematician or computer scientist reaches for daily. Module 8 comes back to induction in a bigger way, using it to prove facts about sequences and summations. Before Module 5, the intermezzi on “More on Natural Induction” (practice proofs) and “Quantifiers as Games” (a game-semantic reading of $\forall/\exists$) are both worth reading.

4.12 Exercises

Organized into **warm-up** (mechanical practice on quantifier manipulation), **standard** (typical assignment-style problems including negation and simple proofs), **challenge** (synthesis involving induction and more nuanced quantifier reasoning), and **connection** (programming and intermezzo pointers). Solutions are in the appendix.

Warm-up

Exercise 1. In the formula $\exists x. \forall y. P(x, y, z)$, identify which variables are free and which are bound. Explain how you determined the status of each variable. Then do the same for $\forall x. (P(x) \rightarrow \exists y. Q(x, y))$.

Exercise 2. Translate each of the following English statements into symbolic form, using the

indicated predicates over the indicated domains:

- (a) “Every cat is either black or white.” (Domain: Cats; predicates: Black, White.)
- (b) “Some student in CS 250 has solved every homework problem.” (Domains: Students, Problems; predicate: Solved(s, p).)
- (c) “No real number is larger than every integer.” (Domains: $\mathbb{R}, \mathbb{Z}$.)
- (d) “Every non-zero real number has a multiplicative inverse.” (Domain: $\mathbb{R}$.)

Exercise 3. For each predicate, write its truth set.

- (a) $P(n)$ = “ n is a positive divisor of 12” over $\mathbb{Z}^+$.
- (b) $P(x)$ = “ $x^2 \leq 9$ ” over $\mathbb{Z}$.
- (c) $P(x)$ = “ $x < x + 1$ ” over $\mathbb{Z}$.

Exercise 4. For each pair, decide whether the first condition is **necessary**, **sufficient**, **both**, or **neither** for the second.

- (a) Being divisible by 4; being divisible by 2.
- (b) Having a valid ticket; being allowed to board the plane.
- (c) Being an equilateral triangle; being a triangle.
- (d) Being a prime; being odd.

Standard

Exercise 5. Negate the following statement and simplify: “For every student s in CS 250, there exists a homework h such that s completed h on time.” Write the negation in both symbols and plain English.

Exercise 6. Let $P(x, y)$ mean “ $x + y = 0$ ” over $\mathbb{Z}$. Determine whether each of the following is true or false, and justify your answer:

- (a) $\forall x. \exists y. P(x, y)$
- (b) $\exists y. \forall x. P(x, y)$

Exercise 7. Negate and fully simplify: $\forall x : \mathbb{Z}. \exists y : \mathbb{Z}. \forall z : \mathbb{Z}. (P(x, y, z) \rightarrow Q(x, z))$. Your answer should push all negations to the innermost atomic predicates.

Exercise 8. Prove the equivalence

$$\forall x. (P(x) \wedge Q(x)) \equiv (\forall x. P(x)) \wedge (\forall x. Q(x))$$

by giving formal proofs in both directions. Then decide whether the analogous claim for $\vee$,

$$\forall x. (P(x) \vee Q(x)) \equiv (\forall x. P(x)) \vee (\forall x. Q(x)),$$

is also an equivalence. If it is, prove both directions. If it isn't, prove one direction and give a counterexample to the other.

Challenge

Exercise 9. Prove that for all $n : \text{Nat}$, $n + 1 = \text{succ } n$, where $1 = \text{succ } 0$. This one is a bit of a trick: figure out whether you actually need induction here, or whether you can do it just by unfolding the definition of $+$. (Hint: think about which equation of $+$ fires when the second argument is $\text{succ } 0$.)

Exercise 10. Take the natural induction rule above and look at it side by side with the BNF $\text{Nat} := 0 \mid \text{succ Nat}$. Identify exactly which part of the rule comes from which case of the BNF. Then, by analogy, write down what the induction rule would look like for the type

$\text{Counter} := \text{zero} \mid \text{tickA Counter} \mid \text{tickB Counter}$

You don’t have to prove anything with it—just write down the rule. This one is a preview of what we’ll do systematically in Module 10: each recursive case of the BNF contributes its own premise to the induction rule, so the **Counter** rule has *two* inductive-step premises (one for **tickA**, one for **tickB**) where natural induction had only one.⁹

Exercise 11. Define multiplication on Nat recursively on the second argument:

$$\begin{aligned} n \cdot 0 &= 0 \\ n \cdot (\text{succ } m) &= (n \cdot m) + n \end{aligned}$$

Prove by induction that $\forall n : \text{Nat}. 0 \cdot n = 0$. (This one *does* need induction—notice that the definition’s first equation gives you $n \cdot 0 = 0$ for free, but the claim $0 \cdot n$ has 0 on the *left*, which is the harder side. Compare this to the $0 + n = n$ proof in the module.)

Exercise 12. Give a concrete domain A and a concrete predicate R on $A \times A$ for which $\forall x. \exists y. R(x, y)$ is true but $\exists y. \forall x. R(x, y)$ is false. Then explain in one or two sentences what intuition this example captures about the difference in quantifier order.

Connection

Exercise 13. (Programming.) Write a Python function `forall_exists(P, A, B)` that takes a predicate P on two arguments and two finite iterables A, B , and returns `True` iff $\forall a \in A. \exists b \in B. P(a, b)$. Also write `exists_forall(P, A, B)` that checks $\exists b \in B. \forall a \in A. P(a, b)$. Test both on $A = B = \{0, 1, 2, 3\}$ with $P(a, b) = (a + b = 3)$, and verify that on this input the first is true but the second is false. Why? (Which b would work for *every* a ? Is there one?)

Exercise 14. (Intermezzo pointer.) The “Quantifiers as Games” intermezzo introduces a game-semantic reading of $\forall$ and $\exists$: $\forall$ is a move by a “universal” player who tries to falsify the formula, and $\exists$ is a move by an “existential” player who tries to verify it. Using this reading, explain in a paragraph why the game for $\forall x. \exists y. y > x$ over $\mathbb{R}$ is a win for the existential player, but the game for $\exists y. \forall x. y > x$ is a loss.

⁹If your rule has three premises, you’re on the right track. If it has two, you forgot a case. If it has four, you’ve invented something interesting and we should talk.

Intermezzo: More on Natural Induction

To illustrate natural induction, let's work through some proofs similar to those in The Natural Number Game, but informally.

For the purposes of these proofs we'll assume that the definition of addition by recursion on the second argument is

$$\begin{aligned}n + 0 &= n \\ n + (\text{succ } m) &= \text{succ } (n + m)\end{aligned}$$

4.13 Proof that $0 + m = m$

We have automatically from the definition of addition by recursion that $n + 0 = n$, but what about the other way? You might wonder why this needs a proof at all—isn't it obvious? The subtlety is that our definition of addition recurses on the *second* argument. The equation $n + 0 = n$ is the base case of that recursion, so it holds by definition. But $0 + m = m$ puts zero in the *first* argument, which is not the recursive position. The definition doesn't directly tell us what happens there, so we need induction.

Theorem: $\forall m. 0 + m = m$

Proof by induction on m .

Case $m = 0$: Need to prove that $0 + 0 = 0$, but this follows immediately by the definition of addition.

Case $m = \text{succ } m'$:

Assuming $0 + m' = m'$.

Shows $0 + \text{succ } m' = \text{succ } m'$.

Chasing the definition and then the induction hypothesis:

$$\begin{aligned}0 + \text{succ } m' &= \text{succ } (0 + m') && \text{by the definition of addition} \\ &= \text{succ } m' && \text{by the induction hypothesis that } 0 + m' = m'\end{aligned}$$

QED

4.14 Proof that $\text{succ } n + m = \text{succ } (n + m)$

Theorem: $\forall n m. \text{succ } n + m = \text{succ } (n + m)$

Proof by induction on m .

Case $m = 0$: $\text{succ } n + 0 = \text{succ } (n + 0)$.

Immediate from the definition of addition.

Case $m = \text{succ } m'$:

Assuming: $(\text{succ } n) + m' = \text{succ } (n + m')$.

Shows: $(\text{succ } n) + (\text{succ } m') = \text{succ } (n + (\text{succ } m'))$.

$$\begin{aligned} (\text{succ } n) + (\text{succ } m') &= \text{succ } (\text{succ } n + m') && \text{by the definition of addition} \\ &= \text{succ } (\text{succ } (n + m')) && \text{by the induction hypothesis} \end{aligned}$$

but the right hand side tells us

$$\text{succ } (n + (\text{succ } m')) = \text{succ } (\text{succ } (n + m')) \quad \text{by the definition of addition}$$

so we end up with

$$\text{succ } (\text{succ } (n + m')) = \text{succ } (\text{succ } (n + m'))$$

which is true by reflexivity.

QED

4.15 Proof that addition is associative

Theorem: $\forall n m r. n + (m + r) = (n + m) + r$

Proof by induction on n .

Case $n = 0$: Need to prove that $\forall m r. 0 + (m + r) = (0 + m) + r$.

Both sides reduce to $m + r$ by the left and right identity properties of 0 (proved in Section 1 and given by definition, respectively):

$$m + r = m + r$$

which is true by reflexivity.

Case $n = \text{succ } n'$: Need to prove that **assuming** $\forall m r. n' + (m + r) = (n' + m) + r$ we can derive $\text{succ } n' + (m + r) = (\text{succ } n' + m) + r$.

$$\text{succ } n' + (m + r) = \text{succ } (n' + (m + r)) \quad \text{by our theorem above}$$

and the right hand side similarly transforms

$$\begin{aligned} (\text{succ } n' + m) + r &= \text{succ } (n' + m) + r && \text{by our theorem above} \\ &= \text{succ } ((n' + m) + r) && \text{by the same theorem} \\ &= \text{succ } (n' + (m + r)) && \text{by the induction hypothesis} \end{aligned}$$

so we have

$$\text{succ } (n' + (m + r)) = \text{succ } (n' + (m + r))$$

which is true by reflexivity.

QED

4.16 Proof that addition is commutative

Theorem: $\forall n m. n + m = m + n$

Proof by induction on n .

Case $n = 0$: Need to prove that $\forall m. 0 + m = m + 0$.

This is immediately solved by the fact that 0 is the left and right identity for addition, so this simplifies to $m = m$ which is true by the definition of $=$ (i.e. by reflexivity).

Case $n = \text{succ } n'$: Need to prove that **assuming** $\forall m. n' + m = m + n'$ we can show $\forall m. (\text{succ } n') + m = m + (\text{succ } n')$.

So by the definition of addition we have

$$m + (\text{succ } n') = \text{succ } (m + n')$$

and by one of the above theorems we have that

$$(\text{succ } n') + m = \text{succ } (n' + m)$$

but

$$\text{succ } (n' + m) = \text{succ } (m + n') \quad \text{by the induction hypothesis}$$

and thus

$$\text{succ } (m + n') = \text{succ } (m + n')$$

which is true by reflexivity.

QED

4.17 Exercises

Exercise 1. Define multiplication on the natural numbers by recursion on the second argument:

$$n \cdot 0 = 0$$

$$n \cdot (\text{succ } m) = n + (n \cdot m)$$

Prove by induction on m that $\forall m. 0 \cdot m = 0$. (As with the addition proof, the right identity is automatic from the definition, and the left identity needs induction.)

Exercise 2. Using the multiplication defined in Exercise 1 and the addition lemmas proved in this chapter, prove by induction on m that $\forall m. 1 \cdot m = m$, where $1 = \text{succ } 0$. You will need to use $0 \cdot m = 0$ from the previous exercise.

Exercise 3. Prove by induction on n that $n + n$ is always even. (“Even” here means: there exists k such that $n + n = k + k$. The witness for the existential is, naturally, n itself—the interesting part is writing the induction step out cleanly using only the addition lemmas.)

Exercise 4. Define the predecessor function on natural numbers by

$$\text{pred } 0 = 0$$

$$\text{pred } (\text{succ } n) = n$$

Prove by induction on n that $\forall n. \text{pred } (\text{succ } n) = n$. (Easy.) Then explain in one sentence why the statement $\forall n. \text{succ } (\text{pred } n) = n$ is *not* provable: where does the induction break down?

Exercise 5. Reflect: in each of the four proofs in this chapter, identify exactly which step “does work” (i.e., uses a lemma that itself required induction) and which steps are pure unfolding of definitions. Why is it that none of these theorems can be proved by definition unfolding alone?

Further reading

For a deeper look at why induction works, where it comes from, and what it can and cannot do:

- Leon Henkin, “On Mathematical Induction,” *American Mathematical Monthly* **67**(4), 323–338 (1960). The classic expository article distinguishing the *principle* of induction from the *axiom*, and showing precisely which Peano axioms are needed to prove commutativity and associativity of addition—the same proofs we just walked through.
- George Pólya, *Induction and Analogy in Mathematics* (volume I of *Mathematics and Plausible Reasoning*), Princeton University Press (1954), particularly the chapter on mathematical induction. Pólya’s gentle, example-driven exposition is aimed at students discovering proof for the first time.
- Edmund Landau, *Foundations of Analysis*, Chelsea (1951; original German 1930), Chapter 1. Landau builds the natural numbers from the Peano axioms and proves commutativity and associativity of addition with full inductive rigor in about ten pages—you can literally check every step. Terse but transparent.

Intermezzo: Quantifiers as Games

In Module 4 we defined quantifiers two ways: as functions into `Bool`, and as Big And / Big Or over a set. Both of those descriptions are perfectly correct, but they’re also a bit static—they tell you what quantifiers *mean* but not what it *feels like* to work with them. There’s a third perspective that is more dynamic, more intuitive, and connects beautifully to both programming and to modern cryptography. It’s the idea that logic is a *game*.

4.18 The game interpretation

Imagine two players sitting across from each other. One is the **Prover** (that’s you—the person trying to show the statement is true). The other is the **Challenger** (think of them as Nature, or an Opponent, or a very skeptical friend). The logical formula being evaluated determines the rules of the game. Prover wins if, at the end of the game, the resulting atomic statement is true. Challenger wins if it’s false.

Here’s how each connective translates into a game move:

- $\exists x. P(x)$: **Prover’s move**. Prover picks a witness x , and then the game continues with $P(x)$. Prover wins if $P(x)$ turns out to be true for the chosen x . This is the “I can find one” quantifier—Prover gets to choose.
- $\forall x. P(x)$: **Challenger’s move**. Challenger picks *any* x they want—the nastiest, most adversarial x they can think of—and then the game continues with $P(x)$. Prover wins if $P(x)$ is true *regardless* of what Challenger picked. This is the “it works for everything” quantifier—Challenger gets to choose, and Prover has to survive.
- $A \wedge B$: **Challenger’s move**. Challenger picks which of A or B to verify. Prover must be able to win *both* games, so it doesn’t matter which one Challenger chooses. Think of it as: “I claim both—go ahead, quiz me on either one.”
- $A \vee B$: **Prover’s move**. Prover picks which of A or B to demonstrate. Only one needs to hold, and Prover gets to pick the one they can actually win. Think of it as: “I claim at least one of these—let me show you which.”
- $A \rightarrow B$: **Challenger provides, Prover responds**. Challenger hands over evidence for A , and Prover must use it to produce evidence for B . If you’re a programmer, this should look familiar: it’s a function. Challenger provides the input, Prover computes the output.
- $\neg A$: **Role swap**. The players switch seats. Prover becomes Challenger, Challenger becomes Prover, and they play the game for A with reversed roles. Negation flips who’s trying to do what.

This might seem like just a cute metaphor, but it’s not—it’s a genuine mathematical framework called *game semantics*¹⁰ and it gives us the exact same truth values as the set-based semantics from Module 4. A formula is true precisely when Prover has a *winning strategy*—a way to play that wins no matter what Challenger does.

4.19 Nested quantifiers as alternating turns

The game interpretation really shines when we look at nested quantifiers, because nested quantifiers correspond to a game with *multiple rounds*—the players alternate turns, and the whole formula describes a back-and-forth.

Let’s revisit the example from Module 4 where we saw that quantifier order matters. Let $P(x, y) = “y > x”$ over $\mathbb{R}$.

Game 1: $\forall x : \mathbb{R}. \exists y : \mathbb{R}. y > x$

1. Challenger picks any real number x . (Challenger’s move—this is $\forall$.)
2. Prover sees what x was chosen and picks y in response. (Prover’s move—this is $\exists$.)
3. Prover wins if $y > x$.

Can Prover always win? Yes! Prover’s strategy is simple: whatever x Challenger picks, respond with $y = x + 1$. Since $x + 1 > x$ for every real number, this strategy works against every possible challenge. The formula is **true**.

The key thing to notice: Prover’s choice of y gets to *depend on* x . Prover moves second, so they have information. Their winning strategy is a *function* from challenges to responses: $f(x) = x + 1$.

Game 2: $\exists y : \mathbb{R}. \forall x : \mathbb{R}. y > x$

1. Prover picks a real number y . (Prover’s move—this is $\exists$.)
2. Challenger sees what y was chosen and picks x in response. (Challenger’s move—this is $\forall$.)
3. Prover wins if $y > x$.

Can Prover win? No. Whatever y Prover commits to, Challenger can just pick $x = y + 1$, and then $y > y + 1$ is false. Prover has to commit *before* seeing the challenge, and no single constant y can be bigger than every real number. The formula is **false**.

See the difference? In Game 1, Prover has a *strategy*—a function that responds to each challenge. In Game 2, Prover needs a *constant*—a single value that works universally. Strategies are much more powerful than constants, which is why $\forall x. \exists y$ is a weaker claim than $\exists y. \forall x$ (the second implies the first, but not vice versa).

If you’re a programmer, the distinction is really clean:

- $\forall x. \exists y. y > x$ is like writing a function:

```
def respond(x):
    return x + 1 # always bigger than the input
```

¹⁰Game semantics was developed seriously in the 1990s by Abramsky, Jagadeesan, Malacaria, Hyland, Ong, and others. It provides a fully compositional semantics for programming languages and logics—not just a teaching analogy.

You get to see the input before choosing your output. Easy.

- $\exists y. \forall x. y > x$ is like declaring a global constant:

$y = ???$ # must be bigger than ALL possible inputs

You have to commit before seeing any input. Impossible (for the reals).

This is the same intuition we built in Module 4, but the game framing makes the “why” visceral: it’s about *who moves first*. The order of quantifiers determines the order of play, and the order of play determines how much information each player has when they make their choice.

4.20 Games and constructive logic

There’s a philosophical point lurking here that connects back to something we touched on in Module 5: the distinction between constructive and non-constructive proofs.

The game interpretation is inherently *constructive*. To win a game, Prover can’t just argue abstractly that a winning move exists somewhere—Prover has to actually *make the move*. When the formula says $\exists x. P(x)$, Prover has to put a concrete x on the table. When it says $A \vee B$, Prover has to point to one of A or B and win that game. No hand-waving allowed.

This is exactly the spirit of constructive mathematics: to prove something exists, you must build it. To prove a disjunction, you must know which side is true.

Classical logic, by contrast, gives Prover an extra power: *strategy-stealing arguments*. In classical logic, Prover is allowed to reason like this: “Suppose I can’t win. Then Challenger must have a winning strategy. But if Challenger has a winning strategy, then [some contradiction arises], so actually I *can* win.” The conclusion is that Prover wins, but the argument never actually described a move for Prover to make. It proved existence without providing a witness.

This connects directly to the law of excluded middle, $p \vee \neg p$. In game terms, this says: “Either Prover can demonstrate p , or Prover can demonstrate $\neg p$.” Classically, this is just assumed—one of the two must be the case. But constructively, Prover has to actually play one of the two games and win it. For many propositions (especially undecidable ones from computability theory) there’s no general way to know which side to pick, so $p \vee \neg p$ isn’t something you get for free.

The link to the Curry–Howard intermezzo is worth flagging explicitly: Prover’s winning strategy *is* a program, in essentially the same sense that Curry–Howard’s “a proof is a program” is. A strategy for $\forall x. \exists y. P(x, y)$ is a function that turns each challenge x into a response y —which is exactly what Curry–Howard says a proof of that formula is. Game semantics and Curry–Howard are two faces of the same computational reading of logic, and students who internalize one usually find the other suddenly clicks.

If you remember the discussion of constructive existence proofs from Module 5—where we said that a constructive proof gives you an algorithm while a non-constructive proof gives you a theorem but no code—this is the same idea wearing different clothes. Game semantics makes it vivid: constructive logic is the version of the game where Prover actually has to play. Classical logic is the version where Prover is sometimes allowed to win by saying “if I couldn’t win, that would be absurd.”¹¹

¹¹If that sounds like cheating, welcome to the intuitionist club. If it sounds perfectly reasonable, welcome to the classical club. Both clubs have cookies; they just disagree about whether the cookies have been constructed or merely shown to exist.

4.21 Why this matters

You might be wondering: is this just a nice way to think about quantifiers, or does it actually *go* somewhere? It goes a lot of places.

Interactive proof systems. In theoretical computer science and cryptography, the Prover-vs-Verifier model is the literal foundation of *interactive proof systems*. A Prover tries to convince a Verifier that a statement is true, and the Verifier is allowed to ask questions (challenges). The statement is “provable” if there’s a Prover strategy that convinces the Verifier, and “sound” if no cheating Prover can fool the Verifier into accepting a false statement. This is essentially the same two-player game we’ve been discussing, just taken very seriously.

Zero-knowledge proofs. An especially remarkable variant: what if Prover wants to convince Verifier that a statement is true *without revealing any information about why it’s true*? This is a zero-knowledge proof, and it’s the cryptographic technology behind privacy-preserving blockchain protocols, anonymous credentials, and a growing chunk of modern cryptography. The game-theoretic structure is essential—the “zero knowledge” property is defined in terms of what Verifier can learn from the interaction.

Complexity theory. One of the landmark results in theoretical computer science is that $IP = PSPACE$: the class of problems solvable by an interactive proof system (where a polynomial-time Verifier interacts with an all-powerful Prover) is exactly the class of problems solvable with polynomial space. This was a shocking result when it was proved in 1992, and the entire framework rests on formalizing computation as a game between Prover and Verifier.

The point is that the two-player game isn’t just a pedagogical metaphor. It’s a lens through which serious mathematics, computer science, and cryptography are actually done. When you see quantifiers as games, you’re not just learning a trick for reading formulas—you’re learning the conceptual vocabulary of a significant part of modern theory.

Exercises

Exercise 1. Consider the statement

$$\forall \epsilon > 0. \exists \delta > 0. \forall x. (|x - 3| < \delta \rightarrow |x^2 - 9| < \epsilon).$$

This is the epsilon-delta definition of continuity for $f(x) = x^2$ at $x = 3$.

Describe the game: who moves first? What does each player choose at each turn? What must Prover achieve in order to win? (You do not need to find Prover’s actual strategy—just describe the structure of the game.)

Exercise 2. In the game for $\exists x. \forall y. (x + y = y)$, what is Prover’s winning move? (Hint: Prover moves first. What value of x makes the inner statement true no matter what Challenger picks for y ?)

Exercise 3. Consider the formula $\forall x \in \mathbb{N}. \exists y \in \mathbb{N}. y > x$ over the natural numbers. Describe the game in detail (who moves first, who moves second, what they choose, what Prover must achieve). Then give Prover’s winning strategy as an explicit function f such that $f(x)$ is Prover’s response when Challenger picks x .

Exercise 4. Now consider $\exists y \in \mathbb{N}. \forall x \in \mathbb{N}. y > x$. Describe the game and explain, in game-theoretic terms, exactly why Prover *cannot* win, no matter what value of y they pick first. Connect your answer to the difference between a strategy and a constant in the discussion above.

Exercise 5. (Quantifier games for arithmetic identities.) Describe the game corresponding to the statement $\forall n \in \mathbb{N}. \forall m \in \mathbb{N}. n + m = m + n$. Both quantifiers are universal, so Challenger gets to pick *both* values. What does Prover have to demonstrate? In what sense is this game “trivial,” even though there are infinitely many possible challenges?

Exercise 6. (Strategies as functions.) The Skolem form of $\forall x. \exists y. R(x, y)$ replaces the existential by a function: $\exists f. \forall x. R(x, f(x))$. Explain how this transformation matches what the game interpretation says about Prover’s strategy. (Hint: in the game for $\forall x. \exists y. R(x, y)$, Prover responds to each x with some y . Bundle all those responses into a single object. What is that object?)

Exercise 7. Consider the game for $\forall x. (P(x) \vee \neg P(x))$ over a finite universe $\{a_1, \dots, a_n\}$. Show that Prover has a winning strategy as long as Prover can decide, for each a_i , whether $P(a_i)$ holds. Now imagine P is a property that requires checking infinitely many cases (like “ x is the code of a Turing machine that halts on every input”). Why does the strategy break down? Connect this to the discussion of constructive vs. classical logic in the chapter.

Further reading

If you want to follow the threads from games into theoretical computer science and modern cryptography:

- Jaakko Hintikka, “Language-Games for Quantifiers,” *American Philosophical Quarterly* Monograph Series no. 2 (1968), 46–72. Hintikka’s own short and accessible exposition of how quantifiers are best read as a two-player game between a Verifier and a Falsifier—the philosophical source of the framework in this chapter.
- Shafi Goldwasser, Silvio Micali, and Charles Rackoff, “[The Knowledge Complexity of Interactive Proof Systems](#),” *SIAM Journal on Computing* **18**(1), 186–208 (1989). The foundational paper on interactive proofs and zero-knowledge. The first few sections, where they set up the Prover/Verifier game, are surprisingly accessible and put a precise mathematical frame around the picture sketched in the chapter.
- Oded Goldreich, [Zero-Knowledge: A Tutorial](#), available on the author’s homepage at the Weizmann Institute. Written explicitly to be the most accessible on-ramp to zero-knowledge proofs for non-specialists. If interactive proofs caught your attention, this is where to go next.

Chapter 5

Informal Proofs and Groups

Reading map

Epp sections. §4.1–4.3 (§4.1–4.4 in the 5th edition): direct proof, proof by counterexample, proofs involving rational numbers and divisibility, and existence proofs.

Prerequisites. Modules 1–4 (especially the $\forall$ and $\exists$ proof rules from Module 4).

Relevant intermezzi. “What Does It Mean for Things to Be ‘The Same’?” (equivalence relations; read after Module 6) ties into the group-theory section here.

Proof-writing reference. The companion chapter “How to Write a Proof, Revisited,” which appears right after Module 7, consolidates prose conventions for the full range of proof techniques—including a technique-selection decision tree and a longer walkthrough using the contrapositive. Worth flagging now as a forward reference for the trickier exercises in this module.

Practice from Epp. §4.1 exercises on direct proofs about even/odd and divisibility; §4.2 exercises on rational-number proofs; §4.3 exercises on divisibility; and any ‘disprove with counterexample’ problems from those sections.

This module is, in a sense, a breather. We’re not introducing new logical machinery—instead we’re getting practice with informal proofs after several modules of formal logic. The relevant sections of Epp are very well written and give plenty of time to practice proofs about numbers, touching on topics I think matter: a games-based approach to quantifiers, constructive vs. non-constructive proofs of $\exists$, and the relationship between writing proofs and writing programs. What I want to add, rather than recapitulating Epp, is to practice the same techniques in a *different* mathematical setting—abstract algebra, specifically groups—so you can see that the patterns are general.

One thing worth saying up front, though: the informal techniques we’re about to practice are not a departure from the formal rules of Modules 3 and 4—they’re *implementations* of them. A direct proof of “for all x , if $P(x)$ then $Q(x)$ ” is $\forall$ -intro combined with $\rightarrow$ -intro: introduce a fresh variable, assume the hypothesis, derive the conclusion. A counterexample to a universal claim is $\exists$ -intro applied to the negation $\neg(\forall x. P(x))$. A proof by cases is $\vee$ -elimination in disguise. The formal rules tell us *why* these techniques are valid; the informal style is how mathematicians actually write. We’ll keep pointing out the correspondence as we go, so you can see the formal machinery underneath the prose. But first, let’s get the number-theory toolkit solid.

5.1 Direct proof

The most basic proof technique is the *direct proof*. The shape of a direct proof for a universal conditional statement — something of the form “for all x , if $P(x)$ then $Q(x)$ ” — is: you assume $P(x)$ for an *arbitrary* x , and then you show that $Q(x)$ follows. That’s it. The whole game is figuring out how to get from $P(x)$ to $Q(x)$, and the trick is almost always: *unpack the definitions*.

Even and odd

Let’s nail down what “even” and “odd” actually mean, precisely enough to use in proofs.

Definition 5.1.1 (Even and odd). An integer n is *even* if $n = 2k$ for some integer k . An integer n is *odd* if $n = 2k + 1$ for some integer k .

These definitions are what we actually *use* in proofs — when we know something is even, we unpack the definition to get the k , and then we do algebra with $2k$. When we need to *show* something is even, we exhibit a k and show the equation holds. This is the “there exists” hidden inside these definitions: “ n is even” really means “ $\exists k \in \mathbb{Z}. n = 2k$.”

Let’s do a worked example.

Claim: If a and b are both odd integers, then $a + b$ is even.

Proof: Let a and b be arbitrary odd integers. By the definition of odd, we have $a = 2j + 1$ for some integer j , and $b = 2k + 1$ for some integer k . Then

$$a + b = (2j + 1) + (2k + 1) = 2j + 2k + 2 = 2(j + k + 1).$$

Since $j + k + 1$ is an integer, $a + b = 2m$ where $m = j + k + 1$, so $a + b$ is even by definition. $\square$

See the pattern? We assumed the hypothesis (both odd), unpacked the definitions (got j and k), did some algebra, and then repacked the result into the definition of even. Assume the hypothesis, unpack the definitions, do the algebra, repack the result—that’s the core of direct proof.

Let’s do another one.

Claim: The sum of any even integer and any odd integer is odd.

Proof: Let a be an arbitrary even integer and b be an arbitrary odd integer. Then $a = 2j$ for some integer j and $b = 2k + 1$ for some integer k . So

$$a + b = 2j + 2k + 1 = 2(j + k) + 1.$$

Since $j + k$ is an integer, $a + b$ has the form $2m + 1$ where $m = j + k$, so $a + b$ is odd. $\square$

Same pattern, different details.

Divisibility

Here’s another definition that works exactly the same way.

Definition 5.1.2 (Divisibility). For integers a and d with $d \neq 0$, we say d *divides* a (written $d \mid a$) if $a = dk$ for some integer k .

So “ $3 \mid 12$ ” because $12 = 3 \cdot 4$, and “ $5 \nmid 7$ ” because there’s no integer k with $7 = 5k$.

Notice this has the same structure as evenness: it’s an existential claim buried inside a definition. So proofs about divisibility work the same way — unpack, do algebra, repack.

Claim: For all integers a, b, c : if $a \mid b$ and $b \mid c$, then $a \mid c$. (Transitivity of divisibility.)

Proof: Let a, b, c be arbitrary integers with $a \mid b$ and $b \mid c$. By definition, $b = aj$ for some integer j , and $c = bk$ for some integer k . Then

$$c = bk = (aj)k = a(jk).$$

Since jk is an integer, we have $c = am$ where $m = jk$, so $a \mid c$. □

Rational numbers

One more definition in this style.

Definition 5.1.3 (Rational number). A real number r is *rational* if $r = a/b$ where a, b are integers and $b \neq 0$.

Showing that a number is rational is really just an existence proof — you need to exhibit the integers a and b . For example, 0.75 is rational because $0.75 = 3/4$, and we’ve exhibited $a = 3$, $b = 4$ ¹.

5.2 Counterexample

Direct proofs let you *prove* universal statements. But what if a universal statement is *false*? How do you disprove it?

A universal statement “for all x , $P(x)$ ” is false if there exists *even one* x where $P(x)$ is false. That one example is called a *counterexample*. This is just the negation of a universal: $\neg(\forall x. P(x)) \equiv \exists x. \neg P(x)$. We saw this equivalence back in Module 4!

So to disprove a universal, you just need to find one concrete value that breaks it. Let’s see this in action.

Claim to disprove: For every integer n , if n is odd then $(n^2 - 1)/2$ is odd.

Disproof: Counterexample: let $n = 1$. Then n is odd, and $(1^2 - 1)/2 = 0/2 = 0$, which is even, not odd. So the claim is false. □

That’s the whole proof. One example. That’s all you need.

Here’s the important asymmetry: to **prove** a universal statement you need to work with an *arbitrary* element — you can’t pick a specific one, because you need the argument to work for all of them. But to **disprove** a universal you just need *one specific* counterexample. This asymmetry shows up everywhere and is worth internalizing.

If you’re a programmer, think of it this way: proving a function is correct means showing it works *for all inputs*. Finding a bug means finding *one bad input*. Testing is the art of finding counterexamples; formal verification is the art of proving universals. That’s why testing can show the presence of bugs but never their absence².

5.3 Existence proofs

Counterexamples are about *disproving* universals, but what about *proving* existentials? If you want to show $\exists x : A. P(x)$, how do you actually do it?

¹Technically you could also use $a = 6$, $b = 8$, or any other representation. There’s no uniqueness requirement in the definition, though you can get uniqueness if you insist on lowest terms.

²This is usually attributed to Dijkstra, and it’s one of those quotes that sounds deep until you realize it’s just the logical asymmetry between $\forall$ and $\exists$ wearing a trench coat.

The most straightforward way is a *constructive existence proof*: you produce a concrete witness and verify that it satisfies the predicate. This is exactly $\exists$ -introduction from Module 4! You hand over a specific element $c : A$ along with a proof that $P(c)$ holds, and you're done.

Claim: There exists a real number $x > 1$ such that $2^x > x^{10}$.

Proof: Let $x = 100$. Then $x > 1$ and we need to check that $2^{100} > 100^{10}$. Well, $100^{10} = (10^2)^{10} = 10^{20}$. And $2^{10} = 1024 > 10^3$, so $2^{100} = (2^{10})^{10} > (10^3)^{10} = 10^{30}$. Since $10^{30} > 10^{20}$, we have $2^{100} > 100^{10}$. $\square$

That's it! We picked a value, plugged it in, and checked. The hard part isn't the proof itself — it's figuring out *which* value to try. But once you've found one that works, the proof is just computation.

A couple of things to note about existence proofs:

- You don't need to find the *best* or *smallest* witness. Any witness that works is fine. We used $x = 100$ above, but $x = 59$ also works.³ The goal is existence, not optimization.
- The witness doesn't have to come from anywhere in particular. You can use trial and error, intuition, a calculator, whatever. As long as you *verify* it rigorously in the proof, nobody cares how you found it.
- This is a *constructive* proof because we actually built the thing we claimed exists. There are also *non-constructive* existence proofs that show something must exist without ever telling you what it is—proof by contradiction can sometimes do this—but constructive proofs are generally preferred when you can get them, especially in computer science where you usually want to actually *compute* the thing.⁴

It's worth making the computational significance of this distinction more explicit. A constructive existence proof hands you a *witness*—a concrete object satisfying the property—and in programming terms that means it's a function that actually returns a value. A non-constructive proof, by contrast, tells you something exists without giving you any way to compute it: you know it's out there, but you can't write a program to find it. This is a real practical distinction, not a philosophical one. If you prove “there exists an optimal schedule” constructively, you have an algorithm; if you prove it non-constructively, you have a theorem but no code. This is the core reason constructive mathematics appeals to computer scientists—every constructive proof is, in a precise sense, a program. (This idea can be made fully rigorous via the Curry–Howard correspondence, which we'll gesture at later in the course, but even informally it's a useful way to think about what your proofs are actually *doing*.)

Now that we have the basic direct proof toolkit for number theory, let's practice these same techniques in a completely different mathematical setting. The point is to show that the proof patterns we just learned — unpack definitions, do algebra, repack — are *general*. They don't just work for even/odd and divisibility; they work for any mathematical structure with precise definitions.

I'm going to talk about *groups*.

³In fact the crossover point is somewhere around $x \approx 58.77$, but nobody asked us for the exact boundary. One working example is all we need.

⁴This is the constructive mathematician in your instructor peeking through again. Hi.

5.4 What is a group?

A group is one of the simplest structures in abstract algebra⁵—an incredibly useful but very simple concept in mathematics that shows up over and over again. It’s also simple enough to start with if you’ve never seen any abstract algebra before.

Here’s the definition:

Definition 5.4.1 (Group). A *group* is a triple $(G, *, e)$ consisting of a set G , a binary operation $*$: $G \times G \rightarrow G$, and a distinguished element $e \in G$, satisfying the following axioms:

(associativity) $\forall a, b, c : G. (a * b) * c = a * (b * c)$

(left-identity) $\forall g : G. e * g = g$

(right-identity) $\forall g : G. g * e = g$

(inverses) $\forall g : G. \exists g^{-1} : G$ such that both

1. (left-inverse) $g^{-1} * g = e$, and

2. (right-inverse) $g * g^{-1} = e$.

So what is this saying? A group is a triple of data: a base set G , a binary operation $*$ on the set, and an element e that acts as the identity for the operation. Associativity tells us that repeated applications of the operation don’t depend on how we parenthesize them, the identity axioms guarantee that e really acts as an identity, and the inverse axioms guarantee that every element has an inverse that can be used to cancel it out.

Sidebar: groups in the wild.

This definition looks abstract, but you’ve been surrounded by groups your whole life. A few places they show up:

- **Symmetries of objects.** Take a square. The rotations and reflections that map it back to itself—eight of them, if you count—form a group under composition. Same for a cube (48 symmetries), a snowflake, a molecule. Any time you hear “this thing has a symmetry,” there’s a group lurking underneath.
- **Permutations.** Shuffling n items is a group. Composing two shuffles gives another shuffle; the identity is “don’t shuffle”; every shuffle can be undone. The Rubik’s cube group is a permutation group on 54 stickers (quotiented by some physical constraints). The same group theory that solves the cube classifies the symmetries of molecules.
- **Cryptography.** Diffie–Hellman key exchange lives in the group of nonzero residues modulo a large prime. Elliptic-curve cryptography uses points on an elliptic curve, which also form a group. The security of these systems rests on the difficulty of “undoing” certain group operations—concretely, the *discrete logarithm problem*. If you’ve ever accessed a site over HTTPS, you’ve used a group.
- **Undo in editors and version control.** The operation “apply a patch” wants to be a group: identity is the empty patch, composition is “apply this, then that,” and inverse is “undo.” Real systems only approximate this (merges make it complicated), but the mental model is group-shaped.

⁵I sometimes think of this as “My First Abstract Algebra”—it’s the training-wheels version of a much larger zoo of algebraic structures.

- **Integer and floating-point arithmetic modulo 2^{32} .** The bit-level addition on a 32-bit CPU is $(\mathbb{Z}/2^{32}\mathbb{Z}, +, 0)$ —a genuine, finite group, right there in your hardware. Overflow wraps around exactly because the inverse of every element exists.

Groups are the minimum algebraic machinery for talking about “invertible operations,” and invertible operations are everywhere in CS—from reversible computing to crypto to functional programming’s lens laws. The definition’s austerity is the point: by asking for *only* associativity, identity, and inverses, we pick up everything that has those three features at once.

Let’s start by proving that some things **are** or are **not** groups.

5.5 The integers under addition form a group

Claim. The integers $\mathbb{Z}$, together with $+$ as the binary operation and 0 as the identity, form a group.

Proof. We need to show that all four axioms hold. We’re allowed to assume the standard definitions of addition, negation, and 0—we’re proving that they form a group, not defining them formally. If you were doing this *inside* a logic (as, say, the Natural Number Game requires), you’d have to first define $\mathbb{Z}$ and all its operations from the ground up. Here we just take them as given.

Associativity. Let $a, b, c : \mathbb{Z}$. Then $(a + b) + c = a + (b + c)$ by the associativity of integer addition.

Left-identity. Let $n : \mathbb{Z}$ be arbitrary. We need $0 + n = n$, but this follows directly from the definition of addition on $\mathbb{Z}$.

Right-identity. Similarly, for arbitrary $n : \mathbb{Z}$, the equation $n + 0 = n$ follows from the definition of addition.

Inverses. We need, for every $n : \mathbb{Z}$, some element $n^{-1} : \mathbb{Z}$ that works as both a left and right inverse. The obvious candidate is $-n$, and our obligations become $\forall n. (-n) + n = 0$ and $\forall n. n + (-n) = 0$. But that’s just subtraction written funny, and since we’ve already defined subtraction to work this way, both equations hold by definition.

All four axioms hold, so $(\mathbb{Z}, +, 0)$ is a group. □

5.6 A worked finite example: rotations of a square

$(\mathbb{Z}, +, 0)$ is the easy confidence-builder, but the axioms were doing so little work there that you might reasonably wonder whether “verify a group” ever amounts to anything. Let’s do a second example where the group is *finite* and the verification has something concrete to chew on.

Imagine a square sitting on a table with its corners labeled 1, 2, 3, 4 clockwise from the top-left. Consider the four rotations that send the square back to itself:

- e : rotate by 0° (don’t do anything).
- r : rotate clockwise by 90° .
- r^2 : rotate by 180° .
- r^3 : rotate by 270° .

Rotating by 360° gets you back to e , so we don’t pick up any new elements past r^3 . Our carrier set is $G = \{e, r, r^2, r^3\}$ with 4 elements.

Our operation is function composition: “ $g * h$ means first apply h , then g .”⁶ Because we’re just adding rotation angles (mod 360°), the operation is completely determined by the table:

$*$	e	r	r^2	r^3
e	e	r	r^2	r^3
r	r	r^2	r^3	e
r^2	r^2	r^3	e	r
r^3	r^3	e	r	r^2

The entry in row g and column h is $g * h$. Read off the patterns: every row is a permutation of G , and so is every column; the table is symmetric about the main diagonal in this particular case (though it wouldn’t be for non-commutative groups, which we’re about to get to).

Now we verify the axioms. This time some of them have content.

Closure. We need $g * h \in G$ for every $g, h \in G$. The table makes this immediate: every entry is one of e, r, r^2, r^3 . Checked by inspection of 16 cells.

Associativity. In principle we could check all $4^3 = 64$ triples (a, b, c) against $(a * b) * c = a * (b * c)$. That would be tedious but valid. The cleaner argument is: $*$ here is function composition, and function composition is associative for *any* functions (we saw this in Module 1). So the associativity axiom is free, inherited from the fact that rotations are functions and functions compose associatively.⁷

Left-identity and right-identity. Reading the first row and first column of the table: $e * g = g$ and $g * e = g$ for every $g \in G$. Both identity axioms hold.

Inverses. We need every element to have an inverse. Read along the rows looking for the identity e :

- $e * e = e$, so $e^{-1} = e$.
- $r * r^3 = e$ and $r^3 * r = e$, so $r^{-1} = r^3$.
- $r^2 * r^2 = e$, so $(r^2)^{-1} = r^2$ (it’s its own inverse).
- $r^3 * r = e$ and $r * r^3 = e$, so $(r^3)^{-1} = r$.

Every element has a two-sided inverse. All four axioms hold; $(G, *, e)$ is a group. $\square$

This is a real finite group, the *cyclic group of order 4*, often written C_4 . We’ll use the notation $C_4 = \{e, r, r^2, r^3\}$ later.

A remark worth sitting with: the multiplication table for C_4 is, up to renaming, the same table as addition in $\mathbb{Z}/4\mathbb{Z}$. Relabel e, r, r^2, r^3 as $0, 1, 2, 3$, and “compose two rotations” becomes “add two residues mod 4.” Same group, different costume. This “two groups that look different but behave identically” phenomenon is what *isomorphism* (which we’ll formalize in a couple of sections) is for. You’ll make this exact correspondence precise in the exercises.

⁶Function-composition order is backwards from how you say it in English, as we noted in Module 1. You can flip the convention if you like, as long as you’re consistent.

⁷This trick works for any group whose elements are functions under composition: symmetry groups, permutation groups, matrix groups under matrix multiplication. The moment you realize associativity is automatic, finite-group verification gets a lot less painful.

5.7 The integers under multiplication do *not* form a group

Disproving a group claim is easier than proving one: we only need a single axiom to fail. Let's show that $(\mathbb{Z}, \cdot, 1)$ has no inverses.

Consider the integer 2. For $(\mathbb{Z}, \cdot, 1)$ to be a group we'd need some integer n with $2 \cdot n = 1$. But for any integer n , the product $2 \cdot n$ is always even,⁸ and 1 is odd. So no integer acts as the multiplicative inverse of 2—one element without an inverse is enough, the axiom fails, and $(\mathbb{Z}, \cdot, 1)$ is not a group. $\square$

This is the group-theory version of the counterexample pattern: disproving a universal (“every element has an inverse”) by exhibiting a single bad case.

5.8 Proof of uniqueness of identity

We can also prove properties about *all* groups, not just a specific group. Like let's prove the uniqueness of identity elements. In other words, that if there are two elements in a group that satisfy the identity axioms then they must be **equal**.

Theorem: Let $(G, *, e)$ be a group and let $e' : G$ satisfy the left and right identity properties. Then $e = e'$.

Proof: We'll prove this by calculation with the properties of identity.

$$\begin{aligned} e &= e * e' && \text{(since } e' \text{ is a right-identity: } e * e' = e) \\ e * e' &= e' && \text{(since } e \text{ is a left-identity: } e * e' = e') \end{aligned}$$

and thus $e = e'$. $\square$

This result tells us something important: the identity element of a group isn't just convenient notation—it's *forced*. Any element that behaves like an identity must be the same as the one we started with. This kind of uniqueness result is typical in algebra: the axioms pin things down more tightly than you might expect.

We've now seen what the group axioms force *inside* a single group. The next move is to look *between* groups—and that is where the subject really opens up.

5.9 Maps between groups: homomorphisms

Each group we've seen so far— $(\mathbb{Z}, +, 0)$, $(\mathbb{Q} \setminus \{0\}, \cdot, 1)$ in the exercises—is interesting on its own. But a huge part of what algebra is *about* is relating one group to another: finding functions between them that respect the group structure. These are called *homomorphisms*, and they are the right notion of “map” once you're doing algebra.

Definition 5.9.1 (Group homomorphism). Let $(G, *_G, e_G)$ and $(H, *_H, e_H)$ be groups. A function $\varphi : G \rightarrow H$ is a *homomorphism* if it respects the group operation: for all $a, b \in G$,

$$\varphi(a *_G b) = \varphi(a) *_H \varphi(b).$$

⁸If you want a bonus challenge, prove this property of multiplication with natural induction! It's not too hard but is a good exercise.

It's a short exercise (and worth doing) to show that a homomorphism automatically sends the identity to the identity, $\varphi(e_G) = e_H$, and sends inverses to inverses, $\varphi(g^{-1}) = \varphi(g)^{-1}$. Both facts fall straight out of the one equation in the definition.

Example 5.9.2 (Reduction mod k). For any positive integer k , the function $\varphi : \mathbb{Z} \rightarrow \mathbb{Z}/k\mathbb{Z}$ defined by $\varphi(n) = n \bmod k$ is a homomorphism from $(\mathbb{Z}, +, 0)$ to $(\mathbb{Z}/k\mathbb{Z}, +, 0)$. The content of this claim is exactly that

$$(a + b) \bmod k = ((a \bmod k) + (b \bmod k)) \bmod k,$$

which is the fact that “addition plays nicely with remainders”—the thing Module 6 will establish when it proves modular addition is well-defined on residue classes. That well-definedness is *exactly* this homomorphism property, just stated in congruence vocabulary rather than function vocabulary. Worth circling back to this example after you've read §6.4.

Once we have homomorphisms, the right notion of “same” for groups is also clear: two groups are the “same” when there is a homomorphism between them that can be undone.

Definition 5.9.3 (Isomorphism). A group homomorphism $\varphi : G \rightarrow H$ is an *isomorphism* if it is a bijection. Two groups are *isomorphic*, written $G \cong H$, if there is an isomorphism between them.

Isomorphic groups are interchangeable for every algebraic purpose: any statement that's true of one, phrased in the language of groups, is true of the other. This is our first concrete instance of a general phenomenon—the intermezzo “What Does It Mean for Things to Be ‘The Same’?” is the essay on exactly this point—but you've already seen the pattern once before: extensional equality of sets (Module 1) is the analogous “right notion of sameness” for sets.

The archetypal group: self-bijections

Here is a forward pointer worth noting. Let A be any set, and let $\text{Sym}(A)$ be the set of bijections $A \rightarrow A$. With function composition as the operation and the identity function id_A as the unit, $\text{Sym}(A)$ is a group. Composition of functions is associative; id_A is a two-sided identity; and every bijection has an inverse which is again a bijection. (Each of these is an easy check against the definitions from Module 1.)

This is the group you should keep in mind when you want a test example. Every group you meet in this course is, in some sense, a group of “symmetries” of something—and the prototype of “symmetry” is exactly “bijection from a thing to itself.”

5.10 Proof by exhaustion

There's one more direct-proof technique that deserves a name because it comes up constantly, especially in computer science: *proof by exhaustion*.

A proof by exhaustion is what you do when the domain you're quantifying over is finite (or can be split into finitely many cases), so you can just *check each case* directly. You literally list them out and verify the predicate on each one.

Claim. Every even integer n with $4 \leq n \leq 12$ can be written as a sum of two prime numbers.

Proof by exhaustion.

- $4 = 2 + 2$ (both prime)
- $6 = 3 + 3$ (both prime)

- $8 = 3 + 5$ (both prime)
- $10 = 3 + 7 = 5 + 5$ (both decompositions work)
- $12 = 5 + 7$ (both prime)

That exhausts the five cases, so the claim holds on the range $\{4, 6, 8, 10, 12\}$. $\square$

The technique is banal in the abstract—“check every case”—but it is *exactly* what you do when proving things like “every binary boolean operator can be expressed in terms of $\neg$ and $\wedge$ ” (the 16-case exercise from the assignment this term). There are finitely many binary boolean operators (sixteen of them, as you proved in Module 2), and you verify the property on each one. That’s proof by exhaustion in action.

Proof by exhaustion is technically just a special case of proof by cases (which we covered formally as $\vee$ -elimination in Module 3): each case is a disjunct in “ $x = x_1$ or $x = x_2$ or $\dots$ or $x = x_{16}$,” and we prove the conclusion from each disjunct. It earns its own name because in practice it’s distinctive: the disjuncts are forced by the finiteness of the domain rather than by some clever case split you chose, and the argument often reduces to a table.

A few cautions:

- Exhaustion only works on *finite* domains (or at most finitely many distinguishable cases, when the domain is infinite but the cases cover it). If the claim is universal over $\mathbb{Z}$, exhaustion is not available to you.
- You have to genuinely cover *every* case. If you list 15 of the 16 binary operators and say “similar argument for the last one,” you haven’t completed the proof.
- The conclusion only applies to the cases you listed. My claim above is specifically about $\{4, 6, 8, 10, 12\}$, not about all even integers. (The unrestricted claim “every even integer ≥ 4 is a sum of two primes” is the famous *Goldbach conjecture*, which is still open as of 2026. Exhaustion on a finite sub-case is not the same as a proof for infinitely many cases, which is a whole other tool—induction—that we’ll come back to in Modules 8–10.)

5.11 Common mistakes

1. **“Suppose $n = 2k$ ” is not a proof that n is even.** This is the error in Exercise 5.13 (and it’s a real thing students write). A direct proof that n is even, starting from some hypothesis about n , has to *derive* an integer k with $n = 2k$ from the hypothesis, not *assume* it. “Suppose $n = 2k$ ” is an assumption, not a conclusion.
2. **Picking a specific example when proving universally.** If the claim is “for all integers n , $n^2 \geq n$,” you cannot “prove” it by saying “take $n = 5$: $25 \geq 5$. ✓” That verifies *one case*, not all cases. For $\forall$ you need an arbitrary element.
3. **Confusing “there exists” with “there exists a *unique*.”** The $\exists$ quantifier says “at least one,” not “exactly one.” If the claim asks for uniqueness, you owe a uniqueness argument in addition to the existence argument.
4. **Inverting “necessary” and “sufficient” on group axioms.** To be a group, the axioms are all *necessary*—you need every one. If you verify three of the four axioms and not the fourth, you haven’t proved “it’s a group”; the best you’ve done is rule out three ways it could fail.

5. **Not actually using the inverse axiom after invoking it.** A very common error in group-theory proofs: students write “let g^{-1} be the inverse of g ” and then never actually use the fact that $g \cdot g^{-1} = e$. If the proof works without that fact, you haven’t used the group structure—you’ve proved something else.
6. **Forgetting one direction of “if and only if.”** A biconditional claim needs two separate arguments: $(\Rightarrow)$ and $(\Leftarrow)$. Students often prove one direction and stop. If the claim is $P \leftrightarrow Q$, you are not done until you have proofs of both $P \rightarrow Q$ and $Q \rightarrow P$.
7. **Trying to read associativity off a group table.** A multiplication table shows closure (every entry is in G), lets you read off the identity (the row/column where $g * e = g$), and lets you read off inverses (find the e in each row). What it *doesn’t* show is associativity— $(a * b) * c = a * (b * c)$ is a statement about *three* elements, and a two-dimensional table has nothing to say about it directly. Either inherit associativity from an underlying structure (function composition, integer addition, matrix multiplication) or check all triples explicitly; don’t assume the table settles it.
8. **Using group axioms on things that aren’t elements of the group.** When writing proofs about a group $(G, *, e)$, every element you name has to live in G . Students sometimes write “let g^{-1} ” when g hasn’t been shown to be in G , or compute a product $g * h$ for some h that was assumed to live in a different structure. If you catch yourself reaching for an inverse, re-check that the element you’re inverting is actually in the group you said you were working in.

5.12 Summary and looking ahead

Key takeaways.

- A *direct proof* assumes the hypothesis, unpacks the definitions, does the algebra, and repacks the conclusion. Most proofs follow this shape.
- A *counterexample* is the universal disprover: one concrete instance is enough to kill a universal claim.
- A *constructive existence proof* exhibits a specific witness and verifies the predicate on it. Constructive proofs correspond to programs that compute the witness.
- *Proof by exhaustion* handles finite case splits mechanically: list every case, check each one, conclude. It’s a special case of $\vee$ -elimination.
- A *group* is a set with an associative operation, an identity, and inverses for every element. The axioms pin down a lot of structure—identity is unique, inverses are unique, $(g^{-1})^{-1} = g$.
- A *homomorphism* $\varphi : G \rightarrow H$ is a function respecting the group operation: $\varphi(a *_G b) = \varphi(a) *_H \varphi(b)$. A bijective homomorphism is an *isomorphism*; isomorphic groups are “algebraically the same.”

What we built this module. This was the first module where we really *wrote proofs*. The unpack-algebra-repack pattern is the single most important habit to build this term; it’s worth more than any specific theorem in this chapter. Groups gave us a second context—beyond

number theory—where the same habit pays off, and the group axioms will recur throughout the course ($\mathbb{Z}/n\mathbb{Z}$ under addition, the nonzero residues mod a prime under multiplication, etc.).

Looking ahead. Module 6 introduces modular arithmetic and proof by cases, both of which lean on the habits we just built. You’ll use direct proof plus case analysis on $n \bmod d$ to prove facts like “every even integer in $\{4, \dots, 12\}$ is a sum of two primes.” After Module 6, the equivalence-relation intermezzo cashes in on the group theory here by showing how modular congruence partitions $\mathbb{Z}$ into *equivalence classes*—and $\mathbb{Z}/n\mathbb{Z}$ is exactly the set of those classes, viewed as a group under the induced operation.

5.13 Exercises

Organized into **warm-up** (mechanical practice on direct proof and counterexample), **standard** (group verification and typical assignment-style problems), **challenge** (reasoning about general groups and proof by exhaustion), and **connection** (programming and forward pointers). Solutions are in the appendix.

Warm-up

Exercise 1. Prove: the product of two rational numbers is rational. (Use the same unpack-algebra-repack pattern from the proofs in this module.)

Exercise 2. Disprove: for every integer n , $n^2 > n$. (Find a counterexample and verify it.)

Exercise 3. Prove directly: the sum of any three consecutive integers is divisible by 3. (Hint: if the first of the three is n , what are the other two?)

Exercise 4. Prove directly: for all integers a, b, c with $a \neq 0$, if $a \mid b$ and $a \mid c$, then $a \mid (b + c)$.

Exercise 5. Disprove the claim: “every prime number is odd.” (This should take one line.)

Standard

Exercise 6. Is $(\mathbb{Z}, -, 0)$ a group (the integers with subtraction as the binary operation and 0 as the proposed identity)? Either verify all four axioms or find one that fails.

Exercise 7. Find the error in the following “proof” that every integer is even:

“Let n be an integer. Suppose $n = 2k$ for some integer k . Then $n = 2k$, which is even by definition. Therefore n is even.”

What proof-technique error does this commit?

Exercise 8. Show that $(\mathbb{Q} \setminus \{0\}, \cdot, 1)$ —the nonzero rationals under multiplication, with identity 1—is a group. Verify all four axioms, and be sure to exhibit the inverse of an arbitrary nonzero rational.

Exercise 9. Show that $(\mathbb{N}, +, 0)$ —the natural numbers under addition—is *not* a group. Which axiom fails, and why?

Exercise 9b. (Isomorphism.) The module claimed that the cyclic group $C_4 = \{e, r, r^2, r^3\}$ of rotations of a square is “the same group, in different costume,” as $(\mathbb{Z}/4\mathbb{Z}, +, 0)$. Make this precise.

Define a function $\varphi : C_4 \rightarrow \mathbb{Z}/4\mathbb{Z}$ by $\varphi(e) = 0$, $\varphi(r) = 1$, $\varphi(r^2) = 2$, $\varphi(r^3) = 3$. Prove: (a) φ is a bijection; (b) φ is a homomorphism—that is, $\varphi(g * h) = \varphi(g) + \varphi(h)$ (addition mod 4) for every $g, h \in C_4$. Part (b) is the work; it amounts to checking that the group-operation table for C_4 matches the addition-mod-4 table entry by entry. Conclude that $C_4 \cong \mathbb{Z}/4\mathbb{Z}$.

Challenge

Exercise 10. Prove that in any group $(G, *, e)$, inverses are unique: if h and h' both satisfy $g * h = e$ and $h * g = e$, and also $g * h' = e$ and $h' * g = e$, then $h = h'$. (Hint: compute $h * g * h'$ in two different ways, analogous to the uniqueness-of-identity proof in this module.)

Exercise 11. Prove by exhaustion: every unary boolean function (there are 4 of them—see Module 2) is expressible as a formula using only the variable p and the connectives $\{\neg, \vee\}$ (i.e. without using $\wedge, \rightarrow$, or the constants T, F). You do not need to show that these are the only such formulas, just that at least one such formula exists for each of the four unary functions.

Exercise 12. Prove: in any group $(G, *, e)$, $(g^{-1})^{-1} = g$ for every $g \in G$. (That is, the inverse of the inverse is the original element.) Make sure to explicitly invoke the inverse axiom at the right moment.

Connection

Exercise 13. (Programming.) For small n , the set $\mathbb{Z}/n\mathbb{Z} = \{0, 1, \dots, n-1\}$ with addition modulo n and identity 0 is a group. Write a Python function `is_group(carrier, op, identity)` that takes a finite list `carrier`, a binary operation `op` (as a callable), and an `identity` element, and checks—by exhaustive brute force—whether the three data form a group. The function should check associativity, the identity axioms, and the existence of inverses. Test it on $\mathbb{Z}/5\mathbb{Z}$ under addition (should return `True`) and on $\mathbb{Z}/5\mathbb{Z}$ under multiplication with identity 1 (should return `False`—why?). This is proof by exhaustion, but automated.

Exercise 13b. (Homomorphisms compose.) Let $\varphi : G \rightarrow H$ and $\psi : H \rightarrow K$ be group homomorphisms.

- (a) Show that the identity function $\text{id}_G : G \rightarrow G$ is a homomorphism from G to itself.
- (b) Show that the composition $\psi \circ \varphi : G \rightarrow K$ is a homomorphism.
- (c) Conclude (informally, no proof required) that if $G \cong H$ and $H \cong K$ then $G \cong K$, so “being isomorphic” is a transitive relation on groups. You’ll need two facts that this module doesn’t prove: that the composition of two bijections is a bijection (we do set this up in the “Cardinality” intermezzo—see the composition lemma there), and that the inverse of a bijective homomorphism is itself a homomorphism (take this on faith for now—it’s a short computation using the fact that φ is bijective and a homomorphism).

Exercise 14. (Intermezzo pointer.) The intermezzo “What Does It Mean for Things to Be ‘The Same’?” (placed after Module 6) introduces equivalence relations—the abstract machinery for saying when two mathematical objects should be treated as the same. As a preview: given a group $(G, *, e)$ and a subgroup $H \subseteq G$ (closed under $*$, containing e , closed under inverses), define the relation $a \sim b$ iff $a^{-1} * b \in H$. Argue informally that this relation is reflexive, symmetric, and transitive—i.e., it’s an equivalence relation on G . You do not need a fully rigorous proof; just check each property by invoking the group axioms.

Chapter 6

Modular Arithmetic and Proofs by Cases

Reading map

Epp sections. §4.5–4.6 (§4.4–4.5 in the 4th edition): divisibility, the quotient-remainder theorem, modular arithmetic, and proofs by cases.

Prerequisites. Module 5 (direct proof and the group axioms).

Relevant intermezzi. “What Does It Mean for Things to Be ‘The Same’?”—the equivalence-relations intermezzo sits right after this module and uses modular congruence as its main example.

Proof-writing reference. “How to Write a Proof, Revisited” (just after Module 7) includes a specific treatment of how to announce and close case proofs, which is the new technique you’ll lean on most in this module. Peek ahead when a proof feels structurally awkward.

Practice from Epp. §4.4 exercises on the quotient-remainder theorem and proofs by cases; §4.5 exercises on floor/ceiling identities and applications.

6.1 The modulus operation

Module 5 gave us the group axioms and a small zoo of examples: the integers under addition, the rationals under multiplication, the non-examples where inverses fail. This module adds a bigger, finite zoo—the modular arithmetic systems $\mathbb{Z}/n\mathbb{Z}$ —and uses them as the setting for a new proof technique, proof by cases.

The star of the show is the modulus operator, sometimes written `mod` or `%`, which gives the remainder after integer division:

```
5 % 2 = 1
9 % 3 = 0
59 % 4 = 3
```

Taking remainders turns the natural numbers—which are *not* a group under addition (no additive inverses) and not a group under multiplication either (no multiplicative inverses)—into a finite system that *is* a group, and sometimes a field. That’s the central algebraic surprise of this module, and everything else we do here flows from it.

6.2 The quotient-remainder theorem

Let's make this precise.

Theorem 6.2.1 (Quotient-Remainder Theorem). For any integer n and any positive integer d , there exist **unique** integers q and r such that

$$n = d \cdot q + r \quad \text{and} \quad 0 \leq r < d.$$

We call q the *quotient* (this is integer division) and r the *remainder* (this is the mod operation).

This theorem formalizes the quotient-remainder decomposition: the decomposition into quotient and remainder always exists and is always unique.

If you're a programmer you already know this in your bones. In Python, `//` gives you q and `%` gives you r . In C, `/` on ints gives you q and `%` gives you r —at least for non-negative values. For negative numbers, C's division truncates toward zero rather than rounding down, so the remainder can be negative; Python's `//` and `%` always match the theorem. So the quotient-remainder theorem is literally just this:

```
n == d * (n // d) + (n % d) # always True!
```

That's it. That's the theorem, in code.

Let's do a worked example. Take $n = 67$ and $d = 4$. Then $67 = 4 \cdot 16 + 3$, so $q = 16$ and $r = 3$. You can check: $4 \times 16 = 64$, and $64 + 3 = 67$. And $0 \leq 3 < 4$, so the remainder condition is satisfied.

Why does this matter? The theorem guarantees that every integer falls into exactly one *residue class* mod d : the remainder is always one of $0, 1, 2, \dots, d-1$. This is why proof by cases using mod works! For example, every integer is either even ($r = 0 \bmod 2$) or odd ($r = 1 \bmod 2$)—there's no third option. The quotient-remainder theorem is what makes that airtight.

6.3 Floor and ceiling

Two functions that show up constantly in both math and programming are *floor* and *ceiling*.

Definition 6.3.1 (Floor and Ceiling). The *floor* of a real number x , written $\lfloor x \rfloor$, is the largest integer that is less than or equal to x —"rounding down." In Python this is `math.floor(x)`, or for positive numbers just `int(x)`.

The *ceiling* of x , written $\lceil x \rceil$, is the smallest integer that is greater than or equal to x —"rounding up." In Python: `math.ceil(x)`.

Some examples:

$$\lfloor 3.7 \rfloor = 3, \quad \lceil 3.7 \rceil = 4, \quad \lfloor 5 \rfloor = 5, \quad \lceil 5 \rceil = 5, \quad \lfloor -2.3 \rfloor = -3, \quad \lceil -2.3 \rceil = -2.$$

One key property: $\lfloor x \rfloor = x$ if and only if x is an integer.¹ If x has any fractional part at all, the floor will be strictly less than x .

There's a nice connection to the quotient-remainder theorem too. For positive integers n and d , the quotient q is exactly $\lfloor n/d \rfloor$. So floor and integer division are really the same thing.

Fun fact: the floor function shows up all over computer science. Array indexing, hash functions, binary search—anywhere you need to convert a real-valued quantity into an integer index, you're probably using floor.

¹This sounds obvious but you'll need to prove it carefully on the assignment!

Sidebar: modular arithmetic, applied.

Modular arithmetic isn't just "clock math." It's everywhere in the systems you use every day:

- **Hash tables.** When you index into a dictionary or hash set, the typical implementation computes `hash(key) % num_buckets` to pick a slot. The `%` is doing real work: it compresses a giant hash value into a bucket index, and the reason collisions behave reasonably is that $(a + b) \% n$ and $(a \% n + b \% n) \% n$ give the same result—that's the homomorphism property from Module 5 applied to $\mathbb{Z} \rightarrow \mathbb{Z}/n\mathbb{Z}$.
- **Checksums and error detection.** Your credit card's last digit is a mod-10 Luhn check. ISBN-10 uses mod 11, ISBN-13 uses mod 10. ISO container numbers use mod 11. The CRC checksums on every network packet are arithmetic in a *binary* finite field $\mathbb{F}_{2^k}$. All of these exist to let the receiver check "did this get corrupted?" in constant time.
- **Cryptography.** RSA encrypts and decrypts in $\mathbb{Z}/n\mathbb{Z}$ where $n = pq$ is a product of two large primes; Diffie–Hellman lives in $(\mathbb{Z}/p\mathbb{Z})^\times$ for a large prime p ; ECDSA lives in the group of points on an elliptic curve over $\mathbb{F}_p$. Every time you open an HTTPS connection, your browser is doing hundreds of mod- p operations in milliseconds.
- **Bitwise tricks.** $x \& (n-1)$ equals $x \% n$ when n is a power of 2—which is why hash-table implementations so often choose a power-of-2 bucket count. Modular arithmetic in hardware is fundamentally cheap when the modulus is 2^k .
- **Pseudo-random numbers.** Classical linear-congruential generators compute $x_{n+1} = (ax_n + c) \bmod m$. Modern RNGs are more sophisticated, but the bones are the same: iterate in a finite group, rely on modular structure to mix bits.
- **Cyclic anything.** Day of the week from day of the year? Mod 7. Which way does a clock hand point? Mod 12 (or 24, or 60). Which frame of a looping animation are you on? Mod `frame_count`. Any time there's a cycle, there's modular arithmetic underneath.

When we say "modular arithmetic is a finite field when the modulus is prime," that abstract statement is what makes RSA possible. When we say " $\mathbb{Z}/n\mathbb{Z}$ is always a group under addition," that's the identity behind every hash table. Abstract algebra feels abstract until the moment you realize your entire software stack is running on it.

6.4 $\mathbb{Z}/n\mathbb{Z}$ is a group

In Module 5 we proved that $(\mathbb{Z}, +, 0)$ is a group. Now we'll see that modular arithmetic gives us *finite* groups—the same axioms, but with a finite set of residues instead of all the integers.

For any positive integer n , define the set of residues $\mathbb{Z}/n\mathbb{Z} = \{0, 1, \dots, n-1\}$, and define modular addition by performing ordinary addition and then taking the modulus:

$$q \oplus r = (q + r) \bmod n.$$

We'll write $\oplus$ here to distinguish it from ordinary $+$, though most texts just overload $+$ and let context disambiguate.

The axioms all work the way you'd hope: associativity comes from associativity of ordinary integer addition, the identity is still 0, and every residue has an additive inverse—namely $r =$

$(n - q) \bmod n$. Checking the inverse is a one-liner: $q \oplus (n - q) = (q + n - q) \bmod n = n \bmod n = 0$. So $(\mathbb{Z}/n\mathbb{Z}, \oplus, 0)$ is a group for every positive n .

6.5 $(\mathbb{Z}/p\mathbb{Z})^\times$ is a group when p is prime

When the modulus is prime, something **really** special happens. It isn't just a group under addition—if you exclude 0, the nonzero residues *also* form a group under multiplication. That is, $((\mathbb{Z}/p\mathbb{Z})^\times, \cdot, 1)$, with operation $q \cdot r = (qr) \bmod p$, satisfies exactly the group axioms we verified for the integers in Module 5.

Associativity, identity (1), and the closure of multiplication on nonzero residues are all straightforward. The surprising axiom is inverses: for every nonzero residue q , there's some residue x with $q \cdot x \equiv 1 \pmod{p}$. This is not obvious—why should such an x exist?—and the proof leans on a classical number-theoretic identity:

Theorem 6.5.1 (Bézout's Identity). For any integers a and b , there exist integers x and y such that $ax + by = \gcd(a, b)$.

We won't prove Bézout here—the intermezzo right after Module 7 shows how the *extended* Euclidean algorithm actually *computes* the coefficients x and y —but the intuition is worth carrying with you: the integer linear combinations $ax + by$ hit exactly the multiples of $\gcd(a, b)$ as x, y range over $\mathbb{Z}$, and when $\gcd(a, b) = 1$ that includes 1 itself. So whenever $\gcd(a, b) = 1$, the equation $ax + by = 1$ has a solution, and Bézout's identity is just naming the x, y that solve it.

We can use this immediately. If q is a nonzero residue mod p and p is prime, then $\gcd(q, p) = 1$, so Bézout gives integers x, y with $qx + py = 1$. Reducing mod p , the py term vanishes and we're left with $qx \equiv 1 \pmod{p}$. So x is the multiplicative inverse of q .

Distributivity of multiplication over addition follows from distributivity in $\mathbb{Z}$, so we won't reprove it. Putting the pieces together: $\mathbb{Z}/p\mathbb{Z}$ is a *finite field*, a set where addition works on all residues and multiplication works on the nonzero residues, both with full group structure. Finite fields are the algebraic backbone of modern cryptography, error-correcting codes, and large chunks of coding theory—see the Wikipedia articles on [finite field arithmetic](#) and the [discrete logarithm problem](#) if you want to dig further.

6.6 The Chinese Remainder Theorem

We'll close the modular-arithmetic toolkit with one more classical result. It's not strictly required for the assignment, but it's beautiful, it will come up later if you take cryptography or number theory, and it's genuinely useful.

Theorem 6.6.1 (Chinese Remainder Theorem, simple form). Let m and n be positive integers with $\gcd(m, n) = 1$ (i.e., m and n are *coprime*). Then for any residues $a \in \mathbb{Z}/m\mathbb{Z}$ and $b \in \mathbb{Z}/n\mathbb{Z}$, there is an integer x satisfying

$$x \equiv a \pmod{m} \quad \text{and} \quad x \equiv b \pmod{n},$$

and x is uniquely determined modulo mn .

Informally: if you know what an unknown integer is modulo several coprime numbers, you can reconstruct it modulo their product. The name comes from an ancient Chinese puzzle: “I have a collection of objects; if I count them in groups of 3, there are 2 left over; in groups of 5, there are 3 left over; in groups of 7, there are 2 left over. How many do I have?”

Let's solve that exact puzzle as a worked example.

Claim. Find x with $x \equiv 2 \pmod{3}$, $x \equiv 3 \pmod{5}$, $x \equiv 2 \pmod{7}$.

Worked solution. One way to solve this is to just enumerate candidates satisfying the first two congruences and filter by the third. Start from $x \equiv 2 \pmod{3}$: candidates are 2, 5, 8, 11, 14, 17, 20, 23, ... Of these, which are $\equiv 3 \pmod{5}$? Check each one: $2 \bmod 5 = 2$, $5 \bmod 5 = 0$, $8 \bmod 5 = 3$. ✓ So 8 works for the first two. And $8 + 15k$ also works for all k (since $15 = \text{lcm}(3, 5)$). The candidates for the first two constraints are 8, 23, 38, 53, 68, 83, ... Which are $\equiv 2 \pmod{7}$? Try each: $8 \bmod 7 = 1$, $23 \bmod 7 = 2$. ✓ So $x = 23$ satisfies all three constraints.

By the CRT, the solution is unique modulo $\text{lcm}(3, 5, 7) = 3 \cdot 5 \cdot 7 = 105$. (The LCM equals the product here because the three moduli are pairwise coprime; the CRT statement is always in terms of LCM, which coincides with the product precisely under that coprimality hypothesis.) So the full set of integer solutions is $\{23, 128, 233, \dots\}$ and $\{23 - 105, 23 - 210, \dots\}$, i.e., $\{23 + 105k \mid k \in \mathbb{Z}\}$.

The general recipe for two coprime moduli m, n is:

1. Use Bézout's identity to find integers u, v with $um + vn = 1$.
2. Then $x = a \cdot vn + b \cdot um$ satisfies $x \equiv a \pmod{m}$ (because $vn \equiv 1 \pmod{m}$ by Bézout and $um \equiv 0 \pmod{m}$) and similarly $x \equiv b \pmod{n}$. ✓

This recipe turns the search into a small amount of algebra, scaled up by however many moduli you're juggling.

Why does this matter? A classical application: instead of doing arithmetic on a single very large modulus N , you can do arithmetic modulo each of its prime factors $p_1, p_2, \dots$ separately, then use the CRT to glue the pieces back together. This is the basis of the RSA speedup used in real-world cryptography. Even if you never do cryptography, the pattern—"break a problem into pieces, solve each piece, combine"—is a recurring theme in computer science.

6.7 Proofs by cases and the connection to or-elimination

Proof by cases is just $\vee$ -elimination from Module 3, dressed up for number-theoretic work. If you have an exhaustive set of cases for your data—like "a number is either 0 or nonzero," or "an integer is positive, zero, or negative," or "an integer is either even or odd"—then the disjunction of those cases is a tautology, and $\vee$ -elimination lets you prove the conclusion by proving it in each case. What's new in this module is that the quotient-remainder theorem gives us a principled way to *generate* these case splits: for any modulus d , every integer lands in exactly one residue class $0, 1, \dots, d - 1$, and that's always an exhaustive disjunction.

Let's do a concrete example. We'll prove that for every integer n , the quantity $n^2 - n + 3$ is odd. The quotient-remainder theorem (with $d = 2$) tells us every integer is either even or odd, so those are our two cases.

Case 1: n is even, so $n = 2k$ for some integer k . Then

$$n^2 - n + 3 = (2k)^2 - 2k + 3 = 4k^2 - 2k + 3 = 2(2k^2 - k + 1) + 1,$$

which is odd.

Case 2: n is odd, so $n = 2k + 1$ for some integer k . Then

$$n^2 - n + 3 = (2k + 1)^2 - (2k + 1) + 3 = 4k^2 + 4k + 1 - 2k - 1 + 3 = 4k^2 + 2k + 3 = 2(2k^2 + k + 1) + 1,$$

which is odd.

So in both cases we get something of the form $2m + 1$, which is odd. Since every integer is either even or odd, we're done. Notice how the quotient-remainder theorem is doing the heavy lifting here—it's the reason we know these two cases are *exhaustive*. We don't have to worry about some integer that's neither even nor odd, because the theorem guarantees there isn't one.

6.8 Common mistakes

1. **Negative remainders.** In math, the quotient-remainder theorem forces $0 \leq r < d$, even when n is negative. So for $n = -7$, $d = 3$, the decomposition is $-7 = 3 \cdot (-3) + 2$, with remainder $r = 2$. Students (and C programmers) often write $-7 = 3 \cdot (-2) + (-1)$, which gives a negative remainder and violates the theorem. Always round the quotient *down* (toward $-\infty$), not toward zero.
2. **“Not every residue has a multiplicative inverse.”** In $\mathbb{Z}/n\mathbb{Z}$ for general n , only residues coprime to n have multiplicative inverses. In particular, 2 has no inverse in $\mathbb{Z}/6\mathbb{Z}$ (because $\gcd(2, 6) = 2 \neq 1$). Students try to invert and hit a wall; the fix is to check that the modulus is *prime*, or at least that the residue is coprime to the modulus.
3. **Confusing $a \equiv b \pmod{n}$ with $a = b \bmod n$.** These look similar but mean different things. $a \equiv b \pmod{n}$ is a relation—it says n divides $a - b$ —and can hold between any two integers. $a = b \bmod n$ is an equation where the right side is an actual number in $\{0, 1, \dots, n - 1\}$. $15 \equiv 3 \pmod{4}$ is true; $15 = 3 \bmod 4$ is also true; but $15 \equiv 7 \pmod{4}$ is also true, and $15 = 7 \bmod 4$ is false.
4. **Forgetting one of the two cases in a parity argument.** A proof “by cases on whether n is even or odd” has to handle both cases. Students sometimes dispatch the even case, write “similarly for odd,” and move on—but the odd case often has an off-by-one that the even case didn't. Write both cases out in full the first few times.
5. **Computing $\lfloor \cdot \rfloor$ on negative numbers the wrong way.** $\lfloor -3.7 \rfloor = -4$, not -3 (which is what you'd get by stripping off the fractional part). Floor rounds *down* on the number line, and -4 is below -3.7 while -3 is above it.

6.9 Summary and looking ahead

Key takeaways.

- The *quotient-remainder theorem* guarantees that every integer n decomposes uniquely as $n = dq + r$ with $0 \leq r < d$. This is what makes “case on $n \bmod d$ ” a valid proof technique.
- $(\mathbb{Z}/n\mathbb{Z}, +, 0)$ is a group for every positive n . $(\mathbb{Z}/p\mathbb{Z})^\times, \cdot, 1$ is a group iff p is prime—then and only then does every nonzero residue have a multiplicative inverse.
- The *Chinese Remainder Theorem*: coprime moduli combine. Knowing $x \bmod m$ and $x \bmod n$ (with $\gcd(m, n) = 1$) determines x uniquely modulo mn .
- *Floor* rounds down on the number line, *ceiling* rounds up. For negative numbers this is *not* the same as “drop the fractional part.”

- *Proof by cases* on modular residues is just $\vee$ -elimination where the disjunction $(n \bmod d = 0) \vee \cdots \vee (n \bmod d = d - 1)$ is exhaustive by the quotient-remainder theorem.

What we built this module. We took integer arithmetic and discovered that it's wrapped around several group structures we already understood from Module 5—addition mod n for any n , and multiplication mod p for primes. The CRT gave us a first taste of “modular reconstruction,” which is the pattern behind fast big-integer arithmetic and RSA. Proof by cases became a concrete, practical technique rather than a formal rule.

Looking ahead. Module 7 introduces the two indirect proof techniques—contradiction and contrapositive—plus the Euclidean algorithm and Hoare logic for program correctness. The *extended* Euclidean algorithm, which turns Bézout's identity into a way to actually compute the coefficients x, y (and hence modular inverses), is handled in an intermezzo just after Module 7. Before you get there, the equivalence-relation intermezzo uses modular congruence as its central example, so it's a good partner to this module.

6.10 Exercises

Organized into **warm-up** (mechanical practice with the operations), **standard** (proofs using cases on parity or modular structure), **challenge** (CRT and uniqueness arguments), and **connection** (programming and forward pointers). Solutions are in the appendix.

Warm-up

Exercise 1. Compute $\lfloor -3.7 \rfloor$, $\lceil -3.7 \rceil$, $\lfloor 4 \rfloor$, and $\lceil 4 \rceil$. (Be careful with the negative value—remember that floor rounds *down*, not toward zero.)

Exercise 2. In $\mathbb{Z}/7\mathbb{Z}$, find the additive inverse of 3. Verify your answer by checking that the sum of 3 and your proposed inverse is 0 under modular addition.

Exercise 3. In $\mathbb{Z}/5\mathbb{Z}$, find the multiplicative inverse of 3. Verify your answer by checking that $3 \cdot x \equiv 1 \pmod{5}$.

Exercise 4. Apply the quotient-remainder theorem: for each pair (n, d) below, find the unique q and r with $n = dq + r$ and $0 \leq r < d$.

(a) $(n, d) = (100, 7)$

(b) $(n, d) = (-100, 7)$

(c) $(n, d) = (23, 11)$

Exercise 5. Compute each of the following:

(a) $17 \bmod 6$

(b) $(-17) \bmod 6$ (use the math convention, not the C convention)

(c) $(5 + 4) \bmod 7$

(d) $(5 \cdot 4) \bmod 7$

Standard

Exercise 6. Prove by cases that for every integer n , the product $n(n+1)$ is even. (Case on whether n is even or odd.)

Exercise 7. Prove: for every integer x , $\lfloor x/2 \rfloor + \lceil x/2 \rceil = x$. (Hint: case on whether x is even or odd.)

Exercise 8. In $\mathbb{Z}/8\mathbb{Z}$, find the additive inverse of every element and present the answers as a table. Then check that each element paired with its inverse sums to 0 under modular addition.

Exercise 9. Build the complete multiplication table for $(\mathbb{Z}/7\mathbb{Z})^\times$ (the nonzero residues mod 7) and use it to read off the multiplicative inverse of each element. (You should get six inverses, one for each of 1, 2, 3, 4, 5, 6.)

Challenge

Exercise 10. Use the Chinese Remainder Theorem recipe to find the smallest positive integer x satisfying

$$x \equiv 1 \pmod{4}, \quad x \equiv 2 \pmod{9}, \quad x \equiv 3 \pmod{25}.$$

(You can do this by enumeration, like the worked example in the module, or by applying the CRT recipe to the first two constraints, then the result to the third.)

Exercise 11. Prove by cases that for every integer n , the product $n(n+1)(n+2)$ is divisible by 6. (Hint: show it's divisible by 2, then show it's divisible by 3, using a case split on $n \bmod 3$ for the second part.)

Exercise 12. Prove: in $\mathbb{Z}/p\mathbb{Z}$ with p prime, if $x^2 \equiv 1 \pmod{p}$ then $x \equiv 1$ or $x \equiv p-1 \pmod{p}$. (Hint: $x^2 - 1 = (x-1)(x+1)$, and in a finite field, $ab = 0$ forces $a = 0$ or $b = 0$.)

Connection

Exercise 13. (Programming.) Write a Python function `mod_inverse(a, n)` that returns the multiplicative inverse of `a` modulo `n` by brute force—enumerate $0, 1, \dots, n-1$ and return the first one that satisfies $a \cdot x \equiv 1 \pmod{n}$, or return `None` if no such x exists. Test it:

- On $a = 3$, $n = 7$ (should return 5, since $3 \cdot 5 = 15 \equiv 1 \pmod{7}$).
- On $a = 2$, $n = 6$ (should return `None`—why?).
- On every nonzero a in $\mathbb{Z}/7\mathbb{Z}$ (should find an inverse for all of them).
- On every nonzero a in $\mathbb{Z}/8\mathbb{Z}$ (some will fail; which?).

Exercise 14. (Intermezzo pointer.) The intermezzo “What Does It Mean for Things to Be ‘The Same’?” (right after this module) uses modular congruence as its main example of an equivalence relation. Without looking ahead, convince yourself that $a \equiv b \pmod{n}$ (defined as “ n divides $a-b$ ”) is reflexive, symmetric, and transitive on $\mathbb{Z}$. Write a short proof of each property.

Intermezzo: What Does It Mean for Things to Be “The Same”?

In Module 1 we defined equivalence relations—relations that are reflexive, symmetric, and transitive—and noted that equality is the canonical example. In Module 6 we met modular congruence: two integers are “the same” mod n when they leave the same remainder. We built $\mathbb{Z}/n\mathbb{Z}$ out of this idea without pausing to ask a deeper question: *what are we really doing when we declare two different things to be “the same”?*

The answer turns out to be one of the most important ideas in mathematics and computer science, and it’s worth making explicit.

6.11 Equivalence as abstraction

Every equivalence relation is a *choice* about which differences to ignore. When you write $17 \equiv 3 \pmod{7}$, you’re saying: “I don’t care about multiples of 7.” The numbers 17 and 3 are genuinely different integers, but for your purposes—working in $\mathbb{Z}/7\mathbb{Z}$ —the difference between them is irrelevant. You’ve chosen to ignore it.

This pattern is everywhere:

- **Modular arithmetic mod n :** ignore multiples of n . Two integers are “the same” if their difference is divisible by n .
- **Case-insensitive string comparison:** ignore capitalization. “Hello” and “hello” are different strings, but if you’re searching a database and don’t care about case, you declare them equivalent.
- **Graph isomorphism:** ignore vertex labels, keep structure. The graph with vertices $\{1, 2, 3\}$ and edges $\{(1, 2), (2, 3)\}$ is “the same” as the graph with vertices $\{a, b, c\}$ and edges $\{(a, b), (b, c)\}$ —they’re both paths of length 2.
- **Extensional equality of programs:** ignore implementation, keep input-output behavior. A bubble sort and a merge sort are “the same function” if you only care about what output they produce for each input.

Each of these is genuinely an equivalence relation—you can verify that each one is reflexive, symmetric, and transitive. (Try it for graph isomorphism if you want a small challenge.) And each one amounts to the same conceptual move: deciding that certain differences don’t matter for the question you’re asking.

This is precisely the idea that the mathematician Hermann Weyl championed. Weyl argued that the essence of mathematical abstraction is *identifying things that differ only in irrelevant ways*. In

his view, a mathematical concept isn't a specific thing—it's what remains after you've stripped away everything you've chosen to ignore. The number 3 isn't any particular collection of three objects (three apples, three dots, three stars). It's what *all* collections of three objects have in common. The concept “three” is an equivalence class.

6.12 Quotient sets

If every equivalence relation is a choice about what to ignore, then we should be able to form a new set that *only remembers what we chose to keep*. That's exactly what a quotient set is.

Definition 6.12.1 (Equivalence Class). Given a set A and an equivalence relation $\sim$ on A , the *equivalence class* of an element $a \in A$ is the set of all elements equivalent to a :

$$[a] = \{b \in A \mid a \sim b\}.$$

Definition 6.12.2 (Quotient Set). The *quotient set* $A/\sim$ is the set of all equivalence classes:

$$A/\sim = \{[a] \mid a \in A\}.$$

The equivalence classes partition A : every element belongs to exactly one class, and distinct classes don't overlap. (This is a theorem you can prove from the reflexivity, symmetry, and transitivity axioms—it's a good exercise.)

Now, concrete examples:

Modular arithmetic. Define $\sim$ on $\mathbb{Z}$ by $a \sim b$ iff $n \mid (a - b)$. The equivalence classes are exactly the residue classes $[0], [1], \dots, [n - 1]$. The quotient set $\mathbb{Z}/\sim$ is $\mathbb{Z}/n\mathbb{Z}$ —exactly the set we worked with in Module 6. This is not a coincidence or an analogy. The notation $\mathbb{Z}/n\mathbb{Z}$ literally means “the integers, quotiented by the equivalence relation of congruence mod n .” We were building a quotient set all along.

Case-insensitive strings. Let A be the set of all strings over the English alphabet, and define $s_1 \sim s_2$ iff s_1 and s_2 differ only in capitalization. Then $[\text{“Hello”}] = \{\text{“Hello”}, \text{“hello”}, \text{“HELLO”}, \text{“hELLO”}, \dots\}$ —all $2^5 = 32$ capitalizations of a five-letter string. The quotient set $A/\sim$ is the set of case-insensitive strings. When Python's `str.lower()` method converts a string to lowercase before comparing, it's choosing a *representative* from each equivalence class.

Constructing the integers. Here's a deeper example. Suppose you only have the natural numbers $\mathbb{N} = \{0, 1, 2, \dots\}$ and you want to *build* the integers. You can't just “add negatives”—you need to define them from what you already have. The trick: represent each integer as a *pair* (a, b) of natural numbers, where the pair is meant to stand for $a - b$. So $(5, 3)$ represents 2, and $(3, 5)$ represents -2 .

But many different pairs represent the same integer: $(5, 3)$, $(6, 4)$, $(7, 5)$, and $(100, 98)$ all represent 2. So we define an equivalence relation on $\mathbb{N} \times \mathbb{N}$:

$$(a, b) \sim (c, d) \quad \text{iff} \quad a + d = b + c.$$

(Notice the trick: we avoid writing $a - b = c - d$, which would require subtraction—the very thing we haven't defined yet. Instead we rearrange to $a + d = b + c$, which only uses addition of natural numbers.)

The integers *are* the quotient set $(\mathbb{N} \times \mathbb{N})/\sim$. The integer 2 is the equivalence class $[(5, 3)] = [(6, 4)] = [(7, 5)] = \dots$. This construction is standard in mathematics, and it's a beautiful instance of the pattern: define what you want as what remains after you've identified everything that should be “the same.”

6.13 The connection to types and programming

In programming, you make equivalence-class decisions constantly—often without realizing it.

Hash maps. When you use a dictionary in Python or a `HashMap` in Java, you’re relying on a notion of key equality. Two keys are “the same” if they produce the same hash *and* compare equal under `__eq__` or `.equals()`. The hash map doesn’t distinguish between two keys that are equal in this sense, even if they’re different objects in memory. This is an equivalence relation on the set of all possible key objects, and the hash map is organized by equivalence classes.

Database normalization. In a relational database, two rows in a table represent the same entity if their primary keys match. You might have a `users` table where the primary key is `user_id`. Two rows with the same `user_id` are “the same user” even if other columns differ (perhaps one was updated more recently). The primary key defines the equivalence relation; normalization is partly about making this choice explicit and consistent.

Interfaces and duck typing. In Python, if two objects both support `__len__` and `__getitem__`, you can use either one wherever you need a sequence. The objects might be implemented completely differently—one a list, one a database cursor—but from the perspective of code that uses the sequence interface, they’re interchangeable. “If it walks like a duck and quacks like a duck...” is an equivalence relation: two objects are equivalent if they support the same operations.

In type theory, this idea is formalized through *quotient types*: you can define a type where certain values are considered equal even though they’re constructed differently. This is the formal version of what every programmer does informally when they implement `__eq__` for a class.

And this connects back to Module 1’s discussion of extensional versus intensional equality. Extensional equality of functions—two functions are equal iff they produce the same output for every input—is a quotient. You take the set of all programs and identify those with the same input-output behavior. What remains is the set of “functions” in the mathematical sense, with implementation details erased. Every time you write a specification for a function (“it should sort the list”) and don’t care how it does it, you’re working in this quotient.

6.14 Weyl’s insight, and why it matters

Hermann Weyl (1885–1955) was one of the twentieth century’s most versatile mathematicians, making deep contributions to analysis, geometry, number theory, and mathematical physics. But some of his most lasting influence was philosophical. Weyl thought carefully about what mathematical objects *are*, and he arrived at a striking answer: we never study raw objects. We study objects *up to equivalence*.

When a geometer studies triangles, she doesn’t care about their position or orientation in the plane—she studies them up to congruence. When a topologist studies shapes, she doesn’t care about bending or stretching—she studies them up to continuous deformation. When a number theorist studies residues mod n , she doesn’t care about the particular integer—she studies them up to congruence mod n . In each case, the mathematician has *chosen an equivalence relation*, and that choice determines the level of abstraction at which the work happens.

The choice of equivalence relation is not a minor technical detail. It’s the most consequential decision you make, because it determines what your theory can see and what it’s blind to. Geometry up to congruence can see lengths and angles; geometry up to similarity can see angles but not lengths; topology can see connectedness but not angles or lengths. Each level of abstraction is useful for different questions, and each one is defined by choosing what to ignore.

This is the same insight that drives good software design. An interface defines what you care

about; everything behind the interface is an implementation detail you’re choosing to ignore. The more carefully you choose what to expose and what to hide, the more robust your abstractions become. A well-designed API is, in a precise sense, a well-chosen equivalence relation on the set of possible implementations.

So the next time you declare two things “the same”—whether in a proof, a program, or a conversation—pause and ask: what am I choosing to ignore? That choice is where the real thinking lives.

Exercise 1. Define an equivalence relation on the set of all Python programs such that two programs are equivalent if and only if they produce the same output for every input. Is this the same as checking whether the source code is identical? Why or why not? (Hint: think about what the equivalence classes look like. How many programs are in the equivalence class of a program that sorts a list?)

Exercise 2. Consider the set of fractions $\{a/b \mid a, b \in \mathbb{Z}, b \neq 0\}$. Define an equivalence relation $\sim$ that captures when two fractions represent the same rational number. (That is, give a precise condition for when $a/b \sim c/d$.) Verify that your relation is reflexive, symmetric, and transitive. What are the equivalence classes? How does this connect to the construction of the integers from pairs of naturals discussed above?

Exercise 3. (Equivalence as cardinality.) Define a relation on sets by $A \sim B$ iff there exists a bijection $f : A \rightarrow B$. Verify that $\sim$ is reflexive, symmetric, and transitive. (For symmetry, you need to invert a bijection; for transitivity, you need to compose two bijections.) What is the equivalence class of $\{a, b, c\}$? What concept does the choice of equivalence class abstract away?

Exercise 4. (Anagrams as a quotient.) Define a relation on the set of English words by: $w_1 \sim w_2$ iff w_2 can be obtained from w_1 by reordering the letters. (So “stop,” “tops,” “pots,” “opts,” and “spot” all sit in the same class.) Verify that $\sim$ is an equivalence relation. List a few elements of the equivalence class of “listen.” What computational object naturally serves as a *representative* of each class?

Exercise 5. (Equivalence classes partition the set.) Prove the following: if $\sim$ is an equivalence relation on A and $a \sim b$, then $[a] = [b]$ as sets. (You will need both directions: $[a] \subseteq [b]$ and $[b] \subseteq [a]$. Use the symmetry and transitivity axioms.) Conclude that two equivalence classes are either equal or disjoint, and hence the equivalence classes *partition* A .

Exercise 6. (Graph isomorphism.) Two simple graphs $G = (V_1, E_1)$ and $H = (V_2, E_2)$ are isomorphic if there is a bijection $\varphi : V_1 \rightarrow V_2$ such that $\{u, v\} \in E_1$ iff $\{\varphi(u), \varphi(v)\} \in E_2$. Show that graph isomorphism is an equivalence relation on the set of all simple graphs. Then exhibit two graphs on four vertices that are isomorphic and two that are not. What “information” does an isomorphism class of graphs remember, and what does it forget?

Further reading

For more on the philosophical and mathematical content of “what does it mean for two things to be the same?”:

- Barry Mazur, “[When is one thing equal to some other thing?](#)” in *Proof and Other Dilemmas: Mathematics and Philosophy* (B. Gold and R. A. Simons, eds.), Mathematical Association of

America (2008). The single best expository essay for undergraduates on equivalence, quotient, and “sameness.” Mazur writes beautifully and assumes almost nothing.

- Steve Awodey, “[Structuralism, Invariance, and Univalence](#),” *Philosophia Mathematica* **22**(1), 1–11 (2014). A short, accessible philosophical essay on what it means for two mathematical structures to be “the same” and on why equivalence (rather than literal equality) is the right notion. The closing section connects to the modern Univalence axiom in homotopy type theory.
- Paul Halmos, *Naive Set Theory*, Springer (1960; reprint 1974), Sections 7–8. Not a paper, but Halmos’s two-page treatment of equivalence relations and partitions is the canonical undergraduate reference and pairs nicely with the philosophical pieces above.

Cumulative Review 2

The point of this review

We're now halfway through the first term, and the toolkit has grown considerably. In Module 4 we added *predicate logic* and a first taste of induction on $\mathbb{N}$; in Module 5 we learned to write *informal proofs* (direct, counterexample, existence, and the group axioms); in Module 6 we introduced *modular arithmetic* and *proof by cases* on residue classes. Adding those to the set-and-propositional-logic foundation from Modules 1–3 gives us enough machinery to prove real, non-trivial theorems—the kind that appear in number theory textbooks.

In this review chapter we pick one classical result, prove it carefully, and then trace through the proof to highlight how every module's tools contribute. The result is:

Theorem 6.14.1. For every integer n , the quantity $n^3 - n$ is divisible by 6.

This is the right level of difficulty for a mid-term review: it is clearly non-trivial (you have to combine at least two separate arguments to get the factor of 6), but it is also clean enough that the proof fits on one page. Let's go.

The proof

We first rewrite the quantity in a more suggestive form:

$$n^3 - n = n(n^2 - 1) = n(n - 1)(n + 1) = (n - 1) \cdot n \cdot (n + 1).$$

So $n^3 - n$ is the product of three consecutive integers.

To show $6 \mid n^3 - n$, it suffices (by a consequence of the Chinese Remainder Theorem, since $\gcd(2, 3) = 1$) to show that both 2 and 3 divide $n^3 - n$. We'll handle each divisibility separately.

Step 1: $2 \mid n^3 - n$

Among any two consecutive integers, one is even. So among $n - 1, n, n + 1$, at least one is even—in fact, *at least* one (and possibly two) are even. Therefore the product $(n - 1) \cdot n \cdot (n + 1)$ is even, i.e., $2 \mid n^3 - n$.

If you want a more formal version of this: proceed by cases on the parity of n (the two-residue-class partition of $\mathbb{Z}$ we get from the quotient-remainder theorem mod 2):

Case 1: n is even. Then $n = 2k$ for some integer k , and $(n - 1) \cdot n \cdot (n + 1) = 2k \cdot (n - 1)(n + 1) = 2 \cdot [k(n - 1)(n + 1)]$. The bracketed quantity is an integer, so 2 divides the product.

Case 2: n is odd. Then $n - 1$ is even, so $n - 1 = 2j$ for some integer j , and the product becomes $2j \cdot n \cdot (n + 1) = 2 \cdot [j \cdot n \cdot (n + 1)]$, which is even.

Either way, the product is even.

Step 2: $3 \mid n^3 - n$

This is the more interesting half. We use *proof by cases on $n \bmod 3$* : by the quotient-remainder theorem, every integer n satisfies exactly one of $n \equiv 0, 1, 2 \pmod{3}$.

Case 1: $n \equiv 0 \pmod{3}$. Then $3 \mid n$, so $3 \mid n(n-1)(n+1)$.

Case 2: $n \equiv 1 \pmod{3}$. Then $n-1 \equiv 0 \pmod{3}$, so $3 \mid (n-1)$, so $3 \mid n(n-1)(n+1)$.

Case 3: $n \equiv 2 \pmod{3}$. Then $n+1 \equiv 0 \pmod{3}$, so $3 \mid (n+1)$, so $3 \mid n(n-1)(n+1)$.

In every case, 3 divides the product.

Combining: $6 \mid n^3 - n$

We've shown that $2 \mid n^3 - n$ and $3 \mid n^3 - n$. Since $\gcd(2, 3) = 1$, the two divisibilities combine: if integers a and b are coprime and both divide m , then ab divides m .

This is the mini-CRT fact we noted in Module 6. Here's a short direct argument: by Bézout's identity (coming in Module 7, but elementary enough to give here), since $\gcd(2, 3) = 1$ there exist integers x, y with $2x + 3y = 1$. Multiplying by m , we get $m = 2xm + 3ym$. If $2 \mid m$ (so $m = 2m'$) and $3 \mid m$ (so $m = 3m''$), then $m = 2xm + 3ym = 2x(3m'') + 3y(2m') = 6(xm'' + ym')$, which is divisible by 6.

So $6 \mid n^3 - n$ for every integer n . □

Tracing the toolkit

Now let's go back and count which modules contributed what to that proof.

- **Module 1** (sets, functions, proof-writing) gave us the disciplined prose style: state the goal, introduce variables, justify each step, conclude clearly. Every “Let $n = 2k$ for some integer k ” and every “By definition of divisibility...” is a habit from Module 1.
- **Module 2** (propositional logic) sat quietly in the background. Every case analysis we did was justified by a tautology of propositional logic—specifically, the disjunction $(n \equiv 0 \pmod{3}) \vee (n \equiv 1 \pmod{3}) \vee (n \equiv 2 \pmod{3})$, combined with $\vee$ -elimination.
- **Module 3** (proof systems) is what makes our casual use of $\vee$ -elimination rigorous. The “proof by cases” we did in Step 2 is formally an application of the $\vee$ -elimination rule from Module 3; each case branch is a subproof. The Gentzen intermezzo sketches how to draw the full derivation as a tree.
- **Module 4** (predicate logic and induction) gave us the quantifier vocabulary. “Every integer n ” is $\forall n : \mathbb{Z}$; “some integer k with $n = 2k$ ” is $\exists k : \mathbb{Z}$. When we write “let n be an arbitrary integer,” we are invoking $\forall$ -introduction. When we write “ $n = 2k$ for some integer k ,” we are using $\exists$ -elimination to extract the witness from the hypothesis “ n is even.”
- **Module 5** (direct proof, groups) provided the overall proof style: unpack the definitions (“even means $n = 2k$ ”), do the algebra, repack the result (“hence $2 \mid n^3 - n$ ”). The uniqueness arguments in Module 5 trained the habit of asking “is the thing I just chose arbitrary?”
- **Module 6** (modular arithmetic and proof by cases) is the star of this proof. The case split on $n \bmod 3$ uses the quotient-remainder theorem to guarantee the three cases are exhaustive, and the final CRT-flavored combination step uses precisely the mini-CRT we highlighted in Module 6. The entire shape of Step 2 is Module-6 shape.

Every module contributed, and dropping any one of them would make the proof harder (or would force us to use a different approach). The moral is that the modules aren't independent silos: each one adds a layer of tooling that sits on top of the previous ones, and a real proof freely draws on all of them at once.

A second worked problem: $30 \mid n^5 - n$

Since the first proof was the clean illustration of the method, let's do a variant that pushes each piece a little further. The same toolkit, applied to a case that needs a nontrivial algebraic identity at one step.

Theorem 6.14.2. For every integer n , the quantity $n^5 - n$ is divisible by 30.

Before starting, note $30 = 2 \cdot 3 \cdot 5$. These three primes are pairwise coprime, so by the same logic as Step 3 of the previous proof, it suffices to show that each of 2, 3, 5 divides $n^5 - n$ separately.

First, factor:

$$n^5 - n = n(n^4 - 1) = n(n^2 - 1)(n^2 + 1) = (n - 1) \cdot n \cdot (n + 1) \cdot (n^2 + 1).$$

The first three factors are three consecutive integers—the same setup that did all of the work last time.

Step 1: $2 \mid n^5 - n$

Among any three consecutive integers at least one is even, so $(n - 1) \cdot n \cdot (n + 1)$ is even, so the full product is even. Exactly as in Step 1 of the $n^3 - n$ proof.

Step 2: $3 \mid n^5 - n$

Again by the same argument: among $n - 1$, n , $n + 1$ at least one is divisible by 3, so the full product is divisible by 3. Identical to Step 2 of the previous proof.

Step 3: $5 \mid n^5 - n$

This is the new ingredient. Three consecutive integers can miss a multiple of 5 entirely—consider $n = 7$, so $\{6, 7, 8\}$. So we can't get away with just the $(n - 1)n(n + 1)$ part; we have to bring the $n^2 + 1$ factor in. Proceed by cases on $n \bmod 5$, using the quotient–remainder theorem to guarantee the five cases are exhaustive.

Case 1: $n \equiv 0 \pmod{5}$. Then $5 \mid n$, so 5 divides the product.

Case 2: $n \equiv 1 \pmod{5}$. Then $n - 1 \equiv 0 \pmod{5}$, so $5 \mid (n - 1)$, so 5 divides the product.

Case 3: $n \equiv 2 \pmod{5}$. Then $n^2 \equiv 4 \pmod{5}$, so $n^2 + 1 \equiv 5 \equiv 0 \pmod{5}$. So $5 \mid (n^2 + 1)$ and 5 divides the product.

Case 4: $n \equiv 3 \pmod{5}$. Then $n^2 \equiv 9 \equiv 4 \pmod{5}$, so again $n^2 + 1 \equiv 0 \pmod{5}$, and 5 divides the product.

Case 5: $n \equiv 4 \pmod{5}$. Then $n + 1 \equiv 0 \pmod{5}$, so $5 \mid (n + 1)$, so 5 divides the product.

The five cases are exhaustive and each concludes the divisibility, so $5 \mid n^5 - n$ for every integer n .

Combining: $30 \mid n^5 - n$

We have $2 \mid n^5 - n$, $3 \mid n^5 - n$, and $5 \mid n^5 - n$. The three primes 2, 3, 5 are pairwise coprime, so the same Bézout-style argument as before lets us combine them one at a time: $2 \cdot 3 = 6$ divides the quantity, then 6 and 5 are coprime so $6 \cdot 5 = 30$ divides it. Hence $30 \mid n^5 - n$ for every integer n . $\square$

What was different this time. Steps 1 and 2 came for free—they were literally the same arguments as in the first proof, because three consecutive integers always contain a multiple of 2 and a multiple of 3. The real work was in Step 3, where the three-consecutive-integers trick *doesn't* cover every residue class mod 5 and we had to bring in an extra algebraic factor, $n^2 + 1$, to mop up the two missing cases. The pattern to notice is that proofs like these grow by cases-needing-another-factor: each prime you want to handle may or may not be covered by the factors you already have, and when it isn't, you reach for another factor of the polynomial.

A harder variant. For every prime p and every integer n , $p \mid n^p - n$. This is Fermat's Little Theorem, and it generalizes both $n^3 - n$ divisible by $6 = 2 \cdot 3$ and $n^5 - n$ divisible by $30 = 2 \cdot 3 \cdot 5$. (Those are the $p = 3$ and $p = 5$ instances, combined with some leftover structure.) The standard proof uses induction (Module 8) plus the binomial theorem, and is worth revisiting once you've done Module 8—you'll see our two worked problems pop out as special cases.

A third angle: induction on n

The two proofs above used *case analysis* (Module 6) as their main engine. That's idiomatic for divisibility results because the case split on $n \bmod d$ handles both signs of n uniformly. But it's not the only tool that works. Here's an alternative proof of the $n^3 - n$ result by *induction on n* (Module 4), done for non-negative n only to keep the setup clean.

Theorem 6.14.3. For every $n \in \mathbb{N}$, $6 \mid n^3 - n$.

Proof. By induction on n .

Base case ($n = 0$): $0^3 - 0 = 0 = 6 \cdot 0$. $\checkmark$

Inductive step. Suppose $6 \mid k^3 - k$, so that $k^3 - k = 6m$ for some integer m . We need to show $6 \mid (k+1)^3 - (k+1)$. Expand:

$$\begin{aligned}(k+1)^3 - (k+1) &= k^3 + 3k^2 + 3k + 1 - k - 1 \\ &= (k^3 - k) + 3k^2 + 3k \\ &= 6m + 3k(k+1) \quad (\text{by IH}).\end{aligned}$$

Now $k(k+1)$ is the product of two consecutive integers, so it is even: write $k(k+1) = 2\ell$ for some integer ℓ (by case analysis on the parity of k , or just by noting that one of two consecutive integers is always even—exactly Step 1 of the earlier proof in miniature). Then

$$(k+1)^3 - (k+1) = 6m + 3 \cdot 2\ell = 6(m + \ell),$$

which is divisible by 6.

By induction, $6 \mid n^3 - n$ for every $n \in \mathbb{N}$. $\square$

Two things worth noting. First, the induction argument *contains a case analysis* inside it—the step where we argued $k(k+1)$ is even. You don't get to escape cases; you just relocate them. Second, for negative n you can either do a separate induction going downward, or argue that $n^3 - n$ is an odd function so the result for negative n follows from the result for positive n . (Exercise:

work out which is cleaner.) The case-split proof earlier handled both signs uniformly, which is why it's the textbook approach. Module 5's habit of "introduce an arbitrary element and unpack the definition" shows up in the inductive step: the entire calculation is one long direct proof inside the inductive case.

The takeaway: the same result can often be proved several ways, and each proof uses a different slice of the toolkit. Case analysis (Module 6) leans on the quotient-remainder theorem; induction (Module 4) leans on the recursive structure of $\mathbb{N}$; a group-theoretic proof (Module 5) might lean on the multiplicative structure of $(\mathbb{Z}/6\mathbb{Z})^\times$. When you see a divisibility result, it's worth asking which approach will be shortest—usually it's case analysis, but not always, and the habit of comparing approaches is itself worth the practice.

Where we're going

Modules 7–10 take the toolkit two more steps forward:

- Module 7 adds *indirect proof* (contradiction and contrapositive), the *Euclidean algorithm* (which makes Bézout's identity computational), and a first taste of *Hoare logic* for program correctness.
- Modules 8 and 9 revisit *induction* in a bigger way: summation identities, strong induction, recurrence solving.
- Module 10 generalizes induction to arbitrary recursive data types (*structural induction*), completing the arc that started with BNF definitions in Module 2.

After Module 10, the third cumulative review chapter ties everything together.

Chapter 7

Indirect Proof, the Euclidean Algorithm, and Hoare Logic

Reading map

Epp sections. §4.7, 4.8, 4.10 (§4.6–4.8 in the 4th edition): indirect proof (contradiction and contrapositive), the Euclidean algorithm, and an introduction to programming logic (Hoare logic).

Prerequisites. Modules 3 (proof systems), 5 (direct proof), and 6 (modular arithmetic—we finally pay off the Bézout-identity debt from Module 6).

Relevant intermezzi. “The Extended Euclidean Algorithm and Bézout Coefficients” immediately follows this module, for anyone who wants the bookkeeping we skip here (and for anyone heading into RSA in CS 251). “Proofs Are Programs” (Curry–Howard; read after this module)—the connection between Hoare logic and the Curry–Howard view of proofs is worth seeing twice.

Proof-writing reference. The companion chapter “How to Write a Proof, Revisited” follows this module directly. You’ve now met every informal proof technique this book uses; that chapter consolidates the prose conventions that go with them and is the natural warm-up before Module 8’s induction-heavy run.

Practice from Epp. §4.6 exercises on contradiction proofs (irrationality of $\sqrt{2}$ and friends); §4.7 exercises on contrapositive; any exercises involving the Euclidean algorithm. Epp’s §4.10 on program correctness is less comprehensive than the Hoare-logic treatment here, but is worth skimming for additional examples.

7.1 More on proofs by contradiction and proofs by contrapositive

Back in Module 3 we derived proof by contradiction as a tautology, starting from modus tollens and working through a bunch of substitution and simplification. That was important for understanding *why* the technique is valid, but now we need to get good at *using* it.

7.1.1 Proof by contradiction, in practice

Let’s recall the shape of proof by contradiction. You want to prove some statement P . Instead of proving it directly, you assume $\neg P$ —that is, you assume the *negation* of what you want to prove—

and then you show that this assumption leads to a contradiction. A contradiction is when you can derive both some statement Q and its negation $\neg Q$ at the same time, which obviously can't happen. Since the assumption $\neg P$ led to nonsense, you conclude that P must be true after all.

The template looks like this:

Goal: prove P .

1. Assume $\neg P$.
2. ... reasoning ...
3. Derive Q .
4. Derive $\neg Q$.
5. Contradiction; therefore P .

When should you reach for this technique? A good rule of thumb: if the statement you're trying to prove is a *negative* statement ("there is no...", "it is not the case that...", " x does not divide y ") then contradiction is often a natural fit. Why? Because assuming the negation of a negative statement gives you a *positive* assumption, which tends to be easier to work with.

Let's do a classic example.

Theorem 7.1.1. $\sqrt{2}$ is irrational.

Proof by contradiction.

Assume, for the sake of contradiction, that $\sqrt{2}$ is *rational*. Then by definition of rational, we can write $\sqrt{2} = \frac{a}{b}$ where a and b are integers with $b \neq 0$ and the fraction is in lowest terms (meaning $\gcd(a, b) = 1$ —hey, look, the stuff from the next section is already showing up).

Squaring both sides we get $2 = \frac{a^2}{b^2}$, so $a^2 = 2b^2$.

This means a^2 is even. But if a^2 is even then a must be even.¹ So we can write $a = 2c$ for some integer c .

Substituting back: $(2c)^2 = 2b^2$, so $4c^2 = 2b^2$, so $b^2 = 2c^2$. But this means b^2 is even, so b is even by the same reasoning.

Now here's the contradiction: we said the fraction $\frac{a}{b}$ was in lowest terms, meaning a and b share no common factors. But we just showed both a and b are even, so they share the common factor 2. Contradiction!

Therefore $\sqrt{2}$ is irrational.

That's a really satisfying proof and one of the oldest in all of mathematics—it goes back to the ancient Greeks, who were apparently quite disturbed by the discovery.

7.1.2 Proof by contrapositive

The other indirect technique is proof by *contrapositive*. This one is specifically for proving implications. So say you want to prove "if P then Q ," written $P \rightarrow Q$. The contrapositive of this statement is $\neg Q \rightarrow \neg P$, and it turns out these are logically equivalent!

We actually already verified this back in Module 3 when we showed that modus tollens is a tautology. But let's just check the equivalence directly with a quick truth table:

¹Why? Because if a were odd, say $a = 2k + 1$, then $a^2 = 4k^2 + 4k + 1$ which is odd. So a can't be odd, meaning it must be even. This is actually a proof by contrapositive hiding inside our proof by contradiction!

P	Q	$P \rightarrow Q$	$\neg Q \rightarrow \neg P$
T	T	T	T
T	F	F	F
F	T	T	T
F	F	T	T

Same column, so they’re equivalent. This means that whenever you want to prove $P \rightarrow Q$, you’re free to instead prove $\neg Q \rightarrow \neg P$. Sometimes this is way easier because flipping the direction and negating everything gives you better things to work with.

Here’s the classic example.

Theorem 7.1.2. For any integer n , if n^2 is even then n is even.

If we tried to prove this directly we’d start with “assume n^2 is even” and then... what? We know $n^2 = 2k$ for some integer k but it’s not obvious how to extract information about n from that.

Instead, let’s prove the contrapositive: if n is **not** even (i.e. n is odd) then n^2 is **not** even (i.e. n^2 is odd).

Proof by contrapositive.

Assume n is odd. Then $n = 2k + 1$ for some integer k . So

$$n^2 = (2k + 1)^2 = 4k^2 + 4k + 1 = 2(2k^2 + 2k) + 1$$

which is odd. See how much cleaner that was? The direct proof was stuck, but the contrapositive version just fell right out. The reason is that “ n is odd” gives us a concrete form to compute with ($n = 2k + 1$), whereas “ n^2 is even” is much harder to decompose.

7.1.3 Contradiction vs. contrapositive: when to use which

You might be wondering: when should I use contradiction and when should I use contrapositive?

Here’s my advice:

- If you’re trying to prove an *implication* ($P \rightarrow Q$) and you’re stuck on the direct proof, try the **contrapositive** first. It’s usually cleaner and more direct than a full contradiction argument.
- If you’re trying to prove a statement that *isn’t* naturally an implication—especially negative or existential statements like “there is no x such that...” or “ x is not divisible by...”—then **contradiction** is your go-to.
- If you’re not sure, contradiction always works as a fallback. You can always just assume the negation and see what breaks. The worst that happens is your proof is a little longer than it needs to be.

There’s a subtlety worth noting for those interested in constructive mathematics. Modus tollens—from $P \rightarrow Q$ and $\neg Q$, conclude $\neg P$ —is constructively valid and doesn’t require excluded middle. But “proof by contrapositive” as a *technique*—proving $\neg Q \rightarrow \neg P$ in order to establish $P \rightarrow Q$ —relies on the equivalence between an implication and its contrapositive, which does require excluded middle. So strictly speaking, both contradiction and contrapositive are classical techniques. However, contrapositive proofs tend to be more transparent and informative, which is why they’re generally preferred when available. In proof assistants like Lean, Agda, or Coq, you’d need to explicitly invoke a classical axiom for either technique, but a contrapositive proof usually

requires it in only one place (at the very end, to flip the direction) while a contradiction proof may obscure the structure more thoroughly.

In your homework this week you'll get to practice both techniques. One tip for the contradiction proofs: when you assume the negation, make sure you actually *use* the assumption somewhere to derive the contradiction. If your proof never references the assumption, something has gone wrong—you've probably actually found a direct proof in disguise!

7.2 GCD, Euclid's algorithm, &c.

Last module we owed a debt: in §6.5 we claimed that multiplicative inverses exist in $\mathbb{Z}/p\mathbb{Z}$ when p is prime, leaning on Bézout's identity without showing how to actually *compute* those coefficients. This section pays most of that debt; the *extended* Euclidean algorithm that actually computes the coefficients is handled in the intermezzo immediately after this module, because it's more bookkeeping than idea, and it doesn't really earn its keep until you want to do RSA (a CS 251 topic).

Definition 7.2.1 (Greatest Common Divisor). For integers a and b that are not both zero, the *greatest common divisor* of a and b , denoted $\gcd(a, b)$, is the greatest *positive* integer that divides both a and b .

For example: $\gcd(12, 24) = 12$, $\gcd(21, 35) = 7$, $\gcd(1024, 1536) = 512$, and $\gcd(18, 0) = 18$. That last one is worth a sentence—more generally, if $a \neq 0$ then $\gcd(a, 0) = |a|$, because every integer divides 0, so the common divisors of a and 0 are exactly the divisors of a , and the greatest positive one is $|a|$. (We leave $\gcd(0, 0)$ undefined.)

How do you *calculate* the gcd when it isn't obvious? The classic solution is *Euclid's algorithm*, based on the observation that if $a = nb + r$ (the quotient-remainder decomposition from Module 6) then $\gcd(a, b) = \gcd(b, r)$ —or, written using the remainder operator, $\gcd(a, b) = \gcd(b, a \bmod b)$. Keep doing that until the right-hand argument is 0; if it ever hits 1, we already know the gcd is 1 and can stop early. Here's a quick Python implementation:

```
def gcd(a, b):
    if b == 1:
        return 1
    elif b == 0:
        return abs(a)
    else:
        return gcd(b, a % b)
```

That's enough to *decide* whether two numbers are coprime, which is the main thing we need for Module 6's claims about inverses: a has an inverse mod n exactly when $\gcd(a, n) = 1$, and now we can check that mechanically. What Euclid's algorithm doesn't do on its own is hand you the inverse. For that we need the *extended* Euclidean algorithm, which tracks a pair of "Bézout coefficients" (x, y) alongside the running gcd and maintains the invariant $ax + by = \gcd(a, b)$. Once you have those, the modular inverse falls out by reducing the Bézout equation mod n . The algorithm itself, a worked trace, and the inverse-extraction step live in the intermezzo after this module. If you're curious about how the bookkeeping actually works, or if you're looking ahead toward RSA, read it now; otherwise it's fine to move on and come back when you need it.

7.3 Programming logic

Up to this point we’ve been using logic to reason about mathematics. But logic is a tool, and one of the most practically important places CS applies it is reasoning about *code*—proving that a program does what its specification says it does. The logic we’re about to meet is called [Hoare logic](#), invented by Tony Hoare in 1969. It’s a fully-fledged logic with its own axioms, inference rules, syntax, and semantics, but its vocabulary is entirely about programs: variables, assignments, loops, conditionals. Epp §4.10 covers this material more informally; what follows here is the formal version.

Every statement in Hoare logic is a *Hoare triple*:

$$\{P\} C \{Q\}.$$

Here P is a proposition about the state of the program *before* C runs (the values of variables, output buffers, etc.), C is the code—a sequence of statements—and Q is a proposition about the state of the program *after* C has finished.

Think of $\{P\} C \{Q\}$ as an implication with a computational step in the middle: if P holds before C runs, then Q holds afterward.

Sidebar: formal verification, for real.

Hoare logic looks academic, but it (or its descendants) is behind some of the most trust-sensitive software in the world. A few places where “ $\{P\} C \{Q\}$ ” turns into real engineering:

- **seL4**, an operating-system microkernel, is fully machine-verified against its specification using Isabelle/HOL. It’s the same microkernel running on the avionics in some military helicopters, in high-assurance routers, and in the secure enclaves of various consumer devices.
- **CompCert**, a verified C compiler developed at Inria, proves that the compiled machine code has the same behavior as the source C program. It’s used in safety-critical avionics systems where “did my compiler introduce a bug?” is a question you can’t afford to answer with a shrug.
- **AWS s2n-tls**, Amazon’s TLS implementation, is continuously checked with SAW, Cryptol, and a symbolic execution pipeline built around Hoare-logic-style specifications. Same for parts of the Linux kernel’s BPF verifier, which does Hoare-style reasoning to decide whether an untrusted program is safe to run inside the kernel.
- **Dafny**, **Frama-C**, **Viper**, and **SPARK Ada** are production verifiers where engineers write pre- and postconditions alongside their code and an SMT solver discharges the proof obligations. Amazon uses Dafny for pieces of AWS; Airbus uses SPARK for flight-control software; Mozilla has used Frama-C on parts of NSS.
- **Separation logic**, a descendant of Hoare logic developed in the 2000s (John Reynolds, Peter O’Hearn, et al.), is the reasoning principle behind Facebook’s Infer static analyzer, which finds null-pointer bugs and memory leaks in large C, Java, and Objective-C codebases by checking Hoare-style specs.

The common thread: Hoare logic turns “the program is correct” from a gut feeling into a proof obligation that a machine can (sometimes, with help) discharge. What you’re about to learn—pre-/postconditions, assignment as substitution, loop invariants—is the conceptual basis for all of the above. The production tools scale it up with automation and additional reasoning principles, but the bones are what’s in this section.

The programming language we’ll use is a simple imperative language, with just a few kinds of statements

Assignment:

```
x := e
where e is a valid arithmetic expression (one that uses +, -, *, /, numbers, or variables
)
```

Sequencing:

```
c1 ; c2
where c1 and c2 are both valid statements
```

Conditionals:

```
if b then c1 else c2
where b is a boolean-valued expression built from comparison operators (>, >=, =, /=, etc
.) and variables,
and c1, c2 are other valid statements in the language
```

While:

```
while b do c
where b is a boolean-valued expression like above, and c is a valid statement
```

Skip:

```
skip
it’s a statement that does nothing
```

This is a deliberately stripped-down language—just enough to explain the basic concepts of proving code correct.

Now here’s our basic rules:

Skip:

$$\frac{}{\text{shows } \{P\} \text{ skip } \{P\}}$$

This rule tells us that **skip** changes nothing. If the proposition P was true before, it’s true after **skip**.

Sequencing:

$$\frac{\text{assumes } \{P\} c_1 \{Q\} \quad \{Q\} c_2 \{R\}}{\text{shows } \{P\} c_1; c_2 \{R\}}$$

This rule tells us that if c_1 takes us from P to Q , and c_2 takes us from Q to R , then we can glue them together with sequencing. This is just transitivity— $P \rightarrow Q$ and $Q \rightarrow R$ give $P \rightarrow R$ —but with the code steps wired in between.

Conditionals:

$$\frac{\text{assumes } \{P \wedge B\} c_1 \{Q\} \quad \{P \wedge \neg B\} c_2 \{Q\}}{\text{shows } \{P\} \text{ if } B \text{ then } c_1 \text{ else } c_2 \{Q\}}$$

This tells us that if Q is true after c_1 runs when P and B are true **and** Q is true after c_2 runs when P is true and B is false, then P implies Q when the conditional statement is run. The way to think of this is that if you take the “true” branch you’re not just assuming P is true but you’re also allowed to assume B is true **because you took the true branch**, and conversely for when you take the false branch.

While:

$$\frac{\text{assumes } \{P \wedge B\} c \{P\}}{\text{shows } \{P\} \text{ while } B \text{ do } c \{P \wedge \neg B\}}$$

The idea is to identify a *loop invariant*—a property preserved by each iteration of the loop body. You show that if the invariant and the loop condition both hold before the body executes, the invariant still holds afterward. When the loop terminates, you know the invariant holds *and* the loop condition is false.

A small but important caveat up front: the rule as stated only proves *partial correctness*—“if the loop terminates, the postcondition holds.” It says nothing about whether the loop actually terminates. For full *total* correctness you need to additionally show that some quantity (called a *variant*) strictly decreases on each iteration and is bounded below, which forces the loop to halt after finitely many steps. Epp and most intro courses stop at partial correctness, and we will too, but keep in mind that an infinite loop trivially satisfies any partial-correctness specification—the rule doesn’t catch “my program is correct except it never finishes.”

The next one is definitely the weirdest, and one that we need to explain a bit.

Assignment:

$$\frac{}{\text{shows } \{P[x/e]\} x := e \{P\}}$$

Here’s the thing to keep pinned to the top of your mind: **the assignment rule is applied backward**. You start with the postcondition P (what you *want* to be true after the assignment) and compute the precondition $P[x/e]$ (what *must* have been true before) by substituting the expression e for the variable x everywhere in P . Every other rule in this module chains facts forward through the program; the assignment rule goes the other direction. It takes getting used to.

Let’s look at an example because it really does make more sense in practice than it does stated abstractly.

Suppose we want to conclude that, after subtracting 1 from x , we have $x > 0$:

{??} $x := x - 1$ { $x > 0$ }

What’s the precondition have to be? Reasoning informally: for the postcondition $x > 0$ to hold *after* the assignment, we need $(x - 1) > 0$ to hold *before* the assignment—that’s just the old value of x being at least 2. Reasoning by the rule: take the postcondition $x > 0$ and substitute $x - 1$ for x , getting $(x - 1) > 0$. Same answer.

So the assignment rule lets us write things like this

{ $x - 1 > 0$ } $x := x - 1$ { $x > 0$ }

The assignment rule as written isn’t super useful on its own — it’s too specific! Like it can’t let us conclude an obvious fact like

{ $x > 0$ } $x := x + 1$ { $x > 0$ }

in other words, that if x is greater than zero incrementing it will keep it greater than zero—exactly the kind of thing we might need to build a loop invariant.

Thankfully, there’s one last rule that’s super useful that helps us glue all of these bits and pieces together:

Strengthening / Weakening:

$$\frac{\begin{array}{c} \text{assumes } \{P_2\} c \{Q_2\} \\ P_1 \rightarrow P_2 \\ Q_2 \rightarrow Q_1 \end{array}}{\text{shows } \{P_1\} c \{Q_1\}}$$

Read it carefully: if we know that $\{P_2\} c \{Q_2\}$, in other words that P_2 implies Q_2 **after** running the code c , and we know that $P_1 \rightarrow P_2$, and we know that $Q_2 \rightarrow Q_1$, we're allowed to glue all these facts together to make the new fact that P_1 implies Q_1 after c is run. Why? Because we can take the P_1 and turn it into a P_2 , since $P_1 \rightarrow P_2$, and then we can take the resulting Q_2 and turn it into a Q_1 since $Q_2 \rightarrow Q_1$.

Another way of thinking of it is that you can always make the pre-condition *stronger* and the post-condition *weaker*.

So in our above example we'd have that the assignment rule gives us

$\{x + 1 > 0\} \ x := x + 1 \ \{x > 0\}$

but since $x > 0$ implies $x + 1 > 0$ we can use our new rule to turn this into

$\{x > 0\} \ x := x + 1 \ \{x > 0\}$

which is what we wanted in the first place!

7.3.1 Putting it all together

Now that we have all the rules on the table, let's do a complete worked example that uses several of them in combination. We'll prove the following Hoare triple:

$\{x = 5\} \ x := x - 1; \ x := x - 1 \ \{x = 3\}$

In other words: if x starts at 5 and we subtract 1 twice, then x ends up at 3. Not exactly a shock, but the point is to see how the formal machinery works end to end.

We'll work backwards from the postcondition, which is generally the right strategy when the assignment rule is involved. The assignment rule tells us what the precondition *must* be for a given assignment and postcondition, so starting at the end and peeling off one assignment at a time is the natural approach. To keep things straight we'll label our two intermediate triples by *program order*—Step A for the first assignment and Step B for the second assignment—even though we'll actually *derive* Step B first and then Step A. That way, when we chain them together in the sequencing step, “Step A” and “Step B” line up with the left-to-right order of the program.

Step B (derived first): Apply the assignment rule to the second assignment.

We want to end up with $\{x = 3\}$ after executing $x := x - 1$. The assignment rule says $\{P[x/e]\} x := e \{P\}$. Here P is $x = 3$ and e is $x - 1$, so $P[x/(x - 1)]$ means we substitute $x - 1$ for x in $x = 3$, giving us $x - 1 = 3$. That produces:

$\{x - 1 = 3\} \ x := x - 1 \ \{x = 3\} \text{ [assignment]}$

So before the second assignment runs, we need $x - 1 = 3$ (equivalently, $x = 4$) to hold.

Step A (derived second): Apply the assignment rule to the first assignment.

Now we need $\{x = 4\}$ to hold after the first $x := x - 1$ (so that Step B has what it needs). $x - 1 = 3$ and $x = 4$ are logically equivalent, so we can use either form; let's use $x = 4$ since it's a bit cleaner. Applying the assignment rule again with P being $x = 4$ and e being $x - 1$:

$P[x/(x - 1)]$ gives us $x - 1 = 4$. So:

$$\{x - 1 = 4\} \ x := x - 1 \ \{x = 4\} \text{ [assignment]}$$

Step 3: Chain the two assignments together with sequencing.

The sequencing rule says:

$$\frac{\text{assumes } \begin{array}{l} \{P\} c_1 \{Q\} \\ \{Q\} c_2 \{R\} \end{array}}{\text{shows } \{P\} c_1; c_2 \{R\}}$$

Our two triples from Steps A and B fit this pattern exactly, with $P = (x - 1 = 4)$, $Q = (x = 4)$, c_1 being the first $x := x - 1$, c_2 being the second $x := x - 1$, and $R = (x = 3)$. But we need the postcondition of Step A to match the precondition of Step B. The postcondition of Step A is $x = 4$ and the precondition of Step B is $x - 1 = 3$, which is the same thing (since $x - 1 = 3$ iff $x = 4$). So sequencing gives us:

$$\{x - 1 = 4\} \ x := x - 1; \ x := x - 1 \ \{x = 3\} \text{ [sequencing]}$$

Step 4: Use strengthening to connect to the stated precondition.

We're almost done, but notice the precondition we derived is $x - 1 = 4$ (i.e. $x = 5$), and the precondition we *want* is $x = 5$. These are logically equivalent, so in particular $x = 5 \rightarrow x - 1 = 4$. We can use the strengthening/weakening rule:

$$\frac{\text{assumes } \begin{array}{l} \{P_2\} c \{Q_2\} \\ P_1 \rightarrow P_2 \\ Q_2 \rightarrow Q_1 \end{array}}{\text{shows } \{P_1\} c \{Q_1\}}$$

with $P_1 = (x = 5)$, $P_2 = (x - 1 = 4)$, $Q_1 = Q_2 = (x = 3)$, and c being the full program. The implication $x = 5 \rightarrow x - 1 = 4$ holds by basic arithmetic, and $x = 3 \rightarrow x = 3$ is trivially true. So we conclude:

$$\{x = 5\} \ x := x - 1; \ x := x - 1 \ \{x = 3\} \text{ [strengthening/weakening]}$$

And that's the complete proof! Let's collect the whole thing in one place for reference:

- (1) $\{x - 1 = 3\} \ x := x - 1 \ \{x = 3\} \text{ [assignment]}$
- (2) $\{x - 1 = 4\} \ x := x - 1 \ \{x = 4\} \text{ [assignment]}$
- (3) $\{x - 1 = 4\} \ x := x - 1; \ x := x - 1 \ \{x = 3\} \text{ [sequencing, (2)+(1)]}$
- (4) $x = 5 \rightarrow x - 1 = 4 \text{ [arithmetic]}$
- (5) $\{x = 5\} \ x := x - 1; \ x := x - 1 \ \{x = 3\} \text{ [strengthening, (4)+(3)]}$

Notice the overall pattern: we used the assignment rule twice (working backwards from the postcondition), glued the results together with sequencing, and then used strengthening to hook up the precondition we actually wanted. This is the same pattern you'll use for the homework problem.

7.3.2 A worked while-loop proof

Straight-line code is the easy case for Hoare logic. The *real* test is a loop, because loops are where programs can actually go wrong in interesting ways. Let's do a full worked example with a while loop so you see how the invariant machinery pays off.

Our program computes the sum $0 + 1 + 2 + \dots + n$ by counting up from 0:

```

{n >= 0}
i := 0;
s := 0;
while i < n do
  i := i + 1;
  s := s + i
{s = n*(n+1)/2}

```

We want to prove that this program really does leave s equal to $n(n+1)/2$, the usual closed form for the first n positive integers.

The hard part of a while-loop proof is choosing the *loop invariant*: a proposition that holds every time control reaches the top of the loop. Our invariant is going to be

$$\text{Inv} := (0 \leq i \leq n) \wedge (s = i(i+1)/2).$$

Read it as: the loop variable i is somewhere between 0 and n , and the running sum s already equals the sum of the first i positive integers. That's the useful fact we want to carry through the loop.

Why these two conjuncts specifically? The second conjunct— $s = i(i+1)/2$ —is the property we actually care about; it's what will give us the postcondition at loop exit. The first conjunct— $0 \leq i \leq n$ —is there to keep i in bounds, so that “the loop has run exactly i times and is about to either run once more or stop” is a coherent thing to say. It's also what lets us claim $i = n$ exactly when the loop exits. Neither conjunct is useless and neither is redundant: drop the first and the arithmetic is unanchored; drop the second and the loop exit tells you nothing about s . A good loop invariant almost always has at least one “bookkeeping” conjunct pinning down the state variables and one “result” conjunct carrying the property you care about.

The proof has three parts, which is typical for while-loop proofs:

1. **Establishment:** after the initialization ($i := 0$; $s := 0$), the invariant holds.
2. **Preservation:** if the invariant and the loop condition both hold before one execution of the body, the invariant still holds after.
3. **Termination:** when the loop exits, the invariant plus the negated loop condition imply the desired postcondition.

Part 1: Establishment. After $i := 0$; $s := 0$ we have $i = 0$ and $s = 0$. Then:

- $0 \leq i = 0 \leq n$ (using the precondition $n \geq 0$). ✓
- $s = 0$ and $i(i+1)/2 = 0 \cdot 1/2 = 0$, so $s = i(i+1)/2$. ✓

So Inv holds after the initialization. Formally this is two applications of the assignment rule plus strengthening, but we'll summarize as:

```

{n >= 0} i := 0; s := 0 {Inv} [assignment x2 + strengthening]

```

Part 2: Preservation. We need to show

```

{Inv ^ (i < n)} i := i + 1; s := s + i {Inv}

```

We work backward from the postcondition, applying the assignment rule twice.

Step 2a. Apply the assignment rule to $s := s + i$. The postcondition is Inv , i.e., $(0 \leq i \leq n) \wedge (s = i(i+1)/2)$. Substituting $s + i$ for s in the postcondition gives

$$(0 \leq i \leq n) \wedge (s + i = i(i+1)/2).$$

Call this *Mid*. So:

$\{\text{Mid}\} \text{ s } := \text{ s } + \text{ i } \{\text{Inv}\} [\text{assignment}]$

Step 2b. Apply the assignment rule to $\text{i} := \text{i} + 1$. The postcondition is now Mid, and we substitute $i + 1$ for i throughout:

$$(0 \leq i + 1 \leq n) \wedge (s + (i + 1) = (i + 1)((i + 1) + 1)/2).$$

Simplify: $0 \leq i + 1 \leq n$ becomes $-1 \leq i \leq n - 1$, which combined with $i \geq 0$ (from the invariant before the body ran) gives $0 \leq i \leq n - 1$, i.e., $i < n$. And the arithmetic equation simplifies as follows:

$$\begin{aligned} s + (i + 1) &= (i + 1)(i + 2)/2 \\ s &= (i + 1)(i + 2)/2 - (i + 1) \\ &= (i + 1)((i + 2)/2 - 1) \\ &= (i + 1) \cdot i/2 \\ &= i(i + 1)/2. \end{aligned}$$

So the required precondition for the body is

$$(0 \leq i \leq n - 1) \wedge (s = i(i + 1)/2),$$

which is exactly Inv combined with $i < n$ —that is, $\text{Inv} \wedge (i < n)$. (Recall $0 \leq i \leq n$ from the invariant and $i < n$ from the loop guard give $0 \leq i \leq n - 1$.) ✓

By sequencing and strengthening we get:

$\{\text{Inv} \wedge (i < n)\} \text{ i } := \text{ i } + 1; \text{ s } := \text{ s } + \text{ i } \{\text{Inv}\} [\text{sequencing} + \text{strengthening}]$

That's the preservation step.

Part 3: Apply the while rule and check termination. The while rule says: if $\{\text{Inv} \wedge B\} c \{\text{Inv}\}$, then $\{\text{Inv}\} \text{ while } B \text{ do } c \{\text{Inv} \wedge \neg B\}$. In our case B is $i < n$ and we just proved $\{\text{Inv} \wedge (i < n)\} c \{\text{Inv}\}$, so:

$\{\text{Inv}\} \text{ while } i < n \text{ do } (i := i + 1; s := s + i) \{\text{Inv} \wedge \neg(i < n)\} [\text{while}]$

Now “ $\text{Inv} \wedge \neg(i < n)$ ” unpacks to $(0 \leq i \leq n) \wedge (s = i(i + 1)/2) \wedge (i \geq n)$. Combining $i \leq n$ and $i \geq n$ forces $i = n$, and then $s = i(i + 1)/2 = n(n + 1)/2$. So the postcondition implies $s = n(n + 1)/2$, which is what we wanted.

Finally, chain everything together with sequencing:

$\{n \geq 0\} \text{ i } := 0; \text{ s } := 0; \text{ while } i < n \text{ do } (\dots) \{s = n*(n+1)/2\}$

A note on termination for this specific example. Recall from the while rule discussion that the rule only gives us partial correctness—it doesn't promise the loop halts. For this program, $n - i$ is a good variant: it starts at n , decreases by 1 each iteration, and can't go below 0 while the loop guard $i < n$ still holds. So after at most n iterations, the loop exits, and we actually have total correctness here. It's worth getting into the habit of pausing after a Hoare-logic proof and asking “okay, but does it terminate?”—often the answer is obviously yes, but noticing that is part of the discipline.

That was a lot, and it's meant as an introduction to how you might prove properties about programs—not something you need to completely absorb right now. In your homework for the week you're going to prove the exciting theorem that $\{x = 0\} x := x + 1; x := x + 1 \{x = 2\}$ or, in other words, that $1 + 1 = 2$.

7.4 Common mistakes

1. **Contradiction without using the assumption.** In a proof by contradiction you assume $\neg P$, derive a contradiction, and conclude P . If the derivation of the contradiction never actually references the assumption $\neg P$, you’ve proved P *directly*, not by contradiction—and you probably should have written a direct proof instead. Watch for this on your own work.
2. **“Same form, different meaning.”** In the $\sqrt{2}$ -is-irrational proof, a student might write $a^2 = 2b^2$ and conclude “ a is even.” The right reasoning chain is a^2 is even $\Rightarrow a$ is even (a separate lemma that itself requires a short proof by cases on the parity of a). Writing “ a is even” without justifying the intermediate “ a^2 is even implies a is even” is skipping a step.
3. **Using proof by contrapositive on the wrong direction.** To prove $P \rightarrow Q$ by contrapositive, you prove $\neg Q \rightarrow \neg P$. Not $\neg P \rightarrow \neg Q$ —that’s the *inverse*, which is not equivalent. Mnemonic: swap *and* negate.
4. **Forgetting that the assignment rule substitutes in the postcondition, not the precondition.** The rule is $\{P[x/e]\} x := e \{P\}$: the postcondition P is what you start with, and you substitute e for x inside P to get the required precondition. Students sometimes try to substitute in the precondition instead, which gives nonsense.
5. **Applying the assignment rule forward.** The assignment rule tells you what precondition is required for a given postcondition—it’s a *backward* rule. You can try to use it forward (“here’s my precondition, what’s the postcondition?”), but that rarely gives you what you want, because the naive forward version produces too-strong postconditions. Always apply it backward.
6. **Loop invariant is too weak.** The invariant has to be strong enough that, combined with $\neg B$ at loop exit, it implies the desired postcondition. An invariant that only says “ $0 \leq i \leq n$ ” is true but doesn’t mention s , and so can’t prove anything about s ’s final value. If your invariant doesn’t *include* the thing you care about, it isn’t doing its job.
7. **Loop invariant is too strong.** At the other extreme, an invariant that includes facts that aren’t actually preserved will fail the preservation step. If you can’t prove $\{Inv \wedge B\} c \{Inv\}$, the invariant was too ambitious and needs to be relaxed.
8. **Forgetting the base case of induction inside a Hoare proof.** When your invariant holds at the start (establishment), that *is* the base case of an induction on how many times the loop has executed. If you don’t verify establishment, you haven’t even started the induction.

7.5 Summary and looking ahead

Key takeaways.

- A *proof by contradiction* assumes $\neg P$, derives $\perp$, and concludes P . Good for proving negative statements (“there is no ...,” “ $\sqrt{2}$ is not rational”).
- A *proof by contrapositive* shows $\neg Q \rightarrow \neg P$ in place of $P \rightarrow Q$. Usually cleaner than contradiction when the goal is an implication.
- The *Euclidean algorithm* computes $\text{gcd}(a, b)$ by repeatedly replacing (a, b) with $(b, a \bmod b)$.

The *extended* version (in the intermezzo after this module) additionally produces Bézout coefficients x, y with $ax + by = \gcd(a, b)$, which is how you compute modular inverses.

- *Hoare logic* reasons about imperative programs via triples $\{P\} c \{Q\}$: if P holds before c runs, then Q holds after. Each language construct has its own proof rule, and the assignment rule is applied *backward*—start from the postcondition and substitute to find the precondition.
- A *loop invariant* is the induction hypothesis for a while loop. It must hold on entry (establishment), be preserved by the body (preservation), and imply the postcondition on exit (termination).

What we built this module. We finished the “how to write a proof” thread that started in Module 1 and grew through Modules 5 and 6. You now have direct proof, counterexample, existence, proof by cases, proof by contradiction, and proof by contrapositive—essentially the full toolkit of informal mathematics. The Hoare-logic material pivoted to the computer-science payoff: the same disciplined case-by-case reasoning we’ve been doing all term can be applied to programs, giving formal guarantees about what code does.

Looking ahead. Modules 8–10 pick up the induction thread. Module 8 revisits natural induction with the full sum-notation toolkit, Module 9 introduces strong induction and the characteristic-equation method for recurrences, and Module 10 generalizes induction to arbitrary recursive data types (structural induction). The intermezzi around here—especially “Proofs Are Programs” (Curry–Howard) and “Other Logics, Other Worlds”—are the payoff for the proof-techniques thread and are worth reading before you start Module 8.

7.6 Exercises

Organized into **warm-up** (mechanical practice), **standard** (indirect-proof and single-Hoare-rule problems), **challenge** (longer proofs, including a while-loop Hoare argument), and **connection** (programming and forward pointers). Solutions are in the appendix.

Warm-up

Exercise 1. Prove by contrapositive: for any integer n , if n^3 is even then n is even. (Hint: the structure is very similar to the n^2 proof above. Start by assuming n is odd and show that n^3 is odd.)

Exercise 2. Trace the Euclidean algorithm on $\gcd(48, 18)$, showing each step. That is, write out each call $\gcd(a, b)$ and the corresponding values of $a \% b$ until you reach the base case. What is $\gcd(48, 18)$?

Exercise 3. Prove the Hoare triple $\{x = 0\} x := x + 3 \{x = 3\}$ using the assignment rule. (This is a single assignment, so you don’t need sequencing—just apply the assignment rule and then use strengthening to connect the precondition you get to the one you want.)

Exercise 4. Apply the assignment rule backward: for each of the following, find the precondition that the assignment rule gives.

- (a) $\{?\} x := x + 5 \{x > 10\}$

- (b) $\{?\} y := 2y \{y = 8\}$
- (c) $\{?\} z := x - y \{z = 0\}$

Standard

Exercise 5. Prove by contradiction: $\sqrt{3}$ is irrational. (Follow the same template as the $\sqrt{2}$ proof, but use the fact that a^2 is divisible by 3 implies a is divisible by 3, which itself follows from a case analysis on $a \bmod 3$.)

Exercise 6. Find the error in the following “proof” by contradiction that $\sqrt{4}$ is irrational:

“Assume for contradiction that $\sqrt{4}$ is rational. Then $\sqrt{4} = a/b$ in lowest terms. Squaring: $4 = a^2/b^2$, so $a^2 = 4b^2$. This means a^2 is even, so a is even. Write $a = 2c$. Then $4c^2 = 4b^2$, so $c^2 = b^2$, so $c = b$. But then $a = 2b$ and the fraction $a/b = 2$, so $\sqrt{4} = 2$. Contradiction!”

Where exactly does the reasoning go wrong? (Hint: the claim being “disproved” is that $\sqrt{4}$ is irrational—which is false, since $\sqrt{4} = 2$ is rational. So any apparent proof that $\sqrt{4}$ is irrational *must* contain a bad inference somewhere. Your job is to locate the exact step where the writer calls something a contradiction that actually isn’t one. Don’t pattern-match on the $\sqrt{2}$ proof—read line by line.)

Exercise 7. Prove the Hoare triple $\{x = 0 \wedge y = 0\} x := x + 1; y := y + 1 \{x + y = 2\}$. Use the assignment rule twice (once for each assignment), chain them with sequencing, and finish with strengthening.

Exercise 8. (Extended Euclidean exercises are in the intermezzo that follows.) For practice with the basic Euclidean algorithm, trace $\text{gcd}(252, 105)$ step by step, showing each call and remainder. What’s the gcd?

Challenge

Exercise 9. Prove by contradiction: there is no largest prime number. (Hint: suppose there is a largest prime p and consider the number $2 \cdot 3 \cdot 5 \cdot 7 \cdots p + 1$, the product of all primes up to p plus one. What can you say about this number?)

Exercise 10. Give a complete Hoare-logic proof of partial correctness for the following program, which computes 2^n into the variable y :

```
{n >= 0}
y := 1;
i := 0;
while i < n do
  y := 2 * y;
  i := i + 1
{y = 2^n}
```

Choose an appropriate loop invariant (hint: it should say something about y in terms of i and n), then carry out the three steps—establishment, preservation, termination—following the worked example in the module. You do not need to prove total correctness, but note what variant would work for it.

Exercise 11. Prove: for every pair of positive integers a and b , $\text{gcd}(a, b) = \text{gcd}(b, a \bmod b)$ (assuming $b \neq 0$). This is the correctness lemma for Euclid’s algorithm. (Hint: show that the set

of common divisors of $\{a, b\}$ equals the set of common divisors of $\{b, a \bmod b\}$, then conclude that the greatest elements of the two sets are the same.)

Connection

Exercise 12. (Programming.) Implement the basic `gcd(a, b)` recursively in Python, matching the pseudocode in §7.2. Test it on the pairs (48, 18), (252, 105), (1024, 1536) and one pair of your own choice. Then: if you’re curious about the extended version that also returns Bézout coefficients, the intermezzo that follows this module has an implementation and a programming exercise of its own.

Exercise 13. (Intermezzo pointer.) The “Proofs Are Programs” intermezzo (Curry–Howard) immediately follows this module. Hoare logic and Curry–Howard are two very different ways of connecting proofs and programs: Hoare logic is about *verifying that a given imperative program meets a specification*, while Curry–Howard says that *a proof in constructive logic **is** a program*. Read the intermezzo and, in a short paragraph, explain which of these two correspondences is more natural for reasoning about the merge-sort algorithm, and why.

Intermezzo: The Extended Euclidean Algorithm and Bézout Coefficients

Module 7 introduced Euclid’s algorithm as a way to compute $\gcd(a, b)$. This intermezzo handles the *extended* version—the one that doesn’t just report $\gcd(a, b)$ but simultaneously produces integers x and y satisfying

$$ax + by = \gcd(a, b).$$

That pair (x, y) is called a set of *Bézout coefficients* for a and b . We met Bézout’s identity back in Module 6 and used it to assert that multiplicative inverses exist in $\mathbb{Z}/p\mathbb{Z}$; this intermezzo is where we actually *compute* them.

We’re treating this as a sidetrip for a reason. The basic Euclidean algorithm is a proof-techniques topic—it’s a clean test case for indirect reasoning and correctness proofs about recursive programs, which is why it lives in Module 7 proper. The *extended* version is mostly bookkeeping on top of that, and it doesn’t really pay for itself until you want to do something with those coefficients. The two places that happens are in modular inverses (we’ll see a little here) and in RSA key generation (which is a CS 251 topic). If neither of those is on your horizon, feel free to skim.

7.7 Why we want Bézout coefficients

The one sentence to carry with you: **Bézout coefficients are how you compute modular inverses.**

Here’s why. Suppose you want $a^{-1} \bmod n$ —that is, some x with $ax \equiv 1 \pmod{n}$. The extended Euclidean algorithm hands you integers x, y with

$$ax + ny = \gcd(a, n).$$

If $\gcd(a, n) = 1$, that equation reads $ax + ny = 1$. Reduce modulo n : the ny term vanishes (it’s divisible by n), leaving $ax \equiv 1 \pmod{n}$. So x is the inverse of $a \bmod n$, and the algorithm has computed it for you.

If $\gcd(a, n) \neq 1$ then the equation gives you $ax + ny = d$ for some $d > 1$, and reducing mod n gives $ax \equiv d \not\equiv 1$, so no inverse exists. This matches what Module 6 told us: a is invertible mod n exactly when a and n are coprime.

The brute-force way to find a modular inverse is to try every residue from 0 to $n - 1$ and see which one works; that’s $O(n)$ and unworkable for large n . The extended Euclidean algorithm runs in $O(\log \min(a, n))$, and “large n ” is exactly the regime where inverses actually matter—for example, RSA keys are a few thousand bits, which means n has several hundred decimal digits and brute force is out of the question.

7.8 The algorithm: back-substitution first

The cleanest way to understand the extended algorithm is to run ordinary Euclid and then *back-substitute* to recover x and y . Once you’ve done that once by hand, the one-pass iterative version makes sense as a rearrangement.

Let’s compute $\gcd(35, 12)$ with Bézout coefficients. Forward Euclid:

$$35 = 2 \cdot 12 + 11$$

$$12 = 1 \cdot 11 + 1$$

$$11 = 11 \cdot 1 + 0$$

So $\gcd(35, 12) = 1$. Good, 35 and 12 are coprime.

Now back-substitute. The step right before the remainder became 0 gave us the gcd; solve it for that gcd:

$$1 = 12 - 1 \cdot 11.$$

Now the next line up gave us $11 = 35 - 2 \cdot 12$. Substitute:

$$1 = 12 - 1 \cdot (35 - 2 \cdot 12) = 12 - 35 + 2 \cdot 12 = 3 \cdot 12 - 1 \cdot 35.$$

So $35 \cdot (-1) + 12 \cdot 3 = 1$: the coefficients are $x = -1$ and $y = 3$. Check: $-35 + 36 = 1$. ✓

Reading off the inverse: since $12 \cdot 3 \equiv 1 \pmod{35}$, we have $12^{-1} \equiv 3 \pmod{35}$. You can sanity-check this with a calculator: $12 \cdot 3 = 36 = 35 + 1$, so it really does reduce to 1 mod 35.

The back-substitution works for the same reason Euclid itself works—at each step, the gcd of the current pair equals the gcd of the previous pair—but now we’re keeping track of *how* to write that gcd as a combination of the original a and b .

7.9 The one-pass version

Doing back-substitution is fine if you’re walking through a single example on paper. A program wants to do it in one pass, updating the coefficients as it goes. The trick is to carry along two pairs (x_0, x_1) and (y_0, y_1) at each step, representing the coefficients for the current a and b .

Here’s the invariant. At every recursive call `egcdRec(a, b, x0, x1, y0, y1)`, the following hold:

- $a = A \cdot x_0 + B \cdot y_0$, where A, B are the original inputs
- $b = A \cdot x_1 + B \cdot y_1$

That is, x_0, y_0 are the current coefficients for a , and x_1, y_1 are the current coefficients for b . When b eventually hits 0, the first pair (x_0, y_0) is what we want, because a is now the gcd.

The initial call is `egcdRec(A, B, 1, 0, 0, 1)`, reflecting the obvious identities $A = A \cdot 1 + B \cdot 0$ and $B = A \cdot 0 + B \cdot 1$.

At each recursive step we compute $q = a \div b$ and replace (a, b) with $(b, a - qb)$. To preserve the invariant, the new coefficients for the new $b = a - qb$ must be $(x_0 - qx_1, y_0 - qy_1)$. (You should check this—plug in the old identities and simplify.) That’s exactly the pattern in the Python code:

```
def egcdRec(a, b, x0, x1, y0, y1):
    if b == 0:
        return (a, x0, y0)
    else:
```

```

    q = a // b
    return egcdRec(b, a % b, x1, x0 - q*x1, y1, y0 - q*y1)

def egcd(a, b):
    return egcdRec(a, b, 1, 0, 0, 1)

def modInverse(a, m):
    g, i, _ = egcd(a, m)
    if g != 1:
        raise ValueError("inverse_does_not_exist")
    return i % m

```

7.10 A traced example

Let's trace `egcd(35, 12)` step by step, so you can see the two coefficient pairs evolve. Each row is the arguments at one call.

call #	a	b	x_0	x_1	y_0	y_1
0	35	12	1	0	0	1
1	12	11	0	1	1	-2
2	11	1	1	-1	-2	3
3	1	0	-1	12	3	-35

At call #3, $b = 0$, so we return $(a, x_0, y_0) = (1, -1, 3)$. That matches what we got by hand. Let's verify the invariant held at every row by plugging back in:

- Call 0: $a = 35 = 35 \cdot 1 + 12 \cdot 0$. ✓ $b = 12 = 35 \cdot 0 + 12 \cdot 1$. ✓
- Call 1: $a = 12 = 35 \cdot 0 + 12 \cdot 1$. ✓ $b = 11 = 35 \cdot 1 + 12 \cdot (-2) = 35 - 24$. ✓
- Call 2: $a = 11 = 35 \cdot 1 + 12 \cdot (-2)$. ✓ $b = 1 = 35 \cdot (-1) + 12 \cdot 3 = -35 + 36$. ✓
- Call 3: $a = 1 = 35 \cdot (-1) + 12 \cdot 3$. ✓

The invariant is preserved at every step—not by accident, but because of the arithmetic that relates (x_0, x_1) to $(x_1, x_0 - qx_1)$ under the substitution $(a, b) \mapsto (b, a - qb)$.

7.11 Looking ahead: where this pays off

In CS 251 (or wherever else you meet cryptographic arithmetic) the extended Euclidean algorithm is the bottom of the stack for RSA key generation. A quick sketch so you see where it's headed, without pretending to teach the whole thing:

1. Pick two large primes p and q and set $n = pq$.
2. Compute $\varphi(n) = (p - 1)(q - 1)$, the size of the multiplicative group mod n .
3. Choose a public exponent e coprime to $\varphi(n)$ —classically $e = 65537$.

4. Compute the private exponent d satisfying $ed \equiv 1 \pmod{\varphi(n)}$. *This step is just a modular inverse, and the only practical way to do it for a $\varphi(n)$ with hundreds of digits is the extended Euclidean algorithm.*
5. Publish (n, e) ; keep (n, d) secret.

Encryption and decryption are then just “raise to the e th power mod n ” and “raise to the d th power mod n ,” and they invert each other by Fermat’s little theorem (or Euler’s theorem, depending on how you slice it). The security of the whole scheme rests on: (a) factoring n being hard, so attackers can’t recover p, q and hence $\varphi(n)$; (b) computing modular inverses being *easy* for the legitimate key-holder, who does know $\varphi(n)$ and runs `egcd`.

That asymmetry—trapdoor one-way function—is what makes RSA work. And the “easy” side of it is exactly this algorithm.

7.12 Exercises

Exercise 1. By hand, compute $\gcd(54, 21)$ and a pair of Bézout coefficients x, y with $54x + 21y = \gcd(54, 21)$. Show your work: forward Euclid first, then back-substitute.

Exercise 2. Use the extended Euclidean algorithm to find integers x and y with $17x + 5y = 1$. Then read off the multiplicative inverse of 17 modulo 5, and the multiplicative inverse of 5 modulo 17. Verify both answers by computing $17 \cdot x \bmod 5$ and $5 \cdot y' \bmod 17$ for the appropriate coefficients.

Exercise 3. Use the extended Euclidean algorithm to compute $7^{-1} \bmod 43$. Then check your answer by computing $7 \cdot x \bmod 43$ directly.

Exercise 4. Trace `egcd(100, 36)` using the table format above, with columns for a, b, x_0, x_1, y_0, y_1 . How many calls does it take to terminate? What are the final Bézout coefficients? Verify them by plugging into $100x + 36y = \gcd(100, 36)$.

Exercise 5. (Programming.) Write a Python function `egcd(a, b)` that implements the extended Euclidean algorithm iteratively (no recursion) and returns a tuple (g, x, y) with $g = \gcd(a, b)$ and $ax + by = g$. Then write `mod_inverse_via_egcd(a, n)` that uses `egcd` to compute $a^{-1} \bmod n$, raising `ValueError` if $\gcd(a, n) \neq 1$. Test it on the same inputs as Module 6 Exercise 13 (the brute-force version) and verify you get the same answers. Which implementation is asymptotically faster, and why?

Exercise 6. (Why the invariant holds.) The one-pass algorithm updates $(x_0, x_1, y_0, y_1) \mapsto (x_1, x_0 - qx_1, y_1, y_0 - qy_1)$ at each step, where $q = a \div b$. Assuming the invariant $a = Ax_0 + By_0$ and $b = Ax_1 + By_1$ held before the update, show that the corresponding invariant for the new a (which is the old b) and the new b (which is $a - qb$) still holds after the update. This is a short algebra exercise; no induction required.

How to Write a Proof, Revisited

The earlier chapter, “How to Write a Proof,” covered the prose habits that work regardless of technique: state the goal, introduce variables, justify each step, conclude clearly. Those habits still apply. You’ve now finished Module 7 and have met the main informal proof techniques for the first seven modules—direct, counterexample, cases, contradiction, contrapositive—and you’re about to start an extended run on induction (Modules 8–10). *Structural* induction on arbitrary recursive data types comes in Module 10 and is arguably a technique in its own right, so if you’re keeping a mental checklist of proof techniques, add that one at the end. Each technique places its own demands on how the proof is structured on the page. This chapter is the upper-level companion: how to choose between techniques, how to write an induction proof, how to handle biconditionals, and the style conventions you’ll need as your proofs get longer.

7.13 Choosing a technique

You’ve probably noticed that most claims can be proved in more than one way. There’s no algorithm for picking the “right” technique, but there are reliable starting points. Here’s a decision tree that’s not universally correct but is usually correct:

- **“For all x , $P(x)$.”** Start with a direct proof: let x be arbitrary, prove $P(x)$. If the direct argument stalls on an internal implication inside P , check whether its contrapositive is easier.
- **“There exists x such that $P(x)$.”** If you can construct a specific x , do so—a *constructive* proof. Otherwise assume no such x exists and try to derive a contradiction—a *non-constructive* proof. Constructive is preferred when available: it tells the reader not just that x exists but what one looks like.
- **“If P then Q .”** Try direct first: assume P , derive Q . If Q feels far from P , try the contrapositive: assume $\neg Q$, derive $\neg P$. If both get tangled, assume $P \wedge \neg Q$ and try for a contradiction.
- **“The claim splits into disjoint cases.”** Use proof by cases. The classic signal is “for all integers n ...” where the argument is different when n is even vs. odd, or “for any integer n , $n \bmod 3 \in \{0, 1, 2\}$.”
- **“For all naturals n , $P(n)$ ” or “for all lists / trees / terms, $P(t)$.”** Use induction. Natural induction for the naturals; strong induction when the IH needs more than the immediate predecessor; structural induction for recursively defined data (Module 10).
- **“ X is impossible.”** Contradiction is often cleanest: assume X , derive a statement you already know is false. The irrationality of $\sqrt{2}$ is the canonical example.

When two techniques both seem to work, pick the one that makes the proof *shorter*. Short proofs are easier to verify and easier to remember.

7.14 A walkthrough: using the contrapositive

Here's a richer walkthrough than the element-chasing pair in the earlier chapter. We'll prove: *for any integer n , if n^2 is even, then n is even.*

First draft (direct attempt):

Suppose n^2 is even. Then $n^2 = 2k$ for some integer k . So $n = \sqrt{2k} \dots$

This is already in trouble. The square root forces us out of the integers, and factoring $n^2 = 2k$ over the integers is exactly the divisibility question we're trying to answer—we'd be assuming what we want to prove. Direct is the wrong tool here.

The flip to the contrapositive is: *if n is odd, then n^2 is odd.* That's a claim we can prove directly—we know exactly what “odd” means as a formula.

Tightened version:

We prove the contrapositive: if n is odd, then n^2 is odd.

Suppose n is odd. By the definition of odd, $n = 2k + 1$ for some integer k . Then

$$n^2 = (2k + 1)^2 = 4k^2 + 4k + 1 = 2(2k^2 + 2k) + 1.$$

Since $2k^2 + 2k$ is an integer, n^2 has the form $2m + 1$ for an integer m , which is the definition of odd.

We have shown that n odd implies n^2 odd. By the logical equivalence of a conditional and its contrapositive, n^2 even implies n even.

Three things worth noting:

1. **Announce the contrapositive up front.** The reader needs to know you're not trying to prove the original statement directly, or they'll be confused about what you're doing.
2. **Invoke the logical equivalence at the end.** You proved $\neg Q \Rightarrow \neg P$; the original claim is $P \Rightarrow Q$. One sentence connects them.
3. **Unpack the definitions, both ways.** You start by unpacking “ n is odd” into a formula, and you end by repacking “ $n^2 = 2m + 1$ ” into “ n^2 is odd.” The unpack-then-repack bookends are a reliable pattern for divisibility-style proofs.

7.15 Writing induction proofs

Induction is the first proof technique in this course whose *structure* is visible on the page. Most techniques before it can be written as a single flowing paragraph; an induction proof has three named pieces that the reader expects to see, in order.

Here's the template.

1. **State $P(n)$ explicitly.** Say what property you're proving, and what variable you're inducting on. “We prove by induction on n that $P(n)$: $1 + 2 + \dots + n = n(n + 1)/2$ for all $n \geq 1$.”
2. **Prove the base case.** Show P holds for the smallest n the claim covers.

3. **Prove the inductive step.** State the *inductive hypothesis* (IH) explicitly: “Suppose $P(k)$ holds for some $k \geq 1$.” Then derive $P(k+1)$ using the IH. The place where you use the IH should be visibly labeled.
4. **Close with the induction principle.** One sentence: “By the principle of mathematical induction, $P(n)$ holds for all $n \geq 1$.”

Here’s the full write-up for the sum identity.

Claim. For all integers $n \geq 1$, $1 + 2 + \cdots + n = n(n+1)/2$.

Proof. *Base case* ($n = 1$). The left-hand side is 1; the right-hand side is $1 \cdot 2/2 = 1$. They agree.

Inductive step. Suppose the claim holds for some $k \geq 1$:

$$1 + 2 + \cdots + k = \frac{k(k+1)}{2}. \quad (\text{IH})$$

We want to show it holds for $k+1$ —that is, $1 + 2 + \cdots + (k+1) = (k+1)(k+2)/2$. Starting from the left-hand side,

$$\begin{aligned} 1 + 2 + \cdots + k + (k+1) &= (1 + 2 + \cdots + k) + (k+1) \\ &= \frac{k(k+1)}{2} + (k+1) && (\text{by IH}) \\ &= \frac{k(k+1) + 2(k+1)}{2} \\ &= \frac{(k+1)(k+2)}{2}, \end{aligned}$$

which is the right-hand side for $n = k+1$.

By the principle of mathematical induction, the claim holds for all $n \geq 1$. $\square$

Three things worth calling out.

1. **Label the IH.** The annotation “(by IH)” at the step where you substitute the hypothesis is not optional decoration. It’s how the reader verifies the step is legal—the IH is the only new assumption you’re allowed to use, and marking where you used it makes your proof machine-checkable in principle.
2. **“Some k ,” not “any k .”** The IH says “suppose $P(k)$ holds for *some* fixed k .” A common student error is to write “suppose $P(k)$ for all k ”—but that’s assuming the thing you’re trying to prove.
3. **Don’t skip the base case.** An induction proof without a base case is a ladder with no bottom rung. There are even famous “paradoxes” (“all horses are the same color”) whose flaw is a missing or mishandled base case.

7.16 Biconditional proofs

A claim of the form “ P if and only if Q ” is really two claims: $P \Rightarrow Q$ and $Q \Rightarrow P$. Your proof needs to visibly handle both.

Template:

($\Rightarrow$) Assume P Therefore Q .

($\Leftarrow$) Assume Q Therefore P .

Both directions may need different techniques—the forward direction might fall to a direct proof while the backward direction wants the contrapositive, or vice versa. That’s fine; label them and write them separately.

As a concrete example, consider: “ n is even if and only if n^2 is even.”

($\Rightarrow$) Suppose n is even. By the definition of even, $n = 2k$ for some integer k , so $n^2 = 4k^2 = 2(2k^2)$, which is even.

($\Leftarrow$) This is the contrapositive direction we proved in §2: if n^2 is even, then n is even.

Together, the two directions establish the biconditional. Don’t try to chain them into a single argument by writing something like “ n even $\Leftrightarrow n = 2k \Leftrightarrow n^2 = 4k^2 \Leftrightarrow n^2$ even,” unless every step is *itself* a biconditional—it is almost always clearer to separate the directions.

7.17 Style conventions as proofs get longer

Once proofs run beyond a handful of sentences, a small stack of conventions starts to earn its keep.

- **End-of-proof markers.** “This completes the proof,” “ $\square$,” and “ $\blacksquare$ ” all work. Pick one and stick with it within a single document.
- **“Without loss of generality” (WLOG).** Useful when the claim is genuinely symmetric in some variables: “WLOG $a \leq b$ ” is fine when a and b play interchangeable roles in the claim. A common abuse is to invoke WLOG when the situation isn’t actually symmetric. If you wrote WLOG, you should be able to explain the symmetry in one sentence; if you can’t, the WLOG is wrong.
- **“Clearly” and “obviously.”** Almost always a bug. If the step really is obvious, the reader can see it without you saying so; if it isn’t, “clearly” won’t help. Replace with the actual reason, even if the reason is “by definition” or “by Lemma 3.”
- **Theorem, proposition, lemma, corollary.** Rough hierarchy: theorems are the headline claims; propositions are mid-tier results; lemmas are auxiliary facts proven on the way; corollaries are short consequences. When your proof needs a sub-result that would distract from the main argument, break it out as a lemma and prove it separately.
- **Paragraph breaks.** Once a proof runs longer than eight or so sentences, break it by phase: setup, main argument, conclusion. Visual structure should mirror logical structure.
- **Don’t reuse variable names.** If n already means “the number of elements in your set,” don’t reuse n as the running index of a summation. Variable collision is one of the cheapest bugs to introduce and one of the hardest to spot in a finished proof—rename early.

7.18 A proof-hygiene checklist

Before you hand in a proof, read it through asking yourself:

- Did you state what you’re proving?
- Did you introduce every variable before using it?

- Did you unpack the definitions of the key terms?
- Can you name the justification for each non-trivial step?
- Did you close every universal (“since x was arbitrary...”)?
- Did you show that your cases are exhaustive, for case proofs?
- Did you state the IH explicitly and mark where you used it, for induction proofs?
- Did you handle both directions, for biconditional proofs?
- Did you conclude clearly with “Therefore...” or a QED marker?
- Did you avoid “clearly” and “obviously”?
- Does your proof read as English you could say out loud, or is it a wall of equations with no connective tissue?

If the answer to any of these is “no,” you probably have a writing fix to make before you have a mathematics one. The mathematics is often easier than the English.

7.19 What’s next

The rest of the book—Modules 8 through 10, plus the type-algebra and lambda-calculus intermezzi—leans hard on the induction template above. Proofs grow longer; the style conventions start to pay rent. Loop back to this chapter when a proof feels like it wants to be longer than its prose can carry, or when you can see the argument in your head but can’t quite write it down on the page.

Intermezzo: Proofs Are Programs

We’ve now seen several different ways to write proofs: numbered-line derivations from Module 3, Gentzen-style inference trees from the trees intermezzo, informal arguments with “assume. . . therefore,” and even machine-checked proofs in Lean via the Natural Number Game. We’ve gotten comfortable with the *mechanics* of proving things.

But here’s a question we haven’t really asked: what IS a proof? Not “how do you write one” but “what kind of mathematical object is it?” We’ve been treating proofs as sequences of steps, or as trees, or as English paragraphs. But is there something deeper going on—some underlying *structure* that all these presentations share?

The answer is yes, and it’s one of the most beautiful ideas in all of mathematics and computer science. It turns out that proofs and programs are the *same thing*.

7.20 What does a proof *look like*?

To see this, we need to think about proofs differently. Instead of asking “how do I convince someone this is true?” we ask: “what *data* does a proof carry?”

There’s a tradition in constructive mathematics called the **Brouwer–Heyting–Kolmogorov interpretation** (BHK for short) that gives a precise answer. It was developed by the Dutch mathematician L.E.J. Brouwer, the logician Arend Heyting, and the great Andrey Kolmogorov, and it says the following:

- **A proof of $A \wedge B$** is a *pair*: a proof of A together with a proof of B . You need both pieces. It’s a tuple!
- **A proof of $A \vee B$** is a *tagged value*: either a proof of A (tagged “left”) or a proof of B (tagged “right”). You have to say *which* side you’re proving, and then actually prove it. It’s a sum type—an **Either**, a tagged union!
- **A proof of $A \rightarrow B$** is a *function*: given any proof of A , it produces a proof of B . Not a specific proof for a specific A —a *method* that works for any proof of A you hand it.
- **A proof of $\forall x : S. P(x)$** is a *function*: given any element x of S , it produces a proof of $P(x)$. A machine that manufactures proofs on demand, one for each possible input.
- **A proof of $\exists x : S. P(x)$** is a *pair*: a specific witness $x \in S$ together with a proof that $P(x)$ holds for that witness. You can’t just say “something exists”—you have to *produce* the thing and *show* it works.
- **A proof of $\perp$ (false)** is. . . nothing. There is no proof of false. It’s the empty type—the type with no values at all.

Read that list again slowly. Does it remind you of anything? Those are *data structures*. Pairs, tagged unions, functions. The BHK interpretation is saying that proofs are made out of the same stuff as programs.

This is not a metaphor. This is not a loose analogy. This is a precise, formal correspondence, and it has a name.

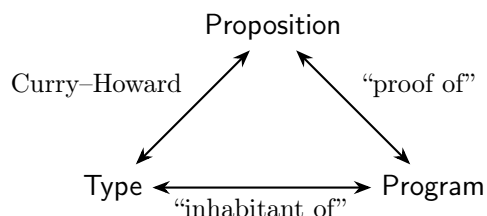
7.21 The Curry–Howard dictionary

The **Curry–Howard correspondence** (sometimes called the Curry–Howard *isomorphism*, because it really is a bijection) is a dictionary that translates between logic and programming. It was discovered independently by the logician Haskell Curry in the 1930s–50s and the logician William Alvin Howard in 1969, though its full implications took decades to unfold.

Here’s the dictionary:

Logic		Programming
Proposition	$\longleftrightarrow$	Type
Proof of a proposition	$\longleftrightarrow$	Program (term) of that type
$A \rightarrow B$	$\longleftrightarrow$	Function type <code>A -> B</code>
$A \wedge B$	$\longleftrightarrow$	Pair/product type <code>(A, B)</code>
$A \vee B$	$\longleftrightarrow$	Sum type <code>Either A B</code>
$\top$ (true)	$\longleftrightarrow$	Unit type <code>()</code> (one trivial value)
$\perp$ (false)	$\longleftrightarrow$	Empty type (no values at all)
$\forall x : S. P(x)$	$\longleftrightarrow$	Dependent function / polymorphism
$\exists x : S. P(x)$	$\longleftrightarrow$	Dependent pair / existential type
Modus ponens	$\longleftrightarrow$	Function application
Implication introduction	$\longleftrightarrow$	Lambda abstraction (defining a function)
Conjunction introduction	$\longleftrightarrow$	Constructing a pair
Conjunction elimination	$\longleftrightarrow$	Projecting from a pair (<code>fst</code> , <code>snd</code>)
Disjunction introduction	$\longleftrightarrow$	Injecting into a sum (<code>Left</code> , <code>Right</code>)
Disjunction elimination	$\longleftrightarrow$	Pattern matching / case analysis
Ex falso ($\perp$ -elimination)	$\longleftrightarrow$	Handling the empty type (vacuously)

The left column is everything we’ve been doing since Module 3. The right column is everything you’ve been doing since your first programming class. They were the same subject the whole time.



Read any side of this triangle as an equivalence: a proof of a proposition and an inhabitant of the corresponding type are literally the same object, and that object is a program.

Let that sink in for a moment. Every time you wrote a function that takes a pair and returns one component, you were doing conjunction elimination. Every time you pattern-matched on a tagged union, you were doing proof by cases. Every time you defined a function, you were introducing an implication.

7.22 A worked example: the same proof, four ways

Let's make this concrete. Consider the logical tautology

$$A \wedge B \rightarrow B \wedge A$$

which says that conjunction is commutative: if A and B are both true, then B and A are both true. Not exactly shocking, but it's a perfect example because the proof is simple enough to see the correspondence clearly.

7.22.1 As a derivation (Module 3 style)

Theorem: Show $A \wedge B \rightarrow B \wedge A$

Proof:

(a) proof that assuming $A \wedge B$, shows $B \wedge A$:

subproof: (i) $A \wedge B$, by assumption

(ii) B , by $\wedge$ -elimination on (i)

(iii) A , by $\wedge$ -elimination on (i)

show $B \wedge A$, by $\wedge$ -introduction with (ii) and (iii)

shows $A \wedge B \rightarrow B \wedge A$ by introduction of implication

7.22.2 As a Gentzen tree

$$\frac{\frac{[A \wedge B]}{B} \wedge E_2 \quad \frac{[A \wedge B]}{A} \wedge E_1}{B \wedge A} \wedge I \quad \rightarrow I$$
$$A \wedge B \rightarrow B \wedge A$$

Read it bottom-up: we want $A \wedge B \rightarrow B \wedge A$, so we use $\rightarrow I$, which means we temporarily assume $A \wedge B$ (in brackets) and need to derive $B \wedge A$. To build $B \wedge A$ we use $\wedge I$, so we need B and A separately. We get B by $\wedge E_2$ on our assumption and A by $\wedge E_1$.

7.22.3 As a Python function

```
def swap(pair):  
    return (pair[1], pair[0])
```

That's it. We take in a pair, extract the second element and the first element, and build a new pair with them in the other order. The function `swap` is the proof. Its type signature—"takes a pair (A, B) , returns a pair (B, A) "—is the theorem.

7.22.4 As Lean 4

```
fun ⟨a, b⟩ => ⟨b, a⟩
```

If you've played the Natural Number Game, this syntax might not look too alien. The angle brackets $\langle \cdot, \cdot \rangle$ are Lean's notation for pairs (and for the `And.intro` constructor). The `fun` keyword introduces a function—implication introduction. Pattern matching on $\langle a, b \rangle$ does both conjunction eliminations at once.

Now here's the punchline: **these are all the same thing**. The derivation, the Gentzen tree, the Python function, and the Lean term are four presentations of *one* underlying mathematical

object. The proof IS the program. The program IS the proof. The type of the program IS the theorem statement. A well-typed program IS a valid proof.

Go back and trace the correspondence step by step. In the Python version, `pair` is the assumption $A \wedge B$. Indexing with `pair[1]` is $\wedge$ -elimination (extracting the second component). Building the tuple `(pair[1], pair[0])` is $\wedge$ -introduction. The `def` itself is $\rightarrow$ -introduction. Every single step in the proof corresponds to a programming operation, and vice versa.

7.23 Why this matters

This correspondence is not just a curiosity. It is the theoretical foundation of an entire field.

Constructive proofs are executable. Under the BHK interpretation, every constructive proof of a proposition carries enough information to actually *compute*. A proof that $\exists x. P(x)$ doesn't just tell you that such an x exists somewhere out there in the platonic realm—it hands you a specific x and shows you why it works. A proof that $A \vee B$ doesn't just say “one of them is true, I'm not telling you which”—it tells you *which one* and proves it. This is why constructive mathematics has always appealed to computer scientists: constructive proofs are programs you can run.

Proof assistants are type checkers. This is the foundation of tools like Lean, Agda, and Coq. When you write a proof in Lean, you're really writing a program. When Lean checks your proof, it's really type-checking your program. The kernel of a proof assistant—the thing you ultimately have to trust—is a type checker. If the program has the right type, the proof is valid. Period. That's why these tools can be so reliable: type checking is a well-understood, mechanizable process.

Excluded middle is non-constructive—and now we can see *why*. Remember all those careful remarks in Modules 3 and 7 about how proof by contradiction and excluded middle are “classical” and not constructively valid? The Curry–Howard correspondence makes the reason visceral. Under BHK, a proof of $A \vee \neg A$ would be a program that, for *any* proposition A whatsoever, either produces a proof of A or produces a proof of $\neg A$. Think about what that would mean: a program that can decide *any* mathematical question. That's absurd! There's no algorithm that can look at an arbitrary proposition and tell you whether it's true or false—that's essentially the halting problem.

So if you add excluded middle as an axiom, what you're doing, computationally, is adding a primitive that magically produces a value without computing it. In programming-language terms, it corresponds to something like `call/cc` (call-with-current-continuation) in Scheme—a control operator that captures the rest of the computation and can jump back to it later. It's computationally meaningful, but it's not a *constructive* program in the ordinary sense. This is the precise sense in which classical logic has “more” theorems than constructive logic: it has a more powerful (and stranger) programming language.

This is the beginning, not the end. You'll see much more of Curry–Howard in CS 251, where we'll work with dependent types and use proof assistants not just for number games but for verifying real mathematical theorems. For now, the takeaway is this: the logical infrastructure you've been building all quarter—conjunction, disjunction, implication, quantifiers, proof rules—is not separate from programming. It IS programming, seen from a different angle.

Exercise 1. Under the Curry–Howard correspondence, what programming construct corresponds to proof by cases (disjunction elimination, $\vee E$)? Hint: think about what you do when you have an `Either` value in a language like Haskell or Python. You need to handle both the `Left` case and the `Right` case. What programming pattern is that?

Exercise 2. The formula $A \rightarrow (B \rightarrow A)$ is a tautology (you can check it with a truth table if you like). Under Curry–Howard, this corresponds to a type: a function that takes an **A** and returns a function from **B** to **A**. Write a Python function with this type signature. What does it do? Why does it make sense that this is a tautology?

Exercise 3. (Currying.) The tautology

$$((A \wedge B) \rightarrow C) \leftrightarrow (A \rightarrow (B \rightarrow C))$$

is one of the most important equivalences in logic, and under Curry–Howard it is the operation of *currying*. Write two Python functions:

- `curry`, taking a function of type $(A, B) \rightarrow C$ and returning a function of type $A \rightarrow (B \rightarrow C)$.
- `uncurry`, doing the reverse.

Verify that `curry` and `uncurry` are inverses of each other. What is this saying logically?

Exercise 4. (Function composition is transitivity.) The tautology

$$(A \rightarrow B) \rightarrow ((B \rightarrow C) \rightarrow (A \rightarrow C))$$

says that implication is transitive. Write a Python function `compose(f, g)` that takes `f : A -> B` and `g : B -> C` and returns a function from **A** to **C**. The proof of transitivity *is* the definition of function composition. Trace which step of the logical proof corresponds to which line of code. (Note the three faces of the same idea: function composition from Module 1, transitivity of implication from Module 3, and a proof of $(A \rightarrow B) \rightarrow ((B \rightarrow C) \rightarrow (A \rightarrow C))$ are a single gadget described from three different angles.)

Exercise 5. (Vacuous truth and the empty type.) Under Curry–Howard, the type `Void` (the empty type) corresponds to the proposition $\perp$. The principle of *ex falso quodlibet* says $\perp \rightarrow A$ for any *A*. Write a Python function (or pseudocode) of type `Void -> A` for any **A**. (Hint: there are no values of type `Void`, so the function never actually has to return anything—you can never call it with a real argument.) Why does the lack of values in `Void` make this trivially valid?

Exercise 6. (The classical/constructive divide.) Try to write a Python function `lem` of type $() \rightarrow \text{Either } A \ (A \rightarrow \text{Void})$ —that is, a function that, for an arbitrary type **A**, returns either a value of type **A** (a “proof of *A*”) or a function from **A** to `Void` (a “proof of $\neg A$ ”). This is the law of excluded middle, $A \vee \neg A$. Explain in one paragraph why no such function exists in general: what would such a function be doing if **A** were the type “codes of Turing machines that halt on every input”?

Further reading

If the proofs-as-programs idea grabs you, the following are the canonical entry points.

- Philip Wadler, “[Propositions as Types](#),” *Communications of the ACM* **58**(12), 75–84 (2015). The accessible undergraduate-friendly tour of the Curry–Howard correspondence, with historical context, examples in modern languages, and Wadler’s characteristic clarity. If you read only one paper this term, read this one.

- William A. Howard, “[The Formulae-as-Types Notion of Construction](#),” in J. R. Hindley and J. P. Seldin (eds.), *To H. B. Curry: Essays on Combinatory Logic, Lambda Calculus and Formalism*, Academic Press (1980); manuscript circulated 1969. The original short note that started everything. Only about a dozen pages, and the first half is genuinely readable once you have seen natural deduction.
- Philip Wadler, “[Proofs are Programs: 19th Century Logic and 21st Century Computing](#),” *Dr. Dobbs’s Journal* (2000). A shorter, more popular precursor to “Propositions as Types,” pitched at working programmers—good as a warm-up before tackling the CACM piece.

Intermezzo: Other Logics, Other Worlds

Propositional logic and first-order logic are powerful, but they are not the only games in town. Over the past century, logicians and computer scientists have invented dozens of specialized logics, each designed to capture a particular kind of reasoning. In this intermezzo we'll take a quick tour of the zoo and then spend some time with one especially useful species: *temporal logic*, the logic of time.

The punchline is this: you already know how to learn a new logic. The methodology is always the same—define a syntax (usually with BNF), define a semantics (what the formulas mean), and then prove things using inference rules. You've done this for propositional logic in Modules 2–3, for first-order logic in Module 4, and for Hoare logic in Module 7. Every logic in this chapter follows exactly the same pattern. The toolkit transfers; only the domain changes.

7.24 The zoo of logics

Here is a very incomplete field guide to some logics you might encounter in the wild.

- **Modal logic** adds two operators to propositional logic: $\Box p$ (“necessarily p ”) and $\Diamond p$ (“possibly p ”). The idea is that there are many possible “worlds,” and $\Box p$ means p is true in *all* of them, while $\Diamond p$ means p is true in *at least one*. If that sounds like $\forall$ and $\exists$ wearing different hats, you're not wrong—modal logic can be seen as a fragment of first-order logic where the quantification is over possible worlds rather than over elements of a set.
- **Temporal logic** is modal logic where the “possible worlds” are moments in time. Instead of “necessarily” and “possibly” you get “always” and “eventually.” We'll spend most of this chapter here.
- **Linear logic** is a logic of *resources*. In classical logic, if you know p you can use that fact as many times as you want—it's free to duplicate. In linear logic, each fact is a resource that gets *consumed* when you use it. If you have one dollar and you spend it, you don't have a dollar anymore. This sounds exotic, but it turns out to be exactly the right logic for reasoning about things like memory management, concurrent access to shared data, and protocol compliance. It's also deeply connected to quantum computing, where the no-cloning theorem says you literally can't duplicate a quantum state.
- **Hoare logic** (Module 7) is a logic of programs. Its “propositions” are Hoare triples $\{P\}C\{Q\}$, its “connectives” are the programming constructs (sequencing, conditionals, while loops), and its “inference rules” are the axioms we learned: assignment, sequencing, the while rule.

The point isn't to learn all of these right now. The point is that you've already internalized the *methodology*. Propositional logic had a BNF for its syntax (Module 2), truth tables and valuations for its semantics (Module 2), and inference rules for its proof system (Module 3). Every logic on

this list has the same three components. Once you see the pattern, picking up a new logic is like picking up a new programming language when you already know how to program—the surface syntax changes, but the deep structure is familiar.

7.25 Temporal logic—logic over time

Let’s zoom in on temporal logic, because it has a beautiful connection to the material we’ve already covered.

In Module 7 we used Hoare logic to reason about programs. One of the key concepts there was the *loop invariant*: a property P that holds before the loop body executes and still holds afterward. The while rule said that if P is preserved by each iteration, then P holds when the loop terminates.

Temporal logic takes this idea and runs with it. Instead of reasoning about a single loop, temporal logic lets you make statements about *entire execution traces*—infinite sequences of states that a system passes through over time. Think of a web server that starts up, handles requests, maybe crashes, restarts, handles more requests, and so on forever. Temporal logic is the language for expressing properties of such systems.

The basic temporal operators are:

Definition 7.25.1 (Temporal operators). Let p be a proposition about the current state of a system.

- $\Box p$ (read “always p ”): p holds at every point in time from now on. This is the temporal version of “necessarily.” Think of it as an invariant that never, ever breaks—not just through one loop iteration, but forever.
- $\Diamond p$ (read “eventually p ”): p holds at some point in the future (possibly right now). Something good is going to happen—you just might have to wait.
- $\bigcirc p$ (read “next p ”): p holds in the very next state. If we think of time as a sequence of discrete steps (like clock ticks, or loop iterations, or program states), this says “one step from now, p is true.”

These operators can be combined with all the usual propositional connectives ($\wedge$, $\vee$, $\rightarrow$, $\neg$), and that’s where things get expressive.

Notice how $\Box$ and $\Diamond$ are related: $\Diamond p \equiv \neg \Box \neg p$. “Eventually p ” means “it’s not the case that p is always false.” And $\Box p \equiv \neg \Diamond \neg p$ —“always p ” means “ p never fails.” This is exactly the duality between $\forall$ and $\exists$ from Module 4, just wearing a temporal costume.

Connection to Hoare logic. Remember the while rule from Module 7?

assumes: $\{P \wedge B\} c \{P\}$
shows: $\{P\} \text{while } B \text{ do } c \{P \wedge \neg B\}$

The invariant P is preserved by every iteration of the loop body. In temporal logic terms, the while rule is establishing something like $\Box P$ *during the loop*: at every step of execution, the invariant holds. When we prove a loop invariant, we’re really doing temporal reasoning in disguise—we’re showing that a property is stable across an entire sequence of state transitions.

The difference is that Hoare logic reasons about programs that terminate (or at least, individual program fragments), while temporal logic reasons about systems that might run forever. A server, an operating system, a network protocol—these aren’t programs that finish and return a value. They’re ongoing processes, and temporal logic is built for them.

7.26 Expressing real properties

The real power of temporal logic shows up when you use it to express properties that practitioners actually care about. Here are some classic examples.

“The server eventually responds to every request.”

$$\Box (request \rightarrow \Diamond response)$$

Read it carefully: *always* (at every point in time), if a request is made, then *eventually* a response will follow. The outer $\Box$ says this must hold at every moment, not just right now. The inner $\Diamond$ says the response doesn’t have to be immediate—it just has to happen at some point in the future. This is called a *liveness* property: something good eventually happens.

“A lock is never held by two threads simultaneously.”

$$\Box \neg (held_1 \wedge held_2)$$

At every point in time, it is not the case that both thread 1 and thread 2 hold the lock. This is a *safety* property: something bad never happens.

“After every login, the user is eventually logged out.”

$$\Box (login \rightarrow \Diamond logout)$$

Same shape as the server example. Every login is eventually followed by a logout. No session lasts forever.

“The system never deadlocks.”

$$\Box \Diamond progress$$

Always, eventually, progress is made. The system might pause temporarily, but it can’t get stuck forever.

These are exactly the kinds of properties that *model checkers* verify automatically. Model checking is one of the most successful applications of formal methods in industry. Amazon Web Services uses TLA+ (a temporal logic) to verify the correctness of their distributed systems. Intel uses model checking to verify chip designs before fabrication—finding a bug after you’ve manufactured a million processors is somewhat more expensive than finding it in a formula. Microsoft has used temporal logic tools to verify device drivers in the Windows kernel. The 2007 Turing Award went to Clarke, Emerson, and Sifakis for their work on model checking.

The reason model checking works so well is precisely the methodology you’ve learned in this course: write down a formal specification (in temporal logic), build a formal model of the system, and then let a computer check that the model satisfies the specification. Syntax, semantics, proof—the same three steps, applied at industrial scale.

7.27 The pattern

Let’s make the pattern explicit by writing down the BNF for (a simple version of) temporal logic. If p ranges over atomic propositions (basic facts about the current state), then the formulas of linear temporal logic (LTL) are:

$\varphi ::= p \mid \neg\varphi \mid \varphi \wedge \varphi \mid \varphi \vee \varphi \mid \varphi \rightarrow \varphi \mid \Box\varphi \mid \Diamond\varphi \mid \bigcirc\varphi$
--

Look familiar? It’s the BNF from Module 2, with three new operators bolted on. The propositional connectives are still there—we’ve just extended the language.

The semantics is defined over infinite sequences of states, sometimes called *Kripke structures* (after the philosopher and logician Saul Kripke). A Kripke structure is a sequence $s_0, s_1, s_2, \dots$ where each s_i is a state—an assignment of truth values to the atomic propositions. Given a sequence and a position i , we define when a formula holds:

- $s_i \models p$ iff p is true in state s_i
- $s_i \models \neg\varphi$ iff $s_i \not\models \varphi$
- $s_i \models \varphi \wedge \psi$ iff $s_i \models \varphi$ and $s_i \models \psi$
- $s_i \models \Box\varphi$ iff $s_j \models \varphi$ for all $j \geq i$
- $s_i \models \Diamond\varphi$ iff $s_j \models \varphi$ for some $j \geq i$
- $s_i \models \bigcirc\varphi$ iff $s_{i+1} \models \varphi$

And there are proof rules for temporal logic, just as there are for propositional and first-order logic. For instance, $\Box$ distributes over $\wedge$ (if something is always true and something else is always true, then their conjunction is always true), and from $\Box\varphi$ you can conclude φ (if something is always true, it’s true right now). These rules can be presented as Gentzen-style inference trees, exactly as in the trees intermezzo.

Same methodology. Same pattern. New domain. The toolkit from Modules 2–3 transfers directly—you just need to learn the new operators and their semantics. If you can read a BNF, define a semantics, and apply inference rules, you can learn any logic.

7.28 Duality everywhere

The relationship $\Diamond p \equiv \neg\Box\neg p$ that we wrote down in passing is not an isolated curiosity. Every logic you have met in this course is secretly running on the same symmetry. Line them up:

Context	Pair 1	Pair 2 (its dual)
Propositional logic (Module 2)	$p \wedge q$	$p \vee q$
via De Morgan	$\neg(p \wedge q)$	$\neg p \vee \neg q$
First-order logic (Module 4)	$\forall x. P(x)$	$\exists x. P(x)$
via quantifier negation	$\neg\forall x. P(x)$	$\exists x. \neg P(x)$
Games for quantifiers (intermezzo)	Proponent’s $\forall$ -move	Opponent’s $\exists$ -move
Algebra of types (intermezzo)	product $A \times B$	sum $A + B$
Types, universally (intermezzo)	arrows <i>into</i> $A \times B$	arrows <i>out of</i> $A + B$
Temporal logic (this chapter)	$\Box\varphi$	$\Diamond\varphi$
via the semantics	$\neg\Box\varphi$	$\Diamond\neg\varphi$

Each row is an instance of the same move: you take a “for-all-like” operator and a “there-exists-like” operator, and pushing negation past one of them flips it into the other. This is not a coincidence of notation; it is a single structural phenomenon appearing in many costumes. Category theory has a precise name for what is happening—these pairs are *adjoint*—and once you see one such pair, the others snap into place as the same thing viewed through different lenses. You don’t need to know that name yet; what matters is noticing the pattern, because it is the kind of thing that tells you there is something deeper waiting.

7.29 Exercises

Exercise 0. (Warm-up.) For each English sentence, write it in LTL using only a single temporal operator (no nesting): (a) “the system is always on” (atom: *on*); (b) “eventually the connection is established” (atom: *connected*); (c) “in the very next step, the buffer becomes non-empty” (atom: *buffer_nonempty*). This is a finger exercise; Exercises 1 and 2 then build up to formulas with multiple operators.

Exercise 1. Express the following properties in temporal logic (using $\Box$, $\Diamond$, $\bigcirc$, and the usual propositional connectives):

- (a) “The program eventually terminates.” (Let *halted* be an atomic proposition meaning “the program has halted.”)
- (b) “The counter is always non-negative.” (Let *counter* ≥ 0 be an atomic proposition.)
- (c) “Every request is eventually followed by a response.” (Let *request* and *response* be atomic propositions.)

Exercise 2. Express each of the following in temporal logic. Be precise; if a property requires more than one operator, use whichever combination you need.

- (a) “Two trains are never on the same section of track at the same time.” (Let *train*₁ and *train*₂ be atomic propositions for “train 1 is on the section” and “train 2 is on the section.”)
- (b) “Once an alarm goes off, it stays on until manually reset.” (Use *alarm* and *reset*.)
- (c) “Every login is eventually followed by a logout, and every logout is preceded by a login.”

Exercise 3. (Duality.) Show that $\neg\Diamond p \equiv \Box\neg p$ directly from the semantics: $s_i \models \neg\Diamond p$ iff there is no $j \geq i$ with $s_j \models p$ iff $s_j \models \neg p$ for every $j \geq i$ iff $s_i \models \Box\neg p$. Compare this argument to the proof of De Morgan’s law $\neg(\exists x. P(x)) \equiv \forall x. \neg P(x)$ from Module 4.

Exercise 4. (Distinguishing $\Diamond\Box p$ and $\Box\Diamond p$.) These two formulas look similar but say very different things. For each one, give a precise English translation (“it is the case that...”), and exhibit an infinite sequence of states $s_0, s_1, s_2, \dots$ that satisfies one but not the other. (Hint: think about what happens if p holds at every *even* state but not at any odd state.)

Exercise 5. (Temporal Hoare logic.) Recall the while rule from Module 7. Express the following property in LTL: “At every state during the loop, the invariant P holds.” How does the LTL formulation make explicit what was implicit in the Hoare rule? In one or two sentences, what does temporal logic give you that Hoare logic does not?

Exercise 6. (Safety vs. liveness.) Classify each of the following formulas as a *safety* property (something bad never happens), a *liveness* property (something good eventually happens), or both:

- (a) $\Box\neg\text{deadlock}$
- (b) $\Box(\text{request} \rightarrow \Diamond\text{response})$
- (c) $\Diamond\Box\text{stable}$
- (d) $\Box\Diamond\text{progress}$

Further reading

If you want to see temporal logic and model checking in real industrial use:

- Chris Newcombe, Tim Rabbat, Fan Zhang, Bogdan Munteanu, Marc Brooker, and Michael Deardeuff, “How Amazon Web Services Uses Formal Methods,” *Communications of the ACM* **58**(4), 66–73 (2015). A wonderfully accessible report by practicing engineers on how AWS uses TLA+ to verify the correctness of distributed systems like S3 and DynamoDB. If you wonder why anyone outside academia cares about temporal logic, this is your answer.
- Edmund M. Clarke, E. Allen Emerson, and Joseph Sifakis, “Model Checking: Algorithmic Verification and Debugging,” *Communications of the ACM* **52**(11), 74–84 (2009). The Turing Award lecture: a retrospective on how model checking was developed and why it works. Written for a general CS audience.
- Amir Pnueli, “The Temporal Logic of Programs,” *18th Annual Symposium on Foundations of Computer Science* (FOCS), 46–57 (1977). The founding paper of program temporal logic. Sections 1–2 are historically essential and undergraduate-readable; later sections get more technical.

Chapter 8

Sequences and Recursion

Reading map

Epp sections. §5.1–5.3: sequences, mathematical induction, and applying induction to sum identities.

Prerequisites. Module 4 (which introduced natural induction) and the “More on Natural Induction” intermezzo (which walked through several inductive proofs in detail).

Relevant intermezzi. The natural-induction intermezzo between Modules 4 and 5 is a direct prerequisite; the type-algebra intermezzo (after Module 10) complements this module by showing another side of the “data definitions = induction principles” story.

Practice from Epp. §5.1 exercises on computing terms of sequences and translating between explicit and recursive definitions; §5.2 exercises on induction proofs of sum identities; §5.3 exercises on induction applied to closed-form computations.

8.1 Definition by recursion

We’ve been building up our toolkit for a while now—logic, sets, functions, induction—and in this module we put a lot of it together to talk about *sequences*. Informally, a sequence is just an ordered list of values, and you’ve seen these before: arrays in C, lists in Python, streams in Haskell. The values come in a definite order—there’s a first element, a second, a third, and so on.

Definition 8.1.1 (Sequence). A *sequence* is a function from the natural numbers (or some subset of them) to some set. If we write $a_1, a_2, a_3, \dots$ what we really mean is that there’s a function $a : \mathbb{N} \rightarrow S$ for some set S , and a_n is just the value of that function at input n . The subscript is the index, the “position” in the list.

If you’re a programmer, think of a_n as `a[n]`—literally the same idea, indexing into a collection. How do we actually *define* a sequence? There are two main approaches.

8.1.1 Explicit formulas

The first way is to just give a formula that lets you compute the n th term directly from n . This is called an *explicit formula* or a *closed-form definition*.

For example, $a_n = 2n + 1$ for $n \geq 0$. That gives us the sequence $1, 3, 5, 7, 9, \dots$ — the odd numbers. If you want the 100th term you just plug in $n = 99$ and get $a_{99} = 199$. You don’t need to know any other terms.

Let's work through a couple more examples from the textbook.

Consider $b_j = \frac{5-j}{5+j}$ for every integer $j \geq 1$. Let's compute the first four terms:

$$b_1 = \frac{5-1}{5+1} = \frac{4}{6} = \frac{2}{3}$$

$$b_2 = \frac{5-2}{5+2} = \frac{3}{7}$$

$$b_3 = \frac{5-3}{5+3} = \frac{2}{8} = \frac{1}{4}$$

$$b_4 = \frac{5-4}{5+4} = \frac{1}{9}$$

And here's another one: $d_m = 1 + \left(\frac{1}{2}\right)^m$ for every integer $m \geq 0$:

$$d_0 = 1 + \left(\frac{1}{2}\right)^0 = 1 + 1 = 2$$

$$d_1 = 1 + \left(\frac{1}{2}\right)^1 = 1 + \frac{1}{2} = \frac{3}{2}$$

$$d_2 = 1 + \left(\frac{1}{2}\right)^2 = 1 + \frac{1}{4} = \frac{5}{4}$$

$$d_3 = 1 + \left(\frac{1}{2}\right)^3 = 1 + \frac{1}{8} = \frac{9}{8}$$

See how the terms are getting closer and closer to 1? That's a *convergent* sequence, but we'll leave that idea for a different class.

8.1.2 Recursive definitions

The other way to define a sequence is *recursively*: you give the first term (or first few terms) and then a rule that builds each subsequent term from the ones that came before.

Instead of the explicit formula $a_n = 2n + 1$ we could equivalently say: $a_0 = 1$ and $a_n = a_{n-1} + 2$ for $n \geq 1$. Same sequence! But a very different way of thinking about it.

The classic example of a recursively defined sequence is the Fibonacci sequence:

```
fib(0) = 0
fib(1) = 1
fib(n) = fib(n-1) + fib(n-2) for n >= 2
```

This should look very familiar! It's *exactly* the same structure as writing a recursive function in Python or C. That's not a coincidence. As we discussed back in Module 4 when we introduced BNF syntax and inductive definitions, recursive definitions are really just another case of defining data inductively. The base case gives you the starting point and the recursive case tells you how to build new data from old data.

In fact, remember how in Module 4 we defined the natural numbers as $\text{Nat} := 0 \mid \text{succ Nat}$? A recursively defined sequence is doing the same thing: defining a_n by cases on whether n is the base case or a successor.

8.1.3 Summation and product notation

There's an important piece of notation we need before we go further: *summation notation*. You've probably seen the big sigma before:

$$\sum_{i=1}^n a_i = a_1 + a_2 + a_3 + \cdots + a_n$$

This is just a compact way of writing “add up all the terms a_i where i goes from 1 to n .” The variable i is the *index* of summation, the bottom tells you where to start, the top tells you where to stop.

If you’re a programmer, this is literally a for-loop:

```
total = 0
for i in range(1, n+1):
    total += a[i]
```

That’s what $\sum$ means. It’s a for-loop.¹

For example:

$$\sum_{i=1}^4 i^2 = 1^2 + 2^2 + 3^2 + 4^2 = 1 + 4 + 9 + 16 = 30$$

And you can also nest these things or get creative with the bounds. The expression

$$\sum_{k=0}^n (-1)^k (k^3 - 1)$$

is just alternating addition and subtraction of the terms $(k^3 - 1)$ because $(-1)^k$ flips sign.

Similarly, there’s *product notation* with the big pi:

$$\prod_{i=1}^n a_i = a_1 \cdot a_2 \cdot a_3 \cdots a_n$$

The most famous example of this is the factorial: $n! = \prod_{i=1}^n i$. Again, same deal as summation but you’re multiplying instead of adding.

8.2 Induction on sequences

Here’s where the fun starts. We introduced the induction principle on natural numbers in Module 4 and worked through proofs about addition in the Intermezzo on Natural Induction. Here we use the same tool to prove things about sequences and sums.

Let’s recall the rule:

```
natural induction
  assumes P(0)
    subproof:
      assumes P(n)
      -----
      shows P(succ n)
  -----
  shows forall n : Nat. P(n)
```

Same rule as before. Nothing new here! The only thing that’s changing is what $P(n)$ is going to be. In the Intermezzo on Natural Induction, $P(n)$ was something like “ $0 + n = n$ ” or “addition is associative.” Now $P(n)$ is going to be equations about sums and sequences.

¹More precisely, the body is pure—no side effects, no mutation beyond the accumulator. If you want to be fancy about it, it’s a *fold*.

8.2.1 The standard template

Here's how an induction proof on a sum identity typically goes. You want to prove that some formula holds for all $n \geq \text{base}$. The template is:

1. **State $P(n)$:** write down exactly what you're trying to prove. This is usually an equation like "the sum of the first n whatevers equals some closed-form expression."
2. **Base case:** verify that $P(\text{base})$ is true by direct computation.
3. **Inductive hypothesis:** assume $P(k)$ is true for some arbitrary $k \geq \text{base}$.
4. **Inductive step:** prove $P(k+1)$ using the assumption that $P(k)$ holds. This is where the actual work happens.

Let's see this in action.

8.2.2 Worked example: sum of the first n integers

We want to prove: for all $n \geq 1$,

$$1 + 2 + 3 + \cdots + n = \frac{n(n+1)}{2}$$

or in summation notation, $\sum_{i=1}^n i = \frac{n(n+1)}{2}$.

Let $P(n)$ be the statement: $\sum_{i=1}^n i = \frac{n(n+1)}{2}$.

Base case ($n = 1$): The left side is just 1. The right side is $\frac{1 \cdot 2}{2} = 1$. They're equal, so $P(1)$ is true.

Inductive hypothesis: Assume $P(k)$ is true for some $k \geq 1$. That is, assume

$$\sum_{i=1}^k i = \frac{k(k+1)}{2}$$

Inductive step: We need to show $P(k+1)$, which says $\sum_{i=1}^{k+1} i = \frac{(k+1)(k+2)}{2}$.

Here's the key trick, and it's the same every time: split the sum into the part we already know about and the new term.

$$\begin{aligned} \sum_{i=1}^{k+1} i &= \left(\sum_{i=1}^k i \right) + (k+1) \\ &= \frac{k(k+1)}{2} + (k+1) && \text{by the inductive hypothesis!} \\ &= \frac{k(k+1)}{2} + \frac{2(k+1)}{2} \\ &= \frac{k(k+1) + 2(k+1)}{2} \\ &= \frac{(k+1)(k+2)}{2} \end{aligned}$$

And that last expression is exactly what $P(k + 1)$ claims. We're done!

See the pattern? You peel off the last term of the sum, apply the inductive hypothesis to the rest, and then do algebra to show you get the right answer. That “peel off the last term” move is the heart of every induction proof on sums.

8.2.3 Setting up: sum of squares

Here's another one that you'll see on the assignment. For all $n \geq 1$:

$$1^2 + 2^2 + \dots + n^2 = \frac{n(n+1)(2n+1)}{6}$$

So $P(n)$ is the statement $\sum_{i=1}^n i^2 = \frac{n(n+1)(2n+1)}{6}$.

The base case is easy: $P(1)$ says $1 = \frac{1 \cdot 2 \cdot 3}{6} = 1$. Check.

For the inductive step, you'd assume $P(k)$ and then look at

$$\sum_{i=1}^{k+1} i^2 = \left(\sum_{i=1}^k i^2 \right) + (k+1)^2 = \frac{k(k+1)(2k+1)}{6} + (k+1)^2$$

and then the game is to do the algebra to show this equals $\frac{(k+1)(k+2)(2k+3)}{6}$. It's the same strategy—peel, substitute, simplify—just with hairier algebra. I'll leave the details to you for the assignment but the key insight is: factor out $(k+1)$ from both terms and then simplify what's left.

8.2.4 Induction is recursion

I want to close with something that I think is really important and ties together a lot of what we've been doing in this course.

Induction and recursion are *the same thing*. Seriously. They're two sides of the same coin.

When you write a recursive function you compute a value by reducing to a smaller case:

```
def sum_to(n):
    if n == 0:
        return 0 # base case
    else:
        return sum_to(n-1) + n # recursive case
```

When you do an induction proof you prove a statement by reducing to a smaller case: you prove $P(k + 1)$ by using $P(k)$. The inductive hypothesis is playing *exactly* the role of the recursive call. The base case of the proof is the base case of the recursion. The inductive step is the recursive step.

Another way to say this is: a recursive function computes a *value* for each natural number by building up from the base case. An inductive proof computes a *proof* for each natural number by building up from the base case. The structure is identical. Induction *is* recursion on proofs.

This is why, back in Module 4, we introduced induction right alongside BNF definitions and recursive structure. They're not separate topics that happen to be in the same chapter—they're the same idea wearing different hats. And once you see that connection, both recursive programming and inductive proofs become a lot less mysterious. You just ask yourself: what's the base case, and how do I build the next one from the previous one?

Sidebar: induction in algorithm design.

Once you see induction-as-recursion, you start seeing it everywhere in CS, especially when arguing that algorithms are correct:

- **Correctness of recursive functions.** To prove a recursive function returns the right answer, you do induction on whatever the recursion is recursing on. Base case: “the function returns the right answer when input is at the base of the recursion.” Inductive step: “assuming the recursive call returns the right answer on a smaller input, the function returns the right answer on the current input.” That’s literally “assume the recursive call works”—which is what every working programmer does intuitively when they write a recursive function.
- **Loop invariants are induction.** Remember the while-loop Hoare rule from Module 7? Establishment is the base case; preservation is the inductive step. A loop invariant is exactly “what $P(n)$ holds after n iterations?” and proving it correct *is* a natural-induction proof on the iteration count.
- **Merge sort, quicksort, binary search.** All of these reduce a problem of size n to one or more subproblems of smaller size. Proving correctness uses *strong induction*—“assume the algorithm works on all smaller inputs”—which we’ll meet properly in Module 9. The runtime analyses (“ $T(n) = 2T(n/2) + n$ ”) are induction on input size too; they just track how long rather than what the answer is.
- **Amortized analysis.** When you say “the amortized cost of **push** on a dynamic array is $O(1)$ even though some pushes are $O(n)$,” you’re really giving an inductive invariant about a *potential function*—the accumulated credit—and proving it’s preserved by each operation. Same shape as a loop invariant.
- **Compiler correctness, type soundness, protocol correctness.** Essentially every proof in programming-language theory or distributed systems is structural induction on syntax trees, execution traces, or message histories. “Progress and preservation” theorems in type soundness are two induction proofs stapled together.

Any time someone says “proof by induction on n ” you can mentally translate that to “here’s a recursive procedure that constructs the proof.” Any time someone says “assume this invariant is preserved by the body of the loop,” you can mentally translate that to “here’s the induction hypothesis.” Two vocabularies, one idea.

8.3 Common mistakes

1. **Forgetting to state $P(n)$ explicitly.** A good induction proof starts by writing down exactly what the claim $P(n)$ is. Students sometimes jump straight to the base case without telling the reader what they’re inducting over, which makes it impossible to check whether the inductive step is proving the right thing.
2. **Inductive step that doesn’t use the inductive hypothesis.** If your inductive step from $P(k)$ to $P(k + 1)$ never actually references $P(k)$, something is off. Either the claim can be proved directly without induction (in which case you should use a direct proof), or you’ve

made an error and you're proving a weaker claim than you think.

3. **Wrong base case.** The base case has to match the lower bound of the claim. “For all $n \geq 5$ ” wants a base case at $n = 5$, not $n = 0$ and not $n = 1$. Students default to $n = 1$ and hope for the best, which gives vacuous base cases for claims that start higher up.
4. **Confusing the sum with the last term.** “Peel off the last term” means splitting $\sum_{i=1}^{k+1} a_i$ into $(\sum_{i=1}^k a_i) + a_{k+1}$. Students sometimes write the split as $a_k + a_{k+1}$ —just the last two terms, omitting the rest of the sum. That’s a very common error that usually shows up as the algebra “not working out.”
5. **Indexing off by one.** Is the sum $\sum_{i=1}^n$ (starting at 1) or $\sum_{i=0}^n$ (starting at 0)? The two differ by whichever term you’re including/excluding at the ends, and the closed form shifts accordingly. Always look at the bounds of summation carefully; an off-by-one there will propagate through the whole proof.
6. **Treating “...” as a proof step.** In writing $1 + 2 + 3 + \dots + n = \frac{n(n+1)}{2}$, the $\dots$ is shorthand for “and I promise the pattern continues,” but it’s not a proof. The induction *is* the proof that the pattern really continues. Students sometimes try to do arithmetic “inside” the dots—for example, writing $1 + 2 + \dots + k + (k+1) = 1 + 2 + \dots + (k+1)$ and calling that a step—which is really just a tautology and contributes nothing. The meaningful step is always the one that applies the inductive hypothesis.

8.4 Summary and looking ahead

Key takeaways.

- A *sequence* is just a function from $\mathbb{N}$ (or a subset) to some set. You can define a sequence explicitly ($a_n = 2n + 1$) or recursively ($a_0 = 1$, $a_n = a_{n-1} + 2$).
- *Summation notation* ($\sum$) and *product notation* ($\prod$) are compact ways to write for-loops in mathematics.
- An induction proof on a sum identity always follows the same template: state $P(n)$, check the base case, assume $P(k)$, and prove $P(k+1)$ by peeling off the last term and applying the IH.
- *Induction is recursion on proofs.* A recursive function computes a value; an inductive proof “computes” a proof. The base case of the code is the base case of the proof; the recursive call is the inductive hypothesis. This is not a metaphor.

What we built this module. We’re now fluent with induction as a proof tool. The sum-formula exercises are finger exercises; the deeper point is one we’ve been sneaking up on since Module 4: **every recursive definition carries an induction principle**. Natural numbers are defined by $\text{Nat} := 0 \mid \text{succ Nat}$, and the shape of that BNF *is* the shape of the natural-induction rule—base case for 0, inductive step across **succ**. The induction principle was never a separate fact we had to memorize; it came bundled with the definition. Modules 9 and 10 will make this slogan progressively more explicit, until in Module 10 we can read the induction principle for *any* BNF-defined type straight off the grammar.

Looking ahead. Before you start Module 9, here's a problem that should feel like it *should* yield to what we just built, but won't:

Prove that every integer $n \geq 2$ is either prime or a product of primes.

Try to set up a natural-induction proof on n . The base case is easy: $n = 2$ is prime, done. The inductive step is where it breaks. Assume every integer up to k is prime or a product of primes, and try to conclude $k + 1$ is too. If $k + 1$ is composite, it factors as $k + 1 = a \cdot b$ with $2 \leq a, b \leq k$ —but which a ? Some random integer in $\{2, \dots, k\}$. Natural induction only hands you the hypothesis for k , one specific value. You need it for the specific a and the specific b , which could land anywhere below $k + 1$. You're stuck.

That stuckness is the itch Module 9 scratches. It introduces *strong induction*—an inductive hypothesis that covers *all* $j \leq k$ at once, which is exactly what the prime-factorization argument wanted—and then turns around and uses the same principle for recurrences that reach back more than one step and for divide-and-conquer algorithms. Module 9 also covers the characteristic-equation method for solving linear recurrences in closed form, which is how we get the Fibonacci formula with φ and $\sqrt{5}$.

8.5 Exercises

Organized into **warm-up** (computation with sequences and summation), **standard** (inductive proofs following the template), **challenge** (tighter arguments and inequalities), and **connection** (programming and forward pointers). Solutions are in the appendix.

Warm-up

Exercise 1. Write a recursive definition for the sequence $a_n = 3n$ (i.e., give a_0 and a rule for a_n in terms of a_{n-1}). Verify your definition by computing the first four terms and checking that they match 0, 3, 6, 9.

Exercise 2. Compute the following sums explicitly:

(a) $\sum_{i=1}^5 (2i + 1)$

(b) $\sum_{k=0}^4 2^k$

(c) $\sum_{j=1}^3 (-1)^{j+1} j^2$

(d) $\prod_{i=1}^4 i$ (yes, this is 4!)

Exercise 3. Translate between the two forms:

- (a) The sequence a_n is defined by $a_0 = 2$ and $a_n = a_{n-1} + 4$ for $n \geq 1$. Give an explicit formula for a_n .
- (b) The sequence $b_n = n^2 + 1$ is defined explicitly for $n \geq 0$. Give a recursive definition.

Exercise 4. Compute the first five terms of the Fibonacci sequence (starting from $\text{fib}(0) = 0$) and verify by hand that $\text{fib}(5) = \text{fib}(4) + \text{fib}(3)$.

Standard

Exercise 5. Prove by induction: for all $n \geq 1$, $\sum_{i=1}^n (2i - 1) = n^2$. (This says the sum of the first n odd numbers is n^2 .) Follow the template: state $P(n)$, check the base case, assume $P(k)$, and prove $P(k + 1)$ by peeling off the last term.

Exercise 6. Prove by induction: for all $n \geq 1$, $\sum_{i=1}^n i^2 = \frac{n(n+1)(2n+1)}{6}$. Do the full algebra carefully—this is a drill-heavy exercise with hairier arithmetic than the n -th-triangular-number proof.

Exercise 7. Prove by induction: for all $n \geq 0$, $\sum_{i=0}^n 2^i = 2^{n+1} - 1$. (This is the formula for a geometric series with ratio 2.)

Exercise 8. Prove by induction: for all $n \geq 1$, $\sum_{i=1}^n i^3 = \left(\frac{n(n+1)}{2}\right)^2$. This striking identity says that the sum of the first n cubes equals the *square* of the n -th triangular number—a surprise that's worth sitting with. (Hint: you already know the sum of the first n integers; use that in your algebra.)

Challenge

Exercise 9. Prove by induction: for all $n \geq 1$, $\sum_{i=1}^n \frac{1}{i(i+1)} = \frac{n}{n+1}$. (Hint: in the inductive step, after you apply the IH, look for a common denominator. The algebra is surprisingly clean.)

Exercise 10. Prove by induction: for all $n \geq 4$, $2^n < n!$. This is an *inequality* rather than an equality, which means the inductive step needs different algebra than the sum proofs. (Hint: the base case is $n = 4$. In the inductive step, assume $2^k < k!$ for $k \geq 4$ and show $2^{k+1} < (k+1)!$. Notice that $2^{k+1} = 2 \cdot 2^k$ and $(k+1)! = (k+1) \cdot k!$, so the IH gives you a factor of 2 on the left and a factor of $(k+1) \geq 5$ on the right.)

Exercise 11. Define a sequence by $T_1 = 1$ and $T_n = T_{n-1} + n$ for $n \geq 2$. (These are the triangular numbers.) Prove by induction that $T_n = \frac{n(n+1)}{2}$. Compare your proof to the worked example in this module—the only difference should be the starting point and the naming.

Connection

Exercise 12. (Programming.) Write a recursive Python function `sum_of_squares(n)` that computes $\sum_{i=1}^n i^2$. Then compare its structure to the induction proof of the sum-of-squares formula (Exercise 6). Which part of your code corresponds to the base case? Which part corresponds to the inductive step? Where does the “inductive hypothesis” show up in the code?

Exercise 13. (Programming.) Write both a recursive and an iterative Python function that computes the n -th Fibonacci number. Test that they agree on $n = 0, 1, 2, \dots, 10$. Then time both versions on $n = 30$. Which is faster, and by how much? Why? (If you want, also write a memoized version and time that.)

Exercise 14. (Intermezzo pointer.) The intermezzo on type algebra (after Module 10) explains that BNF-style recursive data definitions correspond to *initial algebras* of functors, and that induction principles are *unique-morphism* properties of these algebras. As a preview: the natural numbers $\text{Nat} := 0 \mid \text{succ Nat}$ have a single induction principle, and so does each list type, each tree type, etc. Without peeking ahead, speculate on why “one induction principle per recursive type definition” is such a clean correspondence. What does it suggest about how to define induction principles for new types you might encounter later?

Chapter 9

Strong Induction and Recurrences

Reading map

Epp sections. §5.4–5.9: strong induction, recursively defined sequences, solving recurrences by iteration, and second-order linear homogeneous recurrences with constant coefficients (including the characteristic-equation method).

Prerequisites. Module 8 (ordinary induction, summation notation).

Relevant intermezzi. The type-algebra intermezzo after Module 10 uses the geometric-series and generating-function ideas from this module to read closed-form counts out of BNF definitions—so it’s a direct downstream user of §9.3 once you get there. Worth flagging now so you notice the connection when you meet it.

Practice from Epp. §5.4 exercises on strong induction; §5.6 exercises on solving recurrences by iteration; §5.7–5.8 exercises on the characteristic-equation method for second-order linear homogeneous recurrences (both distinct-roots and repeated-root cases).

9.1 Strong induction

Module 8 closed with a cliffhanger: a proof that gets stuck. “Every integer $n \geq 2$ is prime or a product of primes” wants an inductive argument, but when you try natural induction on n you hit a wall—if $k + 1 = a \cdot b$ factors into some a, b between 2 and k , the inductive hypothesis only gives you $P(k)$ and you need $P(a)$ and $P(b)$ for *whichever* smaller numbers those happen to be. That’s the itch this section scratches.

More generally: ordinary induction lets you assume *only* the immediately previous case. You’re proving $P(k + 1)$ and all you get to work with is $P(k)$. But what if your problem reaches back further—to $P(k - 1)$, or $P(k - 3)$, or to some value you can’t pin down in advance because it depends on how a factorization happens to land?

Strong induction is the fix. Instead of just assuming $P(k)$, you get to assume $P(j)$ for **all** j from the base case up to k . You get the whole history, not just the last step.

Here’s the formal rule:

Strong Induction:

```
assumes P(0)
  subproof:
    assumes P(j) for all j <= k
    -----
```

shows $P(k+1)$ shows forall $n : \text{Nat}. P(n)$

Now you might be wondering: is this *really* a valid thing to do? Doesn't assuming more stuff make the proof weaker somehow? Nope! It turns out that strong induction is actually *equivalent* to ordinary induction. You can derive one from the other.¹ The two principles have the same logical strength, but strong induction is often way more convenient when the recursive structure of your problem doesn't just depend on the immediately previous case.

Let's connect this to programming because I think it makes the idea really clear. Ordinary induction is like a recursive function that can *only* call itself on $n - 1$:

```
def f(n):
    if n == 0:
        return base_case
    else:
        return something(f(n-1))
```

Strong induction, on the other hand, is like a recursive function that can call itself on *any* smaller input:

```
def f(n):
    if n == 0:
        return base_case
    else:
        # can use f(n-1), f(n-2), ..., f(0), whatever you need!
        return something(f(n-3), f(n//2))
```

Think of merge sort or binary search! In merge sort you split the list roughly in half and recurse on both halves—you're not just peeling off one element at a time. That's the spirit of strong induction.

Let's do a worked example. Consider the sequence defined by

$$c_0 = 2, \quad c_1 = 2, \quad c_2 = 6, \quad c_k = 3c_{k-3} \text{ for } k \geq 3$$

We want to prove that *all* terms of this sequence are even, i.e. that $2 \mid c_n$ for all $n \geq 0$.

Why do we need strong induction here? Well, the recurrence reaches back *three* steps: c_k depends on c_{k-3} , not c_{k-1} . Ordinary induction would only hand us c_{k-1} which isn't directly useful.

Base cases: We need to check $P(0)$, $P(1)$, and $P(2)$. We have $c_0 = 2$, $c_1 = 2$, $c_2 = 6$, and all of these are even. Good.²

Inductive step: Assume $P(j)$ for all $j \leq k$ where $k \geq 2$. We need to show $P(k+1)$, i.e. that c_{k+1} is even.

Since $k+1 \geq 3$, we can use the recurrence: $c_{k+1} = 3 \cdot c_{(k+1)-3} = 3 \cdot c_{k-2}$.

Now $k-2 \leq k$, so by our strong induction hypothesis c_{k-2} is even. That means $c_{k-2} = 2m$ for some integer m . Therefore $c_{k+1} = 3 \cdot 2m = 6m = 2(3m)$, which is even. Done!

See how we needed to reach back to c_{k-2} and the strong induction hypothesis let us do that? With ordinary induction we'd have been stuck.

¹The trick is that if you have a proof by strong induction for a predicate P , you can convert it to an ordinary-induction proof of the auxiliary predicate $Q(n) := \forall j \leq n. P(j)$. The ordinary-induction step for Q is exactly the strong-induction step for P .

²When you use strong induction, you often need multiple base cases. The number of base cases typically matches how far back the recurrence reaches.

Here's a second example that shows off a different flavor of strong induction—one where you don't reach back a fixed number of steps, but an *unknown* number of steps. This is more like the merge sort analogy: you split the problem roughly in half, and the pieces could land anywhere below you.

Theorem 9.1.1. Every integer $n \geq 2$ is either prime or a product of primes.

Proof. By strong induction on n .

Base case: $n = 2$ is prime. Done.

Inductive step: Assume that every integer j with $2 \leq j \leq k$ is either prime or a product of primes. We need to show this for $k + 1$ (where $k \geq 2$).

There are two cases. If $k + 1$ is prime, we're done immediately. If $k + 1$ is composite, then by definition $k + 1 = a \cdot b$ for some integers a, b with $2 \leq a, b \leq k$. Since both a and b are at most k , our strong induction hypothesis applies to each of them: a is either prime or a product of primes, and so is b . Therefore $k + 1 = a \cdot b$ is a product of primes. $\square$

Notice why ordinary induction would be hopeless here. If $k + 1 = a \cdot b$ is composite, the factors a and b could be *anywhere* between 2 and k —they're not necessarily equal to k or $k - 1$ or any particular value. We need the full strength of the strong induction hypothesis to handle all of them at once. This is a case where the “binary search” analogy is apt: the subproblems aren't one step back, they're roughly half the size, and strong induction is exactly the tool that lets us recurse on arbitrary smaller values.

9.2 Defining sequences recursively

We've been using recursive sequence definitions informally and it's worth being a little more careful about what that means. A *recursively defined sequence* (sometimes called a *recurrence relation*) has two parts:

- **Initial conditions:** explicit values for one or more of the first terms
- **A recurrence relation:** a rule that defines each subsequent term in terms of earlier terms

This is exactly like defining a recursive function in code: you need base cases and a recursive case. If you leave out the base cases your recursion never bottoms out and you get a stack overflow (or, in the math world, an ill-defined sequence).

Let's compute a few terms of a sequence to make sure we're comfortable with the mechanics. Consider

$$d_0 = 3, \quad d_k = k \cdot (d_{k-1})^2 \text{ for } k \geq 1$$

Let's just crank out some values:

$$\begin{aligned} d_0 &= 3 \\ d_1 &= 1 \cdot (d_0)^2 = 1 \cdot 9 = 9 \\ d_2 &= 2 \cdot (d_1)^2 = 2 \cdot 81 = 162 \\ d_3 &= 3 \cdot (d_2)^2 = 3 \cdot 26244 = 78732 \end{aligned}$$

These numbers get big fast! That's the power of squaring in the recurrence. The point is that even though we don't have a nice closed-form formula, we can always compute any term we want as long as we know the previous ones.

Now here's a useful skill: given a proposed *explicit* (closed-form) formula, how do we *verify* that it actually satisfies a recurrence?

For instance, suppose someone hands you the formula

$$s_n = \frac{(-1)^n}{n!}$$

and claims it satisfies the recurrence $s_k = -s_{k-1}/k$ for $k \geq 1$ with $s_0 = 1$.

Let's check. First, $s_0 = (-1)^0/0! = 1/1 = 1$. Good, initial condition checks out. Now we compute $-s_{k-1}/k$:

$$\begin{aligned} -\frac{s_{k-1}}{k} &= -\frac{1}{k} \cdot \frac{(-1)^{k-1}}{(k-1)!} \\ &= \frac{(-1) \cdot (-1)^{k-1}}{k \cdot (k-1)!} \\ &= \frac{(-1)^k}{k!} \\ &= s_k \end{aligned}$$

And that's it! The formula checks out. The key idea is that verifying a closed form is usually way easier than *finding* it in the first place. You just plug the formula into both sides of the recurrence and show they're equal.

9.3 Solving recurrences by iteration

How *do* you find a closed form in the first place? One really useful technique is called *iteration* (or “unrolling”). The idea is simple: you just keep substituting the recurrence into itself until you see a pattern, and then you guess the closed form.

Before we do an example of this, let's recall a formula that shows up everywhere: the *geometric series*. If $r \neq 1$ then

$$1 + r + r^2 + \cdots + r^n = \frac{r^{n+1} - 1}{r - 1}$$

This is going to come in handy when we unroll recurrences because geometric sums pop up all the time.

Let's try iterating a concrete recurrence. Consider the “number of regions” problem: you have a plane, and you're drawing lines through it. Let P_k be the number of regions created by k lines (where no two lines are parallel and no three meet at a point). The k -th line crosses all $k-1$ existing lines, creating k new regions, so:

$$P_1 = 2, \quad P_k = P_{k-1} + k \text{ for } k \geq 2$$

Let's unroll this:

$$\begin{aligned}
P_n &= P_{n-1} + n \\
&= (P_{n-2} + (n-1)) + n \\
&= ((P_{n-3} + (n-2)) + (n-1)) + n \\
&= \dots \\
&= P_1 + 2 + 3 + \dots + n \\
&= 2 + 2 + 3 + \dots + n \\
&= 1 + (1 + 2 + 3 + \dots + n) \\
&= 1 + \frac{n(n+1)}{2}
\end{aligned}$$

where we used the familiar formula $1 + 2 + \dots + n = n(n+1)/2$ at the end. So the closed form is $P_n = 1 + n(n+1)/2$.

Let's sanity check: $P_1 = 1 + 1 \cdot 2/2 = 2$. $P_2 = 1 + 2 \cdot 3/2 = 4$. And indeed if you draw two crossing lines you get 4 regions. Looks right!

Now, to be rigorous, once you've *guessed* the formula by iteration you should really *prove* it by induction. The iteration gives you the conjecture; induction gives you the proof. But in practice, the iteration is often the hard part and the induction proof is pretty mechanical after that.

9.4 Second-order linear homogeneous recurrences

Iteration is a powerful guess-and-check technique, but it depends on you spotting a pattern. For one especially important class of recurrences, we don't need to guess at all—there's a *systematic method* that cranks out the closed form from the recurrence itself. That class is the subject of this section.

Definition 9.4.1 (Second-order linear homogeneous recurrence). A *second-order linear homogeneous recurrence with constant coefficients* is a recurrence of the form

$$a_k = A \cdot a_{k-1} + B \cdot a_{k-2}$$

where A and B are constants (they don't depend on k).

Let's break down the terminology because every word in that name actually means something:

- **Second-order:** the recurrence depends on the *two* previous terms (a_{k-1} and a_{k-2})
- **Linear:** the previous terms show up to the first power only—no a_{k-1}^2 or $a_{k-1} \cdot a_{k-2}$ business
- **Homogeneous:** there's no extra added constant term. Compare $a_k = 2a_{k-1} + 3$ (not homogeneous) vs. $a_k = 2a_{k-1} + a_{k-2}$ (homogeneous)
- **Constant coefficients:** A and B are just numbers, they don't change with k

Why do we care about this specific class? Because there's a *systematic method* for solving them! We don't have to guess and check. Here's the trick.

We're going to *guess* that the solution has the form $a_n = r^n$ for some constant r . Why is this a reasonable guess? The recurrence says “each term is a fixed linear combination of the previous two,”

and pure exponentials are the sequences that satisfy this identically: for any r , the ratio r^k/r^{k-1} is just r , and r^k/r^{k-2} is just r^2 —both constants. So a relation like $r^k = A \cdot r^{k-1} + B \cdot r^{k-2}$ collapses to a single equation in r that doesn't depend on k at all. That's exactly the defining property of homogeneous linear recurrences, which is why the guess works. Any non-exponential guess would leave k in the equation and go nowhere.

So let's substitute $a_n = r^n$ into $a_k = A \cdot a_{k-1} + B \cdot a_{k-2}$:

$$r^k = A \cdot r^{k-1} + B \cdot r^{k-2}$$

Divide both sides by r^{k-2} (assuming $r \neq 0$):

$$r^2 = Ar + B$$

This is the *characteristic equation* of the recurrence. It's just a quadratic! We know how to solve quadratics.

Now here's where it gets interesting. If the characteristic equation has two *distinct* roots r_1 and r_2 , then the general solution is

$$a_n = \alpha \cdot r_1^n + \beta \cdot r_2^n$$

where α and β are constants determined by the initial conditions a_0 and a_1 . In other words, any linear combination of the two solutions is also a solution,³ and we use the initial conditions to pin down which particular linear combination we want.

If the characteristic equation has a *repeated* root r (i.e. $r_1 = r_2 = r$), then the general solution is slightly different:

$$a_n = \alpha \cdot r^n + \beta \cdot n \cdot r^n$$

The extra factor of n shows up because we need two linearly independent solutions and r^n alone only gives us one. But the repeated root case is rarer in practice so let's focus on the distinct roots case.

Let's do the most famous example: the *Fibonacci sequence*.

$$F_0 = 0, \quad F_1 = 1, \quad F_k = F_{k-1} + F_{k-2} \text{ for } k \geq 2$$

This is a second-order linear homogeneous recurrence with $A = 1$ and $B = 1$. The characteristic equation is

$$r^2 = r + 1 \quad \implies \quad r^2 - r - 1 = 0$$

By the quadratic formula:

$$r = \frac{1 \pm \sqrt{5}}{2}$$

So $r_1 = \frac{1+\sqrt{5}}{2}$ (this is the *golden ratio*, often written φ) and $r_2 = \frac{1-\sqrt{5}}{2}$.

The general solution is $F_n = \alpha \cdot \varphi^n + \beta \cdot \left(\frac{1-\sqrt{5}}{2}\right)^n$. Now we use the initial conditions to find α and β :

$$\begin{aligned} F_0 = 0 : \quad \alpha + \beta &= 0 \quad \implies \quad \beta = -\alpha \\ F_1 = 1 : \quad \alpha \cdot \varphi + \beta \cdot \frac{1-\sqrt{5}}{2} &= 1 \end{aligned}$$

³This is one of those things where “linear” in the name really matters. It's exactly the same principle as in linear algebra where solutions to homogeneous systems form a vector space.

Substituting $\beta = -\alpha$:

$$\alpha \left(\varphi - \frac{1-\sqrt{5}}{2} \right) = \alpha \cdot \sqrt{5} = 1 \implies \alpha = \frac{1}{\sqrt{5}}$$

So the closed form for the Fibonacci numbers is

$$F_n = \frac{1}{\sqrt{5}} \left[\left(\frac{1+\sqrt{5}}{2} \right)^n - \left(\frac{1-\sqrt{5}}{2} \right)^n \right]$$

That's pretty remarkable, isn't it? You start with a simple recurrence and end up with the golden ratio and $\sqrt{5}$. The Fibonacci numbers are integers, but the formula involves $\sqrt{5}$ all over the place. The irrational parts cancel out perfectly every time—not by accident: the two roots of the characteristic equation are *conjugate* irrationals ($\frac{1\pm\sqrt{5}}{2}$ differ only in the sign of the $\sqrt{5}$), and when you take a difference of their n th powers the rational parts match up and the irrational parts cancel cleanly for every integer n . This is a general feature of linear recurrences with integer coefficients: even when the roots are irrational or complex, the answer lands on integers. Also, since $\left| \frac{1-\sqrt{5}}{2} \right| < 1$, that second term shrinks to zero as n grows, which means F_n is approximately $\varphi^n / \sqrt{5}$ for large n . So the Fibonacci numbers grow exponentially at the rate of the golden ratio!

It all comes from the characteristic equation method: substitute a guess and solve a quadratic.

Sidebar: recurrences in the wild.

Linear recurrences aren't just a math-class curiosity. The same machinery—and often the same characteristic-equation method—runs underneath a surprising amount of CS and applied science.

- **Algorithm analysis.** The running time of a recursive algorithm is almost always a recurrence. Binary search: $T(n) = T(n/2) + O(1)$. Merge sort: $T(n) = 2T(n/2) + O(n)$. Karatsuba multiplication: $T(n) = 3T(n/2) + O(n)$. Strassen matrix multiplication: $T(n) = 7T(n/2) + O(n^2)$. The *Master Theorem* is essentially a closed-form solution to a whole family of these recurrences, and every asymptotic runtime you'll see in a data-structures class comes from this analysis. (Non-homogeneous, divide-and-conquer recurrences need more machinery than our homogeneous method, but the spirit is the same: set up a recurrence, solve it.)
- **Digital signal processing.** The output of an IIR (infinite impulse response) filter is a linear recurrence on the input: $y_n = a_1 y_{n-1} + a_2 y_{n-2} + \dots + b_0 x_n + b_1 x_{n-1} + \dots$. The *poles* of the filter are exactly the roots of the characteristic equation; if a pole has magnitude ≥ 1 the filter blows up, and if all poles are inside the unit circle it's stable. Audio codecs, speech synthesis, noise cancellation—all running characteristic-equation analyses in real time.
- **Control theory.** A discrete-time feedback controller is usually a linear recurrence, and “will the system go unstable?” is exactly the question of whether any root of the characteristic polynomial lies outside the unit disk. Cruise control, thermostat tuning, robot arm controllers, rocket guidance—all asking the same question Fibonacci asked, just with different coefficients.
- **Population and epidemiological models.** The Leslie matrix for age-structured popu-

lations produces a vector recurrence whose long-term behavior is dictated by the dominant eigenvalue, which is a root of the matrix’s characteristic polynomial (same word, for good reason). SIR models of epidemics in discrete time are recurrence-driven.

- **Linear feedback shift registers.** The pseudo-random generators at the heart of many stream ciphers and CRC checksums are literally linear recurrences over $\mathbb{F}_2$. Their period is determined by—wait for it—the characteristic polynomial.

The thread across all of these: when you have a system that’s defined by “next state is a linear combination of recent states,” the characteristic equation of that recurrence tells you almost everything about how the system behaves. The quadratic we solve for Fibonacci is the toy version of an engineering workhorse.

The repeated-root case

We said earlier that when the characteristic equation has a repeated root r , the general solution picks up an extra factor of n : $a_n = \alpha \cdot r^n + \beta \cdot n \cdot r^n$. The first time you see this, it’s tempting to suspect the extra n is a typo—why does the n appear out of nowhere? The reason is that a second-order recurrence has *two* initial conditions, so the general solution needs *two* independent pieces. When the characteristic equation factors as $(r - r_0)^2$, a single exponential r_0^n only gives you one piece; we need a second linearly independent solution, and $n \cdot r_0^n$ turns out to be it. (Checking it *is* a solution is a direct substitution into $a_k = Aa_{k-1} + Ba_{k-2}$ using $A = 2r_0$ and $B = -r_0^2$ —try it.)

Let’s see this with an example that makes the $\beta \cdot n \cdot r^n$ term unambiguously load-bearing. Consider

$$a_0 = 0, \quad a_1 = 2, \quad a_k = 4a_{k-1} - 4a_{k-2} \text{ for } k \geq 2.$$

The characteristic equation is $r^2 - 4r + 4 = 0$, which factors as $(r - 2)^2 = 0$. Repeated root $r = 2$. The general solution is

$$a_n = \alpha \cdot 2^n + \beta \cdot n \cdot 2^n.$$

Using the initial conditions:

$$\begin{aligned} a_0 = 0 : \quad & \alpha \cdot 1 + \beta \cdot 0 \cdot 1 = \alpha = 0, \\ a_1 = 2 : \quad & \alpha \cdot 2 + \beta \cdot 1 \cdot 2 = 2\beta = 2 \implies \beta = 1. \end{aligned}$$

So the closed form is $a_n = n \cdot 2^n$. Notice that $\alpha = 0$: the pure-exponential piece is *gone*, and the entire solution sits on the $\beta \cdot n \cdot 2^n$ term. If we’d tried the “wrong” general solution $\alpha r^n + \beta r^n$, we would have gotten $\alpha + \beta$ as a single free parameter and couldn’t have matched both $a_0 = 0$ and $a_1 = 2$. The factor of n earns its keep.

Sanity check the recurrence: $a_2 = 4a_1 - 4a_0 = 8$, and $2 \cdot 2^2 = 8$. ✓ $a_3 = 4 \cdot 8 - 4 \cdot 2 = 24$, and $3 \cdot 2^3 = 24$. ✓

9.5 Common mistakes

1. **Too few base cases for strong induction.** If your recurrence reaches back k steps, you need at least k base cases—one for each initial condition—or the inductive step won’t have anything to stand on when it starts. Students often check only the first case and assume the rest will work out. The stamp-postage exercise is a good reality check: you need four base cases (12, 13, 14, 15) because the argument reaches back 4.

2. **Confusing the strong IH with the ordinary IH.** The strong IH assumes $P(j)$ for *all* j up to k , not just $j = k$. Students sometimes write the strong IH as the ordinary one and then fail to derive $P(k + 1)$. When you state the inductive hypothesis, write it as “Assume $P(j)$ for all j with $(\text{base}) \leq j \leq k$ ”—the quantifier over j is the whole point.
3. **Characteristic equation without dividing by r^{k-2} .** The derivation substitutes $a_n = r^n$ and then divides both sides by r^{k-2} (assuming $r \neq 0$). Students sometimes skip the division step and write the characteristic equation as $r^k = Ar^{k-1} + Br^{k-2}$, which is fine but unwieldy, or worse, as something involving k , which is wrong. The characteristic equation should have no k in it.
4. **Missing the repeated-root case.** If the characteristic equation factors as $(r - r_0)^2 = 0$, the general solution is $\alpha r_0^n + \beta n r_0^n$, *not* $\alpha r_0^n + \beta r_0^n$. The second form collapses to $(\alpha + \beta)r_0^n$ —one effective free parameter, not two—so it can’t match two independent initial conditions.
5. **Trying to use characteristic equations on non-homogeneous recurrences.** The method works for recurrences of the form $a_k = Aa_{k-1} + Ba_{k-2}$, without any extra constant term or k -dependent term. If your recurrence is $a_k = 2a_{k-1} + 3$, you can’t directly apply the characteristic equation—the extra $+3$ breaks the assumption. (There are techniques for handling non-homogeneous recurrences, but they’re outside this module.)
6. **Iteration without checking the pattern.** The iteration method requires you to *see* the pattern after unrolling. Students sometimes unroll once or twice and guess. Always unroll at least 3–4 times before committing, and always verify the guess by induction once you have one.

9.6 Summary and looking ahead

Key takeaways.

- *Strong induction* gives you $P(j)$ for *all* $j \leq k$ as the inductive hypothesis, not just $P(k)$. Logically equivalent to ordinary induction, but often much more convenient. Needed whenever the recurrence reaches back more than one step, or when you don’t know in advance how far back the dependency goes (as in divide-and-conquer).
- Strong induction typically needs *multiple base cases*: one per initial condition, or enough cases to cover the reach-back of the recurrence.
- *Iteration* (unrolling) is a way to guess a closed form: substitute the recurrence into itself repeatedly, look for a pattern, then verify the pattern by induction.
- The *characteristic-equation method* solves second-order linear homogeneous recurrences with constant coefficients. Substitute $a_n = r^n$, divide out, solve a quadratic. Distinct roots give $\alpha r_1^n + \beta r_2^n$; repeated roots give $\alpha r^n + \beta n r^n$.
- The Fibonacci numbers have a closed form involving $\sqrt{5}$ and the golden ratio $\varphi = (1 + \sqrt{5})/2$. The $\sqrt{5}$ cancels out perfectly on integer inputs.

What we built this module. Strong induction is the last induction principle we need before we can generalize to arbitrary data types. The characteristic-equation method, in particular,

is the kind of “recipe” result you’ll meet again in algorithm analysis, differential equations, and discrete dynamical systems—once you learn it, you can solve whole families of problems mechanically.

Looking ahead. Module 10 pulls everything together: BNF definitions, induction, recursion, and structural reasoning all converge on the observation that *every recursive data type has its own induction principle*, and the shape of the principle is dictated by the BNF. Natural induction (Module 4) and strong induction (this module) are both special cases. The intermezzo on type algebra (after Module 10) makes this mechanism fully rigorous via initial algebras.

9.7 Exercises

Organized into **warm-up** (mechanical checks on the definitions), **standard** (strong-induction and iteration proofs), **challenge** (characteristic-equation applications including repeated roots), and **connection** (programming and forward pointers). Solutions are in the appendix.

Warm-up

Exercise 1. Compute the first five terms of the following recurrences:

- (a) $a_0 = 1$, $a_n = 2a_{n-1} + 1$ for $n \geq 1$.
- (b) $b_0 = 0$, $b_1 = 1$, $b_n = 2b_{n-1} - b_{n-2}$ for $n \geq 2$.
- (c) $c_0 = 3$, $c_1 = 5$, $c_n = c_{n-1} + c_{n-2}$ for $n \geq 2$. (Fibonacci with different initial conditions—what sequence is this?)

Exercise 2. For each of the following recurrences, state whether it is (i) second-order, (ii) linear, (iii) homogeneous, and (iv) has constant coefficients. (Answer each of the four separately.)

- (a) $a_k = 3a_{k-1} - 2a_{k-2}$
- (b) $a_k = a_{k-1} + 2$
- (c) $a_k = k \cdot a_{k-1}$
- (d) $a_k = a_{k-1}^2 + a_{k-2}$
- (e) $a_k = 5a_{k-1} - 6a_{k-2} + 7$

Exercise 3. Write down the characteristic equation for each of the following second-order linear homogeneous recurrences (without solving):

- (a) $a_k = 5a_{k-1} - 6a_{k-2}$
- (b) $a_k = a_{k-1} + a_{k-2}$
- (c) $a_k = 6a_{k-1} - 9a_{k-2}$

Standard

Exercise 4. Prove by strong induction: every integer amount of postage ≥ 12 cents can be made using 4-cent and 5-cent stamps. (Hint: you will need several base cases—check 12, 13, 14, 15 individually. For the inductive step, use the fact that the amount $k + 1 - 4 = k - 3$ is at least 12 when $k + 1 \geq 16$.)

Exercise 5. Use iteration to find a closed form for $T_1 = 1$, $T_k = T_{k-1} + 3$ for $k \geq 2$. That is, unroll the recurrence a few times until you see the pattern, then write down a formula for T_n . Once you have your conjecture, verify it by induction.

Exercise 6. Find the closed form for $a_0 = 1$, $a_1 = 3$, $a_k = 4a_{k-1} - 3a_{k-2}$ using the characteristic equation method. That is: write down the characteristic equation, find its roots, write the general solution, and then use the initial conditions to solve for the constants.

Exercise 7. Find the closed form for $a_0 = 1$, $a_1 = 5$, $a_k = 5a_{k-1} - 6a_{k-2}$ using the characteristic equation method. (The characteristic equation factors nicely—try to spot it before using the quadratic formula.)

Challenge

Exercise 8. Find the closed form for $a_0 = 2$, $a_1 = 12$, $a_k = 6a_{k-1} - 9a_{k-2}$ using the characteristic equation method. This one has a *repeated root*—pay close attention to the form of the general solution ($\alpha r^n + \beta n r^n$, not αr^n). Verify your closed form by computing a_2 both ways.

Exercise 9. Prove by strong induction: if $T_1 = 1$ and $T_k = T_{\lfloor k/2 \rfloor} + T_{\lceil k/2 \rceil} + k$ for $k \geq 2$, then $T_n \leq 2n \log_2 n + n$ for all $n \geq 1$. (This is the merge-sort-like recurrence: split the input in half, recurse on each half, do linear work to combine. The inductive hypothesis has to handle both halves, and neither is adjacent to k , so strong induction is essential. You can use $\log_2$ manipulations freely.)

Exercise 10. Consider the Tower of Hanoi recurrence: $H_1 = 1$ and $H_n = 2H_{n-1} + 1$ for $n \geq 2$. This is *not* homogeneous, so the characteristic-equation method doesn't directly apply. Use iteration instead to find a closed form, then verify by induction. (Hint: unroll the recurrence and notice that you get a geometric-series-looking quantity at the end.)

Connection

Exercise 11. (Programming.) Implement three versions of a Fibonacci evaluator:

- (a) `fib_closed(n)`: uses the closed form derived in this module (with $\sqrt{5}$ and φ), rounds to the nearest integer at the end.
- (b) `fib_recurrence(n)`: uses the recurrence directly (iterative version, $O(n)$ time).
- (c) `fib_matrix(n)`: uses the matrix form $\begin{pmatrix} F_{n+1} & F_n \\ F_n & F_{n-1} \end{pmatrix} = \begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix}^n$ with fast matrix exponentiation ($O(\log n)$ time).

Compare their outputs for $n = 0, 1, 2, \dots, 30$. Do they all agree? The closed-form version will eventually disagree with the others for large n due to floating-point error—at roughly what n does this first happen on your machine? This is a nice concrete demonstration of the tradeoff between “mathematically elegant” and “numerically stable.”

Exercise 12. (Forward pointer.) Strong induction is the natural proof principle for *structural* induction on general recursive data types (see Module 10). As a preview: if we define binary trees as $\text{Tree} := \text{Leaf} \mid \text{Node Tree Tree}$, then induction on a tree with two subtrees needs an inductive hypothesis for *each* subtree, and neither subtree is a “predecessor” of the whole tree in the ordinary-induction sense—they’re just smaller. This is exactly the situation where strong induction (or its sibling, structural induction) is the right tool. Without proving anything, explain in 2–3 sentences why ordinary induction on $n = \text{height of the tree}$ would let you *simulate* the right argument, and what’s ugly about doing it that way compared to going directly on tree structure.

Chapter 10

Structural Induction

Reading map

Epp sections. This module doesn't correspond directly to an Epp section—Epp's §5.3 covers the closely related topic of recursive definitions on the naturals, but the more general “structural induction on arbitrary recursive data” is a computer-science-flavored topic that most discrete-math texts don't emphasize.

Prerequisites. Modules 4 (natural induction) and 8 (induction is recursion). Module 9 (strong induction) is helpful background for two of the exercises below and for tree-on-tree arguments where the sub-parts aren't adjacent, but the main module text doesn't strictly require it.

Relevant intermezzi. “The Algebra of Types” (immediately after this module) explains why BNF definitions and induction principles line up perfectly—the pattern isn't a coincidence, it's a theorem about initial algebras.

Practice from Epp. The exercises in §5.3 on recursive definitions are the closest match; beyond that, this module is the one where CS 250 most clearly points beyond the textbook.

10.1 The pattern so far

We've arrived at the last module and I want to name something that's been hiding in plain sight all quarter. Let's take a quick tour of what we've done.

Back in Module 2 we introduced BNF as a way to define the syntax of propositional logic:

$$L := T \mid F \mid L \wedge L \mid L \vee L \mid \sim L \mid L \rightarrow L \mid x$$

Then in Module 4 we used the exact same idea to define the natural numbers:

$$\text{Nat} := 0 \mid \text{succ Nat}$$

and we got *natural induction* out of it—for free! Two cases in the BNF, two obligations in the proof: handle zero, handle successor. And in Module 8 we saw that induction is really just recursion on proofs. A recursive function computes a value by reducing to a smaller case; an inductive proof proves a statement by reducing to a smaller case. Same thing, different hat.

Here's what I want to make explicit now. The connection between BNF definitions and induction principles isn't some special coincidence about natural numbers. It's a **general pattern**: *any* BNF definition gives you an induction principle. Every single time. The base cases of the BNF become base cases of your proof, and the recursive cases of the BNF become inductive steps where you get to assume the property holds for the sub-parts.

We've actually been using this pattern all along without naming it. Now let's name it—*structural*

induction—and demonstrate it on two new datatypes that show up constantly in computer science: lists and binary trees.

10.2 Lists

10.2.1 Defining lists inductively

What is a list? You’ve used lists in every programming class you’ve ever taken. Python’s `[1, 2, 3]`, Java’s `ArrayList`, Haskell’s `[a]`—they’re everywhere. But what *is* a list, really, when you strip away all the implementation details?

Here’s the answer, in the same BNF style we’ve been using all quarter:

Definition 10.2.1 (List).
`List A := Nil | Cons(A, List A)`

A list of A ’s is either *empty* (which we call `Nil`) or it’s an element of type A stuck onto the front of another list (which we call `Cons`).¹

Two cases. Sound familiar?

Let’s make sure this connects to what you already know. In Python, the empty list `[]` is our `Nil`. And the list `[1, 2, 3]` is really:

```
# [1, 2, 3] is secretly:
Cons(1, Cons(2, Cons(3, Nil)))
```

In Haskell this is even more explicit. The empty list is `[]` and the `cons` operator is `:`, so you can write `1 : 2 : 3 : []` which is exactly `Cons(1, Cons(2, Cons(3, Nil)))`. This is why the colon operator is so central in Haskell. It’s the recursive case of the datatype.

Now here’s the payoff. Just like the BNF for natural numbers gave us natural induction, the BNF for lists gives us *structural induction on lists*:

```
assumes P(Nil)
  subproof:
    assumes P(xs)
    -----
    shows P(Cons(x, xs))
  -----
shows  $\forall$  xs : List A. P(xs)
```

Look at how perfectly this mirrors the natural induction rule from Module 4! The structure is identical:

- The **base case** corresponds to the base case of the BNF (`Nil`, just like 0 for `Nat`).
- The **inductive step** corresponds to the recursive case of the BNF (`Cons`, just like `succ` for `Nat`), and you get to assume the property holds for the tail `xs` (just like you assumed $P(n)$ when proving $P(\text{succ } n)$).

And this makes sense! A list is basically a natural number with data attached to each successor. `Nil` is like zero, and `Cons` is like successor but it also carries a value. So of course the induction principle looks the same.

¹The name “Cons” comes from Lisp, one of the oldest programming languages, where it was short for “construct.” It’s been the standard name in functional programming ever since.

10.2.2 Recursive functions on lists

Before we can prove anything interesting we need some functions to talk about. Let's define two standard ones: `length` and `append`.

Length. The length of a list is defined recursively, following the two cases of the BNF:

```
length(Nil) = 0
length(Cons(x, xs)) = 1 + length(xs)
```

Or in Python, if you were implementing it from scratch:²

```
def length(lst):
    if lst == []: # Nil case
        return 0
    else: # Cons case
        return 1 + length(lst[1:])
```

Append. The `append` function (which we'll write as `++` because that's what Haskell uses and it's nice and short) concatenates two lists:

```
Nil ++ ys = ys
Cons(x, xs) ++ ys = Cons(x, xs ++ ys)
```

So appending `ys` onto an empty list just gives you `ys`, and appending `ys` onto `Cons(x, xs)` sticks `x` on front and then appends `ys` onto the tail. For example:

```
Cons(1, Cons(2, Nil)) ++ Cons(3, Nil)
= Cons(1, Cons(2, Nil) ++ Cons(3, Nil))
= Cons(1, Cons(2, Nil ++ Cons(3, Nil)))
= Cons(1, Cons(2, Cons(3, Nil)))
```

which is `[1, 2] ++ [3] = [1, 2, 3]`, exactly what you'd expect.

Notice how both of these functions follow the structure of the BNF. Two cases in the datatype, two cases in the function. This is the “recursion is induction” idea from Module 8 showing up again: the shape of your data determines the shape of your code.

10.2.3 Worked proof: length distributes over append

Now let's *prove* something using structural induction on lists. We'll show that:

For all lists `xs` and `ys`:

$$\text{length}(xs ++ ys) = \text{length}(xs) + \text{length}(ys)$$

In other words, the length of two lists concatenated together is the sum of their individual lengths.

We'll do structural induction on `xs`. Our property $P(xs)$ is:

$$\text{for all } ys, \text{length}(xs ++ ys) = \text{length}(xs) + \text{length}(ys).$$

Base case (`xs = Nil`): We need to show that `length(Nil ++ ys) = length(Nil) + length(ys)` for all `ys`.

²Yes, Python has a built-in `len()` function. We're defining it ourselves because we need to reason about it.

The left side:

$$\text{length}(\text{Nil} ++ \text{ys}) = \text{length}(\text{ys}) \quad (\text{by definition of } ++)$$

The right side:

$$\begin{aligned} \text{length}(\text{Nil}) + \text{length}(\text{ys}) &= 0 + \text{length}(\text{ys}) && (\text{by definition of length}) \\ &= \text{length}(\text{ys}) \end{aligned}$$

They're equal. Base case done.

Inductive step ($\text{xs} = \text{Cons}(x, \text{xs}')$): Assume $P(\text{xs}')$ holds, i.e. assume that for all ys ,

$$\text{length}(\text{xs}' ++ \text{ys}) = \text{length}(\text{xs}') + \text{length}(\text{ys}) \quad (\text{IH})$$

We need to show $\text{length}(\text{Cons}(x, \text{xs}') ++ \text{ys}) = \text{length}(\text{Cons}(x, \text{xs}')) + \text{length}(\text{ys})$.

Let's work on the left side:

$$\begin{aligned} &\text{length}(\text{Cons}(x, \text{xs}') ++ \text{ys}) \\ &= \text{length}(\text{Cons}(x, \text{xs}' ++ \text{ys})) && (\text{by definition of } ++) \\ &= 1 + \text{length}(\text{xs}' ++ \text{ys}) && (\text{by definition of length}) \\ &= 1 + \text{length}(\text{xs}') + \text{length}(\text{ys}) && (\text{by IH!}) \end{aligned}$$

And the right side:

$$\begin{aligned} &\text{length}(\text{Cons}(x, \text{xs}')) + \text{length}(\text{ys}) \\ &= (1 + \text{length}(\text{xs}')) + \text{length}(\text{ys}) && (\text{by definition of length}) \\ &= 1 + \text{length}(\text{xs}') + \text{length}(\text{ys}) \end{aligned}$$

They're the same! So $P(\text{Cons}(x, \text{xs}'))$ holds, and by structural induction we conclude $P(\text{xs})$ for all lists xs . $\square$

See the pattern? It's the exact same game as the induction proofs we did on natural numbers in Modules 4 and 8. Unfold the definitions, apply the inductive hypothesis, watch things simplify. The only difference is that instead of peeling off a `succ` we're peeling off a `Cons`.

Making “induction is recursion” concrete on this proof

Back in Module 8 I kept insisting that induction and recursion are the same thing. Let's pin that down on the proof we just finished, because structural induction on lists is the cleanest place to see it.

The statement we proved was “for all lists xs , $P(\text{xs})$ ” where $P(\text{xs})$ is “for all ys , $\text{length}(\text{xs} ++ \text{ys}) = \text{length}(\text{xs}) + \text{length}(\text{ys})$.” Imagine you wanted to *compute* a proof of $P(\text{xs})$ for a specific concrete list xs —say, a list of length three. What would that computation look like? It's a recursive function:

```
# Informal pseudocode: returns a "proof object" that length distributes
# over ++ for this particular xs. Every "return" line corresponds to a
# justified equation in the actual proof.
def length_append_proof(xs):
    if xs == Nil:
        # Base case of the induction:
        # length(Nil ++ ys) = length(ys)
```

```

    # = 0 + length(ys)
    # = length(Nil) + length(ys)
    return "base␣case␣chain␣of␣equalities"
else: # xs == Cons(x, xs_tail)
    # Inductive case. Recurse on the tail -- this recursive call IS
    # the inductive hypothesis:
    IH = length_append_proof(xs_tail)

    # Now use IH to build the proof for Cons(x, xs_tail):
    # length(Cons(x, xs_tail) ++ ys)
    # = length(Cons(x, xs_tail ++ ys)) [def of ++]
    # = 1 + length(xs_tail ++ ys) [def of length]
    # = 1 + length(xs_tail) + length(ys) [by IH]
    # = length(Cons(x, xs_tail)) + length(ys) [def of length]
    return "inductive-step␣chain,␣using␣IH"

```

Reread the worked proof with this pseudocode next to it. The `if xs == Nil` branch is the base case of the induction. The recursive call `length_append_proof(xs_tail)` is the inductive hypothesis—“assume $P(xs')$,” written in the vocabulary of “recurse on the tail.” The final equality chain in the `else` branch is the inductive step. No piece of the proof is missing from the code, and no piece of the code is missing from the proof. They are not two similar things; they are the same object written in two notations.

This is why in proof assistants like Lean, Agda, and Coq the induction principle for a datatype is literally a recursive function—“eliminator” is the technical name—and you write proofs by case-splitting and recursing exactly the way you’d write a program. The “proof” you end up with is a program whose return type is a logical proposition rather than a number. That’s the Curry–Howard correspondence, which the earlier intermezzo gestured at and which you’ll see more of in CS 251.

10.3 Binary trees

10.3.1 Defining binary trees inductively

Now let’s look at a slightly more interesting datatype. You’ve likely seen binary trees in a data structures class (or you will soon). Binary search trees, heaps, parse trees, &c. They’re one of the most important data structures in all of computer science.

Here’s the BNF definition:

Definition 10.3.1 (Binary Tree).
`BinTree := Leaf | Node(BinTree, BinTree)`

A binary tree is either a *leaf* (an empty tree, a dead end) or it’s a *node* with a left subtree and a right subtree.³

Again, two cases. But notice something new: the recursive case `Node` has **two** recursive sub-parts, not one. A cons cell had one tail; a node has two children. This is going to affect the induction principle.

In Haskell you’d write this as:

```
data BinTree = Leaf | Node BinTree BinTree
```

³Some definitions of binary trees store data at the nodes, like `Node(A, BinTree, BinTree)`. We’re keeping it simple here and just tracking the shape of the tree. Everything we prove about the shape applies equally well when you add data.

And in Python, maybe something like:

```
class Leaf:
    pass

class Node:
    def __init__(self, left, right):
        self.left = left
        self.right = right
```

What's the induction principle? Here's where the two subtrees make things interesting:

```
assumes P(Leaf)
subproof:
    assumes P(left)
        P(right)
    -----
    shows P(Node(left, right))
-----
shows  $\forall t : \text{BinTree}. P(t)$ 
```

See the difference from list induction? In the inductive step you get **two** inductive hypotheses—one for the left subtree and one for the right subtree—because the BNF has two recursive occurrences. Compare this to lists, where **Cons** had one recursive sub-part so you got one inductive hypothesis, which was the same as natural induction where **succ** had one recursive sub-part. The number of inductive hypotheses always matches the number of recursive sub-parts in the BNF. Always.

10.3.2 Recursive functions on binary trees

Let's define a function that counts the number of internal *nodes* in a binary tree (i.e. the non-leaf nodes):

```
size(Leaf) = 0
size(Node(left, right)) = 1 + size(left) + size(right)
```

A leaf contributes nothing, and a node contributes itself plus whatever's in its subtrees. Simple enough. In Python:

```
def size(t):
    if isinstance(t, Leaf):
        return 0
    else: # Node
        return 1 + size(t.left) + size(t.right)
```

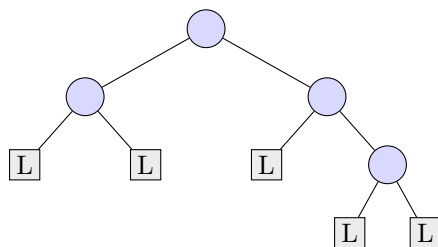
We'll also want to count the number of *internal edges*—edges connecting one node to another node or a node to a leaf. Each node is connected to its two children, so let's define:

```
edges(Leaf) = 0
edges(Node(left, right)) = 2 + edges(left) + edges(right)
```

Why 2? Because each **Node** creates exactly two edges: one going down to its left child and one going down to its right child.⁴

⁴If you draw a binary tree on paper, every line you draw corresponds to one of these edges. A node with two children means two lines going down from that node.

Before we prove anything, let's look at a concrete binary tree so we have something to count. The tree `Node(Node(Leaf, Leaf), Node(Leaf, Node(Leaf, Leaf)))` drawn out looks like this (blue circles are internal Nodes and gray squares marked L are Leafs):



Count by hand: there are 4 internal nodes (the blue circles) and 5 leaves (the gray squares), and you can trace 8 edges going down to children (2×4 , one pair per internal node). That's the pattern we're about to prove in general. The shapes *L* stand for **Leaf** and the unmarked circles are **Node** constructors.

10.3.3 Worked proof: counting edges in a binary tree

Now let's prove something about binary trees using structural induction. We'll show that:

For all binary trees t :

$$\text{edges}(t) = 2 \cdot \text{size}(t)$$

In other words, a binary tree with n internal nodes has exactly $2n$ edges (counting edges to leaf children). This makes intuitive sense: every node contributes exactly 2 edges (to its children), and leaves contribute 0.

We'll do structural induction on t . Our property $P(t)$ is the statement " $\text{edges}(t) = 2 \cdot \text{size}(t)$."

Base case ($t = \text{Leaf}$): We need $\text{edges}(\text{Leaf}) = 2 \cdot \text{size}(\text{Leaf})$. The left side is 0 and the right side is $2 \cdot 0 = 0$. Done.

Inductive step ($t = \text{Node}(\text{left}, \text{right})$): Assume $P(\text{left})$ and $P(\text{right})$, i.e. assume:

$$\text{edges}(\text{left}) = 2 \cdot \text{size}(\text{left}) \quad (\text{IH for left})$$

$$\text{edges}(\text{right}) = 2 \cdot \text{size}(\text{right}) \quad (\text{IH for right})$$

We need to show $\text{edges}(\text{Node}(\text{left}, \text{right})) = 2 \cdot \text{size}(\text{Node}(\text{left}, \text{right}))$.

Left side:

$$\begin{aligned} \text{edges}(\text{Node}(\text{left}, \text{right})) &= 2 + \text{edges}(\text{left}) + \text{edges}(\text{right}) && (\text{by definition of edges}) \\ &= 2 + 2 \cdot \text{size}(\text{left}) + 2 \cdot \text{size}(\text{right}) && (\text{by both IHs!}) \end{aligned}$$

Right side:

$$\begin{aligned} 2 \cdot \text{size}(\text{Node}(\text{left}, \text{right})) &= 2 \cdot (1 + \text{size}(\text{left}) + \text{size}(\text{right})) && (\text{by definition of size}) \\ &= 2 + 2 \cdot \text{size}(\text{left}) + 2 \cdot \text{size}(\text{right}) \end{aligned}$$

They're equal! Both sides give us $2 + 2 \cdot \text{size}(\text{left}) + 2 \cdot \text{size}(\text{right})$, so $P(\text{Node}(\text{left}, \text{right}))$ holds. By structural induction, $P(t)$ for all binary trees t . $\square$

Notice how we used **both** inductive hypotheses in the inductive step—one for each subtree. That's the characteristic feature of tree induction: you get two IHs and you typically need both.

10.4 The general recipe

Let’s spell out the pattern once and for all, because at this point you’ve seen it enough times that it should click.

Here’s the recipe. Given **any** BNF definition of a datatype, you automatically get an induction principle. The correspondence is:

- Each **base case** of the BNF gives you a **base case** of the proof. You need to show your property holds for each non-recursive constructor.
- Each **recursive case** of the BNF gives you an **inductive step**. For each recursive constructor, you get to *assume* the property holds for all recursive sub-parts, and you need to *show* it holds for the whole thing.

That’s it. That’s the whole recipe. Let’s verify it against everything we’ve seen this quarter:

Natural numbers: $\text{Nat} := 0 \mid \text{succ } \text{Nat}$. One base case (0), one recursive case (**succ**) with one sub-part. So you get one base case and one inductive step with one IH. That’s natural induction from Module 4.

Lists: $\text{List } A := \text{Nil} \mid \text{Cons}(A, \text{List } A)$. One base case (**Nil**), one recursive case (**Cons**) with one recursive sub-part (the tail). So you get one base case and one inductive step with one IH. Structurally identical to natural induction! And that makes sense: a list is basically a natural number that carries data.

Binary trees: $\text{BinTree} := \text{Leaf} \mid \text{Node}(\text{BinTree}, \text{BinTree})$. One base case (**Leaf**), one recursive case (**Node**) with **two** recursive sub-parts. So you get one base case and one inductive step with **two** IHs. That’s the new thing we saw in this module.

And you could keep going! Imagine a datatype with two base cases and three recursive cases. You’d get two base cases and three inductive steps in your proof. Or consider propositional logic formulas from Module 2: T and F are base cases, $\sim L$ is a recursive case with one sub-part, and $L \wedge L$, $L \vee L$, $L \rightarrow L$ are recursive cases with two sub-parts each. So to prove something about all propositional formulas you’d handle the base cases (T , F , variables) and then do inductive steps for each connective, getting the appropriate number of inductive hypotheses each time.⁵

This extends to any datatype you can dream up—rose trees, abstract syntax trees, regular expressions, whatever. If you can write down a BNF for it, you get a structural induction principle for it. You’ll see more of this in CS 251 when we work with graphs, automata, and other structures.

Sidebar: structural induction in programming, for real.

If you go on to any course in programming languages, compilers, type theory, or formal verification, structural induction will be the single most-used proof technique you meet. Not an exaggeration. A few concrete places it shows up:

- **Compilers.** Every pass—parsing, type-checking, optimization, code generation—walks a recursive datatype (the abstract syntax tree, or the intermediate representation, or the control-flow graph). When the compiler team proves a transformation preserves meaning, they do structural induction on the AST. “If optimization O is correct on each sub-expression, it’s correct on the whole expression” is a structural induction step.

⁵If you go on to study programming languages or compilers, this is *exactly* how you prove theorems about programs: by structural induction on the syntax of the language. The BNF of the programming language gives you the induction principle, and then you grind through the cases. It’s this same recipe, just with bigger grammars.

- **Type systems.** “Progress and preservation” is the standard soundness theorem for a typed programming language, and both halves are structural inductions on typing derivations (or on program syntax, or both). This is how we know a well-typed program “can’t go wrong.” You’ll see these proofs laid out in books like *Types and Programming Languages* (Pierce) and *Software Foundations* (Pierce et al.), where they’re written down rigorously in Coq and are sometimes dozens of cases long.
- **Interpreters and evaluators.** Writing an interpreter is structural recursion on the abstract syntax; proving the interpreter correct is structural induction on the same syntax. The definitions of `eval` and $P(e)$ have the same shape, one returning a value and the other returning a proof.
- **Static analyzers.** Tools like Facebook’s Infer (separation logic), Clang Static Analyzer, SpotBugs, and ESLint all walk syntax trees checking an invariant. When the tool’s authors prove the analysis is sound, they do structural induction on program syntax.
- **Regular expressions and automata.** Every identity about regexes—“ $(a|b)^* = a^*(ba^*)^*$,” for example—is proved by structural induction on the regex grammar, or by induction on the automaton it compiles to. CS 251’s formal languages material is essentially one long structural-induction exercise.
- **DSLs and embedded languages.** Any time you write a small language inside another language—SQL queries, GraphQL schemas, a configuration grammar, a shader language—you’re defining a BNF. Anything you want to prove or enforce about programs in that DSL is a structural induction on that grammar.

The recipe “write the BNF, get the induction principle, prove case by case” is the workhorse of programming-language theory and formal verification. In real practice it scales up to grammars with dozens of productions and proofs with hundreds of cases—but the pattern is exactly the one in this module. Once you see the shape, every textbook in the field starts reading like a variation on the same theme.

10.4.1 Example: structural induction on propositional formulas

Let’s actually carry out that propositional-logic example instead of just waving at it. Recall the BNF from Module 2:

$$L := T \mid F \mid x \mid \sim L \mid L \wedge L \mid L \vee L \mid L \rightarrow L$$

Define a function `size` that counts the number of connectives in a formula:

```
size(T) = 0
size(F) = 0
size(x) = 0
size(~L) = 1 + size(L)
size(L1 ^ L2) = 1 + size(L1) + size(L2)
size(L1 v L2) = 1 + size(L1) + size(L2)
size(L1 -> L2) = 1 + size(L1) + size(L2)
```

Seven cases, following the seven cases of the BNF—exactly the recipe we just described. Now let’s prove a simple property to see the machinery work on a grammar with multiple base cases and multiple recursive cases.

Claim. For all formulas f , $\text{size}(f) \geq 0$.

We proceed by structural induction on f .

Base cases. If $f = T$, $f = F$, or $f = x$ (a variable), then $\text{size}(f) = 0 \geq 0$. Done.

Inductive step (negation): $f = \sim L$. Assume $\text{size}(L) \geq 0$ (IH). Then:

$$\begin{aligned} \text{size}(\sim L) &= 1 + \text{size}(L) && \text{(by definition of size)} \\ &\geq 1 + 0 = 1 \geq 0 && \text{(by IH)} \end{aligned}$$

Inductive step (binary connectives): $f = L_1 \star L_2$ where $\star$ is any of $\wedge$, $\vee$, $\rightarrow$. Assume $\text{size}(L_1) \geq 0$ (IH1) and $\text{size}(L_2) \geq 0$ (IH2). Then:

$$\begin{aligned} \text{size}(L_1 \star L_2) &= 1 + \text{size}(L_1) + \text{size}(L_2) && \text{(by definition of size)} \\ &\geq 1 + 0 + 0 = 1 \geq 0 && \text{(by IH1 and IH2)} \end{aligned}$$

All cases handled. By structural induction, $\text{size}(f) \geq 0$ for every formula f . $\square$

The proof itself is almost trivially easy—and that’s the point. What matters is the *shape*: three base cases (matching the three non-recursive alternatives in the BNF), one inductive case with one IH (for $\sim$), and three inductive cases with two IHs each (for $\wedge$, $\vee$, $\rightarrow$). That shape was determined entirely by the BNF. We just followed the recipe.

When the recursive case has variable structure: rose trees

All the datatypes we’ve handled so far have recursive cases that carry a *fixed* number of recursive sub-parts: **Cons** has one tail, **Node** has two subtrees, binary connectives have two subformulas. The recipe “one IH per recursive occurrence” is easy to apply when the “number of occurrences” is a small constant.

What about datatypes where a single constructor holds a *whole list* of recursive sub-parts? The classic example is a *rose tree*—a tree where each internal node has an arbitrary number of children, not just two:

```
RoseTree A := Leaf(A) | Branch(List (RoseTree A))
```

A **Leaf** holds a value of type A ; a **Branch** holds a list of rose-tree children (the list might be empty, or might have a hundred). Any filesystem directory structure, HTML DOM tree, or s-expression has roughly this shape.

The induction principle for rose trees needs to handle the **Branch** case carefully: you don’t get “an IH per child” because you don’t know how many children there are. Instead, you get an IH that says “the property holds for *every* element of the **List**.” Concretely:

```
assumes P(Leaf(a)) for every a : A
subproof:
  assumes P(t) for every t in children [the list]
  -----
  shows P(Branch(children))
-----
shows  $\forall t : \text{RoseTree } A. P(t)$ 
```

“For every t in the list of children, $P(t)$ holds” is a universal over the list, not a fixed finite set of hypotheses. When you use the IH, you use it on whichever specific child you’re currently reasoning about.

Let’s see this with a small example. Define the *size* of a rose tree to be the total number of Leafs:

```

size(Leaf(a)) = 1
size(Branch(cs)) = sum of size(c) over each c in cs
                  (which equals 0 if cs is Nil)

```

And define the *count of Branch nodes*:

```

branches(Leaf(a)) = 0
branches(Branch(cs)) = 1 + sum of branches(c) over each c in cs

```

Claim. For every rose tree t , $\text{size}(t) \geq 1$.

Proof by structural induction on t .

Base case ($t = \text{Leaf}(a)$): $\text{size}(\text{Leaf}(a)) = 1 \geq 1$. ✓

Inductive step ($t = \text{Branch}(cs)$): The inductive hypothesis says $\text{size}(c) \geq 1$ for every c in the list cs .

We split on whether cs is empty.

- If $cs = \text{Nil}$, then $\text{size}(\text{Branch}(\text{Nil})) = 0$ by the definition (empty sum). But wait— $0 \not\geq 1$. Our claim is false for empty-branch rose trees! The definition of **RoseTree** allows **Branch(Nil)**—a “branch with no children”—and such a tree has size 0.

The proof *should* fail here, and the lesson is genuine: rose trees admit **Branches** with no children, and those have size 0. The right claim for **RoseTree** as defined is something like “every rose tree has at least one **Leaf** *or* is an empty **Branch**.” Alternatively, if we want the $\text{size} \geq 1$ claim to hold, we have to rule out empty branches at the BNF level: e.g., require that **Branch** carry a *non-empty* list of children. Either fix is available; they make different mathematical objects.

This is a feature, not a bug, of structural induction: the structure of the BNF forces you to confront edge cases that might otherwise hide. Students who say “obviously a tree has at least one leaf” have usually forgotten the **Branch(Nil)** case.

Let’s instead prove something that actually is true of the datatype as given.

Claim. For every rose tree t , $\text{size}(t) = \text{branches}(t) + \ell(t)$, where $\ell(t)$ is the number of **Leaf** nodes in t (an easy separate recursive definition—try writing it).

This is the rose-tree analogue of the edges/size identity we proved for binary trees. The argument goes through with the IH applied termwise to the sum over children. Working it out carefully is a good exercise and is left as such below (Exercise 3b, expanded).

The general lesson: **datatypes whose recursive cases contain lists of sub-parts need an IH that’s universally quantified over the list**. The recipe still works; you just read “one IH per recursive occurrence” as “one IH per element of the list of occurrences.” The number isn’t pinned down in advance, but the principle is the same.

10.5 Wrapping up: the big picture

Let’s take a step back and look at what we’ve actually built over this whole course. There’s been a single thread running through every module, and now we can finally state it cleanly.

The thread is a three-step pattern:

1. **Define your data inductively** using BNF. Specify the base cases and the recursive cases.
2. **Get an induction principle for free** that mirrors the structure of your definition. Base cases in the BNF become base cases in the proof. Recursive cases become inductive steps.

3. **Use that principle to prove things** about all instances of your data by handling each case.

We did this for propositional logic formulas in Module 2. We did this for natural numbers in Module 4. We did this for sequences in Module 8. In Module 9, we saw how *strong* induction generalizes the basic induction principle to handle recurrences that reach back more than one step, and the characteristic equation technique gave us tools for solving those recurrences in closed form. And now in this module we’ve done it for lists and binary trees.

The specific data changes but the *method* is always the same. Define inductively, get an induction principle, prove by cases. And as we saw in Module 8, this is the exact same thing as writing recursive programs: the structure of your data determines the structure of your code, which determines the structure of your proofs. Recursion, induction, and case analysis are facets of a single method depending on whether you’re computing a value or proving a theorem.

If there’s one thing I want you to take away from this course, it’s this: when you encounter some new kind of structured data, the first question you should ask is “what’s the BNF?” Once you have that, you know how to define functions on it (by recursion following the cases), and you know how to prove things about it (by induction following the cases). Everything else is details.

This is the foundation for CS 251 and beyond—type theory, programming language theory, algorithm analysis, formal verification. The datatypes get bigger and the proofs get harder, but the recipe stays the same. That recipe will carry you further than you might expect.⁶

10.6 Common mistakes

1. **Wrong number of inductive hypotheses.** The induction principle has exactly one IH per recursive sub-part in each recursive case. Students sometimes only use *one* IH when proving a property of `Node(left, right)`—forgetting that they get *both* $P(\text{left})$ and $P(\text{right})$. Conversely, students sometimes *invent* IHs they don’t have (e.g. claiming an IH for a parent tree when doing induction on a child). Always: IH count matches recursive-sub-part count in the BNF.
2. **Missing a BNF case.** A structural induction proof has to handle every alternative in the BNF. If your grammar has seven alternatives (like the BNF for propositional formulas) and your proof handles five of them, you’ve proved a weaker statement than you think. For long grammars, explicitly list the cases at the top of the proof and check them off as you go.
3. **Skipping the unfolding step.** The inductive step usually has the shape “unfold the definition on the left, apply the IH, unfold the definition on the right, see they match.” Students sometimes skip the unfolding and try to cite the IH directly on an expression where the IH doesn’t actually apply—because the LHS expression has a `Cons` or `Node` wrapped around it and you have to *unwrap* the definition first before the IH becomes applicable.
4. **Generalizing the wrong variable.** When proving “for all `xs` and `ys`, ...”, you usually induct on one of them with the *other* universally quantified inside the predicate $P(\text{xs})$. If you fix a particular `ys` and then try to induct on `xs`, you might find your proof goes through—but you’ve only proved the result for that particular `ys`. Concretely: the *right* predicate for induction on `xs` in the length-append proof is

$$P(\text{xs}) := \forall \text{ys}. \text{length}(\text{xs} ++ \text{ys}) = \text{length}(\text{xs}) + \text{length}(\text{ys}).$$

⁶Okay, I lied a little. In CS 251 you’ll see datatypes where the recursion is more complicated—mutual recursion, nested datatypes, indexed families—and the induction principles get correspondingly fancier. But the core idea really is the same. Promise.

The *wrong* predicate is the same statement with some specific `ys` (say, `Nil`) silently fixed. The second version can be induced on and the algebra goes through, but the thing you’ve proved is just “the equation holds when `ys = Nil`”—not the universal claim. The quantifier has to live *inside* P , otherwise the inductive hypothesis doesn’t give you anything about other values of `ys`.

5. **Proving a lemma you didn’t state.** In the reverse-is-involution exercise (and others like it) you often need a side lemma—something like `reverse(xs ++ ys) = reverse(ys) ++ reverse(xs)`—before the main theorem can go through. Students sometimes use such a lemma mid-proof without proving it. If you need a lemma, prove it separately first (or note it and come back to it). Don’t assume a useful-looking equation.
6. **Forgetting that Leaf is a tree.** Base cases sometimes get short-changed because they seem trivial. But a “trivial” base case is still a required case; write it down explicitly and verify. The one-line base case is sometimes where the *whole argument* hinges—for example, in the leaves-vs-size proof, `leaves(Leaf) = 1` but `size(Leaf) = 0`, and you need that $0 + 1$ relationship to see why the inductive hypothesis delivers the right answer.

10.7 Summary and looking ahead

Key takeaways.

- *Structural induction*: every BNF definition gives you an induction principle, one case per BNF alternative, with one IH per recursive sub-part in each case.
- Natural induction is structural induction on `Nat := 0 | succ Nat`. List induction is structural induction on `List := Nil | Cons(A, List)`. Tree induction gives you *two* IHs (one per subtree). Propositional-formula induction gives you seven cases. Same recipe every time.
- The shape of your data determines the shape of your code (recursive functions), which determines the shape of your proofs (induction principles). Recursion, induction, and case analysis are three views of a single pattern.
- Serious structural induction proofs often involve *lemma chains*: the main theorem depends on a helper lemma, which depends on a smaller helper, each proved by its own structural induction.
- Once you can name the shape of a data type, you can also name the “right notion of sameness” for values of that type by abstracting over some of the structure—lists up to reordering become multisets, binary trees up to relabeling become shapes, formulas up to logical equivalence become Boolean functions.
- That is the same move we already saw for sets (extensional equality, Module 1), formulas (logical equivalence, Module 2), and groups (isomorphism, Module 5). The intermezzo after Module 6 is the essay on the pattern.

What we built this module. This is the capstone of the first term. The three-step pattern—(1) define your data inductively via BNF, (2) get an induction principle for free, (3) use it to prove things by case analysis—has been present from Module 2 (the BNF for propositional logic) through Module 4 (natural induction) to here. We’ve finally named it and seen it work

on several concrete data types. This is the pattern that will carry you through CS 251 and beyond.

Looking ahead. This is the end of the first-term notes. The intermezzi after this module (“The Algebra of Types,” “The Lambda Calculus”) round out the picture by showing that data types form a rich algebraic structure of their own and that there’s a tiny universal programming language— λ -calculus—underneath everything we’ve been doing. In CS 251 you’ll see mutual recursion, nested datatypes, indexed families, and eventually dependent types; the core recipe “BNF gives you induction” generalizes to all of it. And if you go further—into programming language theory, formal verification, or type theory—this module is the hinge on which the rest hangs.

If there’s one thing to internalize from the whole first term, it’s this: when you encounter some new kind of structured data, the first question to ask is “what’s the BNF?” The answer tells you how to define functions on it and how to prove things about it. Everything else is details.

10.8 Exercises

Organized into **warm-up** (mechanical practice with recursive definitions), **standard** (structural-induction proofs on lists and trees), **challenge** (deeper proofs, including the negation-free monotone result), and **connection** (programming and forward pointers). Solutions are in the appendix.

Warm-up

Exercise 1. Define a recursive function **reverse** on lists, following the two cases of the BNF. (Hint: you will need to use **append** (**++**) in the recursive case.) Then compute **reverse**(**Cons**(1, **Cons**(2, **Cons**(3, **Nil**)))) step by step, showing each unfolding of the definition.

Exercise 2. Compute the following by hand, unfolding the definitions step by step:

- (a) **length**(**Cons**(*a*, **Cons**(*b*, **Nil**)))
- (b) **Cons**(1, **Cons**(2, **Nil**)) **++** **Cons**(3, **Nil**)
- (c) **size**(**Node**(**Leaf**, **Node**(**Leaf**, **Leaf**)))
- (d) **edges**(**Node**(**Leaf**, **Node**(**Leaf**, **Leaf**)))

Exercise 3. For each of the following BNFs, state the shape of the structural induction principle: how many base cases and how many inductive steps, and how many IHs in each inductive step.

- (a) **Color** := **Red** | **Green** | **Blue**
- (b) **RoseTree** := **Leaf**(*A*) | **Branch**(**List** **RoseTree**) (hmm, this one is tricky because the recursive case contains a list of subtrees, not a fixed number—answer informally)
- (c) **Expr** := **Const**($\mathbb{Z}$) | **Var**(**Name**) | **Add**(**Expr**, **Expr**) | **Mul**(**Expr**, **Expr**) | **Neg**(**Expr**)

Standard

Exercise 4. Define a function `leaves(t)` that counts the number of leaves in a binary tree (a `Leaf` counts as 1, a `Node` counts the leaves in both subtrees). Then prove by structural induction: $\text{leaves}(t) = \text{size}(t) + 1$ for all binary trees t . (You will need both inductive hypotheses in the inductive step, just like in the edges proof above.)

Exercise 5. Define a function `vars(f)` that returns the number of variable occurrences in a propositional formula (following the BNF from Module 2). For instance, $\text{vars}(x \wedge y) = 2$ and $\text{vars}(x \wedge x) = 2$ (we’re counting occurrences, not distinct variables). Then prove by structural induction that for every formula f , $\text{vars}(f) \leq \text{size}(f) + 1$ (where `size` is the connective-counting function defined in the module).

Exercise 6. Prove by structural induction that `append` is associative on lists:

$$(xs ++ ys) ++ zs = xs ++ (ys ++ zs).$$

Induct on xs , with ys and zs universally quantified inside the predicate.

Exercise 7. Define `map(f, xs)` recursively on lists: `map(f, Nil) = Nil` and `map(f, Cons(x, xs')) = Cons(f(x), map(f, xs'))`. Prove by structural induction that $\text{length}(\text{map}(f, xs)) = \text{length}(xs)$ for every list xs .

Exercise 7b. (Rose trees.) Using the `RoseTree` datatype from the rose-tree subsection, prove by structural induction: for every rose tree t , the total number of constructors appearing in t (counting each `Leaf` and each `Branch` once) equals $\text{branches}(t) + \text{size}(t)$. Start by writing a recursive definition of the “total constructor count” function, then do the induction. The `Leaf` case is immediate. The `Branch(cs)` case is where the variable-arity IH actually gets exercised: you need to apply the IH to every child in the list cs at once, not to a single “previous” child. State the IH precisely before using it—“for every child c occurring in cs , the property holds”—and keep that quantifier visible in your proof.

Challenge

Exercise 8. Prove by structural induction that `reverse` is an involution:

$$\text{reverse}(\text{reverse}(xs)) = xs \quad \text{for every list } xs.$$

You will need two helper lemmas:

1. $xs ++ \text{Nil} = xs$ for every xs . (Proof by induction on xs .)
2. $\text{reverse}(xs ++ ys) = \text{reverse}(ys) ++ \text{reverse}(xs)$. (Proof by induction on xs , using the first lemma in the base case.)

Prove both helpers, then use them to prove the main result. This is a good example of how real structural induction proofs often involve a chain of lemmas.

Exercise 9. (The plan’s request: a real semantic property.) Say a propositional formula is *negation-free* if it is built from T , F , variables, and the connectives $\wedge$, $\vee$ only (i.e., the BNF excludes $\neg$ and $\rightarrow$). Say a boolean function f is *monotone in variable x* if flipping x from F to T

(with all other variables held fixed) never flips the output of f from T to F . Prove by structural induction on the negation-free BNF:

Every negation-free formula is monotone in every variable.

Hint: you'll need the sub-lemma that $\wedge$ and $\vee$ preserve monotonicity—if two functions are monotone in x , so is their conjunction and their disjunction. You can also look back at Module 2 Exercise 10, where you proved that $\{\wedge, \vee\}$ is not functionally complete using exactly this monotonicity idea.

Exercise 10. A binary tree is *perfect* if every non-leaf node has two subtrees of the same height. Define `height`(t) recursively: `height(Leaf)` = 0 and `height(Node( $l$ ,  $r$ ))` = $1 + \max(\text{height}(l), \text{height}(r))$. Prove by structural induction: for every perfect binary tree t , `leaves`(t) = $2^{\text{height}(t)}$. (Hint: the definition of “perfect” is itself inductive: `Leaf` is perfect, and `Node( $l$ ,  $r$ )` is perfect if l and r are both perfect and have the same height. Induct on the structure of the perfect tree.)

Connection

Exercise 11. (Programming.) Implement `List` and `BinTree` as Python classes following the BNFs. Then implement `length`, `append`, `reverse`, `size`, `edges`, `leaves`, `height` as methods or free functions. Write unit tests that check the theorems proved in the module and exercises: for instance, verify that `length(xs ++ ys)` = `length(xs)` + `length(ys)` holds on a handful of random lists. Use Python's `random` and maybe `hypothesis` (if you want a proper property-based testing library) to stress-test your implementations.

Exercise 12. (Intermezzo pointer.) The “Algebra of Types” intermezzo that follows this module explains that BNF definitions correspond to *polynomial functors*, and that the associated induction principles are cases of a general categorical theorem. As a warm-up, count the “shape” of each of the following datatypes by writing down a polynomial-like expression. For example, `List A := Nil | Cons(A, List A)` corresponds to $L = 1 + A \cdot L$ (where 1 stands for the `Nil` case with no data and $A \cdot L$ stands for `Cons` with an A and a sub-list). Do the same for:

- (a) `Nat := 0 | succ Nat`
- (b) `BinTree := Leaf | Node(BinTree, BinTree)`
- (c) `Maybe A := Nothing | Just(A)` (not recursive!)

Why do you think the intermezzo calls these “polynomial” functors?

Intermezzo: The Algebra of Types

Something strange has been happening throughout this course, and it's time we talked about it.

In Module 1 we learned that $|A \times B| = |A| \cdot |B|$ —the cardinality of a Cartesian product is the product of the cardinalities. In the sets intermezzo we learned that $|B^A| = |B|^{|A|}$ —function spaces have exponential cardinality. And in Module 10 we defined data types using BNF, building lists and trees out of simpler pieces.

These facts live in different chapters, but they are secretly the same idea. Types obey the laws of algebra. Not metaphorically, not “sort of”—they obey the *actual* laws of high-school algebra, the ones with $+$ and $\times$ and exponents. Once you see this, a lot of things click into place.

10.9 Types are like numbers

Let's build the dictionary.

Product types are multiplication. A pair (a, b) where $a \in A$ and $b \in B$ has type $A \times B$. How many inhabitants does $A \times B$ have? For each choice of a (there are $|A|$ many) and each choice of b (there are $|B|$ many), we get a distinct pair. So $|A \times B| = |A| \cdot |B|$. A tuple is multiplication.

Sum types are addition. Suppose we have a type $A + B$ that means “either a value of type A , tagged with `Left`, or a value of type B , tagged with `Right`.” In Haskell this is `Either A B`; in Python you might use a tagged union or a class hierarchy. How many inhabitants? Every element of A gives one (tagged `Left`), and every element of B gives another (tagged `Right`). So $|A + B| = |A| + |B|$. A tagged union is addition.

Function types are exponentiation. A function $f : A \rightarrow B$ assigns to each element of A an element of B . That's $|B|$ choices for each of the $|A|$ inputs, so $|A \rightarrow B| = |B|^{|A|}$. We saw this in the sets intermezzo—it's why the function space is written B^A . A function is exponentiation.

The unit type is 1. Let 1 denote a type with exactly one element (in Haskell, `()`; in Python, `None`). Then $A \times 1 \cong A$ —a pair of an A and a “nothing” is just an A , since the second component carries no information. And $1^A \cong 1$ —there's only one function from A to a one-element set (the function that sends everything to the single element).

The empty type is 0. Let 0 denote a type with *no* elements (in Haskell, `Void`; in Python, a class you can never instantiate). Then $A \times 0 \cong 0$ —you can't build a pair if the second component has no possible values. And $A + 0 \cong A$ —“either an A or an impossible thing” is just an A . And for any nonempty A , $0^A \cong 0$ —there are no functions from A to an empty set (you'd have to send the elements of A somewhere, but there's nowhere to send them).

Look at these equations again. $A \times 1 = A$. $A + 0 = A$. $A \times 0 = 0$. Those are the rules of arithmetic you learned in grade school, but now they're facts about data types. Types form an algebra, and it's the same algebra you already know.

10.10 The laws of algebra hold

Let’s verify that the familiar identities from arithmetic actually hold for types. In each case, “ $\cong$ ” means “there’s a bijection between the two types”—they have the same number of inhabitants, and you can convert back and forth without losing information.

Multiplicative identity: $A \times 1 \cong A$. A pair where one component is trivial is just the other component. In Haskell: `(a, ())` is isomorphic to `a`.

Additive identity: $A + 0 \cong A$. Either `A Void` is isomorphic to `A`—the `Right` case is impossible (you can’t construct a `Void`), so you’re always in the `Left` case.

Distributivity: $A \times (B + C) \cong (A \times B) + (A \times C)$. A pair of an `A` with “either a `B` or a `C`” is the same as “either a pair of `A` and `B`, or a pair of `A` and `C`.” This is the distributive law $a(b + c) = ab + ac$, in type form.

Exponent of a product: $A \rightarrow (B \times C) \cong (A \rightarrow B) \times (A \rightarrow C)$. A function that returns a pair is the same as a pair of functions. If I ask you for a machine that takes an input and produces two outputs, you can either build one machine with two output slots or build two separate machines—same thing.

Exponent of a sum: $(A + B) \rightarrow C \cong (A \rightarrow C) \times (B \rightarrow C)$. A function from “either an `A` or a `B`” is the same as a pair of functions: one that handles `A` inputs and one that handles `B` inputs. In Python:

```
# A function from Either is a pair of handlers:
def handle_either(x):
    if isinstance(x, Left):
        return handle_a(x.value) # the A -> C part
    else:
        return handle_b(x.value) # the B -> C part
```

Wait a moment. A function from a sum is a pair of handlers—one for each case. Isn’t that exactly *proof by cases*? In Module 3 we learned the disjunction elimination rule: if you want to prove something from $A \vee B$, you handle the `A` case and handle the `B` case separately. That’s the logical version of $(A + B) \rightarrow C \cong (A \rightarrow C) \times (B \rightarrow C)$ —or-elimination *is* the fact that a function from a sum is a pair of functions. The algebra is telling us something about logic.

10.11 Data types as polynomials

Here is where things get a little bit magical.

In Module 10 we defined lists with a BNF:

```
List A := Nil | Cons(A, List A)
```

In our algebraic notation, `Nil` is the unit type `1` (one way to be empty) and `Cons(A, List A)` is a product $A \times L$ where `L` is the type of lists. The `|` in the BNF is a sum. So the definition of lists is:

$$L = 1 + A \cdot L$$

That’s a polynomial equation! Let’s be reckless and solve it like one. Rearranging:

$$L - A \cdot L = 1 \quad \implies \quad L(1 - A) = 1 \quad \implies \quad L = \frac{1}{1 - A}$$

Now, you might be having a reasonable objection right now, which is: what does it even *mean* to divide types or subtract types? Fair enough. But let's push ahead and see what happens. From Module 9, we know the formula for a geometric series:

$$\frac{1}{1-x} = 1 + x + x^2 + x^3 + \dots$$

So if we just blindly expand:

$$L = 1 + A + A^2 + A^3 + A^4 + \dots$$

And what is this saying? A list of A 's is:

- 1: the empty list (one way), or
- A : a single element (one A), or
- $A^2 = A \times A$: two elements (a pair of A 's), or
- $A^3 = A \times A \times A$: three elements, or
- ...and so on forever.

That's *exactly right*. A list is either empty, or has one element, or has two, or three, or any finite number of elements. The algebra of types—this insane-looking manipulation where we divided by $(1 - A)$ as if types were numbers—told us the truth. The geometric series from Module 9 is secretly a statement about data structures.

Let's try another one. Binary trees from Module 10 were defined as:

```
Tree := Leaf | Node(Tree, Tree)
```

In algebraic notation:

$$T = 1 + T^2$$

This is a quadratic equation! Rearranging: $T^2 - T + 1 = 0$, and the quadratic formula gives

$$T = \frac{1 \pm \sqrt{1-4}}{2} = \frac{1 \pm \sqrt{-3}}{2}$$

which involves complex numbers and seems completely unhinged. But remarkably, if you take the right root and expand it as a power series, you get the *Catalan numbers*—which are exactly the number of distinct binary trees with n internal nodes. The algebra of types knows about combinatorics.

We won't push this further here, but the point is: these aren't parlor tricks. There is a rigorous mathematical theory—the theory of *combinatorial species*—that makes all of this precise. The equation $L = 1 + A \cdot L$ is a legitimate algebraic equation in that theory, the “division” is a real operation, and the geometric series expansion is a theorem, not a coincidence.

What we're seeing is a remarkable convergence: the sets intermezzo (cardinality of function spaces), Module 9 (geometric series), and Module 10 (BNF definitions of data types) are all aspects of a single algebraic framework. Types are numbers, data definitions are polynomial equations, and the power series you learned in Module 9 enumerate the shapes of data structures.

10.12 Aside: type constructors come with maps

Notice what `List`, `Tree`, and `Maybe` actually *are*. None of them is a type on its own. Each is a construction that *eats* a type A and produces a new type: `List A`, `Tree A`, `Maybe A`. You could call them “type-level functions”—and that is exactly how they behave.

Type-level functions of this kind always come bundled with a companion data-level operation. Suppose someone hands you a function $f : A \rightarrow B$ and a list `xs : List A`. There is an obvious thing you can do: produce a new list of B ’s by applying f to every element. That’s `map f xs`. And `map` has the type

$$\text{map} : (A \rightarrow B) \rightarrow \text{List } A \rightarrow \text{List } B.$$

`Tree` has the analogous `map`: given $f : A \rightarrow B$ and a tree of A ’s, replace every label with f applied to it. `Maybe` has one too: `map f Nothing = Nothing`, `map f (Just a) = Just (f a)`. Every reasonable type constructor you have met—and a few you haven’t—comes with its own `map`.

These `maps` all satisfy two laws, which are easy enough to check on each example that we won’t do it here:

- **Identity law:** `map id xs = xs`. Mapping the identity function changes nothing.
- **Composition law:** `map (g . f) xs = map g (map f xs)`. Mapping a composition is the composition of two maps.

The combined gadget—a type constructor F together with a `map` obeying these two laws—has a name: F is called a *functor*. You will meet functors formally in later coursework in category theory and in languages like Haskell, where they are a foundational abstraction. For now the only thing to notice is that you have been using them for the entire course; most of what we’ve called a “data structure” is a functor in disguise.

10.13 The Curry–Howard connection

There’s one more layer. In the Curry–Howard intermezzo we learned that types correspond to propositions: a type is “true” if it’s inhabited (has at least one value) and “false” if it’s empty. Under this correspondence:

Algebra	Types	Logic
$A \cdot B$	$A \times B$ (product)	$A \wedge B$ (conjunction)
$A + B$	$A + B$ (sum)	$A \vee B$ (disjunction)
B^A	$A \rightarrow B$ (function)	$A \rightarrow B$ (implication)
1	Unit	$\top$ (true)
0	Void	$\perp$ (false)

So the algebraic identities we verified in Section 2 are also *logical tautologies*:

- $A \times 1 \cong A$ is $A \wedge \top \equiv A$ (“ A and true” is just A)
- $A + 0 \cong A$ is $A \vee \perp \equiv A$ (“ A or false” is just A)
- $A \times (B + C) \cong (A \times B) + (A \times C)$ is the distribution of $\wedge$ over $\vee$
- $(A + B) \rightarrow C \cong (A \rightarrow C) \times (B \rightarrow C)$ is or-elimination

- $A \times 0 \cong 0$ is $A \wedge \perp \equiv \perp$ (“ A and false” is false)

Three parallel worlds—arithmetic, type theory, and logic—and they’re all obeying the same laws. The algebra of numbers, the algebra of types, and the algebra of propositions are three views of one structure.

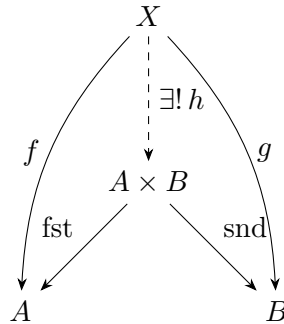
That structure has a name, by the way. It’s called a *rig* (a “ring without negation”—because you can’t subtract types or negate propositions in general). But you don’t need to know the name to appreciate the fact: the math you already know, from all across this course, fits together like a puzzle. Cardinality is counting. Types are numbers. BNF definitions are polynomial equations. Proofs are programs. And the geometric series from Module 9 secretly counts the shapes of lists.

10.14 The shape of definitions

There is a second pattern running through everything we’ve done in this intermezzo, and it’s worth unearthing before we stop. When we introduced the product type $A \times B$, we didn’t really *define* it by saying “a value of $A \times B$ is a pair of this particular form.” What we did—whether we realized it or not—was describe $A \times B$ by saying what you can *do* with it:

- Given a value of $A \times B$, you can extract an A (the first projection, **fst**).
- Given a value of $A \times B$, you can extract a B (the second projection, **snd**).
- If you have a way of building an A from some X and a way of building a B from the same X , you have a unique way of building an $A \times B$ from X .

The third bullet is worth staring at. It says: to give a function $X \rightarrow A \times B$ is the same thing as to give a pair of functions, $X \rightarrow A$ and $X \rightarrow B$. Pictorially:

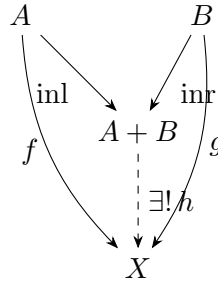


The dashed arrow is the unique $h : X \rightarrow A \times B$ determined by f and g —namely $h(x) = (f(x), g(x))$. The picture is a promise: whatever X and whatever f, g you give me, the pair (f, g) collapses into a *single* arrow into $A \times B$, and the way you get f and g back is by following **fst** and **snd** out of $A \times B$. That promise is the whole content of what $A \times B$ “is.”

This is a funny kind of definition. We did not list the elements of $A \times B$. We did not give a construction. We described $A \times B$ by saying, essentially, “it is the type such that mapping out of any X into it is the same as a pair of maps into its pieces.” And here is the payoff: that description pins $A \times B$ down *up to isomorphism*, which is exactly how much precision a working mathematician ever wants.

The same trick works for every type-former we’ve talked about:

- A function out of $A + B$ is a pair of functions, one from A and one from B . This is or-elimination (Section 2); it’s also the mirror-image of the product story, with arrows reversed.
- A function into $A \rightarrow B$ from X is the same as a function from $X \times A$ to B . (“Curry” it.) The type $A \rightarrow B$ is pinned down by what you can do *out of the variable* into it.
- The unit type 1 is the type such that for every X there is exactly one function $X \rightarrow 1$. The empty type 0 is the type such that for every X there is exactly one function $0 \rightarrow X$. Those are the shortest definitions of these types in all of mathematics, and they are correct.



This mode of definition—“specify the object by what arrows into or out of it are the same as”—is called a *universal property*. Each of the identities we proved in Section 2 ($A \times 1 \cong A$, $A + 0 \cong A$, distributivity, currying, exponent-of-a-product) is really a statement that two universal properties land in the same place.

We have spent this intermezzo calling the resulting structure an “algebra.” It has a second name, the one it gets when one takes this universal-property perspective seriously: *category theory* is the mathematics of these patterns, and products, sums, unit, and empty are its simplest examples. You are already doing category theory. You just aren’t using the vocabulary yet. CS 251 and any abstract-algebra course you take in the future will pick this thread back up.

10.15 Exercises

Exercise 1. Using the algebra of types, simplify the type $(A \rightarrow B) \times (A \rightarrow C)$. What single type is it isomorphic to? What does this isomorphism correspond to as a logical proposition (under the Curry–Howard correspondence)?

Exercise 2. The BNF for natural numbers is $\text{Nat} = \text{Zero} \mid \text{Succ}(\text{Nat})$, which in our algebraic notation is $N = 1 + N$. Write this as a “polynomial equation” and “solve” for N by expanding as a series. Does the series $1 + 1 + 1 + 1 + \dots$ make sense as a description of the natural numbers? (Hint: what does $1^n = 1$ mean for the n th term?)

Exercise 3. (Distributivity, programmatically.) Use the algebra of types to simplify $(A + B) \times (C + D)$. What single “polynomial” do you get? Convert your answer back into a Python data structure: write a class hierarchy or tagged union with one constructor for each summand of the result. What logical tautology does this correspond to under Curry–Howard?

Exercise 4. (Solving for non-empty lists.) Define non-empty lists by the BNF

$$\text{NEList } A = A \mid A \times \text{NEList } A$$

that is, “either a single A , or an A followed by a non-empty list of A .” Translate this to a polynomial equation in L (where $L = \text{NList } A$), solve for L algebraically, and expand the result as a series. What kinds of values appear in the series, and how does that match the structure of a non-empty list?

Exercise 5. (The `Maybe` type.) The type `Maybe A` (in Haskell) is defined as `Just A | Nothing`. In algebraic notation: $\text{Maybe } A = A + 1$.

- (a) What is $|\text{Maybe } A|$ in terms of $|A|$?
- (b) What is $|\text{Maybe } (\text{Maybe } A)|$ in terms of $|A|$?
- (c) Express $\text{Maybe } (\text{Maybe } A)$ as an algebraic expression and simplify. Why does the answer match the cardinality formula in (b)?

Exercise 5b. (Functor laws for lists.) In Python, implement `list_map(f, xs)` that returns a new list obtained by applying `f` to every element of `xs`. Do not use the built-in `map`; write the recursion or the list comprehension yourself.

- (a) Verify informally, by running on a sample list, that `list_map(lambda x: x, xs) == xs` (the identity law).
- (b) Verify that `list_map(lambda x: g(f(x)), xs) == list_map(g, list_map(f, xs))` (the composition law) for concrete `f` and `g` of your choosing.
- (c) Explain, in one sentence, why the composition law is not a mere convenience: what would go wrong computationally or conceptually if it failed?

Exercise 6. (A function that returns a pair is a pair of functions.) Verify the isomorphism

$$A \rightarrow (B \times C) \cong (A \rightarrow B) \times (A \rightarrow C)$$

by writing two Python functions: one that converts the left side into the right side, and one that converts the right side into the left side. Verify (informally, by trying out a few examples) that they are inverses of each other. What does this say in pure-arithmetic terms about the equation $(b \cdot c)^a = b^a \cdot c^a$?

Further reading

If the algebra of types appeals to you, here is where to go next. The connection to combinatorics is deep and beautiful.

- Conor McBride, “The Derivative of a Regular Type is its Type of One-Hole Contexts,” unpublished manuscript (2001), available on the author’s homepage at strictlypositive.org. A short, witty, surprisingly elementary derivation of why *differentiating* an algebraic data type literally gives you its zipper. Connects high-school calculus to programming in a way undergraduates love.

- Brent Yorgey, “Species and Functors and Types, Oh My!” *Haskell Symposium 2010*. Written explicitly as an accessible bridge between Joyal’s combinatorial species and the algebraic data types programmers already know. The most undergraduate-friendly entry point to species I know of.
- Philippe Flajolet and Robert Sedgewick, *Analytic Combinatorics*, Cambridge University Press (2009), Chapter I (“Symbolic Methods”). The full book PDF is freely available on Sedgewick’s Princeton homepage. Chapter I introduces the symbolic method—sums, products, sequences as type combinators with generating functions—at exactly the level a strong undergrad can handle.

Intermezzo: The Lambda Calculus

We’ve reached the end of CS 250. Over the past ten modules you’ve learned how to define data with BNF, how to prove things about data with induction, and—in the intermezzi on Curry–Howard and the algebra of types—how proofs and programs are secretly the same thing. But there’s a question we left hanging all the way back in Module 1 that I want to come back to now.

Remember the question? We asked: if we want to understand what computers can do *ever*—not just what today’s chips can handle—we need an abstract model of computation, one that doesn’t depend on hardware or programming language. We said we’d leave the answer for CS 251. But we can give a preview now, and it turns out that almost everything you need to understand the answer has been hiding in this course the whole time.

10.16 A mathematical model of computation

In the 1930s, before electronic computers existed, a mathematician named Alonzo Church asked a foundational question: what does it mean to *compute* something? His answer was the **lambda calculus**—the simplest possible programming language. Not a practical language you’d use to build a web app, but a theoretical one, stripped down to the absolute minimum needed to express any computation at all.

How minimal? The entire language has exactly three constructs. Here’s the BNF:

$$M := x \mid \lambda x. M \mid M M$$

That’s it. Three things:

- **Variables** (x): names that stand for values. You’ve seen these since Module 2.
- **Abstraction** ($\lambda x. M$): a function definition. The λ says “I’m defining a function whose parameter is x and whose body is M .” If you’ve written `def f(x): return M` in Python or `fun x -> M` in OCaml, you already know what this is. The λ is just the world’s oldest anonymous function syntax.
- **Application** ($M M$): a function call. Put a function next to its argument and it runs.

No numbers. No booleans. No if-statements. No loops. No data structures. Just variables, function definitions, and function calls. You’re probably thinking: how can you compute *anything* with that? The answer is one of the most surprising facts in all of computer science. You can build everything from functions alone.

10.17 Computing with pure functions

The lambda calculus has one rule of computation, called **beta reduction**. It says: when you apply a function to an argument, substitute the argument for the parameter in the body. In symbols:

$$(\lambda x. M) N \longrightarrow M[N/x]$$

where $M[N/x]$ means “ M with every occurrence of x replaced by N .” That’s it. That’s the entire execution model.

Example 10.17.1 (The identity function). The term $\lambda x. x$ is a function that takes its argument and returns it unchanged. Apply it to 5:

$$(\lambda x. x) 5 \longrightarrow x[5/x] = 5$$

The identity function applied to 5 gives 5. Not very exciting, but it illustrates the rule.

Example 10.17.2 (Applying a function twice). Consider the term $\lambda f. \lambda x. f (f x)$. This takes a function f and a value x , and applies f twice. If f is “add 1” and x is 0:

$$\begin{aligned} (\lambda f. \lambda x. f (f x)) \text{ add1 } 0 &\longrightarrow (\lambda x. \text{add1 } (\text{add1 } x)) 0 \\ &\longrightarrow \text{add1 } (\text{add1 } 0) \\ &= \text{add1 } 1 \\ &= 2 \end{aligned}$$

Does this remind you of anything? In Module 4, we defined the natural numbers with a successor function `succ` and a base case 0. Applying `succ` twice to 0 gives 2. The idea of “iterate a function n times” is about to become very important.

Notice something familiar about the BNF for lambda terms. It has a recursive structure: the body of an abstraction is itself a lambda term, and both parts of an application are lambda terms. By the pattern from Module 10, this BNF gives us a principle of *structural induction on lambda terms*—three cases, one for each construct. This is exactly how programming language theorists prove properties of languages: by induction on the syntax. The tools you learned in Module 10 are the same tools the researchers use.

10.18 Church encodings—building everything from nothing

Here is where things get genuinely mind-bending. Since the lambda calculus has no built-in data types, Church figured out how to *encode* data as functions. The strategy is elegant: represent a piece of data by what you can *do with it*.

Church booleans

What can you do with a boolean? You use it to choose between two alternatives. So let’s define booleans as choosers:

$$\begin{array}{ll} \text{true} = \lambda t. \lambda f. t & \text{(given two options, pick the first)} \\ \text{false} = \lambda t. \lambda f. f & \text{(given two options, pick the second)} \end{array}$$

A boolean *is* an if-statement. Given two branches, it picks one. In fact, the “if” function is almost trivial:

$$\text{if} = \lambda b. \lambda t. \lambda f. b \ t \ f$$

It just hands the two branches to the boolean and lets the boolean decide. Let’s verify, using “ $\equiv$ ” for unfoldings-by-definition and “ $\longrightarrow$ ” for actual beta-reduction steps:

$$\begin{aligned} \text{if true } a \ b &\equiv (\lambda b. \lambda t. \lambda f. b \ t \ f) \ \text{true } a \ b \\ &\longrightarrow (\lambda t. \lambda f. \text{true } t \ f) \ a \ b \\ &\longrightarrow (\lambda f. \text{true } a \ f) \ b \\ &\longrightarrow \text{true } a \ b \\ &\equiv (\lambda t. \lambda f. t) \ a \ b \\ &\longrightarrow (\lambda f. a) \ b \\ &\longrightarrow a. \end{aligned}$$

It works. And this connects directly to Module 2: we’re encoding T and F—the truth values of propositional logic—as pure functions. The logical constants have become computational objects.

Church numerals

What can you do with a natural number n ? You can use it to *repeat* something n times. So let’s define a natural number as a function that iterates its argument:

$$\begin{aligned} \mathbf{0} &= \lambda f. \lambda x. x && \text{(apply } f \text{ zero times)} \\ \mathbf{1} &= \lambda f. \lambda x. f \ x && \text{(apply } f \text{ once)} \\ \mathbf{2} &= \lambda f. \lambda x. f \ (f \ x) && \text{(apply } f \text{ twice)} \\ \mathbf{3} &= \lambda f. \lambda x. f \ (f \ (f \ x)) && \text{(apply } f \text{ three times)} \\ &\vdots \\ \mathbf{n} &= \lambda f. \lambda x. f^n \ x && \text{(apply } f \text{ } n \text{ times)} \end{aligned}$$

Stop and look at this. A Church numeral $\mathbf{n}$ takes a function f and a starting value x , and applies f to x exactly n times. Zero applies f zero times (just returns x). One applies f once. And so on.

Now here’s the connection I want you to see. In Module 4 we defined the natural numbers by BNF:

`Nat ::= 0 | succ(Nat)`

Two cases: a base case (0) and a recursive case (apply succ one more time). Church numerals are *the same definition in disguise*. The base case is x (the “zero” of the iteration). The recursive case is “apply f one more time.” Church numeral $\mathbf{n}$ is literally $\text{succ}^n(0)$ with succ renamed to f and 0 renamed to x . The BNF and the Church encoding are two faces of the same coin.

With this insight, the successor function writes itself:

$$\text{succ} = \lambda n. \lambda f. \lambda x. f \ (n \ f \ x)$$

Given a Church numeral n , this produces a new numeral that applies f one extra time: n already applies f n times to x , and then we wrap one more f around the result. That’s “apply f one more time”—exactly the recursive case of the BNF.

And addition? Apply f a total of $m + n$ times by first applying it n times, then m more:

$$\text{add} = \lambda m. \lambda n. \lambda f. \lambda x. m \ f \ (n \ f \ x)$$

The term $n \ f \ x$ applies f n times to x . Then $m \ f$ applies f m more times to that result. Total: $m + n$ applications.

A word on efficiency, since a student reading this for the first time should not come away with the wrong mental model: Church encodings are *theoretically* complete but *practically* horrendous. Evaluating the Church numeral **n** requires n function applications; evaluating $\text{add } \mathbf{m} \ \mathbf{n}$ requires $m+n$; multiplication and exponentiation blow up from there. Real programming languages represent integers as bit patterns because “apply f a billion times” is not a plan you want to run on actual silicon. The point of Church encodings isn’t that you’d compute this way. It’s that you *could*—which tells you that lambda-expressions alone, without any built-in notion of “number,” are already enough to express every computable function. That universality is the result. Efficiency is an entirely separate engineering concern.

10.19 Why this matters

We started this course by asking whether Python and C can solve the same problems, and whether some future computer might solve problems that today’s computers can’t. The lambda calculus gives us the tools to answer these questions.

The Church–Turing thesis. In the same decade that Church invented the lambda calculus, Alan Turing independently invented a different model of computation—the Turing machine, a kind of abstract tape-reading automaton. The two models look nothing alike. But Church and Turing proved that they compute exactly the same class of functions. Every function computable by a Turing machine can be computed in the lambda calculus, and vice versa. This result has been replicated for every “reasonable” model of computation anyone has ever proposed: register machines, cellular automata, Post systems, RAM machines, quantum computers. They all compute the same things. The **Church–Turing thesis** is the claim that this class—the class of functions computable by any of these models—is *the* right notion of “computable.”

This is the answer to Module 1’s question. Python and C can solve the same problems because they are both models of computation at least as powerful as the lambda calculus (and no more powerful). No future chip will change this. The limits of computation are not engineering constraints—they are mathematical facts, and the lambda calculus is one of the cleanest ways to see them.

The lambda calculus is the ancestor of every functional language. Haskell, ML, OCaml, Lisp, Scheme, Clojure, Erlang, F#—all of them are, at their core, the lambda calculus with syntactic sugar on top. When you write a lambda expression in Python (`lambda x: x + 1`), you are writing a Church-style abstraction. The notation is literally named after Church’s λ .

Under Curry–Howard, lambda terms are proofs. In the previous intermezzo, we saw that proofs correspond to programs and propositions correspond to types. The programs in that correspondence are lambda terms. So the simplest possible model of computation is also the simplest possible language of mathematical proof. A lambda term of type $A \rightarrow B$ is simultaneously a function from A to B and a proof that A implies B . Defining a function *is* introducing an implication. Applying a function *is* modus ponens. The act of computing and the act of reasoning are, at bottom, the same act.

BNF, induction, Curry–Howard, and the lambda calculus form a single story. The BNF for lambda terms ($M := x \mid \lambda x. M \mid M \ M$) gives you structural induction on terms. Structural

induction lets you prove properties of programs. Curry–Howard tells you those programs are proofs. And the Church–Turing thesis tells you those programs capture all of computation. The entire course—from the first BNF in Module 2 to the structural induction of Module 10, from the proof rules of Module 3 to the Curry–Howard correspondence—was building toward this point. Logic, proof, computation, and data are not four separate subjects. They are four views of one subject. CS 251 will make that precise.

10.20 Exercises

Exercise 1. Evaluate $(\lambda x. \lambda y. x) 3 7$ by applying beta reduction step by step. What does this function do? (Hint: the expression $(\lambda x. \lambda y. x) 3 7$ means $((\lambda x. \lambda y. x) 3) 7$ —application is left-associative.)

Exercise 2. Write a Church encoding for the boolean operation AND. Hint: $\text{and} = \lambda a. \lambda b. a b \text{ false}$. Verify that this definition is correct by evaluating it on all four input combinations: and true true , and true false , and false true , and and false false . For each case, show the beta reduction steps and confirm the result matches the expected truth value.

Exercise 3. Evaluate $(\lambda x. \lambda y. y) 3 7$ step by step using beta reduction. Compare your answer to Exercise 1. What single concept do these two functions, together, encode? (Hint: think back to the Church booleans in Section 3.)

Exercise 4. (Successor of a Church numeral.) Recall that $\text{succ} = \lambda n. \lambda f. \lambda x. f (n f x)$. Apply succ to the Church numeral $\mathbf{2} = \lambda f. \lambda x. f (f x)$ and reduce step by step. Verify that the result is exactly the Church numeral $\mathbf{3} = \lambda f. \lambda x. f (f (f x))$.

Exercise 5. (Church-encoded OR.) Write a Church encoding for the boolean operation OR. (Hint: $\text{or} = \lambda a. \lambda b. a \text{ true } b$.) Verify it on all four input combinations by reducing each case step by step.

Exercise 6. (Multiplication of Church numerals.) Define multiplication on Church numerals as

$$\text{mult} = \lambda m. \lambda n. \lambda f. m (n f).$$

Apply this to $\mathbf{2}$ and $\mathbf{3}$ (in some order) and reduce. Verify that the result is the Church numeral for 6. In one or two sentences, explain why the body $m (n f)$ correctly encodes multiplication. (Hint: $n f$ is the function “apply f n times.” What does m do to that?)

Exercise 7. Translate the Python function `lambda x: lambda y: x + y` into a lambda-calculus term. Then translate the lambda-calculus term $\lambda f. \lambda g. \lambda x. f (g x)$ back into Python. What standard programming concept is this term?

Further reading

The lambda calculus is the gateway to a huge swath of theoretical computer science. Here are three excellent next steps:

- Henk Barendregt, “[The Impact of the Lambda Calculus in Logic and Computer Science](#),” *Bulletin of Symbolic Logic* **3**(2), 181–215 (1997). The standard expository overview of why the lambda calculus matters across logic and computer science, written by the field’s leading authority. The first half is fully undergraduate-accessible.
- Peter Selinger, *[Lecture Notes on the Lambda Calculus](#)*, arXiv:0804.3434 (revised 2013). The cleanest free introduction I know of—covers Church encodings, beta reduction, the Church–Rosser theorem, and fixed-point combinators at exactly undergraduate pitch.
- Felice Cardone and J. Roger Hindley, “[History of Lambda-calculus and Combinatory Logic](#),” in D. Gabbay and J. Woods (eds.), *Handbook of the History of Logic*, vol. 5, Elsevier (2009). A historical narrative of how Church, Curry, and Turing arrived at the lambda calculus and combinatory logic—gives the Church–Turing-thesis story in human terms.
- And of course: Philip Wadler, “[Propositions as Types](#),” *Communications of the ACM* **58**(12), 75–84 (2015). Already cited in the Curry–Howard intermezzo, but it earns a second mention here because it tells the lambda-calculus story from the proofs-as-programs angle.

Cumulative Review 3

The point of this review

This is the last review chapter of the first term, and its job is to tie the whole course together. At this point you’ve seen:

- Sets, functions, relations, and the extensional view of equality (Module 1).
- Propositional logic with a BNF syntax, truth-table semantics, and a natural-deduction proof system (Modules 2, 3).
- Predicate logic with quantifiers and a first induction principle (Module 4).
- The practice of informal proof—direct, counterexample, existence, cases, contradiction, contrapositive—with a side trip into groups and modular arithmetic (Modules 5, 6, 7).
- Induction on $\mathbb{N}$, summation identities, strong induction, recurrence solving (Modules 8, 9).
- Structural induction on arbitrary recursive data types (Module 10).

That is a lot of material. But it all rests on a single pattern, which is the theme we named explicitly in Module 10 and have been building towards since Module 2: *define your data with a BNF grammar; derive an induction principle that mirrors the BNF; use it to prove theorems by case-and-recursion analysis*. Everything else in the course is a specialization or a consequence of that pattern.

In this review, we pick a capstone theorem that touches every major thread from the term and walk through its proof end to end, stopping to point at which module every step comes from. The theorem is non-trivial but still fits on a couple of pages. Let’s go.

The capstone theorem

Recall from Module 2 the BNF for propositional logic formulas:

$$L := T \mid F \mid x \mid \neg L \mid L \wedge L \mid L \vee L \mid L \rightarrow L.$$

We’ll prove the following fact:

Theorem 10.20.1 (DNF exists). For every propositional formula f over the variables $x_1, \dots, x_n$, there exists a logically equivalent formula f' in disjunctive normal form (i.e., f' is a disjunction of conjunctions of literals).

In English: every propositional formula can be rewritten as DNF. This is really a theorem about *universality*: propositional logic is rich enough that every truth function on n inputs has a DNF representation. It's also, in disguise, a statement about functional completeness: it says $\{\wedge, \vee, \neg\}$ is functionally complete, which we hinted at in Module 2 but only proved for the case of finite truth tables in Module 3.

We'll prove it by *structural induction* on the BNF. That means handling one case per alternative—seven cases—and in each case, assuming we already have DNF representations of the sub-formulas (the inductive hypotheses) and constructing a DNF representation of the whole formula.

The proof

Fix a set of variables $\{x_1, \dots, x_n\}$. We prove by structural induction on f that there exists a DNF formula f' with $f \equiv f'$.

Base case 1: $f = T$. The constant T is already in DNF: it's a “disjunction of one term, the empty conjunction.” Or, if you want to avoid the empty conjunction, note that $T \equiv x_1 \vee \neg x_1$, which is a disjunction of two single-literal conjunctions, hence in DNF. Either way, a DNF equivalent exists.

Base case 2: $f = F$. The constant F is the “empty disjunction,” which is trivially in DNF. If you'd rather avoid empty disjunctions, $F \equiv x_1 \wedge \neg x_1$ is a single single-literal-times-literal conjunction, which counts as a (one-disjunct) DNF.

Base case 3: $f = x_i$. A single variable is a literal, hence a one-literal conjunction, hence a one-disjunct DNF. Done.

Inductive case: $f = \neg g$. Inductive hypothesis (IH): there exists a DNF formula g' with $g \equiv g'$. By IH, g' has the form

$$g' = C_1 \vee C_2 \vee \dots \vee C_k,$$

where each C_i is a conjunction of literals. Then $\neg g \equiv \neg g'$, and by De Morgan's laws (Module 2),

$$\neg(C_1 \vee \dots \vee C_k) \equiv \neg C_1 \wedge \neg C_2 \wedge \dots \wedge \neg C_k.$$

Each $\neg C_i$ is a disjunction of literals (by another application of De Morgan, pushing the $\neg$ inside each conjunction and flipping each literal—see the De Morgan box in Module 2). So we have transformed $\neg g$ into a *conjunctive* normal form (CNF) formula. To convert CNF to DNF, we use the $\wedge$ -over- $\vee$ distributivity law $A \wedge (B \vee C) \equiv (A \wedge B) \vee (A \wedge C)$ (verify by truth table if you haven't before—it's one of the basic propositional equivalences), and iterating gives us

$$(D_{1,1} \vee D_{1,2}) \wedge \dots \wedge (D_{k,1} \vee \dots) \equiv \bigvee_{(\text{choices})} (\text{AND of one } D_{i,j} \text{ per clause}),$$

which is a disjunction of conjunctions of literals—that is, DNF. So $\neg g$ has a DNF equivalent.

Inductive case: $f = g \wedge h$. IHs: $g \equiv g'$ and $h \equiv h'$, both in DNF. Write

$$g' = G_1 \vee \dots \vee G_a, \quad h' = H_1 \vee \dots \vee H_b.$$

By distributivity of $\wedge$ over $\vee$,

$$g' \wedge h' \equiv \bigvee_{i=1}^a \bigvee_{j=1}^b (G_i \wedge H_j).$$

Each $G_i \wedge H_j$ is a conjunction of literals (being a conjunction of two conjunctions of literals), so the whole expression is a disjunction of conjunctions of literals—DNF. Done.

Inductive case: $f = g \vee h$. IHs: $g \equiv g'$ and $h \equiv h'$, both in DNF. Then $g' \vee h'$ is already in DNF: concatenate the two lists of disjuncts, and you have a single disjunction of conjunctions of literals.

Inductive case: $f = g \rightarrow h$. IHs as before. First rewrite using the equivalence $g \rightarrow h \equiv \neg g \vee h$ (proved in Module 2). Now apply the $\neg g$ case above (which gives a DNF for $\neg g$) and the $g \vee h$ case (which combines two DNFs into a DNF), and we're done.

All seven BNF cases handled. By structural induction (Module 10), every propositional formula has a DNF equivalent. $\square$

Tracing the toolkit

Let's trace which modules contributed what:

- **Module 1** (sets, proof writing): ambient vocabulary (“let f be a formula,” “by definition of DNF”), the proof style, and the extensional reading of logical equivalence (“same truth table”).
- **Module 2** (propositional logic): the BNF grammar we're inducting on, the truth-table semantics that underpins $\equiv$, De Morgan's laws, the definition of DNF, and the crucial equivalence $g \rightarrow h \equiv \neg g \vee h$.
- **Module 3** (proof systems): the background justification for distributivity and De Morgan—both are tautologies that can be derived in the formal proof system. We used them as black boxes here, but each use is a fully formalizable derivation.
- **Module 4** (predicate logic and induction): the statement “for every formula f , there exists a DNF equivalent f' ” is a $\forall\exists$ statement. Every step of the proof is, implicitly, an application of $\forall$ -introduction (“let f be arbitrary”) and $\exists$ -introduction (“construct f' ”).
- **Module 5** (direct proof, constructive existence): the proof is *constructive*—we don't just claim that f' exists, we describe how to build it from f . That means the proof is implicitly an algorithm: “to convert a formula to DNF, recurse on its structure, applying distributivity and De Morgan in each case.” This is the Curry–Howard payoff: the proof *is* a program.
- **Module 6** (proof by cases): the seven-case structure of the structural induction is a multi-way case analysis on the BNF alternatives. This generalizes the parity and mod-3 case splits from Module 6 to a case split on “which BNF alternative built this formula.”
- **Module 7** (indirect proof, Hoare logic): not directly used, but the habit of “formalize the algorithm, verify it step by step” comes from here. If you wanted to, you could turn our DNF-conversion proof into a Hoare-logic verification of an actual DNF-conversion program.

- **Module 8** (induction is recursion): this is the hinge. The seven BNF cases exactly match the seven cases of a recursive `toDNF` function, and the inductive hypotheses are exactly the recursive calls. Reading the proof as code, it says: “recursively convert the subformulas, then use distributivity to combine the results.”
- **Module 9** (strong induction): not strictly needed here (ordinary structural induction works because sub-formulas are strictly smaller than the whole formula), but the intuition “recurse on anything smaller” that comes from strong induction is exactly what’s happening when we apply the IH to a sub-formula.
- **Module 10** (structural induction): the ambient proof technique. One case per BNF alternative, one IH per recursive sub-part in each case—the seven-case shape was dictated by the BNF of propositional logic, which we wrote down in Module 2 and have been carrying around ever since.

Every single module shows up. That’s the point. *The course’s threads converge.*

The meta-theorem

Here’s the thing worth pausing on. The proof we just did isn’t only about propositional logic; its shape generalizes. For *any* recursive data type—lists, trees, abstract syntax trees, whatever—the same proof template works:

The structural-induction recipe. To prove that every value of type T has property P :

1. Write down the BNF for T .
2. For each non-recursive case, prove P directly by computation.
3. For each recursive case, assume P on all recursive sub-parts (these are the IHs), then prove P on the whole.

The number of IHs in each recursive case matches the number of recursive sub-parts in the BNF.

That’s the whole technique. You now know how to prove theorems about propositional formulas, natural numbers, lists, binary trees, arithmetic expressions, XML trees, programming-language syntax, and any other recursive data type you might meet in CS 251 and beyond. The data types get bigger, the cases get more numerous, but the recipe is always the same.

What you take away from the first term

If you internalize just three things from this course, these are the three:

1. **Proofs are essays.** A proof is a piece of writing that explains, step by step, why a claim is true. State the goal; introduce variables; justify every step; end with a clear conclusion. The “How to Write a Proof” chapter after Module 1 is the single most re-read-able part of the book.

2. **The same fact wears many hats.** Truth tables, formal derivations, set identities, and semantic arguments can all prove the same propositional-logic fact. Direct proof, contrapositive, and contradiction can all prove the same implication. Ordinary induction, strong induction, and structural induction are all special cases of the same recursive pattern. When you get stuck, change your hat.
3. **BNF gives you induction.** When you meet a new kind of structured data, the first question is “what’s the BNF?” The answer tells you how to define functions on it and how to prove theorems about it. Everything else is just turning the crank.

Where you’re going

This is the last page of the first-term notes. CS 251 takes the tools built here and applies them to *computation itself*: automata, formal languages, Turing machines, the halting problem, NP-completeness, and eventually the limits of what computers can do. The three takeaways above will carry you through every one of those topics. If you remember them, you’ll be fine.

Thanks for reading. Now go try Module 10 Exercise 8 (the reverse-is-an-involution one, with its chain of helper lemmas) if you haven’t yet. It’s the most satisfying proof in the whole book, and you now have every tool you need to do it.

Appendix A

Notation Summary

Symbol	Name	Introduced in
<i>Propositional Logic</i>		
$\wedge$	Conjunction (and)	Module 2
$\vee$	Disjunction (or)	Module 2
$\neg$ or $\sim$	Negation (not)	Module 2
$\rightarrow$	Implication (if... then)	Module 2
$\leftrightarrow$	Biconditional (if and only if)	Module 2
$\equiv$	Logical equivalence	Module 2
$\oplus$	Exclusive or (XOR)	Module 2
T, F	Truth values	Module 2
$\perp$	Absurdity / falsum	Gentzen Intermezzo
<i>Quantifiers and Predicates</i>		
$\forall$	For all (universal quantifier)	Module 4
$\exists$	There exists (existential quantifier)	Module 4
$P(x)$	Predicate applied to x	Module 4
<i>Sets</i>		
$\{ \}$	Set braces	Module 1
$\in, \notin$	Element of, not element of	Module 1
$\subseteq$	Subset	Module 1
$\cap$	Intersection	Module 1
$\cup$	Union	Module 1
$\setminus$	Set difference	Module 1
$\overline{A}$	Complement	Module 1
$\emptyset$	Empty set	Module 1
$ A $	Cardinality	Module 1
$\mathcal{P}(A)$	Power set	Module 1
$A \times B$	Cartesian product	Module 1
B^A	Function space ($A \rightarrow B$)	Cardinality Intermezzo
$\{x \mid P(x)\}$	Set-builder notation	Module 1
<i>Functions and Relations</i>		
$f : A \rightarrow B$	Function from A to B	Module 1
$a R b$	a is related to b by R	Module 1
$\cong$	Isomorphism / bijection exists	Cardinality Intermezzo

Symbol	Name	Introduced in
<i>Number Theory</i>		
$\mathbb{N}$	Natural numbers	Module 1
$\mathbb{Z}$	Integers	Module 5
$\mathbb{Q}$	Rationals	Module 5
$\mathbb{R}$	Reals	Module 4
$a \mid b$	a divides b	Module 5
$\gcd(a, b)$	Greatest common divisor	Module 7
$\mathbb{Z}/n\mathbb{Z}$	Integers mod n	Module 6
$\lfloor x \rfloor, \lceil x \rceil$	Floor, ceiling	Module 6
<i>Sequences and Sums</i>		
a_n	n th term of a sequence	Module 8
$\sum$	Summation	Module 8
$\prod$	Product	Module 8
$n!$	Factorial	Module 8
<i>Data Structures</i>		
Nil, Cons	List constructors	Module 10
Leaf, Node	Binary tree constructors	Module 10
<i>Proof Notation</i>		
$\therefore$	Therefore	Module 3
$\square$ or QED	End of proof	Throughout
$\{P\} C \{Q\}$	Hoare triple	Module 7

Appendix B

Glossary

This glossary gives one-paragraph definitions of the main concepts introduced in the book. It complements the *Notation Summary* appendix, which is a symbol reference rather than a concept reference. If you're looking up a symbol, check Notation first; if you're looking up a term, this is the right place. Entries are alphabetical.

A

Argument (in logic). A sequence of premises followed by a conclusion. An argument is *valid* if, whenever all the premises are simultaneously true, the conclusion is also true. Validity is about form, not content—a valid argument can have false premises and a false conclusion, as long as the inference itself is sound. Introduced in Module 3.

Assignment rule (Hoare logic). The proof rule $\{P[x/e]\} x := e \{P\}$: to conclude P holds after the assignment, you need P with e substituted for x to hold before. Always applied backward from the postcondition. See Module 7.

Associativity. A binary operation $*$ is associative if $(a*b)*c = a*(b*c)$ for all a, b, c . Addition, multiplication, and function composition are all associative. Subtraction is not. Appears first in Module 5 in the group axioms.

B

Backus-Naur Form (BNF). A notation for defining the syntax of a language or a recursive data type. Each alternative on the right-hand side of $:=$ is a way to build an instance of the type; alternatives are separated by $|$. Every BNF definition yields an induction principle of the same shape. Introduced in Module 2, used throughout Modules 4, 8, 9, and 10.

Base case. In an inductive proof, the non-recursive case that you verify by direct computation. For natural induction, the base case is $P(0)$; for list induction, it is $P(\text{Nil})$; in general, there is one base case for each non-recursive alternative in the BNF.

Bijection. A function $f : A \rightarrow B$ that is both injective and surjective—a perfect pairing between A and B . Equivalently, a function with an inverse. Bijections are the relation we use to say two sets have the same cardinality, finite or infinite (Cardinality intermezzo).

Biconditional ($\leftrightarrow$). The logical connective “if and only if.” $p \leftrightarrow q$ is shorthand for $(p \rightarrow q) \wedge (q \rightarrow p)$ and is true exactly when p and q have the same truth value. See Module 2.

Bound variable. A variable that is captured by a quantifier or a function parameter, local to the scope of that binder. Renaming a bound variable (to a fresh name) doesn't change the meaning

of a formula. See Module 4.

C

Cardinality. The “size” of a set. For finite sets, $|A|$ is the number of elements; for infinite sets, cardinality is defined up to bijection—two sets have the same cardinality iff there is a bijection between them. Countable vs. uncountable infinity is the cleanest example of different cardinalities. See Module 1 and the Cardinality intermezzo.

Cartesian product. $A \times B = \{(a, b) \mid a \in A, b \in B\}$, the set of ordered pairs. Generalizes to $A \times B \times C$, etc. Used to define functions and relations (both are subsets of a Cartesian product). See Module 1.

Category theory. The mathematics of objects and structure-preserving arrows between them, with composition as the central operation. It is the general setting in which *product*, *sum*, *functor*, and *universal property* (see separate entries) live. Named in passing in the typealgebra intermezzo; formally developed in later coursework.

Characteristic equation. The quadratic $r^2 = Ar + B$ obtained by substituting $a_n = r^n$ into a second-order linear homogeneous recurrence $a_k = Aa_{k-1} + Ba_{k-2}$. Its roots determine the closed form. See Module 9.

Chinese Remainder Theorem (CRT). If m and n are coprime, then knowing $x \bmod m$ and $x \bmod n$ uniquely determines x modulo mn . Generalizes to any finite set of pairwise-coprime moduli. See Module 6.

Closure. A set S is closed under an operation $*$ if applying $*$ to elements of S produces another element of S . The group axioms implicitly require closure (the operation has type $G \rightarrow G \rightarrow G$). See Module 5.

Completeness (of a proof system). For a proof system relative to a semantics: every semantically valid formula has a derivation. For propositional logic, every tautology has a natural-deduction proof. Partner to *soundness*. See Module 3.

Composition (of functions). Given $f : A \rightarrow B$ and $g : B \rightarrow C$, the composition $g \circ f : A \rightarrow C$ is defined by $(g \circ f)(a) = g(f(a))$. Associative, with the identity function as a two-sided unit. Introduced in Module 1 and used throughout the course, including the Curry–Howard intermezzo (as transitivity of implication) and Module 5 (the archetypal group is bijections under composition).

Conjunction ($\wedge$). The logical connective “and.” $p \wedge q$ is true exactly when both p and q are true. Under the Curry–Howard correspondence, conjunction corresponds to the product type. See Module 2.

Conjunctive Normal Form (CNF). A formula in propositional logic is in CNF when it is a conjunction of clauses, each of which is a disjunction of literals. Every formula has an equivalent CNF form. See Module 3. Contrast with *disjunctive normal form*.

Contradiction. A formula that is false under every assignment of truth values. The canonical example is $p \wedge \neg p$. Proof by contradiction assumes $\neg P$ and derives a contradiction to conclude P . See Modules 2, 3, and 7.

Contrapositive. The contrapositive of $P \rightarrow Q$ is $\neg Q \rightarrow \neg P$. An implication and its contrapositive are logically equivalent (in classical logic). Proving the contrapositive of a claim is often cleaner than proving it directly. See Module 7.

Converse. The converse of $P \rightarrow Q$ is $Q \rightarrow P$. An implication and its converse are *not* logically equivalent in general. The *converse error*—concluding P from $P \rightarrow Q$ and Q —is one of the two classic fallacies. See Module 3.

Countable set. A set that is either finite or can be put into bijection with $\mathbb{N}$. The integers and the rationals are countable; the real numbers are not. See the Cardinality intermezzo.

Counterexample. A single concrete instance that falsifies a universal claim. To disprove “for all x , $P(x)$,” it suffices to exhibit one x with $\neg P(x)$. See Module 5.

D

De Morgan’s laws. $\neg(p \wedge q) \equiv \neg p \vee \neg q$ and $\neg(p \vee q) \equiv \neg p \wedge \neg q$. Generalized to quantifiers: $\neg \forall x. P(x) \equiv \exists x. \neg P(x)$ and $\neg \exists x. P(x) \equiv \forall x. \neg P(x)$. See Modules 2 and 4.

Disjunction ($\vee$). The logical connective “or” (inclusive—true when at least one operand is true). Under the Curry–Howard correspondence, disjunction corresponds to the sum type. See Module 2.

Disjunctive Normal Form (DNF). A formula in propositional logic is in DNF when it is a disjunction of terms, each of which is a conjunction of literals. Every truth table can be mechanically converted to a DNF formula (one disjunct per T row). See Module 3. Contrast with *conjunctive normal form*.

Disjoint sets. Two sets A and B are disjoint if $A \cap B = \emptyset$ —they share no elements. See Module 1.

Divisibility. For integers a and d with $d \neq 0$, we say d divides a (written $d \mid a$) if $a = dk$ for some integer k . See Module 5.

Domain (of a function). The set of inputs on which a function is defined. For $f : A \rightarrow B$, the domain is A . See Module 1.

E

Empty set. The set with no elements, written $\emptyset$ or $\{\}$. It is a subset of every set. $|\emptyset| = 0$. See Module 1.

Equivalence relation. A relation that is reflexive, symmetric, and transitive. Equality is the canonical example; modular congruence ($a \equiv b \pmod{n}$) is another. Equivalence relations partition their underlying set into equivalence classes. See Modules 1 and 6 and the Equivalence intermezzo.

Euclidean algorithm. An algorithm for computing $\gcd(a, b)$ by repeatedly applying the identity $\gcd(a, b) = \gcd(b, a \bmod b)$ until the second argument reaches 0. The *extended* version also computes Bézout coefficients x, y with $ax + by = \gcd(a, b)$. See Module 7.

Excluded middle. The axiom $p \vee \neg p$. Every formula is either true or false, full stop. Classical logic accepts it; constructive logic does not. Proof by contradiction and double-negation elimination both depend on it. See Modules 2, 3, and 7, and the Curry–Howard intermezzo.

Existential quantifier ($\exists$). $\exists x : A. P(x)$ is true iff at least one element of A satisfies P . Its introduction rule requires a concrete witness. See Module 4.

Extensional equality. Two sets (or functions) are extensionally equal if they have the same elements (or produce the same outputs on all inputs). Contrasts with intensional equality, which cares about how objects are defined rather than what they contain. See Module 1.

F

Fallacy. An argument form that looks plausible but is actually invalid. The two classics in propositional logic are the *converse error* and the *inverse error*. See Module 3.

Fibonacci sequence. The sequence $F_0 = 0, F_1 = 1, F_n = F_{n-1} + F_{n-2}$. Its closed form, derived via the characteristic equation, involves the golden ratio $\varphi = (1 + \sqrt{5})/2$. See Module 9.

Finite field. A field—a set with addition, subtraction, multiplication, and division (by nonzero elements)—with finitely many elements. $\mathbb{Z}/p\mathbb{Z}$ is a finite field iff p is prime. See Module 6.

First-order logic. The logic of *predicate* formulas, extending propositional logic with the quantifiers $\forall$ and $\exists$. The workhorse logic for stating and proving most of ordinary mathematics. See Module 4.

Floor and ceiling. $\lfloor x \rfloor$ is the greatest integer $\leq x$ (rounding down). $\lceil x \rceil$ is the least integer $\geq x$ (rounding up). For negative numbers, floor is *not* the same as truncating toward zero. See Module 6.

Free variable. A variable that is not bound by any quantifier in its scope. A formula with free variables doesn't have a definite truth value until you specify what those variables refer to. See Module 4.

Function. A total, single-valued assignment $f : A \rightarrow B$: every $a \in A$ has exactly one output $f(a) \in B$. Dropping totality gives a *partial function*. See Module 1.

Functional completeness. A set of boolean connectives is functionally complete if every boolean function can be expressed using only those connectives. $\{\wedge, \vee, \neg\}$ is functionally complete; so is $\{\text{NAND}\}$; $\{\wedge, \vee\}$ alone is not (it only yields monotone functions). See Modules 2 and 3.

Functor. A type constructor F together with a `map` operation that, given $f : A \rightarrow B$, produces $F(f) : F(A) \rightarrow F(B)$, subject to the identity and composition laws. `List`, `Tree`, and `Maybe` are all functors. Named in the typealgebra intermezzo; developed fully in later coursework.

G

Greatest Common Divisor (GCD). For integers a, b not both zero, $\text{gcd}(a, b)$ is the largest positive integer that divides both. Computed by the Euclidean algorithm. See Module 7.

Group. A set G with an associative binary operation $*$, an identity element e (with $e * g = g * e = g$), and an inverse g^{-1} for every element (with $g * g^{-1} = g^{-1} * g = e$). $(\mathbb{Z}, +, 0)$ and $(\mathbb{Q} \setminus \{0\}, \cdot, 1)$ are groups. See Module 5.

H

Hoare logic. A formal logic for reasoning about imperative programs. Its basic statement is a triple $\{P\} c \{Q\}$: if P holds before c runs, then Q holds after. Each language construct has its own inference rule. See Module 7.

Hoare triple. A statement of the form $\{P\} c \{Q\}$ where P (the *precondition*) and Q (the *postcondition*) are logical formulas and c is a program. See Module 7.

Homomorphism. A structure-preserving map. For groups, a function $\varphi : G \rightarrow H$ is a homomorphism if $\varphi(a * b) = \varphi(a) * \varphi(b)$ for all $a, b \in G$; this forces $\varphi(e_G) = e_H$ and $\varphi(g^{-1}) = \varphi(g)^{-1}$. The notion of “map” once you’re doing algebra. See Module 5.

I

Identity element. In a group (or any structure with a binary operation), an element e satisfying $e * g = g * e = g$ for every g . Unique if it exists. See Module 5.

Identity function. The function $\text{id}_A : A \rightarrow A$ defined by $\text{id}_A(a) = a$. The two-sided unit for function composition. See Module 1.

Implication ($\rightarrow$). The logical connective $p \rightarrow q$, true except when p is true and q is false. Often read “if p then q .” The rows where p is false give *vacuous truth*. See Module 2.

Induction, natural. The proof principle: to prove $\forall n : \mathbb{N}. P(n)$, prove $P(0)$ and prove that $P(n)$ implies $P(\text{succ } n)$. Equivalent to the well-ordering principle. See Module 4 and Module 8.

Induction, strong. A variant of natural induction where the inductive hypothesis gives you $P(j)$ for *all* $j \leq k$, not just $j = k$. Logically equivalent to ordinary induction, but more convenient for recurrences that reach back more than one step. See Module 9.

Induction, structural. The general form of induction for arbitrary recursive data types. Every BNF definition yields its own structural induction principle, with one case per alternative. See Module 10.

Inductive hypothesis (IH). In an inductive proof, the assumption you get to make in the inductive step— $P(k)$ in ordinary induction, or $P(j)$ for all $j \leq k$ in strong induction, or P applied to each recursive sub-part in structural induction.

Inference rule. A schema of the form “given these premises, conclude this conclusion.” Written with the premises above a line and the conclusion below. The rules of a proof system. See Modules 3 and 4, and the Gentzen intermezzo.

Injection (injective function). A function $f : A \rightarrow B$ such that $f(a_1) = f(a_2)$ implies $a_1 = a_2$ —no two inputs share an output. See the Cardinality intermezzo.

Intersection ($\cap$). $A \cap B = \{x \mid x \in A \wedge x \in B\}$, the set of elements in both A and B . See Module 1.

Intensional equality. Equality based on how an object is defined or constructed, not just what it contains. Contrasts with extensional equality. Two programs computing the same function are extensionally equal but may be intensionally different. See Module 1.

Inverse (of a group element). The element g^{-1} such that $g * g^{-1} = g^{-1} * g = e$. Unique if it exists. See Module 5.

Inverse error. The fallacy of concluding $\neg Q$ from $P \rightarrow Q$ and $\neg P$. Not valid. See Module 3.

Irrational number. A real number that is not rational (not expressible as a/b for integers a, b). $\sqrt{2}$ is the classic example. See Module 7.

Isomorphism (of groups). A bijective group homomorphism. Two groups are *isomorphic*, $G \cong H$, when an isomorphism between them exists; isomorphic groups are interchangeable for every algebraic purpose. The right notion of sameness for groups. See Module 5.

L

Lambda calculus. A minimal universal computation model built from just three things: variables, function abstraction ($\lambda x. e$), and application ($e_1 e_2$). Everything programmable can be encoded as λ -terms. See the Lambda Calculus intermezzo.

Literal. In a propositional formula, either a variable or its negation. The building blocks of CNF and DNF clauses. See Module 3.

Logical equivalence. Two formulas are logically equivalent (written $\equiv$) if they have the same truth table—or equivalently, if the biconditional between them is a tautology. The right notion of sameness for propositional formulas. See Module 2.

Loop invariant. In a Hoare-logic proof of a while loop, a proposition that holds every time control reaches the top of the loop. The invariant, combined with the negation of the loop guard, must imply the desired postcondition. See Module 7.

M

Metalanguage. The language in which we make claims *about* a formal system, as opposed to the *object language* whose expressions the formal system generates. When we say “the formula $p \wedge q \rightarrow p$ is a tautology,” the formula is in the object language; the word “tautology” lives in the metalanguage. Keeping these layers distinct is a habit the course starts in Module 3.

Modal logic. The logic of “necessity” ($\Box$) and “possibility” ($\Diamond$) over a structure of “possible worlds.” Temporal logic is the special case where the worlds are moments in time. See the temporal-logic intermezzo.

Modular arithmetic. Arithmetic in $\mathbb{Z}/n\mathbb{Z}$, the integers modulo n . Addition, subtraction, and multiplication all work on residues mod n ; division works too if n is prime. See Module 6.

Modus ponens. The inference rule: from p and $p \rightarrow q$, conclude q . The elimination rule for $\rightarrow$. See Module 3.

Modus tollens. The inference rule: from $p \rightarrow q$ and $\neg q$, conclude $\neg p$. Derivable from modus ponens via the contrapositive. See Module 3.

Monotone function. A boolean function f is monotone if flipping any input from F to T never flips the output from T to F . Every formula built from $\{\wedge, \vee\}$ alone is monotone; $\neg$ is not. See Modules 2 and 10.

N

Natural deduction. A style of proof system where each logical connective has introduction and elimination rules, and proofs are written as trees. The Gentzen intermezzo is all about this style.

Negation ($\neg$). The logical connective that flips true to false and vice versa. Also written $\sim$ in code-style notation. See Module 2.

Normal form. A canonical way of writing a formula. CNF and DNF are normal forms for propositional logic. See Module 3.

P

Partial function. A function-like assignment that may be undefined on some inputs. Every total function is a partial function; not vice versa. See Module 1.

Partition. A decomposition of a set into disjoint non-empty subsets whose union is the whole set. Equivalence relations and partitions are in bijective correspondence. See the Equivalence intermezzo.

Power set. For a set A , the power set $\mathcal{P}(A)$ is the set of all subsets of A . If $|A| = n$ then $|\mathcal{P}(A)| = 2^n$. See Module 1.

Precondition / postcondition. The logical assertions before and after a program in a Hoare triple $\{P\} c \{Q\}$. P is the precondition; Q is the postcondition. See Module 7.

Predicate. A function from a set to $\{\text{True}, \text{False}\}$, i.e., $P : A \rightarrow \{T, F\}$. Predicates are the building blocks of statements in first-order logic. See Module 4.

Propositional logic. The logic of boolean variables and the connectives $\wedge, \vee, \neg, \rightarrow$. The simplest nontrivial logic. See Module 2.

Proof by cases. A proof technique where the situation is split into finitely many exhaustive, mutually exclusive cases, and each case is handled separately. Formally, $\vee$ -elimination. See Modules 3 and 6.

Proof by contradiction. A proof technique where you assume the negation of what you want to prove, derive a contradiction, and conclude the original claim. Requires excluded middle. See Modules 3 and 7.

Proof by contrapositive. A proof technique where, to prove $P \rightarrow Q$, you instead prove $\neg Q \rightarrow \neg P$. Requires excluded middle (via the equivalence of an implication with its contrapositive). See Module 7.

Proof by exhaustion. A proof technique where you enumerate every case in a finite domain and verify the claim on each one. A special case of proof by cases, but distinctive in practice. See Module 5.

Proof tree. A visual representation of a natural-deduction proof, with the conclusion at the root and premises branching upward. See the Gentzen intermezzo.

Q

Quantifier. $\forall$ (universal, “for all”) and $\exists$ (existential, “there exists”) are the two quantifiers in first-order logic. They bind variables over a specified domain. See Module 4.

Quotient-remainder theorem. For any integer n and positive integer d , there exist unique integers q, r with $n = dq + r$ and $0 \leq r < d$. The foundation of modular arithmetic. See Module 6.

R

Rational number. A real number expressible as a/b for integers a, b with $b \neq 0$. The set $\mathbb{Q}$ of rationals is countable. See Module 5.

Recurrence relation. A recursive definition of a sequence: initial values plus a rule expressing each subsequent term in terms of earlier ones. See Module 9.

Reflexive relation. A relation R on A is reflexive if aRa for every $a \in A$. One of the three properties of an equivalence relation. See Module 1.

Relation. A subset of $A \times B$. A relation between A and B records which pairs (a, b) are “related.” Functions are a special case of relations. See Module 1.

Residue class. The equivalence class of an integer under congruence mod n . The residue classes form the set $\mathbb{Z}/n\mathbb{Z}$. See Module 6 and the Equivalence intermezzo.

Rig. A set with $+$ and $\cdot$ satisfying the usual arithmetic laws but *without* requiring additive inverses: a “ring without negation.” Data types under sum and product form a rig (you can’t subtract types). See the typealgebra intermezzo.

S

Sequence. A function from $\mathbb{N}$ (or a subset) to some set. Can be defined explicitly or recursively. See Module 8.

Set. An unordered, duplicate-free collection of elements. The primary data structure of discrete math. See Module 1.

Set-builder notation. The notation $\{x \mid x \in S, P(x)\}$ for “the set of all x in S such that $P(x)$.” See Module 1.

Soundness (of a proof system). For a proof system relative to a semantics: every derivable formula is semantically valid. For propositional logic, every derivable formula is a tautology. Partner to *completeness*. See Module 3.

Subset ($\subseteq$). $A \subseteq B$ iff every element of A is also an element of B . Equivalently, $\forall x. (x \in A \rightarrow x \in B)$. See Module 1.

Surjection (surjective function). A function $f : A \rightarrow B$ such that every $b \in B$ has some $a \in A$ with $f(a) = b$. See the Cardinality intermezzo.

Symmetric relation. A relation R on A is symmetric if $a R b$ implies $b R a$ for all a, b . See Module 1.

T

Tautology. A formula that is true under every assignment of truth values. $p \vee \neg p$ is the canonical example. See Module 2.

Temporal logic. A modal logic whose “worlds” are moments in time; the modal operators become $\square$ (“always”), $\diamond$ (“eventually”), and $\bigcirc$ (“next”). The basis of model checking. See the temporal-logic intermezzo.

Transitive relation. A relation R on A is transitive if $a R b$ and $b R c$ imply $a R c$, for all a, b, c . See Module 1.

Truth set. For a predicate P on a set A , the truth set is $\{x \in A \mid P(x)\}$, the set of elements on which P is true. See Module 4.

Truth table. A table listing the truth value of a formula for every assignment of truth values to its variables. The semantics of propositional logic, and a decision procedure for tautology-checking. See Module 2.

U

Union ($\cup$). $A \cup B = \{x \mid x \in A \vee x \in B\}$, the set of elements in A or B (or both). See Module 1.

Universal quantifier ($\forall$). $\forall x : A. P(x)$ is true iff every element of A satisfies P . Its introduction rule requires generalizing over an arbitrary element. See Module 4.

Uncountable set. A set that cannot be put into bijection with $\mathbb{N}$. The real numbers are uncountable, as proved by Cantor’s diagonal argument. See the Cardinality intermezzo.

Universal property. A way of specifying a mathematical object by describing the arrows into or out of it rather than its internal construction. $A \times B$ is pinned down by the fact that functions $X \rightarrow A \times B$ correspond to pairs of functions $X \rightarrow A$ and $X \rightarrow B$; $A + B$, 1 , 0 , and $A \rightarrow B$ each have analogous characterizations. See the typealgebra intermezzo.

V

Vacuous truth. The logical convention that $F \rightarrow q$ is true for every q . Follows from the definition of $\rightarrow$: the implication only promises something when its antecedent is true. See Module 2.

Valid argument. An argument whose conclusion must be true whenever all its premises are true. See *argument*.

Venn diagram. A visual representation of set operations: each set is a circle, and you shade the region corresponding to the set you’re describing. Not a rigorous proof tool, but useful for intuition. See Module 1.

Appendix C

Further Reading

Throughout this book, most intermezzi ended with a “Further reading” section pointing at accessible next steps if the topic caught your interest. This appendix collects those entries in one place, organized by topic, for readers who want the whole reading list at a glance. Each entry is reproduced from the intermezzo where it originally appeared, so there is some overlap when a single source is relevant to multiple topics.

A few of the entries below are genuinely foundational papers that everyone in computer science should read at some point; others are gentle on-ramps for specific subtopics. If you only have time for one or two, the entries marked **[start here]** are the ones to pick up first.

Natural Deduction and Proof Theory

Related intermezzo: Inference Rules as Trees (Module 3).

- **[start here]** Frank Pfenning, *Natural Deduction* (lecture notes for CMU 15-317, “Constructive Logic”). The single best gentle introduction to natural deduction in tree form, with exactly the discipline of introduction and elimination rules used in the intermezzo.
- Jan von Plato, “Gentzen’s Proof Systems: Byproducts in a Work of Genius,” *Bulletin of Symbolic Logic* **18**(3), 313–367 (2012). A historical and conceptual essay on how Gentzen actually invented natural deduction and the sequent calculus, written for a logic-curious but non-specialist audience.
- Dag Prawitz, *Natural Deduction: A Proof-Theoretical Study*, Almqvist & Wiksell (1965); Dover reprint (2006). The first chapter is the canonical exposition of normalization—“every detour can be removed”—which is the proof-theory version of evaluating a program. Stop at the end of Chapter I; later chapters are more technical.

Cantor, Infinity, and the Diagonal Argument

Related intermezzo: Cardinality and Function Spaces (Module 1).

- Robert Gray, “Georg Cantor and Transcendental Numbers,” *American Mathematical Monthly* **101**(9), 819–832 (1994). A clean historical account of Cantor’s *original* 1874 uncountability argument (which was not the diagonal argument!) and how the diagonal version came later. Excellent for seeing how the ideas in the intermezzo actually developed.

- **[start here]** Martin Davis, “[What is a Computation?](#)” in L. A. Steen (ed.), *Mathematics Today: Twelve Informal Essays*, Springer (1978). Davis walks you from Cantor’s diagonal argument straight to the unsolvability of the halting problem in a way that requires almost no prior CS background—perfect if you want to see the forward look from this chapter cashed out.
- Raymond Smullyan, *Diagonalization and Self-Reference*, Oxford Logic Guides 27, Oxford University Press (1994). A whole book—but the early chapters are gentle, and they show how Cantor, Gödel, and Tarski are all variations on a single self-referential theme.

Curry–Howard and Proofs as Programs

Related intermezzi: Proofs Are Programs (Module 7), The Lambda Calculus (end of book).

- **[start here]** Philip Wadler, “[Propositions as Types](#),” *Communications of the ACM* **58**(12), 75–84 (2015). The accessible undergraduate-friendly tour of the Curry–Howard correspondence, with historical context, examples in modern languages, and Wadler’s characteristic clarity. If you read only one paper from this list, read this one.
- William A. Howard, “[The Formulae-as-Types Notion of Construction](#),” in J. R. Hindley and J. P. Seldin (eds.), *To H. B. Curry: Essays on Combinatory Logic, Lambda Calculus and Formalism*, Academic Press (1980); manuscript circulated 1969. The original short note that started everything. Only about a dozen pages, and the first half is genuinely readable once you have seen natural deduction.
- Philip Wadler, “[Proofs are Programs: 19th Century Logic and 21st Century Computing](#),” *Dr. Dobbs’s Journal* (2000). A shorter, more popular precursor to “Propositions as Types,” pitched at working programmers—good as a warm-up before tackling the CACM piece.

The Lambda Calculus

Related intermezzo: The Lambda Calculus (end of book).

- Henk Barendregt, “[The Impact of the Lambda Calculus in Logic and Computer Science](#),” *Bulletin of Symbolic Logic* **3**(2), 181–215 (1997). The standard expository overview of why the lambda calculus matters across logic and computer science, written by the field’s leading authority. The first half is fully undergraduate-accessible.
- **[start here]** Peter Selinger, *Lecture Notes on the Lambda Calculus*, arXiv:0804.3434 (revised 2013). The cleanest free introduction I know of—covers Church encodings, beta reduction, the Church–Rosser theorem, and fixed-point combinators at exactly undergraduate pitch.
- Felice Cardone and J. Roger Hindley, “[History of Lambda-calculus and Combinatory Logic](#),” in D. Gabbay and J. Woods (eds.), *Handbook of the History of Logic*, vol. 5, Elsevier (2009). A historical narrative of how Church, Curry, and Turing arrived at the lambda calculus and combinatory logic—gives the Church–Turing-thesis story in human terms.

The Algebra of Types

Related intermezzo: The Algebra of Types (end of book).

- **[start here]** Conor McBride, “The Derivative of a Regular Type is its Type of One-Hole Contexts,” unpublished manuscript (2001), available on the author’s homepage at strictlypositive.org. A short, witty, surprisingly elementary derivation of why *differentiating* an algebraic data type literally gives you its zipper. Connects high-school calculus to programming in a way undergraduates love.
- Brent Yorgey, “Species and Functors and Types, Oh My!” *Haskell Symposium 2010*. Written explicitly as an accessible bridge between Joyal’s combinatorial species and the algebraic data types programmers already know. The most undergraduate-friendly entry point to species I know of.
- Philippe Flajolet and Robert Sedgewick, *Analytic Combinatorics*, Cambridge University Press (2009), Chapter I (“Symbolic Methods”). The full book PDF is freely available on Sedgewick’s Princeton homepage. Chapter I introduces the symbolic method—sums, products, sequences as type combinators with generating functions—at exactly the level a strong undergrad can handle.

Temporal Logic and Model Checking

Related intermezzo: Other Logics, Other Worlds (Module 7).

- **[start here]** Chris Newcombe, Tim Rabbat, Fan Zhang, Bogdan Munteanu, Marc Brooker, and Michael Deardeuff, “How Amazon Web Services Uses Formal Methods,” *Communications of the ACM* **58**(4), 66–73 (2015). A wonderfully accessible report by practicing engineers on how AWS uses TLA+ to verify the correctness of distributed systems like S3 and DynamoDB. If you wonder why anyone outside academia cares about temporal logic, this is your answer.
- Edmund M. Clarke, E. Allen Emerson, and Joseph Sifakis, “Model Checking: Algorithmic Verification and Debugging,” *Communications of the ACM* **52**(11), 74–84 (2009). The Turing Award lecture: a retrospective on how model checking was developed and why it works. Written for a general CS audience.
- Amir Pnueli, “The Temporal Logic of Programs,” *18th Annual Symposium on Foundations of Computer Science (FOCS)*, 46–57 (1977). The founding paper of program temporal logic. Sections 1–2 are historically essential and undergraduate-readable; later sections get more technical.

Equivalence, Sameness, and Structuralism

Related intermezzo: What Does It Mean for Things to Be “The Same”? (Module 6).

- **[start here]** Barry Mazur, “[When is one thing equal to some other thing?](#)” in *Proof and Other Dilemmas: Mathematics and Philosophy* (B. Gold and R. A. Simons, eds.), Mathematical Association of America (2008). The single best expository essay for undergraduates on equivalence, quotient, and “sameness.” Mazur writes beautifully and assumes almost nothing.

- Steve Awodey, “[Structuralism, Invariance, and Univalence](#),” *Philosophia Mathematica* **22**(1), 1–11 (2014). A short, accessible philosophical essay on what it means for two mathematical structures to be “the same” and on why equivalence (rather than literal equality) is the right notion. The closing section connects to the modern Univalence axiom in homotopy type theory.
- Paul Halmos, *Naive Set Theory*, Springer (1960; reprint 1974), Sections 7–8. Not a paper, but Halmos’s two-page treatment of equivalence relations and partitions is the canonical undergraduate reference and pairs nicely with the philosophical pieces above.

Game Semantics, Interactive Proofs, and Zero Knowledge

Related intermezzo: Quantifiers as Games (Module 4).

- Jaakko Hintikka, “Language-Games for Quantifiers,” *American Philosophical Quarterly* Monograph Series no. 2 (1968), 46–72. Hintikka’s own short and accessible exposition of how quantifiers are best read as a two-player game between a Verifier and a Falsifier—the philosophical source of the framework in the intermezzo.
- Shafi Goldwasser, Silvio Micali, and Charles Rackoff, “The Knowledge Complexity of Interactive Proof Systems,” *SIAM Journal on Computing* **18**(1), 186–208 (1989). The foundational paper on interactive proofs and zero-knowledge; widely available via Silvio Micali’s MIT homepage and elsewhere. The first few sections, where they set up the Prover/Verifier game, are surprisingly accessible and put a precise mathematical frame around the picture sketched in the chapter.
- **[start here]** Oded Goldreich, [Zero-Knowledge: A Tutorial](#), available on the author’s homepage at the Weizmann Institute. Written explicitly to be the most accessible on-ramp to zero-knowledge proofs for non-specialists. If interactive proofs caught your attention, this is where to go next.

Induction and the Peano Axioms

Related intermezzo: More on Natural Induction (Module 4).

- **[start here]** Leon Henkin, “On Mathematical Induction,” *American Mathematical Monthly* **67**(4), 323–338 (1960). The classic expository article distinguishing the *principle* of induction from the *axiom*, and showing precisely which Peano axioms are needed to prove commutativity and associativity of addition—the same proofs the intermezzo walks through.
- George Pólya, [Induction and Analogy in Mathematics](#) (volume I of *Mathematics and Plausible Reasoning*), Princeton University Press (1954), particularly the chapter on mathematical induction. Pólya’s gentle, example-driven exposition is aimed at students discovering proof for the first time.
- Edmund Landau, *Foundations of Analysis*, Chelsea (1951; original German 1930), Chapter 1. Landau builds the natural numbers from the Peano axioms and proves commutativity and associativity of addition with full inductive rigor in about ten pages—you can literally check every step. Terse but transparent.

Modular Arithmetic and Finite Fields

Related module: Module 6.

- *Wikipedia: [Finite field arithmetic](#)*. Useful starting reference for how finite fields are actually computed with in software and hardware.
- *Wikipedia: [Finite field: Discrete logarithm](#)*. The discrete logarithm problem in a finite field is the hard problem underlying Diffie–Hellman key exchange and a surprising amount of modern cryptography.

Appendix D

Solutions to Exercises

Module 1

Warm-up

Exercise 1. Let $A = \{1, 2, 3, 4\}$ and $B = \{3, 4, 5, 6\}$.

$$\begin{aligned}A \cap B &= \{3, 4\}, \\A \cup B &= \{1, 2, 3, 4, 5, 6\}, \\A \setminus B &= \{1, 2\}, \\|A \times B| &= |A| \cdot |B| = 4 \times 4 = 16.\end{aligned}$$

Exercise 2. The set $\{1, \{1\}, \{1, \{1\}\}\}$ has three distinct elements: 1, $\{1\}$, and $\{1, \{1\}\}$. Therefore $|\{1, \{1\}, \{1, \{1\}\}\}| = 3$.

Exercise 3. For the first set, we want natural numbers n with $n^2 < 20$. Checking small values: $0^2 = 0$, $1^2 = 1$, $2^2 = 4$, $3^2 = 9$, $4^2 = 16$ all satisfy the condition, while $5^2 = 25$ does not. So $\{n \mid n \in \mathbb{N}, n^2 < 20\} = \{0, 1, 2, 3, 4\}$.

For the second, we want integers n with $-3 \leq n < 3$. The endpoint 3 is excluded but -3 is included, so we get $\{n \mid n \in \mathbb{Z}, -3 \leq n < 3\} = \{-3, -2, -1, 0, 1, 2\}$.

Exercise 4. Let $S = \{1, 2, \{1\}, \{1, 2\}\}$.

- (a) $1 \in S$: **true**. 1 is literally one of the four listed elements.
- (b) $\{1\} \in S$: **true**. The set $\{1\}$ is also one of the listed elements—the third one.
- (c) $\{1\} \subseteq S$: **true**. Every element of $\{1\}$ —just 1—is in S .
- (d) $\{1, 2\} \in S$: **true**. The set $\{1, 2\}$ is the fourth listed element.
- (e) $\{1, 2\} \subseteq S$: **true**. Both 1 and 2 are elements of S . Note that (d) and (e) are both true but for different reasons—(d) says “ $\{1, 2\}$ appears as a single element of S ,” while (e) says “each of 1 and 2 appears as an element of S .”
- (f) $\{\{1\}\} \subseteq S$: **true**. The only element of $\{\{1\}\}$ is $\{1\}$, and $\{1\} \in S$.

- (g) $2 \subseteq S$: **type error**. Being a subset is a relation between two *sets*, and 2 is a number, not a set. (If you insisted on interpreting 2 as the von Neumann natural number $\{0, 1\}$, you could make sense of the question, but in this course we treat the natural numbers as primitive, not as sets.)

Exercise 5.

$$\mathcal{P}(\{a, b, c\}) = \{\emptyset, \{a\}, \{b\}, \{c\}, \{a, b\}, \{a, c\}, \{b, c\}, \{a, b, c\}\}.$$

That's $8 = 2^3$ elements, as expected. And

$$\{a, b\} \times \{1, 2, 3\} = \{(a, 1), (a, 2), (a, 3), (b, 1), (b, 2), (b, 3)\},$$

which has $2 \times 3 = 6$ elements.

Standard

Exercise 6.

- (a) $a R b$ iff $a + b$ is even.

- *Reflexive*: $a + a = 2a$, which is always even. ✓
- *Symmetric*: $a + b = b + a$, so if $a + b$ is even then $b + a$ is even. ✓
- *Transitive*: Suppose $a + b$ and $b + c$ are both even. Then $a + c = (a + b) + (b + c) - 2b$ is a sum of even numbers minus an even number, hence even. ✓

All three properties hold, so R is an equivalence relation.

- (b) $a R b$ iff $a \leq b$.

- *Reflexive*: $a \leq a$. ✓
- *Symmetric*: Fails. $1 \leq 2$ but $2 \not\leq 1$. ✗
- *Transitive*: If $a \leq b$ and $b \leq c$ then $a \leq c$. ✓

Not an equivalence relation (symmetry fails).

- (c) $a R b$ iff $|a - b| \leq 1$.

- *Reflexive*: $|a - a| = 0 \leq 1$. ✓
- *Symmetric*: $|a - b| = |b - a|$. ✓
- *Transitive*: Fails. $|0 - 1| = 1 \leq 1$ and $|1 - 2| = 1 \leq 1$, but $|0 - 2| = 2 > 1$. ✗

Not an equivalence relation (transitivity fails).

Exercise 7.

- (a) $f(x) = 1/x$. *Not a function* on $\mathbb{R}$: totality fails at $x = 0$, where the assignment gives no output. (It *is* a function on $\mathbb{R} \setminus \{0\}$, so if the domain were restricted it would work.)
- (b) $f(x) = \sqrt{x}$, non-negative branch. *Not a function* on $\mathbb{R}$: totality fails for negative x because $\sqrt{x}$ is not a real number when $x < 0$. (It *is* a function on $\{x \in \mathbb{R} \mid x \geq 0\}$.)

- (c) $\{(x, y) \mid y^3 = x\}$. *Is a function.* Every real x has a unique real cube root, so totality and uniqueness both hold. The rule is $x \mapsto \sqrt[3]{x}$.
- (d) $\{(x, y) \mid y^2 = x\}$. *Not a function.* Uniqueness fails: $x = 4$ has two outputs, $y = 2$ and $y = -2$. Totality also fails for negative x . (This is the example from the good-vs-bad proof in the module, just in “relation” clothing.)

Exercise 8. We prove $A \cap (B \cup C) = (A \cap B) \cup (A \cap C)$ by showing each side is a subset of the other.

$(\subseteq)$. Let $x \in A \cap (B \cup C)$. Then $x \in A$ and $x \in B \cup C$, so $x \in A$ and either $x \in B$ or $x \in C$. We split into two cases. If $x \in B$, then $x \in A$ and $x \in B$, so $x \in A \cap B$, hence $x \in (A \cap B) \cup (A \cap C)$. If instead $x \in C$, then $x \in A \cap C$ by the same reasoning, hence $x \in (A \cap B) \cup (A \cap C)$. Either way, x is in the right-hand side.

$(\supseteq)$. Let $x \in (A \cap B) \cup (A \cap C)$. Then either $x \in A \cap B$ or $x \in A \cap C$. In either case, $x \in A$ (since both $A \cap B$ and $A \cap C$ are subsets of A). In the first case, $x \in B$, so $x \in B \cup C$. In the second, $x \in C$, so again $x \in B \cup C$. Either way, $x \in A$ and $x \in B \cup C$, so $x \in A \cap (B \cup C)$.

Since each side is contained in the other, they are equal. $\square$

Exercise 9. *Claim:* If $A \subseteq B$ then $\mathcal{P}(A) \subseteq \mathcal{P}(B)$. *True.*

Proof. Assume $A \subseteq B$. To show $\mathcal{P}(A) \subseteq \mathcal{P}(B)$, let S be an arbitrary element of $\mathcal{P}(A)$. By definition of power set, this means $S \subseteq A$. So every element of S is in A , and therefore—since $A \subseteq B$ —every element of S is in B . That is, $S \subseteq B$, which means $S \in \mathcal{P}(B)$. Since S was arbitrary, $\mathcal{P}(A) \subseteq \mathcal{P}(B)$. $\square$

The key move here is the “two-level” reasoning: to argue about subsets of power sets, you have to peel off one layer (the outer $\subseteq$) and then use another layer (the inner $\subseteq$).

Exercise 9b. With $f(n) = n + 1$ and $g(n) = 2n$:

$$(g \circ f)(n) = g(f(n)) = g(n + 1) = 2(n + 1) = 2n + 2,$$

$$(f \circ g)(n) = f(g(n)) = f(2n) = 2n + 1.$$

These are not the same function—for example, at $n = 0$ the first gives 2 and the second gives 1. In particular, composition is *not* commutative in general.

For associativity, applying both sides to n :

$$((f \circ f) \circ g)(n) = (f \circ f)(g(n)) = f(f(2n)) = f(2n + 1) = 2n + 2,$$

$$(f \circ (f \circ g))(n) = f((f \circ g)(n)) = f(2n + 1) = 2n + 2.$$

Both sides send n to $2n + 2$, confirming associativity at this n . Since n was arbitrary, the two compositions agree as functions.

Challenge

Exercise 10. We argue by counting. Every element of $A \cup B$ falls into exactly one of three categories:

- elements of A that are not in B (there are $|A \setminus B|$ of them),
- elements of B that are not in A (there are $|B \setminus A|$ of them),
- elements in both A and B (there are $|A \cap B|$ of them).

These three categories are disjoint and their union is $A \cup B$, so

$$|A \cup B| = |A \setminus B| + |B \setminus A| + |A \cap B|.$$

Now, A itself is the union of the first and third categories, so $|A| = |A \setminus B| + |A \cap B|$, and similarly $|B| = |B \setminus A| + |A \cap B|$. Adding,

$$|A| + |B| = |A \setminus B| + |B \setminus A| + 2|A \cap B|.$$

This is exactly $|A \cup B|$ with the intersection counted *twice*. Subtracting one copy gives

$$|A \cup B| = |A| + |B| - |A \cap B|,$$

as required. □

The moral: $|A| + |B|$ overcounts the elements that are in both sets, and inclusion–exclusion corrects for the overcounting.

Exercise 11. A relation on A is a subset of $A \times A$. If $|A| = n$, then $|A \times A| = n^2$, so

$$\text{number of relations on } A = |\mathcal{P}(A \times A)| = 2^{n^2}.$$

For reflexive relations: a reflexive relation must contain every pair (a, a) for $a \in A$ —that’s n forced pairs. The remaining $n^2 - n$ pairs (the off-diagonal pairs) may be freely chosen to be in or out, independently. So the number of reflexive relations is

$$2^{n^2-n}.$$

For example, with $n = 2$: $2^4 = 16$ relations total, and $2^{4-2} = 2^2 = 4$ reflexive ones.

Exercise 12. The flaw is the phrase “if $a R b$ ”—the argument assumes that *every* a has *some* b with $a R b$. That’s an extra assumption; the definition of reflexive doesn’t get a free ride on it. If there’s an element $a \in A$ that is not related to anything at all, the argument never gets started for a , and we have no obligation to conclude $a R a$.

Concrete counterexample on $A = \{1, 2, 3\}$: take $R = \{(1, 2), (2, 1), (1, 1), (2, 2)\}$. This is symmetric (the pairs $(1, 2)$ and $(2, 1)$ are both in R) and transitive (check each chain: from $(1, 2)$ and $(2, 1)$ we get $(1, 1)$, which is in R ; from $(2, 1)$ and $(1, 2)$ we get $(2, 2)$, in R ; the diagonal pairs chain trivially). But R is *not* reflexive: $(3, 3) \notin R$. The element 3 is not related to anything, so the bad argument never reaches it.

The additional assumption needed to save the argument is what’s sometimes called *seriality* or “left-totality”: for every $a \in A$, there exists some b with $a R b$. Under that extra assumption, symmetric and transitive do imply reflexive.

Connection

Exercise 13.

```
def cartesian(xs, ys):
    result = []
    for x in xs:
        for y in ys:
            result.append((x, y))
    return result

print(cartesian([1, 2], ['a', 'b', 'c']))
# [(1, 'a'), (1, 'b'), (1, 'c'), (2, 'a'), (2, 'b'), (2, 'c')]
```

The output matches the six pairs from Exercise 5. The nested loop visits each (x, y) combination exactly once, so it does $m \cdot n$ appends and (if you're counting inner loop iterations as "comparisons") $m \cdot n$ work total. This is the computational content of $|A \times B| = |A| \cdot |B|$.

Exercise 14. There are $2^3 = 8$ functions from $\{a, b, c\}$ to $\{0, 1\}$. Each function is determined by its three outputs, and each output is one of two choices, giving $2 \times 2 \times 2 = 8$ total:

f	$f(a)$	$f(b)$	$f(c)$
f_1	0	0	0
f_2	0	0	1
f_3	0	1	0
f_4	0	1	1
f_5	1	0	0
f_6	1	0	1
f_7	1	1	0
f_8	1	1	1

The answer is 2^3 and not 3^2 because the *codomain* (here $\{0, 1\}$, of size 2) is the base of the exponent and the *domain* (here $\{a, b, c\}$, of size 3) is the exponent itself. Intuitively, each of the 3 inputs independently gets 2 choices, so the total is $2 \cdot 2 \cdot 2 = 2^3$. This is why the set of functions from A to B is sometimes written B^A —the notation encodes the counting formula. The intermezzo on cardinality picks up this thread and runs with it.

Intermezzo: Cardinality and Function Spaces

Exercise 1.

$$\mathcal{P}(\{1, 2, 3\}) = \{\emptyset, \{1\}, \{2\}, \{3\}, \{1, 2\}, \{1, 3\}, \{2, 3\}, \{1, 2, 3\}\}.$$

That is $2^3 = 8$ elements.

Exercise 2. There are $2^3 = 8$ functions from $\{a, b, c\}$ to $\{0, 1\}$. Each function is the characteristic function of a subset of $\{a, b, c\}$:

Subset	$f(a)$	$f(b)$	$f(c)$
$\emptyset$	0	0	0
$\{a\}$	1	0	0
$\{b\}$	0	1	0
$\{c\}$	0	0	1
$\{a, b\}$	1	1	0
$\{a, c\}$	1	0	1
$\{b, c\}$	0	1	1
$\{a, b, c\}$	1	1	1

Exercise 3.

Proof. Define $f : \mathbb{N} \rightarrow E$ by $f(n) = 2n$, where $E = \{0, 2, 4, 6, \dots\}$.

Injective: If $f(n) = f(m)$, then $2n = 2m$, so $n = m$.

Surjective: Let $e \in E$. Then $e = 2k$ for some $k \in \mathbb{N}$, and $f(k) = 2k = e$.

Since f is both injective and surjective, it is a bijection, proving that the even natural numbers are countably infinite. $\square$

Exercise 4. One clean bijection is $f : (0, 1) \rightarrow \mathbb{R}$ defined by

$$f(x) = \tan\left(\pi x - \frac{\pi}{2}\right).$$

The argument $\pi x - \pi/2$ is a linear map sending $(0, 1)$ to the open interval $(-\pi/2, \pi/2)$, on which $\tan$ is a strictly increasing bijection onto $\mathbb{R}$ (it goes from $-\infty$ to $+\infty$ and is continuous with nonzero derivative). The composition of two bijections is a bijection, so f is bijective.

Injective: if $f(x) = f(y)$, then the monotonicity of both $x \mapsto \pi x - \pi/2$ and $\tan$ on $(-\pi/2, \pi/2)$ forces $x = y$.

Surjective: for any $r \in \mathbb{R}$, the value $x = \frac{1}{\pi}(\arctan r + \frac{\pi}{2})$ lies in $(0, 1)$ and satisfies $f(x) = r$.

(An equivalent rational-function witness is $g(x) = \frac{1}{x} - \frac{1}{1-x}$ on $(0, 1)$. As $x \rightarrow 0^+$ we have $1/x \rightarrow +\infty$ and $1/(1-x) \rightarrow 1$, so $g(x) \rightarrow +\infty$; as $x \rightarrow 1^-$ we have $1/x \rightarrow 1$ and $1/(1-x) \rightarrow +\infty$, so $g(x) \rightarrow -\infty$. A short derivative check shows g is strictly decreasing on $(0, 1)$, hence a continuous strictly-monotone bijection onto $\mathbb{R}$.)

The takeaway is that “size” for infinite sets is not about how much room they appear to take up in the real line: the tiny interval $(0, 1)$ has exactly as many points as the whole line.

Exercise 5. Suppose, for contradiction, that there is a surjection $f : A \rightarrow \mathcal{P}(A)$. Define

$$D = \{a \in A \mid a \notin f(a)\}.$$

D is a subset of A , so $D \in \mathcal{P}(A)$. Because f is surjective, some $d \in A$ satisfies $f(d) = D$. Now ask the fatal question: is $d \in D$?

Case 1: $d \in D$. By the definition of D , membership in D means $d \notin f(d) = D$. So $d \notin D$, contradicting our case assumption.

Case 2: $d \notin D$. Then $d \notin f(d)$, which is exactly the condition for membership in D . So $d \in D$, again a contradiction.

Either way we get a contradiction, so no such surjection f exists. Since the identity $A \rightarrow A$ composed with the obvious injection $a \mapsto \{a\}$ embeds A into $\mathcal{P}(A)$ (so $|A| \leq |\mathcal{P}(A)|$) while no surjection goes the other way, we have $|\mathcal{P}(A)| > |A|$ strictly. $\square$

The same diagonal structure as Cantor’s real-numbers proof is visible here: D is the set that *disagrees* with each candidate $f(a)$ on the single question “is a in you?”—so it cannot equal any $f(a)$, no matter which a we pick.

Exercise 6. Fix $n \in \mathbb{N}$ and consider how g and f_n compare at the input n :

$$g(n) = f_n(n) + 1 \neq f_n(n).$$

Two functions $\mathbb{N} \rightarrow \mathbb{N}$ are equal iff they agree on *every* input. Since g and f_n disagree at the input n , $g \neq f_n$ as functions. This holds for every n , so g is not anywhere in the list $f_0, f_1, f_2, \dots$.

Consequently, any countable list of functions $\mathbb{N} \rightarrow \mathbb{N}$ is missing at least one function (namely the g built from it). So the set of all functions $\mathbb{N} \rightarrow \mathbb{N}$ is not countable.

This is the same diagonal move as in Cantor’s theorem: to beat a proposed enumeration, construct an object that differs from the n th entry in the n th slot. The same trick drives the proof that the halting problem is undecidable: one builds a program that does the *opposite* of what the n th program does on input n , and that program is guaranteed not to appear in any enumeration of “always halting” programs.

Exercise 7. We list the eight functions from $\{0, 1, 2\}$ to $\{a, b\}$, one per row, together with the subset of $\{0, 1, 2\}$ whose characteristic function maps to a -versus- b in the same pattern (if we treat a as “in the subset” and b as “out”):

Function	$f(0)$	$f(1)$	$f(2)$	Subset
f_1	b	b	b	$\emptyset$
f_2	a	b	b	$\{0\}$
f_3	b	a	b	$\{1\}$
f_4	b	b	a	$\{2\}$
f_5	a	a	b	$\{0, 1\}$
f_6	a	b	a	$\{0, 2\}$
f_7	b	a	a	$\{1, 2\}$
f_8	a	a	a	$\{0, 1, 2\}$

There are exactly $2^3 = 8$ rows. The eight rows cover every possible assignment of an output in $\{a, b\}$ to each of the three inputs— 2 choices per input and 3 independent inputs gives $2 \cdot 2 \cdot 2 = 8$ functions. Each row corresponds to a distinct subset of $\{0, 1, 2\}$, and the map “function $\mapsto$ subset” is a bijection—which is exactly the statement that $\mathcal{P}(\{0, 1, 2\}) \cong \{a, b\}^{\{0, 1, 2\}}$.

Module 2

Warm-up

Exercise 1. Truth table for $(p \rightarrow q) \wedge (q \rightarrow p)$:

p	q	$p \rightarrow q$	$q \rightarrow p$	$(p \rightarrow q) \wedge (q \rightarrow p)$
T	T	T	T	T
T	F	F	T	F
F	T	T	F	F
F	F	T	T	T

This matches the biconditional $p \leftrightarrow q$, which is true exactly when p and q have the same truth value.

Exercise 2. Truth table for $\neg(p \vee q)$:

p	q	$p \vee q$	$\neg(p \vee q)$
T	T	T	F
T	F	T	F
F	T	T	F
F	F	F	T

Truth table for $(\neg p) \wedge (\neg q)$:

p	q	$\neg p$	$\neg q$	$(\neg p) \wedge (\neg q)$
T	T	F	F	F
T	F	F	T	F
F	T	T	F	F
F	F	T	T	T

The output columns are identical, confirming De Morgan's law $\neg(p \vee q) \equiv (\neg p) \wedge (\neg q)$.

Exercise 3. A unary boolean operation takes one boolean input (2 possible values) and produces one boolean output. The number of such operations is $2^2 = 4$. They are:

1. Identity: $p \mapsto p$.
2. Negation: $p \mapsto \neg p$.
3. Constant true: $p \mapsto T$ (sometimes called “top” or “always true”).
4. Constant false: $p \mapsto F$ (“bottom” or “always false”).

Exercise 4. Build each truth table and read off the pattern:

- (a) $p \vee \neg p$ is T on both rows, so it's a **tautology** (this is the law of excluded middle).
- (b) $p \wedge \neg p$ is F on both rows, so it's a **contradiction**.
- (c) $p \rightarrow (q \rightarrow p)$: if the “outer” p is T then $q \rightarrow p$ is T regardless of q , so the implication is T ; if the outer p is F then the outer implication is vacuously T . **Tautology**.
- (d) $(p \rightarrow q) \rightarrow p$: checking all four rows, $(p = T, q = T)$ gives $T \rightarrow T = T$; $(p = T, q = F)$ gives $F \rightarrow T = T$; $(p = F, q = T)$ gives $T \rightarrow F = F$; $(p = F, q = F)$ gives $T \rightarrow F = F$. So the formula is true when p is true and false when p is false—it's logically equivalent to p itself. **Contingent**. (Interesting footnote: the formula $((p \rightarrow q) \rightarrow p) \rightarrow p$, with one extra implication wrapped around the outside, *is* a tautology in classical logic. It's called Peirce's law, and it fails in constructive logic—it's one of the standard tests for whether a logic is classical.)
- (e) $(p \wedge q) \rightarrow (p \vee q)$: whenever the antecedent $p \wedge q$ is true, both p and q are true, so $p \vee q$ is also true; in the other rows the implication is vacuously true. **Tautology**.

Exercise 5.

- (a) $\neg p \wedge q \vee r$ groups as $((\neg p) \wedge q) \vee r$. At $p = T, q = F, r = T$: $((F) \wedge F) \vee T = F \vee T = T$.
- (b) $p \rightarrow q \rightarrow r$ groups (right-associatively) as $p \rightarrow (q \rightarrow r)$. At $p = T, q = F, r = T$: $q \rightarrow r = F \rightarrow T = T$, then $p \rightarrow T = T$. If you grouped it left-associatively instead as $(p \rightarrow q) \rightarrow r$, you would get $(T \rightarrow F) \rightarrow T = F \rightarrow T = T$ —same answer in this row by coincidence, but they differ in general. Try $p = F, q = T, r = F$ to see the split: right-assoc gives $F \rightarrow (T \rightarrow F) = F \rightarrow F = T$; left-assoc gives $(F \rightarrow T) \rightarrow F = T \rightarrow F = F$.
- (c) $\neg p \vee q \wedge \neg r$ groups as $(\neg p) \vee (q \wedge (\neg r))$. At $p = T, q = F, r = T$: $F \vee (F \wedge F) = F \vee F = F$.

Standard

Exercise 6. We claim $p \oplus q \equiv (p \wedge \neg q) \vee (\neg p \wedge q)$. Truth table:

p	q	$p \wedge \neg q$	$\neg p \wedge q$	$(p \wedge \neg q) \vee (\neg p \wedge q)$
T	T	F	F	F
T	F	T	F	T
F	T	F	T	T
F	F	F	F	F

This matches the XOR truth table (true when exactly one of p, q is true). This is exactly what the DNF recipe would produce: the truth table has two T rows, namely (T, F) and (F, T) , and those are the two disjuncts.

Exercise 7. We build one combined truth table and read off both sides:

p	q	r	$q \wedge r$	$p \rightarrow (q \wedge r)$	$p \rightarrow q$	$p \rightarrow r$	$(p \rightarrow q) \wedge (p \rightarrow r)$
T	T	T	T	T	T	T	T
T	T	F	F	F	T	F	F
T	F	T	F	F	F	T	F
T	F	F	F	F	F	F	F
F	T	T	T	T	T	T	T
F	T	F	F	T	T	T	T
F	F	T	F	T	T	T	T
F	F	F	F	T	T	T	T

Columns 5 and 8 agree on every row, so $p \rightarrow (q \wedge r) \equiv (p \rightarrow q) \wedge (p \rightarrow r)$. □

Exercise 8. Let $p | q$ denote NAND ($\neg(p \wedge q)$).

- (a) *NOT from NAND:* $\neg p \equiv p | p$. Check: $p | p = \neg(p \wedge p) = \neg p$. ✓
- (b) *AND from NAND:* $p \wedge q \equiv (p | q) | (p | q)$. That is, take NAND and then apply our NOT-from-NAND trick to undo the negation:

$$(p | q) | (p | q) = \neg(p | q) = \neg(\neg(p \wedge q)) = p \wedge q. \checkmark$$

- (c) *OR from NAND:* By De Morgan, $p \vee q \equiv \neg(\neg p \wedge \neg q)$. Rewrite each $\neg$ as NAND-with-itself and the inner $\wedge$ as NAND applied twice (or, more efficiently, observe that $\neg(\neg p \wedge \neg q) = (\neg p) | (\neg q)$). So

$$p \vee q \equiv (p | p) | (q | q).$$

Check with $p = T, q = F$: $(T | T) | (F | F) = F | T = \neg(F \wedge T) = \neg F = T$. ✓

Since NAND can build $\neg$, $\wedge$, and $\vee$, and those three together are functionally complete, so is NAND alone.

Exercise 9. The three T rows are (T, F, F) , (F, T, F) , (F, F, T) . The DNF recipe gives one conjunct per T row, using the variable if the row has T and its negation if the row has F:

$$f(x, y, z) \equiv (x \wedge \neg y \wedge \neg z) \vee (\neg x \wedge y \wedge \neg z) \vee (\neg x \wedge \neg y \wedge z).$$

You can verify this by plugging in (T, F, F) : $T \wedge T \wedge T = T$ in the first disjunct, and F in the other two, so the disjunction is T . Similarly for the other two hot rows. On all five cold rows, every disjunct has at least one literal evaluating to F , so the overall disjunction is F .

Challenge

Exercise 10. We show $\{\wedge, \vee\}$ is not functionally complete by exhibiting a function that cannot be expressed.

(a) *Monotonicity is preserved by $\wedge$ and $\vee$.* Call a function $f : \{T, F\}^n \rightarrow \{T, F\}$ *monotone* if whenever $\vec{x} \leq \vec{y}$ coordinate-wise (with $F \leq T$), we have $f(\vec{x}) \leq f(\vec{y})$ —flipping an input from F to T cannot flip the output from T to F . By induction on the structure of formulas using only $\wedge$ and $\vee$:

- *Base cases:* A variable x_i is monotone (its output agrees with its input, which only goes up when the input goes up). The constants T and F are monotone (output never changes at all).
- *Inductive step:* Suppose f and g are both monotone (induction hypothesis). We show $f \wedge g$ and $f \vee g$ are monotone. Take $\vec{x} \leq \vec{y}$. By IH, $f(\vec{x}) \leq f(\vec{y})$ and $g(\vec{x}) \leq g(\vec{y})$. Then $f(\vec{x}) \wedge g(\vec{x}) \leq f(\vec{y}) \wedge g(\vec{y})$ because $\wedge$ is monotone in each argument (you can check this on the truth table: $T \wedge T = T$, and weakening either input to F weakens the output to F , never strengthens). The same argument works for $\vee$.

So every formula built from $\wedge$, $\vee$, and variables represents a monotone boolean function.

(b) *A non-monotone function.* The unary function $\neg$ is not monotone: flipping the input from F to T flips the output from T to F , which is a strict decrease.

(c) *Conclusion.* If $\{\wedge, \vee\}$ were functionally complete, we could write a formula for $\neg$ using only $\wedge$ and $\vee$, hence (by (a)) $\neg$ would be monotone. But (b) says it's not. Contradiction, so $\{\wedge, \vee\}$ is not functionally complete.

Exercise 11. ($\Rightarrow$). Suppose φ is a tautology. Then for every truth assignment, φ evaluates to T . So $\neg\varphi$ evaluates to F on every assignment, which is exactly the definition of a contradiction.

($\Leftarrow$). Suppose $\neg\varphi$ is a contradiction. Then for every truth assignment, $\neg\varphi$ evaluates to F , so φ evaluates to T on every assignment, so φ is a tautology.

The two directions are basically the same observation run both ways, which is typical of “iff” proofs where negation is involved.

Exercise 12.

- Constants:** There are exactly 2 constant binary functions—the one that always outputs T and the one that always outputs F .
- Depending on only one input:** Such a function $f(x, y)$ ignores one of its two arguments. If it ignores y , it behaves like a unary function of x ; the non-constant unary functions are identity and negation, giving 2 options. Similarly, if it ignores x , we get the identity and negation of y . That's $2 + 2 = 4$ functions that depend on exactly one argument.
- Depending on both:** $16 - 2 - 4 = 10$ functions depend on both arguments.

Sanity check: the ten that depend on both include the familiar $\wedge$, $\vee$, $\rightarrow$, $\leftrightarrow$, $\oplus$ (XOR), NAND, NOR, and three more—the two “left-minus-right” operations $p \wedge \neg q$ and $\neg p \wedge q$, and a few others depending on how you slice them. The exact inventory is a fun bookkeeping exercise that the above counting bypasses.

Connection

Exercise 13.

```
from itertools import product

def is_tautology(f, n):
    for assignment in product([False, True], repeat=n):
        if not f(*assignment):
            return False
    return True

# Logically equivalent to p -> q; not a tautology (fails at p=T, q=F)
```

```
print(is_tautology(lambda p, q: (not p) or q, 2)) # False

# p or not p; always true
print(is_tautology(lambda p, q: p or (not p), 2)) # True
```

The key insight is that truth-table semantics gives us a decision procedure: to check if a propositional formula is a tautology, we can just evaluate it on all 2^n assignments. This works because there are only finitely many assignments. It blows up exponentially with n , of course, and one of the most famous open problems in computer science—P vs. NP—asks whether we can do fundamentally better than this brute-force check in the general case.

Exercise 14. The program would have a type signature something like

$$\text{find_or_refute} : (P : \mathbb{N} \rightarrow \text{Bool}) \rightarrow (\exists n. P(n) = T) + (\forall n. P(n) = F)$$

where $+$ is a disjoint union (“either an example or a proof of no example”). The classical law of excluded middle would tell us this is always decidable—one or the other must be true, end of story. But *writing an actual program* that produces one of the two outputs in finite time is impossible in general: you would have to search through infinitely many naturals to rule out the existential. This is the difference between classical existence (“one of the two is true”) and constructive existence (“here is which one, with a finite recipe”). The Curry–Howard intermezzo makes this precise: in constructive logic, a proof of $A \vee B$ really is a program that computes which disjunct holds. Classical logic’s excluded middle breaks that guarantee, which is what makes it powerful *and* what makes it non-computable in general. You’ll see this in action when you do the Natural Number Game.

Module 3

Warm-up

Exercise 1. Truth table for $p \wedge q \rightarrow q$:

p	q	$p \wedge q$	$p \wedge q \rightarrow q$
T	T	T	T
T	F	F	T
F	T	F	T
F	F	F	T

Every row evaluates to T, so $p \wedge q \rightarrow q$ is a tautology.

Exercise 2.

- (a) $p \rightarrow q$ and $\neg p \vee q$. Both columns are T, F, T, T across the four rows, so they **are equivalent**. (This is the equivalence the module noted when discussing functional completeness.)
- (b) $p \rightarrow q$ and $q \rightarrow p$. At $(p = T, q = F)$: $p \rightarrow q = F$ but $q \rightarrow p = T$. **Not equivalent**.
- (c) $p \wedge (p \rightarrow q)$ and $p \wedge q$. A formula at $(p = T, q = T)$: both T . At $(p = T, q = F)$: $T \wedge F = F$ on both sides. At $(p = F, q = T)$: $F \wedge (\dots) = F$ on both sides. At $(p = F, q = F)$: similarly F . **Equivalent**. This is sometimes called “absorption” of implication under conjunction.

Exercise 3.

- (a) **Modus ponens.** From p and $p \rightarrow q$, conclude q . Valid.
- (b) **Modus tollens.** From $p \rightarrow q$ and $\neg q$, conclude $\neg p$. Valid.
- (c) **Neither.** This is the *converse error*. From $p \rightarrow q$ and q , you *cannot* conclude p —the sprinklers example from the module. (Counterexample: p false, q true.)
- (d) **Neither.** This is the *inverse error*. From $p \rightarrow q$ and $\neg p$, you cannot conclude $\neg q$. (Counterexample: p false, q true.)

Exercise 4. Start with $\neg(p \wedge q) \vee r$. Apply De Morgan to the first disjunct:

$$\neg(p \wedge q) \vee r \equiv (\neg p \vee \neg q) \vee r \equiv \neg p \vee \neg q \vee r.$$

The right side is a single disjunction of literals, which is a conjunction of one clause—so it is trivially in CNF: a single clause $(\neg p \vee \neg q \vee r)$. That's the CNF.

Exercise 5. For DNF we need a disjunction of conjunctions of literals. Starting again from $\neg(p \wedge q) \vee r$, apply De Morgan and then rewrite:

$$\neg(p \wedge q) \vee r \equiv \neg p \vee \neg q \vee r.$$

This is a disjunction of three single-literal terms, and each single literal is already a conjunction of one literal, so this is also in DNF: $(\neg p) \vee (\neg q) \vee r$.

In this particular example the CNF and DNF look syntactically identical, which can happen when the formula simplifies to just a clause of literals. (The point of the two exercises is that in general they would be different; try starting from $\neg(p \wedge q) \vee (r \wedge s)$ to see a case where they diverge.)

Standard

Exercise 6. Premises: $p \rightarrow q$, $r \rightarrow \neg q$, r . Conclusion: $\neg p$.

From r (premise 3) and $r \rightarrow \neg q$ (premise 2), by modus ponens we obtain $\neg q$. From $\neg q$ and $p \rightarrow q$ (premise 1), by modus tollens we obtain $\neg p$.

Thus the argument is **valid**. (Equivalently, one can build the full truth table and verify that in every critical row—where all three premises are true—the conclusion $\neg p$ is also true.)

Exercise 7. The rows with output 1 are $(P, Q, R) = (1, 0, 1)$ and $(P, Q, R) = (0, 1, 1)$. The DNF expression is:

$$(P \wedge \neg Q \wedge R) \vee (\neg P \wedge Q \wedge R).$$

Circuit description: Two AND gates, one for each minterm. The first AND gate takes P , $\neg Q$ (via a NOT gate on Q), and R . The second AND gate takes $\neg P$ (via a NOT gate on P), Q , and R . The outputs of both AND gates feed into a single OR gate, which produces the final output.

Exercise 8. Theorem. Assuming $p \rightarrow q$ and $p \rightarrow r$, show $p \rightarrow (q \wedge r)$.

Proof.

1. $p \rightarrow q$ (assumption)
2. $p \rightarrow r$ (assumption)
3. Subproof: assuming p , show $q \wedge r$:
 - (3a) p (assumption)

- | | |
|---------------------------------|--|
| (3b) q | (modus ponens on (3a) and (1)) |
| (3c) r | (modus ponens on (3a) and (2)) |
| (3d) $q \wedge r$ | ($\wedge$ -introduction on (3b) and (3c)) |
| 4. $p \rightarrow (q \wedge r)$ | ($\rightarrow$ -introduction on (3)) |

The subproof is what $\rightarrow$ -introduction needs: you assume p inside a contained block, derive the goal $q \wedge r$, and then discharge the assumption by wrapping the whole thing in $p \rightarrow$.

Exercise 9. Theorem. From $p \vee q$, $\neg p$, $q \rightarrow r$, show r .

Proof.

- | | |
|----------------------|--|
| 1. $p \vee q$ | (assumption) |
| 2. $\neg p$ | (assumption) |
| 3. $q \rightarrow r$ | (assumption) |
| 4. q | (disjunctive syllogism on (1) and (2)) |
| 5. r | (modus ponens on (4) and (3)) |

Challenge

Exercise 10. The “proof” only checks the rows of the truth table where p and q have the same truth value (both T or both F). In those rows, $p \rightarrow q$ and $q \rightarrow p$ do happen to agree. But the proof ignores the two rows where p and q differ.

When $p = T$ and $q = F$: $p \rightarrow q = F$ but $q \rightarrow p = T$. When $p = F$ and $q = T$: $p \rightarrow q = T$ but $q \rightarrow p = F$.

Since the two formulas differ on these rows, they are **not** logically equivalent. The error is an incomplete case analysis—the proof conveniently skipped exactly the cases that would reveal the formulas are different.

Exercise 11. (a) *Excluded middle $\Rightarrow$ double-negation elimination.* Goal: assuming $p \vee \neg p$ is available, derive $\neg\neg p \rightarrow p$.

- | | |
|---|--|
| 1. $p \vee \neg p$ | (excluded middle) |
| 2. Subproof: assuming $\neg\neg p$, show p : | |
| (2a) $\neg\neg p$ | (assumption) |
| (2b) Subproof (case p of (1)): assuming p , p follows by proof-by-assumption. | |
| (2c) Subproof (case $\neg p$ of (1)): assuming $\neg p$, | |
| (i) $\neg p$ | (assumption) |
| (ii) $\perp$ | ($\neg$ -elimination on (2a) and (i)) |
| gives p by $\perp$ -elimination. | |
| Shows p , by $\vee$ -elimination on (1), (2b), (2c). | |
| 3. $\neg\neg p \rightarrow p$ | ($\rightarrow$ -introduction on (2)) |

(b) *Double-negation elimination $\Rightarrow$ excluded middle.* Goal: assuming $\neg\neg q \rightarrow q$ is available for every q , derive $p \vee \neg p$ for arbitrary p . The standard move is to prove $\neg\neg(p \vee \neg p)$ constructively, then apply double-negation elimination with $q := (p \vee \neg p)$.

1. Subproof: assuming $\neg(p \vee \neg p)$, derive $\perp$:

(1a) $\neg(p \vee \neg p)$ (assumption)

(1b) Subproof: assuming p , derive $\perp$:

(i) p (assumption)

(ii) $p \vee \neg p$ ($\vee$ -introduction, left)

gives $\perp$ by $\neg$ -elimination on (1a) and (ii).

(1c) $\neg p$ ($\neg$ -introduction on (1b))

(1d) $p \vee \neg p$ ($\vee$ -introduction, right, on (1c))

Shows $\perp$, by $\neg$ -elimination on (1a) and (1d).

2. $\neg\neg(p \vee \neg p)$ ($\neg$ -introduction on (1))

3. $\neg\neg(p \vee \neg p) \rightarrow (p \vee \neg p)$ (assumed DNE rule)

4. $p \vee \neg p$ (modus ponens on (2) and (3))

So the two rules are interderivable. Adding either one to constructive logic gives you the same classical logic.

Exercise 12. By the DNF recipe from the module, you get one disjunct for every row of the truth table where the output is T . There are 2^n rows total. So the number of disjuncts is exactly the number of T -rows, which is between 0 and 2^n .

- *Zero disjuncts* happens when the formula is a contradiction—every row outputs F . You get the empty disjunction, which by convention is F .
- *Exactly 2^n disjuncts* happens when the formula is a tautology—every row outputs T . You get the full disjunction of all minterms, which simplifies to T .

In between, you get contingent formulas with anywhere from 1 to $2^n - 1$ disjuncts.

Connection

Exercise 13.

```
from itertools import product

def dnf(f, n):
    var_names = [f"x{i}" for i in range(n)]
    terms = []
    for assignment in product([False, True], repeat=n):
        if f(*assignment):
            literals = []
            for name, val in zip(var_names, assignment):
                literals.append(name if val else f"not_{name}")
            terms.append("(" + "_and_".join(literals) + ")")
    if not terms:
        return ""
```

```

    return "False"
    return "⊔or⊔".join(terms)

# XOR in two variables
print(dnf(lambda x, y: x != y, 2))
# -> (not x0 and x1) or (x0 and not x1)

# exactly-one on three variables (matches Module 2 Exercise 9)
def exactly_one(x, y, z):
    return (x and not y and not z) or (not x and y and not z) or (not x and not y and z)
print(dnf(exactly_one, 3))
# -> three three-literal conjunctions, one per T row

```

The output is a complete DNF formula derived mechanically from the truth table—exactly the recipe the module describes.

Exercise 14. A proof tree for the theorem from Exercise 8 (assuming $p \rightarrow q$ and $p \rightarrow r$, show $p \rightarrow (q \wedge r)$) looks like this:

$$\frac{\frac{[p]^{(1)} \quad p \rightarrow q}{q} \rightarrow E \quad \frac{[p]^{(1)} \quad p \rightarrow r}{r} \rightarrow E}{\frac{q \wedge r}{p \rightarrow (q \wedge r)} \wedge I} \rightarrow I, \text{ discharging } (1)$$

The key correspondence is: the numbered subproof in Exercise 8 becomes the *discharged assumption* $[p]$ at the leaves of the tree, and the outer proof becomes the tree rooted at $p \rightarrow (q \wedge r)$. Every application of $\rightarrow$ -introduction corresponds to discharging an assumption in the tree, which in the linear-proof notation corresponded to closing a subproof. The Gentzen intermezzo makes this correspondence precise and shows why proof trees are often clearer than linear proofs once the proofs get long.

Intermezzo: Inference Rules as Trees

Exercise 1. Goal: $A \wedge (B \wedge C) \rightarrow (A \wedge B) \wedge C$. The only undischarged assumption is $[A \wedge (B \wedge C)]$, which gets discharged by the final $\rightarrow I$. Read the tree bottom-up: to get $(A \wedge B) \wedge C$ we need $A \wedge B$ and C ; to get $A \wedge B$ we need A and B ; each leaf $[A \wedge (B \wedge C)]$ is projected appropriately.

$$\frac{\frac{\frac{[A \wedge (B \wedge C)]}{A} \wedge E_1 \quad \frac{\frac{[A \wedge (B \wedge C)]}{B \wedge C} \wedge E_2 \quad B}{B} \wedge E_1}{A \wedge B} \wedge I \quad \frac{\frac{[A \wedge (B \wedge C)]}{B \wedge C} \wedge E_2 \quad C}{C} \wedge E_2}{(A \wedge B) \wedge C} \wedge I \rightarrow I$$

All three occurrences of $[A \wedge (B \wedge C)]$ at the leaves refer to the *same* discharged assumption—the bracketed formula becomes available everywhere in the subtree above the final $\rightarrow I$ and is closed off by that rule.

Exercise 2. Goal: $(A \rightarrow B) \rightarrow ((B \rightarrow C) \rightarrow (A \rightarrow C))$. There are three $\rightarrow I$ steps stacked at the bottom of the tree, discharging the assumptions $[A]$, $[B \rightarrow C]$, and $[A \rightarrow B]$ in that order (innermost first).

$$\begin{array}{c}
\frac{[A] \quad [A \rightarrow B]}{B} \rightarrow E \quad [B \rightarrow C] \rightarrow E \\
\frac{\frac{C}{A \rightarrow C} \rightarrow I}{(B \rightarrow C) \rightarrow (A \rightarrow C)} \rightarrow I \\
\frac{(A \rightarrow B) \rightarrow ((B \rightarrow C) \rightarrow (A \rightarrow C))}{(A \rightarrow B) \rightarrow ((B \rightarrow C) \rightarrow (A \rightarrow C))} \rightarrow I
\end{array}$$

The innermost $\rightarrow I$ discharges $[A]$, leaving $[A \rightarrow B]$ and $[B \rightarrow C]$ still open. The next discharges $[B \rightarrow C]$, and the outermost discharges $[A \rightarrow B]$. Each assumption gets closed by exactly one $\rightarrow I$, and the order matches the nesting of the implications in the goal.

Exercise 3. Goal: $(p \rightarrow q) \rightarrow (\neg q \rightarrow \neg p)$. Treat $\neg p$ as $p \rightarrow \perp$ and $\neg q$ as $q \rightarrow \perp$. The tree then uses three $\rightarrow I$ steps in total: one to build $\neg p$ (a.k.a. $p \rightarrow \perp$), one to build $\neg q \rightarrow \neg p$, and one to build the outer implication.

$$\begin{array}{c}
\frac{[p] \quad [p \rightarrow q]}{q} \rightarrow E \quad [\neg q] \rightarrow E \\
\frac{\frac{\perp}{\neg p} \rightarrow I \text{ (discharging } [p])}{\neg q \rightarrow \neg p} \rightarrow I \text{ (discharging } [\neg q]) \\
\frac{\neg q \rightarrow \neg p}{(p \rightarrow q) \rightarrow (\neg q \rightarrow \neg p)} \rightarrow I \text{ (discharging } [p \rightarrow q])
\end{array}$$

The three discharged assumptions are $[p]$, $[\neg q]$, and $[p \rightarrow q]$, closed in that order. The middle step applies what is formally $\rightarrow E$ between q and $\neg q$ (using the reading $\neg q = q \rightarrow \perp$); the rule it realises is “negation elimination,” i.e., from q and $\neg q$ derive $\perp$.

Exercise 4. Goal: $p \vee p \rightarrow p$. We assume $[p \vee p]$ at the top. For $\vee E$ we need an arrow $p \rightarrow p$ for each branch—and both branches are p , so both arrows are the same trivial $\rightarrow I$ on a one-line subderivation.

$$\begin{array}{c}
\frac{[p \vee p] \quad \frac{[p]^1}{p \rightarrow p} \rightarrow I^1 \quad \frac{[p]^2}{p \rightarrow p} \rightarrow I^2}{p} \vee E \\
\frac{p}{p \vee p \rightarrow p} \rightarrow I
\end{array}$$

The superscripts 1, 2 distinguish the two independent $[p]$ assumptions that each trivial branch discharges; the final $\rightarrow I$ discharges the outer $[p \vee p]$. The whole proof amounts to saying “from p or p , either way you get p .” As a program under Curry–Howard, this is exactly `either(id, id)`—pattern match on the tagged union and apply the identity in both cases.

Exercise 5. Take Exercise 8 from Module 3: “assuming $p \rightarrow q$ and $p \rightarrow r$, show $p \rightarrow (q \wedge r)$.” In `lstlisting` format, the proof was a subproof assuming p , deriving q by modus ponens with $p \rightarrow q$, deriving r by modus ponens with $p \rightarrow r$, combining them by $\wedge I$, and then $\rightarrow I$ to discharge. The Gentzen tree is:

$$\begin{array}{c}
\frac{[p] \quad p \rightarrow q}{q} \rightarrow E \quad \frac{[p] \quad p \rightarrow r}{r} \rightarrow E \\
\frac{q \wedge r}{p \rightarrow (q \wedge r)} \wedge I \rightarrow I
\end{array}$$

The two occurrences of $[p]$ are the *same* discharged assumption—the tree is a directed acyclic graph, not strictly a tree, in that one leaf can be referenced more than once. The assumptions $p \rightarrow q$ and $p \rightarrow r$ sit as undischarged leaves (they are premises, not local assumptions).

Translating back to `lstlisting`, the tree becomes:

(a) $p \rightarrow q$, by assumption
 (b) $p \rightarrow r$, by assumption
 (c) subproof: assuming p , show $q \wedge r$
 (i) p , by assumption
 (ii) q , by modus ponens on (i) and (a)
 (iii) r , by modus ponens on (i) and (b)
 shows $q \wedge r$, by $\wedge$ -introduction on (ii) and (iii)
 shows $p \rightarrow (q \wedge r)$, by $\rightarrow$ -introduction on (c)

The two formats are interchangeable, but the tree makes the dependency structure between steps more visible: you can literally see that both q and r trace back to the same leaf $[p]$, whereas the listing names a fresh label p inside the subproof and trusts the reader to track the dependency by name.

Module 4

Warm-up

Exercise 1. In the formula $\exists x. \forall y. P(x, y, z)$:

- x is **bound** by $\exists$.
- y is **bound** by $\forall$.
- z is **free** (it does not appear in the scope of any quantifier binding z).

In the formula $\forall x. (P(x) \rightarrow \exists y. Q(x, y))$:

- x is **bound** by the outer $\forall x$.
- y is **bound** by the inner $\exists y$ (its scope is just $Q(x, y)$).
- There are no other variables, so no free variables.

Exercise 2.

- (a) $\forall c : \text{Cats}. (\text{Black}(c) \vee \text{White}(c)).$
- (b) $\exists s : \text{Students}. \forall p : \text{Problems}. \text{Solved}(s, p).$ (Note the $\forall$ is *inside* the $\exists$ —there must be a *single* student who solved *all* problems.)
- (c) $\neg \exists r : \mathbb{R}. \forall n : \mathbb{Z}. r > n$, or equivalently $\forall r : \mathbb{R}. \exists n : \mathbb{Z}. r \leq n.$
- (d) $\forall r : \mathbb{R}. (r \neq 0 \rightarrow \exists s : \mathbb{R}. r \cdot s = 1).$

Exercise 3.

- (a) Positive divisors of 12 in $\mathbb{Z}^+$: $\{1, 2, 3, 4, 6, 12\}.$
- (b) Integers with $x^2 \leq 9$: we need $-3 \leq x \leq 3$, so $\{-3, -2, -1, 0, 1, 2, 3\}.$
- (c) Integers with $x < x + 1$: every integer satisfies this, so the truth set is all of $\mathbb{Z}$ (equivalently, the predicate is always true, so it's a tautology on $\mathbb{Z}$).

Exercise 4.

- (a) Divisible by 4 / divisible by 2. If n is divisible by 4 then $n = 4k = 2(2k)$, so n is divisible by 2; that makes divisibility by 4 **sufficient** for divisibility by 2. The reverse fails (2 is divisible by 2 but not 4), so divisibility by 4 is not necessary. **Sufficient but not necessary.**
- (b) Valid ticket / boarding. You need a valid ticket to board (necessary), but having a ticket alone doesn't guarantee boarding—you also need to clear security, be at the gate on time, etc. **Necessary but not sufficient.**
- (c) Equilateral triangle / triangle. Every equilateral triangle is a triangle, so equilateral implies triangle: sufficient. The reverse fails (scalene triangles are not equilateral), so not necessary. **Sufficient but not necessary.**
- (d) Prime / odd. Not every prime is odd (2 is prime and even), so prime is not sufficient for odd. And not every odd number is prime (9 is odd and composite), so prime is not necessary for odd. **Neither.**

Standard

Exercise 5. Original statement in symbols:

$$\forall s : \text{CS250}. \exists h : \text{HW}. \text{OnTime}(s, h).$$

Negation (pushing $\neg$ inward, flipping quantifiers):

$$\exists s : \text{CS250}. \forall h : \text{HW}. \neg \text{OnTime}(s, h).$$

In English: “There is a student in CS 250 who did not complete any homework on time.”

Exercise 6. Let $P(x, y)$ mean “ $x + y = 0$ ” over $\mathbb{Z}$.

- (a) $\forall x. \exists y. P(x, y)$. **True.** For any $x \in \mathbb{Z}$, pick $y = -x$. Then $x + (-x) = 0$.
- (b) $\exists y. \forall x. P(x, y)$. **False.** This claims there is a single y such that $x + y = 0$ for *all* x . That would require $y = -x$ for every x simultaneously, which is impossible.

Exercise 7. Starting from $\forall x. \exists y. \forall z. (P(x, y, z) \rightarrow Q(x, z))$, push the negation step by step:

$$\begin{aligned} \neg(\forall x. \exists y. \forall z. (P \rightarrow Q)) &\equiv \exists x. \neg(\exists y. \forall z. (P \rightarrow Q)) \\ &\equiv \exists x. \forall y. \neg(\forall z. (P \rightarrow Q)) \\ &\equiv \exists x. \forall y. \exists z. \neg(P \rightarrow Q) \\ &\equiv \exists x. \forall y. \exists z. (P \wedge \neg Q), \end{aligned}$$

using $\neg(P \rightarrow Q) \equiv P \wedge \neg Q$ at the last step. So:

$$\exists x : \mathbb{Z}. \forall y : \mathbb{Z}. \exists z : \mathbb{Z}. (P(x, y, z) \wedge \neg Q(x, z)).$$

Exercise 8. For $\wedge$: the equivalence $\forall x. (P(x) \wedge Q(x)) \equiv (\forall x. P(x)) \wedge (\forall x. Q(x))$ holds. Both directions:

$(\Rightarrow)$. Assume $\forall x. (P(x) \wedge Q(x))$. Take arbitrary a in the domain. By $\forall$ -elimination, $P(a) \wedge Q(a)$. By $\wedge$ -elimination, $P(a)$. Since a was arbitrary, $\forall x. P(x)$. By the same argument, $\forall x. Q(x)$. Hence $(\forall x. P(x)) \wedge (\forall x. Q(x))$.

$(\Leftarrow)$. Assume $(\forall x. P(x)) \wedge (\forall x. Q(x))$. Take arbitrary a . By $\wedge$ -elim, $\forall x. P(x)$, so $P(a)$ by $\forall$ -elim. Similarly, $Q(a)$. By $\wedge$ -intro, $P(a) \wedge Q(a)$. Since a was arbitrary, $\forall x. (P(x) \wedge Q(x))$.

For $\vee$: the analogous claim is **not** an equivalence. One direction holds: $(\forall x. P(x)) \vee (\forall x. Q(x)) \rightarrow \forall x. (P(x) \vee Q(x))$. Proof: if the left side holds, then (say) $\forall x. P(x)$, so for any a we have $P(a)$, so $P(a) \vee Q(a)$, so $\forall x. (P(x) \vee Q(x))$; similarly for the other disjunct.

But the other direction fails. Counterexample: let the domain be $\mathbb{Z}$, let $P(x)$ be “ x is even,” and let $Q(x)$ be “ x is odd.” Then $\forall x. (P(x) \vee Q(x))$ is true (every integer is even or odd), but neither $\forall x. P(x)$ nor $\forall x. Q(x)$ is true, so $(\forall x. P(x)) \vee (\forall x. Q(x))$ is false.

Challenge

Exercise 9. We want to show $n + 1 = \text{succ } n$, where $1 = \text{succ } 0$. No induction needed—just unfold:

$$\begin{aligned} n + 1 &= n + \text{succ } 0 \\ &= \text{succ } (n + 0) && \text{by the second equation of } + \\ &= \text{succ } n && \text{by the first equation of } + \end{aligned}$$

This is pure definitional unfolding. The trick the hint was pointing at: when the second argument is $\text{succ } 0$, the second equation of $+$ fires, reducing to $\text{succ } (n + 0)$, and then the *first* equation of $+$ (which handles $+0$) finishes the job. No induction required because the argument we’re unfolding has already been built out of the two definition cases.

Exercise 10. The BNF $\text{Nat} := 0 \mid \text{succ Nat}$ has two cases, and the induction rule correspondingly has two premises:

- The premise $P(0)$ comes from the *base* case of the BNF.
- The premise “assuming $P(n)$, show $P(\text{succ } n)$ ” comes from the *recursive* case—with an inductive hypothesis for each recursive Nat occurrence (there’s exactly one here).

By analogy, $\text{Counter} := \text{zero} \mid \text{tickA Counter} \mid \text{tickB Counter}$ has three cases—one base and two recursive—so the induction rule has three premises:

```

assumes P(zero)
  subproof:
    assumes P(c)
    -----
    shows P(tickA c)
  subproof:
    assumes P(c)
    -----
    shows P(tickB c)
-----
shows  $\forall c : \text{Counter}. P(c)$ 

```

One premise per constructor; an IH for each recursive Counter occurrence.

Exercise 11. We prove $\forall n : \text{Nat}. 0 \cdot n = 0$ by induction on n .

Base case ($n = 0$). $0 \cdot 0 = 0$ by the first equation of $\cdot$. ✓

Inductive step ($n = \text{succ } n'$). IH: $0 \cdot n' = 0$. Goal: $0 \cdot (\text{succ } n') = 0$.

$$\begin{array}{ll}
 0 \cdot (\text{succ } n') = (0 \cdot n') + 0 & \text{by the second equation of } \cdot \\
 = 0 + 0 & \text{by the IH} \\
 = 0 & \text{by the first equation of } +
 \end{array}$$

By the induction principle, $\forall n : \text{Nat}. 0 \cdot n = 0$. □

Notice how the proof needs the IH at line 2—without it we’d be stuck with an unsimplified $(0 \cdot n') + 0$ and no way to continue. This is exactly the kind of case Common Mistakes #5 warns against: if you find yourself writing an inductive step that never touches the IH, something’s wrong.

Exercise 12. Take $A = \mathbb{N}$ and let $R(x, y)$ be “ $y > x$.”

Then $\forall x. \exists y. R(x, y)$ is true: for any natural x , choose $y = x + 1$, and $x + 1 > x$.

But $\exists y. \forall x. R(x, y)$ is false: this would demand a single natural y that is greater than *every* natural x , including y itself. No such y exists.

The intuition: in the $\forall\exists$ form, the witness y is allowed to *depend on* x . In the $\exists\forall$ form, the witness y is fixed first and has to work against every subsequent x . Being allowed to see x before choosing y is strictly more powerful—you can adapt your answer to the question.

Connection

Exercise 13.

```
def forall_exists(P, A, B):
    return all(any(P(a, b) for b in B) for a in A)

def exists_forall(P, A, B):
    return any(all(P(a, b) for a in A) for b in B)

A = B = {0, 1, 2, 3}
P = lambda a, b: a + b == 3

print(forall_exists(P, A, B)) # True
print(exists_forall(P, A, B)) # False
```

Why it splits:

$\forall\exists$ *is true*. Given any $a \in \{0, 1, 2, 3\}$, pick $b = 3 - a$, which is also in $\{0, 1, 2, 3\}$. Then $a + b = 3$. ✓ So for every a there’s a witness b —but the witness depends on a (it’s different for $a = 0$ versus $a = 3$).

$\exists\forall$ *is false*. A single fixed b would have to satisfy $a + b = 3$ for *every* $a \in \{0, 1, 2, 3\}$. That would force $b = 3 - a$ simultaneously for $a = 0, 1, 2, 3$, i.e., $b = 3, 2, 1, 0$ all at once. No single value of b works. ✓

The whole point: in $\forall\exists$, the witness can depend on the input; in $\exists\forall$, it has to be uniform.

Exercise 14. The game for $\forall x. \exists y. y > x$ over $\mathbb{R}$: the universal player (“Challenger”) picks an x , then the existential player (“Prover”) picks a y and wins if $y > x$. Prover’s winning strategy is trivial: whatever x Challenger picks, Prover responds with $y = x + 1$. Since Prover gets to see x before choosing y , and since there’s always something bigger than any given real, Prover always wins.

The game for $\exists y. \forall x. y > x$: here the order is reversed. Prover picks y *first*, then Challenger sees y and picks any x they like. Prover wins if $y > x$. But Challenger can always pick $x = y + 1$ (or

y itself, or anything at least as large), which falsifies $y > x$. Prover has no winning move, because no real number is bigger than every other. Prover always loses.

The difference: in the first game, Prover benefits from *seeing the opponent's move* before committing. In the second, Prover must commit *first* to a uniform answer that will work against every possible challenge. The same predicate, the same domain—only the order of commitment changes—and that's enough to flip the outcome.

Intermezzo: More on Natural Induction

Exercise 1. Theorem: $\forall m. 0 \cdot m = 0$.

Proof by induction on m .

Case $m = 0$: Need to prove $0 \cdot 0 = 0$, which is immediate from the first equation of multiplication.

Case $m = \text{succ } m'$:

Assuming $0 \cdot m' = 0$.

Shows $0 \cdot (\text{succ } m') = 0$.

$$\begin{aligned} 0 \cdot (\text{succ } m') &= 0 + (0 \cdot m') && \text{by the definition of multiplication} \\ &= 0 + 0 && \text{by the induction hypothesis} \\ &= 0 && \text{by the definition of addition} \end{aligned}$$

QED

Exercise 2. Theorem: $\forall m. 1 \cdot m = m$, where $1 = \text{succ } 0$.

Proof by induction on m .

Case $m = 0$: Need to prove $1 \cdot 0 = 0$, which is immediate from the first equation of multiplication.

Case $m = \text{succ } m'$:

Assuming $1 \cdot m' = m'$.

Shows $1 \cdot (\text{succ } m') = \text{succ } m'$.

$$\begin{aligned} 1 \cdot (\text{succ } m') &= 1 + (1 \cdot m') && \text{by the definition of multiplication} \\ &= 1 + m' && \text{by the induction hypothesis} \\ &= (\text{succ } 0) + m' \\ &= \text{succ } (0 + m') && \text{by the succ-shift lemma in Section 2} \\ &= \text{succ } m' && \text{by the left-identity lemma in Section 1} \end{aligned}$$

QED

Notice that the final two steps together depend on the prior theorems in the chapter: the succ-shift lemma (Section 2) lets us move succ past addition, and the left-identity lemma (Section 1) collapses $0 + m'$ to m' .

Exercise 3. Theorem: $\forall n. \exists k. n + n = k + k$.

The witness for the existential is, naturally, n itself: taking $k = n$ makes the required equation $n + n = n + n$, which is true by reflexivity. So there is in fact a one-line proof:

$$\exists k. n + n = k + k \quad \text{by the witness } k = n \text{ and reflexivity.}$$

But the problem asks for an inductive proof, and writing it out usefully exercises the addition lemmas.

Proof by induction on n .

Case $n = 0$: Need to produce k with $0 + 0 = k + k$. Take $k = 0$; then $0 + 0 = 0 + 0$ by reflexivity.

Case $n = \text{succ } n'$:

Assuming $\exists k. n' + n' = k + k$ —say $n' + n' = k_0 + k_0$ for some k_0 .

Shows $\exists k'. (\text{succ } n') + (\text{succ } n') = k' + k'$.

We claim $k' = \text{succ } n'$ works. Compute:

$$\begin{aligned} (\text{succ } n') + (\text{succ } n') &= \text{succ } ((\text{succ } n') + n') && \text{by the definition of addition} \\ &= \text{succ } (\text{succ } (n' + n')) && \text{by the succ-shift lemma in Section 2} \end{aligned}$$

On the other hand

$$(\text{succ } n') + (\text{succ } n') = (\text{succ } n') + (\text{succ } n')$$

by reflexivity, so taking $k' = \text{succ } n'$ gives $(\text{succ } n') + (\text{succ } n') = k' + k'$.

QED

The trivial witness argument (“take $k = n$ ”) is enough here, but the inductive elaboration exposes the succ-shift lemma as the workhorse that keeps the algebra honest as n grows by one.

Exercise 4. Theorem: $\forall n. \text{pred } (\text{succ } n) = n$.

Proof. Immediate from the second equation of pred : $\text{pred } (\text{succ } n) = n$ by definition. No induction is required, because the left-hand side already has the shape $\text{pred } (\text{succ } \cdot)$, which the definition handles directly. (*If one insists on writing it as an induction on n , both the base and the step case close by the same definitional unfolding.*)

QED

Why $\forall n. \text{succ } (\text{pred } n) = n$ is not provable. The statement fails at $n = 0$: by the first equation of pred , $\text{pred } 0 = 0$, so $\text{succ } (\text{pred } 0) = \text{succ } 0 = 1 \neq 0$. So even the base case of any induction blows up—zero’s predecessor is defined to be zero, and adding succ back cannot recover the original.

Exercise 5. Proof of $0 + m = m$ (Section 1). The base case ($0 + 0 = 0$) is pure unfolding of the first equation of addition. The inductive step does one unfolding ($0 + \text{succ } m' = \text{succ } (0 + m')$ by the second equation of addition) and *uses the induction hypothesis* to replace $0 + m'$ with m' . The induction hypothesis is the only step in the whole proof that “does work”—everything else is definitional.

Proof of $\text{succ } n + m = \text{succ } (n + m)$ (Section 2). Same shape: the base case is pure unfolding, and the inductive step does two unfoldings (one on each side) and applies the induction hypothesis to align them. Again, the induction hypothesis is the only step that cannot be replaced by an unfolding.

Proof of associativity (Section 3). Here the base case already uses a prior theorem (left-identity of 0); the inductive step uses both the succ-shift lemma from Section 2 and the induction hypothesis. Two lemmas and one IH do all the work.

Proof of commutativity (Section 4). The base case invokes left- and right-identity of 0. The inductive step uses the succ-shift lemma and the induction hypothesis. As in the associativity proof, the lemmas proved earlier in the chapter are doing the heavy lifting.

Why definitional unfolding alone cannot finish any of these theorems. Each theorem makes a claim about $0 + m$, $\text{succ } n + m$, or $n + m$ where the *first* argument is either zero, a successor, or a variable. Addition is defined by recursion on the *second* argument. So definitional unfolding can only peel succ off the second argument; the first argument stays opaque. To prove a statement

about the first argument, we must use induction on something—either the second argument (which exposes the recursive structure of $+$) or the first argument (which requires a lemma like succ-shift to translate between the two). There is no way around this because the theorems describe behaviour that the definition hides.

Intermezzo: Quantifiers as Games

Exercise 1. The game has three rounds, corresponding to the three quantifiers $\forall \epsilon, \exists \delta, \forall x$:

1. **Round 1** (Challenger's move, $\forall \epsilon$): Challenger picks any $\epsilon > 0$. Challenger's goal is to make the task as hard as possible, so they will try to pick a very small ϵ .
2. **Round 2** (Prover's move, $\exists \delta$): Prover sees the chosen ϵ and picks a $\delta > 0$ in response. Prover's choice of δ can depend on ϵ .
3. **Round 3** (Challenger's move, $\forall x$): Challenger picks any x satisfying $|x - 3| < \delta$, trying to find an x that breaks Prover's claim.

Prover wins if, for the chosen x , $|x^2 - 9| < \epsilon$. The fact that Prover has a winning strategy—pick δ small enough based on ϵ —is precisely what it means for $f(x) = x^2$ to be continuous at $x = 3$.

Exercise 2. Prover moves first (this is the $\exists x$) and picks $x = 0$. Then Challenger picks any y they like (this is the $\forall y$). Prover wins if $x + y = y$, which becomes $0 + y = y$ —true for every y . So no matter what Challenger picks, Prover wins.

The winning move is $x = 0$ because 0 is the additive identity.

Exercise 3. The game for $\forall x \in \mathbb{N}. \exists y \in \mathbb{N}. y > x$ has Challenger go first (picking x), then Prover (picking y). Prover wins if $y > x$.

Prover's winning strategy is the function $f(x) = x + 1$. Whatever x Challenger throws, Prover responds with $x + 1$, and $x + 1 > x$ holds for every natural number. Since Prover moves second, the response is allowed to depend on x —which is exactly what makes f a strategy rather than a constant.

Exercise 4. The game for $\exists y \in \mathbb{N}. \forall x \in \mathbb{N}. y > x$ reverses the order: Prover commits to y first, then Challenger picks x .

Prover cannot win. Whatever y Prover commits to, Challenger responds with $x = y$ (or $y + 1$, or any larger value), making $y > x$ false. Because Prover has to move first, there is no way to use information about Challenger's upcoming move—so no single fixed y can be larger than *every* natural number.

In the game-theoretic terms of the chapter: Prover needs a *constant* y that beats every possible x , not a *strategy*. A constant does not get to depend on Challenger's choice. No fixed natural number is larger than every natural number (take the number itself as a counter-choice), so no constant works. This is the same argument that made $\exists y. \forall x. y > x$ false over $\mathbb{R}$ —only now it's over $\mathbb{N}$, and the verdict is unchanged.

Exercise 5. The game for $\forall n \in \mathbb{N}. \forall m \in \mathbb{N}. n + m = m + n$:

1. **Round 1** (Challenger's move, $\forall n$): Challenger picks any natural number n .
2. **Round 2** (Challenger's move, $\forall m$): Challenger picks any natural number m .
3. Prover must demonstrate that $n + m = m + n$ for the specific n, m that Challenger chose.

Both quantifiers are universal, so Prover never gets to choose anything: Challenger controls every move. Prover’s “strategy” is therefore an answer to the question “given n and m , why is $n + m = m + n$?”

The game is trivial in the sense that no strategy is needed—but only because the underlying arithmetic fact ($n + m = m + n$) is *already true* for every (n, m) . Prover doesn’t have to *do* anything in the game; Prover just has to *be right*. And Prover is right precisely because commutativity of addition has been proved (by induction, in the chapter).

What infinitely many challenges confirm is that infinitely many instances of the theorem all hold. Had we phrased the theorem incorrectly (say $n + m = m - n$), any single challenge that evaluated the two sides to different numbers would be enough for Challenger to win. A winning strategy for Prover in a $\forall\forall$ game is exactly a proof of the inner statement that works for all inputs.

Exercise 6. In the game for $\forall x. \exists y. R(x, y)$, Challenger picks x and Prover responds with some y . Prover’s winning strategy is a rule that specifies, for each possible x , which y to play. That rule is a *function* $f : \text{dom}(x) \rightarrow \text{dom}(y)$. Packaging all of Prover’s responses into a single object gives exactly such an f .

The Skolem form $\exists f. \forall x. R(x, f(x))$ says: there exists a function f such that, for every x , $R(x, f(x))$ holds. That is word-for-word the statement “Prover has a winning strategy.” The outer existential “ $\exists f$ ” existentially quantifies over Prover’s strategy; the inner $\forall x. R(x, f(x))$ says the strategy beats every Challenger move x .

So the Skolem transformation is the proof-theoretic shadow of the game-theoretic fact: winning strategies for $\forall x. \exists y$ are functions $x \mapsto y$. Witness-producing existentials become witness-producing functions, and the “function” is exactly Prover’s strategy.

Exercise 7. *Finite universe.* The game for $\forall x. (P(x) \vee \neg P(x))$ over $\{a_1, \dots, a_n\}$:

1. Challenger picks a_i from the finite universe.
2. Prover plays the $\vee$ game for $P(a_i) \vee \neg P(a_i)$: Prover chooses one disjunct and must win that sub-game.

If Prover can decide, for each a_i , whether $P(a_i)$ holds, then Prover’s strategy is: for each a_i Challenger might pick, pre-compute whether $P(a_i)$ is true or false, and play the corresponding disjunct. There are only finitely many a_i ’s, so a lookup table of n bits encodes the winning strategy. Prover always wins.

Infinite universe with an undecidable predicate. Now let the universe be $\mathbb{N}$ and $P(e)$ be “ e codes a Turing machine that halts on every input.” The game asks Prover to decide, for an arbitrary e given by Challenger, whether $P(e)$ or $\neg P(e)$. Because totality of Turing machines is undecidable (a standard result from computability theory), no algorithm can produce the right answer for every e . Prover has no computable strategy.

Connection to constructive versus classical logic. A constructive proof of $\forall x. (P(x) \vee \neg P(x))$ would have to come with, for each x , a decision procedure producing one of the two disjuncts. For decidable P (finite cases, or checkable inequalities), such a procedure exists and the formula is constructively valid. For undecidable P , no such procedure exists, and the formula is not constructively provable—even though in classical logic it is a tautology (excluded middle $p \vee \neg p$ applied to $P(x)$). Classical logic lets Prover win by saying “one of the two disjuncts must be true,” without committing to which; constructive logic requires Prover to actually play one of them, and the game can only be won when Prover can compute the answer.

Module 5

Warm-up

Exercise 1.

Proof. Let r_1 and r_2 be rational numbers. By definition there exist integers a_1, b_1, a_2, b_2 with $b_1 \neq 0$ and $b_2 \neq 0$ such that $r_1 = a_1/b_1$ and $r_2 = a_2/b_2$. Then

$$r_1 \cdot r_2 = \frac{a_1}{b_1} \cdot \frac{a_2}{b_2} = \frac{a_1 a_2}{b_1 b_2}.$$

Since $a_1 a_2$ and $b_1 b_2$ are integers and $b_1 b_2 \neq 0$, the product $r_1 \cdot r_2$ is rational. $\square$

Exercise 2. The claim “for every integer n , $n^2 > n$ ” is **false**.

Counterexample: $n = 0$. Then $n^2 = 0$ and $n = 0$, so $n^2 = n$, not $n^2 > n$.

(Another counterexample: $n = 1$, since $1^2 = 1$.)

Exercise 3. Let n be an arbitrary integer. The three consecutive integers starting at n are n , $n + 1$, and $n + 2$. Their sum is

$$n + (n + 1) + (n + 2) = 3n + 3 = 3(n + 1).$$

Since $n + 1$ is an integer, $3n + 3 = 3k$ where $k = n + 1$, so the sum is divisible by 3 by definition. $\square$

Exercise 4. Let a, b, c be integers with $a \neq 0$, $a \mid b$, and $a \mid c$. By definition, there exist integers j and k such that $b = aj$ and $c = ak$. Then

$$b + c = aj + ak = a(j + k).$$

Since $j + k$ is an integer, $b + c = am$ where $m = j + k$, so $a \mid (b + c)$. $\square$

Exercise 5. Counterexample: 2 is prime (its only positive divisors are 1 and 2), but 2 is even. So the claim “every prime is odd” is false.

Standard

Exercise 6. $(\mathbb{Z}, -, 0)$ is **not** a group. Associativity fails: $(a - b) - c \neq a - (b - c)$ in general.

Concrete example:

$$\begin{aligned}(1 - 2) - 3 &= -1 - 3 = -4, \\ 1 - (2 - 3) &= 1 - (-1) = 2.\end{aligned}$$

Since $-4 \neq 2$, the operation is not associative, so $(\mathbb{Z}, -, 0)$ is not a group.

(Left-identity also fails: $0 - n = -n$, not n . Any one axiom failing is enough to disqualify.)

Exercise 7. The “proof” that every integer is even begins by writing “Suppose $n = 2k$ for some integer k ”—but $n = 2k$ is exactly the conclusion that n is even. The proof assumes what it is trying to prove. This is *begging the question* (circular reasoning).

A correct proof would need to show that *every* integer n can be written as $n = 2k$ for some integer k . But this is false: odd integers exist (e.g., $n = 3$ cannot be written as $2k$ for any integer k). The flaw is not a small gap in an otherwise valid argument; the claim itself is false, and the “proof” hides this by assuming the conclusion as its starting point.

Exercise 8. We verify the four group axioms for $(\mathbb{Q} \setminus \{0\}, \cdot, 1)$.

Closure (needed first, since the definition says $$ goes $G \rightarrow G \rightarrow G$):* If a/b and c/d are nonzero rationals, their product is $(ac)/(bd)$, which is nonzero (since $a, c \neq 0$ means $ac \neq 0$). So $\mathbb{Q} \setminus \{0\}$ is closed under multiplication.

Associativity: Inherited from rational multiplication: $(r_1 \cdot r_2) \cdot r_3 = r_1 \cdot (r_2 \cdot r_3)$ for all rationals.

Left-identity: $1 \cdot r = r$ for every rational r , so in particular for every nonzero rational.

Right-identity: $r \cdot 1 = r$ similarly.

Inverses: Let $r = a/b$ be an arbitrary nonzero rational (so a, b are nonzero integers). Define $r^{-1} = b/a$, which is well-defined and nonzero because $a \neq 0$. Then

$$r \cdot r^{-1} = \frac{a}{b} \cdot \frac{b}{a} = \frac{ab}{ba} = 1,$$

and similarly $r^{-1} \cdot r = 1$. So every element has a two-sided inverse.

All four axioms hold, so $(\mathbb{Q} \setminus \{0\}, \cdot, 1)$ is a group. $\square$

Exercise 9. $(\mathbb{N}, +, 0)$ is **not** a group. The inverse axiom fails: the element $1 \in \mathbb{N}$ has no inverse. We would need some $m \in \mathbb{N}$ with $1 + m = 0$, but the natural numbers start at 0, so $1 + m \geq 1 > 0$ for every $m \in \mathbb{N}$. No such m exists.

(Associativity, closure, and identity all hold for $(\mathbb{N}, +, 0)$, but inverses are the stumbling block—which is exactly why the integers $\mathbb{Z}$ exist: you get them by “freely adjoining inverses” to $\mathbb{N}$.)

Challenge

Exercise 10. Let h, h' both be inverses of g , so $h * g = g * h = e$ and $h' * g = g * h' = e$.

Consider the expression $h * g * h'$. We evaluate it two ways using associativity:

$$\begin{array}{ll} h * g * h' = (h * g) * h' = e * h' = h' & \text{using } h * g = e \text{ and left-identity,} \\ h * g * h' = h * (g * h') = h * e = h & \text{using } g * h' = e \text{ and right-identity.} \end{array}$$

Both equal $h * g * h'$, so $h = h'$. $\square$

This is structurally the same trick as the uniqueness-of-identity proof in the module: squeeze a carefully chosen expression between two identities.

Exercise 11. There are four unary boolean functions: identity, negation, constant true, and constant false (Module 2 Exercise 3). We exhibit a formula for each using only $\neg$ and $\vee$ applied to the variable p .

- **Identity** ($p \mapsto p$): just the formula p .
- **Negation** ($p \mapsto \neg p$): the formula $\neg p$.
- **Constant true** ($p \mapsto T$): the formula $p \vee \neg p$. Truth table: $T \vee F = T$ and $F \vee T = T$. $\checkmark$
- **Constant false** ($p \mapsto F$): the formula $\neg(p \vee \neg p)$. Truth table: $\neg T = F$ on both rows. $\checkmark$

Each of the four unary functions has a formula in $\{\neg, \vee\}$, so by exhaustion all unary functions are expressible.

Exercise 12. Let $(G, *, e)$ be a group and let $g \in G$. By the inverse axiom, there is an element g^{-1} satisfying

$$g^{-1} * g = e \quad \text{and} \quad g * g^{-1} = e.$$

Applying the inverse axiom to g^{-1} (since g^{-1} is itself an element of G), there is an element $(g^{-1})^{-1}$ satisfying

$$(g^{-1})^{-1} * g^{-1} = e \quad \text{and} \quad g^{-1} * (g^{-1})^{-1} = e.$$

From the second defining equation of g 's inverse, $g * g^{-1} = e$, so g is a right-inverse for g^{-1} . From the first defining equation of $(g^{-1})^{-1}$'s inverse, $(g^{-1})^{-1} * g^{-1} = e$, so $(g^{-1})^{-1}$ is a left-inverse for g^{-1} . By the uniqueness of inverses (Exercise 10), any two inverses of the same element are equal: so

$$(g^{-1})^{-1} = g. \quad \square$$

Connection

Exercise 13.

```
def is_group(carrier, op, identity):
    # Closure
    for a in carrier:
        for b in carrier:
            if op(a, b) not in carrier:
                return False
    # Associativity
    for a in carrier:
        for b in carrier:
            for c in carrier:
                if op(op(a, b), c) != op(a, op(b, c)):
                    return False
    # Identity (both sides)
    for a in carrier:
        if op(identity, a) != a or op(a, identity) != a:
            return False
    # Inverses (for each a, some b with a*b = b*a = identity)
    for a in carrier:
        if not any(op(a, b) == identity and op(b, a) == identity
                    for b in carrier):
            return False
    return True

Z5 = list(range(5))
print(is_group(Z5, lambda x, y: (x + y) % 5, 0)) # True
print(is_group(Z5, lambda x, y: (x * y) % 5, 1)) # False: 0 has no inverse
```

Why the multiplication version fails: the element 0 has no multiplicative inverse in $\mathbb{Z}/5\mathbb{Z}$ because $0 \cdot x \equiv 0 \not\equiv 1 \pmod{5}$ for every x . If you remove 0 and try $(\mathbb{Z}/5\mathbb{Z})^* = \{1, 2, 3, 4\}$ under multiplication mod 5, you do get a group—you'll see this in Module 6 when we talk about multiplicative inverses mod a prime.

Exercise 13b.

- (a) For every $a, b \in G$, $\text{id}_G(a *_G b) = a *_G b = \text{id}_G(a) *_G \text{id}_G(b)$, so id_G is a homomorphism.
- (b) We check the defining equation of a homomorphism for $\psi \circ \varphi$. Let $a, b \in G$. Then

$$(\psi \circ \varphi)(a *_G b) = \psi(\varphi(a *_G b)) = \psi(\varphi(a) *_H \varphi(b)) = \psi(\varphi(a)) *_K \psi(\varphi(b)) = (\psi \circ \varphi)(a) *_K (\psi \circ \varphi)(b),$$

using φ 's homomorphism property at the second equality and ψ 's at the third.

- (c) If $\varphi : G \rightarrow H$ and $\psi : H \rightarrow K$ are isomorphisms (bijective homomorphisms), then $\psi \circ \varphi$ is a homomorphism by (b) and is a bijection because the composition of bijections is a bijection (Exercise 6.14). So $G \cong K$.

Exercise 14. Let $(G, *, e)$ be a group, H a subgroup, and define $a \sim b$ iff $a^{-1} * b \in H$.

Reflexive: $a^{-1} * a = e \in H$ (since subgroups contain the identity), so $a \sim a$.

Symmetric: Suppose $a \sim b$, i.e., $a^{-1} * b \in H$. Since H is closed under inverses, $(a^{-1} * b)^{-1} \in H$. In a group, $(xy)^{-1} = y^{-1}x^{-1}$ (an identity you may know or may have to derive), so $(a^{-1} * b)^{-1} = b^{-1} * a$, which is in H . Therefore $b \sim a$.

Transitive: Suppose $a \sim b$ and $b \sim c$, so $a^{-1} * b \in H$ and $b^{-1} * c \in H$. Since H is closed under the group operation, $(a^{-1} * b) * (b^{-1} * c) \in H$. By associativity and the inverse axiom, $(a^{-1} * b) * (b^{-1} * c) = a^{-1} * (b * b^{-1}) * c = a^{-1} * e * c = a^{-1} * c$, which is in H . Therefore $a \sim c$.

All three properties hold, so $\sim$ is an equivalence relation. This is the definition of the *left cosets* of H in G , and the equivalence classes partition G into pieces all of the same size. The equivalence intermezzo will explain what “partition” means and why the piece-sizes come out equal.

Module 6

Warm-up

Exercise 1.

$$\begin{aligned} \lfloor -3.7 \rfloor &= -4, & \lceil -3.7 \rceil &= -3, \\ \lfloor 4 \rfloor &= 4, & \lceil 4 \rceil &= 4. \end{aligned}$$

(Note: $\lfloor -3.7 \rfloor = -4$, not -3 , because floor rounds *down*, not toward zero.)

Exercise 2. The additive inverse of 3 in $\mathbb{Z}/7\mathbb{Z}$ is $(7 - 3) \bmod 7 = 4$.

Verification: $3 + 4 = 7 \equiv 0 \pmod{7}$. ✓

Exercise 3. We need $3x \equiv 1 \pmod{5}$. Try $x = 2$:

$$3 \cdot 2 = 6 \equiv 1 \pmod{5}.$$

So the multiplicative inverse of 3 in $\mathbb{Z}/5\mathbb{Z}$ is 2.

Exercise 4.

(a) $100 = 7 \cdot 14 + 2$, so $q = 14$, $r = 2$.

(b) $-100 = 7 \cdot (-15) + 5$, so $q = -15$, $r = 5$. (Note that $7 \cdot (-15) = -105$, and $-105 + 5 = -100$. Check: $0 \leq 5 < 7$. ✓)

(c) $23 = 11 \cdot 2 + 1$, so $q = 2$, $r = 1$.

Exercise 5.

(a) $17 \bmod 6 = 5$ (since $17 = 6 \cdot 2 + 5$).

(b) $(-17) \bmod 6 = 1$ (since $-17 = 6 \cdot (-3) + 1$).

(c) $(5 + 4) \bmod 7 = 9 \bmod 7 = 2$.

(d) $(5 \cdot 4) \bmod 7 = 20 \bmod 7 = 6$.

Standard

Exercise 6. We prove $n(n+1)$ is even for every integer n by cases on parity.

Case 1: n is even. Then $n = 2k$ for some integer k , and $n(n+1) = 2k(n+1) = 2m$ where $m = k(n+1)$ is an integer. So $n(n+1)$ is even.

Case 2: n is odd. Then $n+1$ is even: $n+1 = 2j$ for some integer j . So $n(n+1) = n \cdot 2j = 2(nj)$, which is even.

In both cases, $n(n+1)$ is even. By the quotient-remainder theorem, every integer is in one of the two cases, so the proof is complete. $\square$

(This is actually the simplest case of Exercise 11, which shows $n(n+1)(n+2)$ is divisible by 6: here we're getting the "divisible by 2" factor.)

Exercise 7. We show $\lfloor x/2 \rfloor + \lceil x/2 \rceil = x$ for every integer x , by cases on parity.

Case 1: x is even. Then $x = 2k$ for some integer k . So $x/2 = k$, an integer, and both floor and ceiling of an integer equal the integer. Thus $\lfloor x/2 \rfloor + \lceil x/2 \rceil = k + k = 2k = x$. $\checkmark$

Case 2: x is odd. Then $x = 2k+1$ for some integer k . So $x/2 = k+1/2$, which is not an integer. Then $\lfloor x/2 \rfloor = k$ (the largest integer $\leq k+1/2$) and $\lceil x/2 \rceil = k+1$ (the smallest integer $\geq k+1/2$). Their sum is $k + (k+1) = 2k+1 = x$. $\checkmark$

Both cases give $\lfloor x/2 \rfloor + \lceil x/2 \rceil = x$, so the claim holds for every integer. $\square$

Exercise 8. Additive inverses in $\mathbb{Z}/8\mathbb{Z}$ (using $-a \equiv (8-a) \pmod{8}$):

a	$-a$
0	0
1	7
2	6
3	5
4	4
5	3
6	2
7	1

Checks: $0+0=0$, $1+7=8 \equiv 0$, $2+6=8 \equiv 0$, $3+5=8 \equiv 0$, $4+4=8 \equiv 0$. The remaining three rows are the same by symmetry (additive inverses are symmetric: if $a+b=0$ then $b+a=0$). Note that 4 is its own inverse—this happens for the "half" element in $\mathbb{Z}/2k\mathbb{Z}$ whenever n is even.

Exercise 9. Multiplication table for $(\mathbb{Z}/7\mathbb{Z})^\times = \{1, 2, 3, 4, 5, 6\}$ under multiplication mod 7:

$\cdot$	1	2	3	4	5	6
1	1	2	3	4	5	6
2	2	4	6	1	3	5
3	3	6	2	5	1	4
4	4	1	5	2	6	3
5	5	3	1	6	4	2
6	6	5	4	3	2	1

Inverses (find the 1 in each row):

- $1^{-1} = 1$

- $2^{-1} = 4$ (since $2 \cdot 4 = 8 \equiv 1$)
- $3^{-1} = 5$ (since $3 \cdot 5 = 15 \equiv 1$)
- $4^{-1} = 2$
- $5^{-1} = 3$
- $6^{-1} = 6$ (since $6 \cdot 6 = 36 \equiv 1$)

Note the symmetry of the table—it's symmetric about the diagonal because multiplication is commutative.

Challenge

Exercise 10. We want x with $x \equiv 1 \pmod{4}$, $x \equiv 2 \pmod{9}$, $x \equiv 3 \pmod{25}$. By CRT, a unique solution exists modulo $4 \cdot 9 \cdot 25 = 900$.

Proceed by combining constraints two at a time.

Step 1: combine the first two. Candidates for $x \equiv 1 \pmod{4}$: 1, 5, 9, 13, 17, 21, 25, 29, ... Of these, which are $\equiv 2 \pmod{9}$? Check: $1 \bmod 9 = 1$, $5 \bmod 9 = 5$, $9 \bmod 9 = 0$, $13 \bmod 9 = 4$, $17 \bmod 9 = 8$, $21 \bmod 9 = 3$, $25 \bmod 9 = 7$, $29 \bmod 9 = 2$. ✓ So $x \equiv 29 \pmod{36}$ for the first two constraints.

Step 2: combine with the third. Candidates from the first two constraints: 29, 65, 101, 137, 173, 209, 245, 281, 317, 353. Which is $\equiv 3 \pmod{25}$? $29 \bmod 25 = 4$, $65 \bmod 25 = 15$, $101 \bmod 25 = 1$, $137 \bmod 25 = 12$, $173 \bmod 25 = 23$, $209 \bmod 25 = 9$, $245 \bmod 25 = 20$, $281 \bmod 25 = 6$, $317 \bmod 25 = 17$, $353 \bmod 25 = 3$. ✓

So $x = 353$ is the smallest positive solution. (You can double-check: $353 \bmod 4 = 1$, $353 \bmod 9 = 2$ (since $9 \cdot 39 = 351$, $353 - 351 = 2$), $353 \bmod 25 = 3$ (since $25 \cdot 14 = 350$). ✓)

By CRT, every solution has the form $353 + 900k$.

Exercise 11. We show $6 \mid n(n+1)(n+2)$ for every integer n .

Part 1: divisible by 2. Among any two consecutive integers, one is even. So among $n, n+1$, at least one is even, which makes $n(n+1)(n+2)$ even.

Part 2: divisible by 3. Case on $n \bmod 3$.

- If $n \equiv 0 \pmod{3}$, then $3 \mid n$, so $3 \mid n(n+1)(n+2)$.
- If $n \equiv 1 \pmod{3}$, then $n+2 \equiv 0 \pmod{3}$, so $3 \mid n+2$, so $3 \mid n(n+1)(n+2)$.
- If $n \equiv 2 \pmod{3}$, then $n+1 \equiv 0 \pmod{3}$, so $3 \mid n+1$, so $3 \mid n(n+1)(n+2)$.

In all three cases, $3 \mid n(n+1)(n+2)$.

Since $n(n+1)(n+2)$ is divisible by both 2 and 3, and $\gcd(2, 3) = 1$, it is divisible by $2 \cdot 3 = 6$.

□

(The step “divisible by a and b with $\gcd(a, b) = 1$ implies divisible by ab ” is a mini-CRT result—you can prove it directly from Bézout's identity.)

Exercise 12. Suppose $x^2 \equiv 1 \pmod{p}$ with p prime. Then $x^2 - 1 \equiv 0 \pmod{p}$, i.e., $(x-1)(x+1) \equiv 0 \pmod{p}$. Equivalently, $p \mid (x-1)(x+1)$.

Key fact: if p is prime and $p \mid ab$, then $p \mid a$ or $p \mid b$. (This is the definition of “prime” in the ring-theoretic sense, and it follows from unique factorization.) Applied here, either $p \mid (x-1)$ or $p \mid (x+1)$.

- If $p \mid (x - 1)$, then $x \equiv 1 \pmod{p}$.
- If $p \mid (x + 1)$, then $x \equiv -1 \equiv p - 1 \pmod{p}$.

So $x \equiv 1$ or $x \equiv p - 1 \pmod{p}$, as claimed. $\square$

This fails when the modulus is not prime. In $\mathbb{Z}/8\mathbb{Z}$, for instance, $3^2 = 9 \equiv 1$ and $5^2 = 25 \equiv 1$, so you have four solutions to $x^2 \equiv 1 \pmod{8}$ rather than two. The extra solutions exist because 8 has nontrivial factorizations.

Connection

Exercise 13.

```
def mod_inverse(a, n):
    for x in range(n):
        if (a * x) % n == 1:
            return x
    return None

print(mod_inverse(3, 7)) # 5
print(mod_inverse(2, 6)) # None

# Every nonzero a in Z/7Z should have an inverse:
for a in range(1, 7):
    print(f"{a}^{-1} mod 7 = {mod_inverse(a, 7)}")
# 1->1, 2->4, 3->5, 4->2, 5->3, 6->6

# In Z/8Z, not every nonzero a has an inverse:
for a in range(1, 8):
    print(f"{a}^{-1} mod 8 = {mod_inverse(a, 8)}")
# 1->1, 2->None, 3->3, 4->None, 5->5, 6->None, 7->7
```

The elements 2, 4, 6 in $\mathbb{Z}/8\mathbb{Z}$ have no multiplicative inverse because each shares a factor of 2 with 8: $\gcd(2, 8) = 2$, $\gcd(4, 8) = 4$, $\gcd(6, 8) = 2$. Recall the pattern from the theory: a is invertible mod n iff $\gcd(a, n) = 1$. Since 8 is not prime, it has nontrivial common factors with half the nonzero residues, and those elements are non-invertible. This is precisely why $(\mathbb{Z}/n\mathbb{Z})^\times$ is interesting only when n is prime: there, every nonzero element is invertible, and you get a genuine finite field.

Exercise 14. Define $a \equiv b \pmod{n}$ iff $n \mid (a - b)$.

Reflexive: $a - a = 0$ and $n \mid 0$ (since $0 = n \cdot 0$), so $a \equiv a \pmod{n}$. $\checkmark$

Symmetric: Suppose $a \equiv b \pmod{n}$, so $n \mid (a - b)$, i.e., $a - b = nk$ for some integer k . Then $b - a = -nk = n(-k)$, and $-k$ is an integer, so $n \mid (b - a)$. Therefore $b \equiv a \pmod{n}$. $\checkmark$

Transitive: Suppose $a \equiv b \pmod{n}$ and $b \equiv c \pmod{n}$, so $n \mid (a - b)$ and $n \mid (b - c)$. That is, $a - b = nj$ and $b - c = nk$ for some integers j, k . Adding, $a - c = (a - b) + (b - c) = nj + nk = n(j + k)$, which is divisible by n . Therefore $a \equiv c \pmod{n}$. $\checkmark$

All three properties hold, so $\equiv \pmod{n}$ is an equivalence relation on $\mathbb{Z}$, with equivalence classes $\{0 + n\mathbb{Z}, 1 + n\mathbb{Z}, \dots, (n - 1) + n\mathbb{Z}\}$. These classes are exactly the elements of $\mathbb{Z}/n\mathbb{Z}$. The “quotient construction” of $\mathbb{Z}/n\mathbb{Z}$ is: take $\mathbb{Z}$, declare two integers equivalent iff they differ by a multiple of n , and let the elements of $\mathbb{Z}/n\mathbb{Z}$ be the equivalence classes.

Intermezzo: What Does It Mean for Things to Be “The Same”?

Exercise 1. Define $P_1 \sim P_2$ iff for every input x , $P_1(x) = P_2(x)$ (they produce the same output on every input).

This is **not** the same as checking whether the source code is identical. Two programs with completely different code can produce the same outputs. For example:

```
def f(x): return x + x
def g(x): return 2 * x
```

These are equivalent under the relation $\sim$ (they produce the same output for every input), but their source code is different. The equivalence class of a sorting program contains every correct sorting implementation—bubble sort, merge sort, quicksort, insertion sort, etc.—all distinct programs, but all “the same” under $\sim$.

This is *extensional equality* (Module 1): two functions are equal if they agree on all inputs, regardless of how they compute their results.

Exercise 2. Define $a/b \sim c/d$ iff $ad = bc$ (cross-multiplication).

Reflexive: $a/b \sim a/b$ iff $ab = ba$, which holds by commutativity of multiplication. ✓

Symmetric: If $a/b \sim c/d$ then $ad = bc$, so $cb = da$ (the same equation rearranged), hence $c/d \sim a/b$. ✓

Transitive: Suppose $a/b \sim c/d$ and $c/d \sim e/f$. Then $ad = bc$ and $cf = de$. Multiply the first equation by f : $adf = bcf$. Multiply the second by b : $cfb = deb$. So $adf = bcf = deb$, which gives $af \cdot d = be \cdot d$. Since $d \neq 0$, we can cancel to get $af = be$, hence $a/b \sim e/f$. ✓

The equivalence class of $1/2$ is $\{1/2, 2/4, 3/6, -1/(-2), -2/(-4), \dots\}$ —all fractions that reduce to the same value. The rational numbers $\mathbb{Q}$ are these equivalence classes: just as the integers can be constructed as equivalence classes of pairs of natural numbers (with $(a, b) \sim (c, d)$ iff $a + d = b + c$), the rationals are constructed as equivalence classes of pairs of integers.

Exercise 3. Define $A \sim B$ iff there exists a bijection $f : A \rightarrow B$.

Reflexive: The identity function $\text{id}_A : A \rightarrow A$ is a bijection (it is its own inverse), so $A \sim A$. ✓

Symmetric: Suppose $A \sim B$, witnessed by a bijection $f : A \rightarrow B$. Every bijection has an inverse, so $f^{-1} : B \rightarrow A$ exists and is itself a bijection. Thus $B \sim A$. ✓

Transitive: Suppose $A \sim B$ and $B \sim C$, witnessed by bijections $f : A \rightarrow B$ and $g : B \rightarrow C$. The composition $g \circ f : A \rightarrow C$ is a bijection (composition of bijections is a bijection, with inverse $f^{-1} \circ g^{-1}$), so $A \sim C$. ✓

The equivalence class of $\{a, b, c\}$ is the class of all *three-element sets*: $\{1, 2, 3\}$, $\{\alpha, \beta, \gamma\}$, $\{\text{Alice}, \text{Bob}, \text{Carol}\}$, and every other set that can be put into bijective correspondence with $\{a, b, c\}$.

This equivalence class is what is abstracted away: the *identities* of the elements, the names, the order. What remains is pure *cardinality*—the number 3. In fact, one standard way to define the natural number n is as the equivalence class of all sets with n elements under bijection; you have just rediscovered the Fregean definition of number.

Exercise 4. Define $w_1 \sim w_2$ iff w_2 can be obtained from w_1 by reordering the letters.

Reflexive: Every word is a reordering of itself (leave the letters alone). ✓

Symmetric: If w_2 is a reordering of w_1 , then w_1 is obtained from w_2 by the inverse permutation—also a reordering. ✓

Transitive: If w_2 is a reordering of w_1 and w_3 is a reordering of w_2 , then w_3 is obtained from w_1 by composing the two reorderings—still a reordering. ✓

A few elements of the equivalence class of “listen”: “silent,” “tinsel,” “enlist,” “inlets,” and “listen” itself.

The natural *representative* of each class is the multiset of letters—or, equivalently, the letters sorted into a canonical order (say, alphabetical). “listen,” “silent,” and “enlist” all sort to “eilnst,” so you can recognise anagrams of one another by sorting their letters and comparing. This is exactly the standard algorithm for anagram detection, and it is computing a canonical representative of the equivalence class.

Exercise 5.

Proof. Assume $\sim$ is an equivalence relation on A and $a \sim b$. We show $[a] = [b]$ by double inclusion.

$[a] \subseteq [b]$. Let $x \in [a]$. Then $a \sim x$ by definition of $[a]$. From $a \sim b$ and symmetry, $b \sim a$. From $b \sim a$ and $a \sim x$, by transitivity, $b \sim x$. Hence $x \in [b]$.

$[b] \subseteq [a]$. Let $x \in [b]$. Then $b \sim x$. From $a \sim b$ and $b \sim x$, by transitivity, $a \sim x$. Hence $x \in [a]$.

Combining the two inclusions gives $[a] = [b]$. $\square$

Partition. We show that any two equivalence classes are either equal or disjoint. Take classes $[a]$ and $[b]$ and suppose they are not disjoint: some c satisfies $c \in [a] \cap [b]$, so $a \sim c$ and $b \sim c$. By symmetry and transitivity, $a \sim b$, and by the lemma just proved, $[a] = [b]$. So distinct classes are disjoint, and every element $a \in A$ belongs to at least one class (namely $[a]$, by reflexivity). The classes therefore partition A .

Exercise 6. Define $G \sim H$ iff there is a bijection $\varphi : V_1 \rightarrow V_2$ such that $\{u, v\} \in E_1$ iff $\{\varphi(u), \varphi(v)\} \in E_2$.

Reflexive: The identity $\text{id}_{V_1} : V_1 \rightarrow V_1$ is a bijection, and $\{u, v\} \in E_1$ iff $\{\text{id}(u), \text{id}(v)\} = \{u, v\} \in E_1$. So $G \sim G$. $\checkmark$

Symmetric: If φ witnesses $G \sim H$, then φ^{-1} is also a bijection, and $\{u', v'\} \in E_2$ iff $\{\varphi^{-1}(u'), \varphi^{-1}(v')\} \in E_1$ (apply the defining equivalence at $u = \varphi^{-1}(u')$, $v = \varphi^{-1}(v')$ and use $\varphi \circ \varphi^{-1} = \text{id}$). So $H \sim G$. $\checkmark$

Transitive: If φ witnesses $G \sim H$ and ψ witnesses $H \sim K$, then $\psi \circ \varphi$ is a bijection, and

$$\{u, v\} \in E_1 \iff \{\varphi(u), \varphi(v)\} \in E_2 \iff \{\psi(\varphi(u)), \psi(\varphi(v))\} \in E_3.$$

So $G \sim K$. $\checkmark$

Isomorphic example on four vertices. The path P_4 with vertices $\{1, 2, 3, 4\}$ and edges $\{\{1, 2\}, \{2, 3\}, \{3, 4\}\}$ is isomorphic to the path with vertices $\{a, b, c, d\}$ and edges $\{\{a, b\}, \{b, c\}, \{c, d\}\}$; the bijection $1 \mapsto a, 2 \mapsto b, 3 \mapsto c, 4 \mapsto d$ witnesses this.

Non-isomorphic example. The four-cycle C_4 with edges $\{\{1, 2\}, \{2, 3\}, \{3, 4\}, \{4, 1\}\}$ and the path P_4 above are *not* isomorphic, even though both have four vertices and three or four edges. C_4 has four edges and P_4 has three, so no bijection between their vertex sets can match the edge structure. (Even more robustly: every vertex of C_4 has degree 2, while P_4 has two vertices of degree 1 and two of degree 2. Any isomorphism must preserve the degree sequence, and these differ.)

An isomorphism class of graphs *remembers* the structural data: the number of vertices, the number of edges, the degree sequence, whether the graph is connected, whether it has cycles, and so on—anything that depends only on the abstract shape of the graph. It *forgets* everything that depends on vertex labels: the names of the vertices, the order they appear, the particular identifiers used. This is the standard abstraction of graph theory: we study graphs “up to isomorphism,” because the labels are incidental to the structural questions.

Module 7

Warm-up

Exercise 1.

Proof. We prove the contrapositive: if n is odd, then n^3 is odd.

Assume n is odd. Then $n = 2k + 1$ for some integer k . Compute:

$$\begin{aligned} n^3 &= (2k + 1)^3 \\ &= 8k^3 + 12k^2 + 6k + 1 \\ &= 2(4k^3 + 6k^2 + 3k) + 1. \end{aligned}$$

Since $4k^3 + 6k^2 + 3k$ is an integer, n^3 has the form $2m + 1$ and is therefore odd.

By contrapositive, if n^3 is even then n is even. □

Exercise 2. Trace of the Euclidean algorithm on $\text{gcd}(48, 18)$:

$$\begin{aligned} \text{gcd}(48, 18) : \quad & 48 = 2 \cdot 18 + 12, \quad \text{so } \text{gcd}(48, 18) = \text{gcd}(18, 12). \\ \text{gcd}(18, 12) : \quad & 18 = 1 \cdot 12 + 6, \quad \text{so } \text{gcd}(18, 12) = \text{gcd}(12, 6). \\ \text{gcd}(12, 6) : \quad & 12 = 2 \cdot 6 + 0, \quad \text{so } \text{gcd}(12, 6) = 6. \end{aligned}$$

Answer: $\text{gcd}(48, 18) = 6$.

Exercise 3. We want to derive $\{x = 0\} x := x + 3 \{x = 3\}$.

Step 1 (assignment rule). The assignment rule says: to derive $\{?\} x := x + 3 \{x = 3\}$, substitute $x + 3$ for x in the postcondition $x = 3$. This yields the precondition $x + 3 = 3$, giving:

$$\{x + 3 = 3\} x := x + 3 \{x = 3\}.$$

Step 2 (strengthening). We need to show $x = 0 \implies x + 3 = 3$. This holds by elementary arithmetic. By the rule of consequence (precondition strengthening):

$$\{x = 0\} x := x + 3 \{x = 3\}.$$

Exercise 4. In each case, substitute the right-hand side of the assignment for the variable in the postcondition.

- (a) $\{?\} x := x + 5 \{x > 10\}$: the postcondition is $x > 10$. Substituting $x + 5$ for x gives $x + 5 > 10$, i.e., $x > 5$. So $\{x > 5\} x := x + 5 \{x > 10\}$.
- (b) $\{?\} y := 2y \{y = 8\}$: substitute $2y$ for y in $y = 8$ to get $2y = 8$, i.e., $y = 4$. So $\{y = 4\} y := 2y \{y = 8\}$.
- (c) $\{?\} z := x - y \{z = 0\}$: substitute $x - y$ for z in $z = 0$ to get $x - y = 0$, i.e., $x = y$. So $\{x = y\} z := x - y \{z = 0\}$.

Standard

Exercise 5.

Proof that $\sqrt{3}$ is irrational. Assume for contradiction that $\sqrt{3}$ is rational. Then $\sqrt{3} = a/b$ in lowest terms (so $\gcd(a, b) = 1$) with a, b integers and $b > 0$. Squaring, $3 = a^2/b^2$, so $a^2 = 3b^2$. Therefore $3 \mid a^2$.

Sub-lemma: if $3 \mid a^2$ then $3 \mid a$. Proof by cases on $a \bmod 3$. If $a \equiv 0 \pmod{3}$, done. If $a \equiv 1 \pmod{3}$, then $a^2 \equiv 1 \pmod{3}$. If $a \equiv 2 \pmod{3}$, then $a^2 \equiv 4 \equiv 1 \pmod{3}$. So the only way for $3 \mid a^2$ is $a \equiv 0 \pmod{3}$, i.e., $3 \mid a$. ✓

So $3 \mid a$. Write $a = 3c$ for some integer c . Substituting into $a^2 = 3b^2$: $9c^2 = 3b^2$, so $b^2 = 3c^2$. By the same sub-lemma applied to b , $3 \mid b$.

But now $3 \mid a$ and $3 \mid b$, so $\gcd(a, b) \geq 3 > 1$, contradicting our assumption that the fraction was in lowest terms. Therefore $\sqrt{3}$ is irrational. $\square$

Exercise 6. The “proof” that $\sqrt{4}$ is irrational follows the structure of the classic $\sqrt{2}$ proof by contradiction. It assumes $\sqrt{4} = a/b$ with $\gcd(a, b) = 1$, squares both sides to get $4b^2 = a^2$, and derives that a is even, say $a = 2c$. Substituting gives $4b^2 = 4c^2$, so $b^2 = c^2$, hence $b = c$ (since $b, c > 0$). The proof then claims “Contradiction!”

But there is no contradiction. The derivation $b = c$ together with $a = 2c$ gives $a = 2b$, so $\sqrt{4} = a/b = 2b/b = 2$. This is a perfectly true statement— $\sqrt{4}$ really is 2—not a contradiction. The proof claims a true result is contradictory.

(A secondary issue: the step from $c^2 = b^2$ to $c = b$ requires $c, b \geq 0$, which happens to hold here but isn’t justified. However, the main error is that the argument produces no contradiction at all, because $\sqrt{4}$ is rational.)

Exercise 7. We work backward from the postcondition $x + y = 2$, peeling off one assignment at a time.

Step 1 (backward through $y := y + 1$). Substitute $y + 1$ for y in $x + y = 2$ to get $x + (y + 1) = 2$, i.e., $x + y = 1$:

$\{x + y = 1\} \ y := y + 1 \ \{x + y = 2\} \text{ [assignment]}$

Step 2 (backward through $x := x + 1$). Substitute $x + 1$ for x in $x + y = 1$ to get $(x + 1) + y = 1$, i.e., $x + y = 0$:

$\{x + y = 0\} \ x := x + 1 \ \{x + y = 1\} \text{ [assignment]}$

Step 3 (sequencing). Chain the two:

$\{x + y = 0\} \ x := x + 1; y := y + 1 \ \{x + y = 2\} \text{ [sequencing]}$

Step 4 (strengthening). From the stated precondition $x = 0 \wedge y = 0$, we have $x + y = 0 + 0 = 0$, so the stronger precondition implies the one we derived:

$\{x = 0 \wedge y = 0\} \ x := x + 1; y := y + 1 \ \{x + y = 2\} \text{ [strengthening]}$

Done. $\square$

Exercise 8. We run the extended Euclidean algorithm on $a = 17$, $b = 5$:

$$17 = 3 \cdot 5 + 2,$$

$$5 = 2 \cdot 2 + 1,$$

$$2 = 2 \cdot 1 + 0.$$

So $\gcd(17, 5) = 1$. Back-substitute:

$$\begin{aligned} 1 &= 5 - 2 \cdot 2 \\ &= 5 - 2 \cdot (17 - 3 \cdot 5) \\ &= 5 - 2 \cdot 17 + 6 \cdot 5 \\ &= 7 \cdot 5 - 2 \cdot 17. \end{aligned}$$

So $17 \cdot (-2) + 5 \cdot 7 = 1$, i.e., $x = -2$, $y = 7$.

Multiplicative inverse of 17 mod 5: $x = -2 \equiv 3 \pmod{5}$. Check: $17 \cdot 3 = 51 \equiv 1 \pmod{5}$.
 $\checkmark$ (Alternatively, $17 \equiv 2 \pmod{5}$, and $2 \cdot 3 = 6 \equiv 1 \pmod{5}$.)

Multiplicative inverse of 5 mod 17: $y = 7$. Check: $5 \cdot 7 = 35 = 2 \cdot 17 + 1 \equiv 1 \pmod{17}$. $\checkmark$

Challenge

Exercise 9.

Proof by contradiction that there is no largest prime. Assume for contradiction that there is a largest prime, call it p . Then every prime is at most p , so the list of all primes is $2, 3, 5, 7, \dots, p$ —a finite list.

Consider the number

$$N = 2 \cdot 3 \cdot 5 \cdot 7 \cdots p + 1,$$

i.e., the product of all primes up to p , plus 1. This N is an integer greater than 1, so it has at least one prime factor. Let q be any prime factor of N .

Case 1: $q \leq p$. Then q is one of the primes in the product $2 \cdot 3 \cdots p$, so q divides that product. But q also divides $N = (\text{product}) + 1$. So q divides the difference $N - (\text{product}) = 1$. The only positive divisor of 1 is 1 itself, but $q \geq 2$ (primes are at least 2). Contradiction.

Case 2: $q > p$. Then q is a prime strictly greater than p . But we assumed p was the largest prime, so no prime can be bigger. Contradiction.

Either way we reach a contradiction. So the assumption “there is a largest prime” is false, i.e., there are infinitely many primes. $\square$

This is the classical argument, attributed to Euclid (around 300 BCE), and it’s one of the oldest proofs in the book.

Exercise 10. Loop invariant: $\text{Inv} := (0 \leq i \leq n) \wedge (y = 2^i)$.

Establishment. After $y := 1$; $i := 0$, we have $y = 1 = 2^0$ and $i = 0$, and $0 \leq 0 \leq n$ (using the precondition $n \geq 0$). So Inv holds.

Preservation. We show $\{\text{Inv} \wedge (i < n)\} y := 2y; i := i + 1 \{\text{Inv}\}$ by working backward from Inv .

Step 1 (through $i := i + 1$). Substitute $i + 1$ for i in Inv :

$$(0 \leq i + 1 \leq n) \wedge (y = 2^{i+1}).$$

Step 2 (through $y := 2y$). Substitute $2y$ for y in the above:

$$(0 \leq i + 1 \leq n) \wedge (2y = 2^{i+1}),$$

which simplifies to $(0 \leq i + 1 \leq n) \wedge (y = 2^i)$.

This is implied by $\text{Inv} \wedge (i < n)$: from $0 \leq i \leq n$ and $i < n$, we get $0 \leq i \leq n - 1$, and adding 1 gives $1 \leq i + 1 \leq n$, which implies $0 \leq i + 1 \leq n$. And $y = 2^i$ is already part of Inv . $\checkmark$

Termination (exit condition). When the loop exits, $\text{Inv} \wedge \neg(i < n) = \text{Inv} \wedge (i \geq n)$. Combined with $i \leq n$ from Inv , we get $i = n$, and then $y = 2^i = 2^n$. So the postcondition $y = 2^n$ holds.

Putting the three parts together via the while rule and sequencing, we get

$$\{n \geq 0\} y := 1; i := 0; \text{while } (i < n) \text{ do } (y := 2y; i := i + 1) \{y = 2^n\}. \square$$

Variant for total correctness: $n - i$ strictly decreases on each iteration (since i increases by 1) and is bounded below by 0 (since the guard forces $i < n$, i.e., $n - i > 0$). After at most n iterations the loop exits.

Exercise 11. We show $\text{gcd}(a, b) = \text{gcd}(b, a \bmod b)$ for positive integers a, b with $b \neq 0$.

Write $a = bq + r$ with $r = a \bmod b$, so $0 \leq r < b$. We show the common divisors of $\{a, b\}$ and the common divisors of $\{b, r\}$ are the same set—from which it follows that the *greatest* common divisors coincide.

($\subseteq$): Let d be a common divisor of a and b . Then $d \mid a$ and $d \mid b$. Since $r = a - bq$, we have r is an integer linear combination of a and b , so $d \mid r$. Therefore d divides both b and r , so d is a common divisor of $\{b, r\}$.

($\supseteq$): Let d be a common divisor of b and r . Then $d \mid b$ and $d \mid r$. Since $a = bq + r$, a is an integer linear combination of b and r , so $d \mid a$. Therefore d divides both a and b .

The two sets of common divisors are equal, so their greatest elements are equal: $\text{gcd}(a, b) = \text{gcd}(b, r) = \text{gcd}(b, a \bmod b)$. $\square$

This lemma is what makes Euclid's algorithm work: each step replaces the pair (a, b) with a pair $(b, a \bmod b)$ that has strictly smaller second coordinate but the same gcd. The algorithm terminates because the second coordinate strictly decreases and can't go below 0, and it's correct because the gcd is preserved at each step.

Connection

Exercise 12.

```
def egcd(a, b):
    if b == 0:
        return (a, 1, 0)
    g, x1, y1 = egcd(b, a % b)
    # g = b*x1 + (a//b)*y1
    # a//b = a - (a//b)*b, so
    # g = b*x1 + (a - (a//b)*b)*y1
    # = a*y1 + b*(x1 - (a//b)*y1)
    return (g, y1, x1 - (a // b) * y1)

def mod_inverse_via_egcd(a, n):
    g, x, _ = egcd(a % n, n)
    if g != 1:
        raise ValueError(f"{a} has no inverse mod {n}")
    return x % n

# Same tests as Module 6 Exercise 13
print(mod_inverse_via_egcd(3, 7)) # 5
try:
    print(mod_inverse_via_egcd(2, 6))
except ValueError as e:
    print(e) # 2 has no inverse mod 6
```

The extended Euclidean algorithm is asymptotically much faster than brute force. Brute force takes $O(n)$ time to find the inverse (it checks up to n candidates). Euclid’s algorithm takes $O(\log n)$ steps because each iteration at least halves the smaller argument (the ratio is actually governed by the Fibonacci numbers, but $O(\log n)$ captures the bound). For small n both work fine; for cryptographic sizes (where n is hundreds of digits long), brute force would take longer than the age of the universe, while Euclid’s algorithm finishes in milliseconds.

Exercise 13. Hoare logic is better suited for reasoning about merge sort as a *specific imperative program* that mutates arrays—establishing things like “after `merge(xs, lo, mid, hi)`, the subarray `xs[lo..hi]` is sorted.” The proof is about the state of memory before and after each step, which is exactly what Hoare triples express.

Curry–Howard is better suited for reasoning about merge sort as a *pure mathematical function* that takes a list and returns a sorted list. Under Curry–Howard, a proof that merge sort is correct *becomes* an implementation of merge sort in the same proof assistant, with the return type guaranteeing the sortedness. There’s no separate “program” and “proof”—the proof *is* the program.

Both correspondences are real and useful, and the choice usually tracks whether your merge sort is destructively mutating an array (Hoare) or functionally building a new list (Curry–Howard). For the imperative version in C, Hoare logic is the natural tool; for the functional version in Haskell or Lean, Curry–Howard is. This is not a contradiction: the two views are complementary, and one of the interesting frontiers in programming-language research is figuring out how to combine them (separation logic, for example, extends Hoare logic with “ownership” of memory regions in a way that starts to look Curry–Howard-ish).

Intermezzo: Proofs Are Programs

Exercise 1. Disjunction elimination ($\vee E$) corresponds to **pattern matching** on a sum type, i.e., case analysis on an `Either` value. Given a value of type `Either A B`, you provide two handlers:

- One for the `Left a` case (handling a proof of A), and
- One for the `Right b` case (handling a proof of B).

This is exactly proof by cases: to prove C from $A \vee B$, you show $A \rightarrow C$ and $B \rightarrow C$ separately, then use whichever case applies. In Haskell:

```
either :: (a -> c) -> (b -> c) -> Either a b -> c
either f g (Left a) = f a
either f g (Right b) = g b
```

Exercise 2. The type $A \rightarrow (B \rightarrow A)$ corresponds to a function that takes an a and returns a function that ignores its argument and returns a :

```
def f(a):
    return lambda b: a
```

This is the “constant function factory”—given any value of type A , it produces a function from B to A that ignores its input and always returns the original a .

As a logical tautology, $A \rightarrow (B \rightarrow A)$ says: “if A is true, then no matter what B is, A is still true.” This is obvious logically—having a proof of A means you have a proof of A regardless of what else is going on—and the program makes it computationally concrete: given evidence for A , just hold onto it and ignore whatever B -evidence someone hands you.

Exercise 3.

```

def curry(f):
    # f has type (A, B) -> C
    return lambda a: lambda b: f((a, b))

def uncurry(g):
    # g has type A -> (B -> C)
    return lambda pair: g(pair[0])(pair[1])

```

Inverses. Starting with $f : (A \times B) \rightarrow C$:

$$\begin{aligned}
 \text{uncurry}(\text{curry}(f))(a, b) &= \text{curry}(f)(a)(b) \\
 &= (\lambda a. \lambda b. f((a, b)))(a)(b) \\
 &= f((a, b)),
 \end{aligned}$$

so $\text{uncurry}(\text{curry}(f)) = f$ on every pair. In the other direction, starting with $g : A \rightarrow (B \rightarrow C)$:

$$\begin{aligned}
 \text{curry}(\text{uncurry}(g))(a)(b) &= \text{uncurry}(g)((a, b)) \\
 &= g(a)(b),
 \end{aligned}$$

so $\text{curry}(\text{uncurry}(g)) = g$ on every a and b . The two are inverses, witnessing the isomorphism $(A \times B) \rightarrow C \cong A \rightarrow (B \rightarrow C)$.

Logically, this is the tautology

$$((A \wedge B) \rightarrow C) \leftrightarrow (A \rightarrow (B \rightarrow C)).$$

“Assume A and B at once, prove C ” is the same thing as “Assume A and prove ‘assume B and prove C .’ ” The way you build a proof of the two-step implication from a proof of the single-step one—and vice versa—is exactly currying and uncurrying. Function currying in programming and the currying equivalence in logic are the same operation on proofs/programs.

Exercise 4.

```

def compose(f, g):
    # f : A -> B, g : B -> C, result : A -> C
    return lambda a: g(f(a))

```

The logical proof that $A \rightarrow B \rightarrow (B \rightarrow C) \rightarrow (A \rightarrow C)$ goes like this:

1. Assume $A \rightarrow B$. (The argument f .)
2. Assume $B \rightarrow C$. (The argument g .)
3. Subproof: assume A , show C . (The body `lambda a: ...`)
 - (i) A , by assumption. (The parameter a .)
 - (ii) B , by modus ponens on (i) and (1). (The subexpression $f(a)$.)
 - (iii) C , by modus ponens on (ii) and (2). (The expression $g(f(a))$.)
 - (iv) Shows C .
4. $A \rightarrow C$, by $\rightarrow$ I on the subproof.

Each logical step lines up with a syntactic piece of **compose**: modus ponens becomes function application, implication introduction becomes the defining **lambda**, and the result $g(f(a))$ is the proof of C that the subproof produces. Function composition is the computational content of transitivity.

Exercise 5.

```
def absurd(v):
    # v : Void --- impossible to construct, so this branch is never taken
    raise RuntimeError("unreachable:⊥Void⊥has⊥no⊥values")
```

The function has type $\text{Void} \rightarrow A$ for any A . The body raises an error, but this body never actually runs: there are no values of type **Void**, so the parameter v can never be supplied. The function is vacuously well-defined because its domain of “real” inputs is empty.

In a language like Haskell with a genuine empty type, you can often write this function by pattern-matching on the empty set of constructors:

```
absurd :: Void -> a
absurd v = case v of {} -- no cases, because Void has no constructors
```

The empty **case** makes the vacuity explicit: there is no input to handle.

Why the lack of values makes this trivially valid: to check that **absurd** really has type $\text{Void} \rightarrow A$, a type-checker must verify that for every value $v \in \text{Void}$, the body **absurd**(v) evaluates to a value in A . That universal quantifier is over an empty set, so it is vacuously satisfied. No actual return value is ever required.

Logically, this is the principle *ex falso quodlibet*: from $\perp$ you may conclude anything. There is no proof of $\perp$, so this “implication” never gets discharged, but formally it is a function from an empty input space to any output space.

Exercise 6.

No such function can be written in general.

Suppose, for contradiction, that we had a total function **lem** of type $() \rightarrow \text{Either } A \text{ } (A \rightarrow \text{Void})$ that worked for *every* type A . Specialise to

$A = \text{“codes of Turing machines that halt on every input.”}$

A value of type A is a witness that some specific machine is total. A value of $A \rightarrow \text{Void}$ is a function that takes any purported total machine and derives a contradiction—equivalently, a proof that no total machine exists at this code.

For a given machine code e , calling **lem**() must (in finite time) return either:

- a **Left**: a witness that e is in fact total, or
- a **Right**: a proof that e is *not* total.

But this is exactly a decision procedure for whether e codes a total Turing machine. By a standard computability-theory argument (this is the Π_2^0 -complete totality problem, harder than even the halting problem), no algorithm decides this.

So **lem** cannot be implemented as a total computable function. In classical logic, $A \vee \neg A$ is a tautology because we accept “one of the two must be true” without caring which; but from the Curry–Howard perspective, a proof of $A \vee \neg A$ would be *code*, and the code would have to tell us which disjunct holds. No such code can exist in general, which is the computational reason excluded middle is rejected in constructive logic.

Intermezzo: Other Logics, Other Worlds

Exercise 1.

- (a) “The program eventually terminates”: $\Diamond \text{halt}$.
- (b) “The counter is always non-negative”: $\Box(\text{counter} \geq 0)$.
- (c) “Every request is eventually followed by a response”: $\Box(\text{request} \rightarrow \Diamond \text{response})$.

Exercise 2.

- (a) “Two trains are never on the same section at the same time”:

$$\Box \neg(\text{train}_1 \wedge \text{train}_2).$$

A classic mutual-exclusion safety property.

- (b) “Once an alarm goes off, it stays on until manually reset.” Using only $\Box$, $\Diamond$, $\bigcirc$ (no “until” operator), we can express the “stays on” part by saying: in every state where the alarm is on and has not yet been reset, the alarm is still on in the next state:

$$\Box((\text{alarm} \wedge \neg \text{reset}) \rightarrow \bigcirc \text{alarm}).$$

This captures the rule that the only way the alarm can turn off is via a reset.

- (c) “Every login is eventually followed by a logout, and every logout is preceded by a login.” The forward half is $\Box(\text{login} \rightarrow \Diamond \text{logout})$, by analogy with the request/response example. The backward half—“a logout is preceded by a login”—is not expressible in plain LTL starting at $t = 0$, because LTL’s operators run forward in time. The natural forward-only reformulation is: “logout is never the first thing to happen,” i.e., $\neg \text{logout}$ at time 0 together with $\Box((\neg \text{login_so_far}) \rightarrow \neg \text{logout})$, where login_so_far is a derived state proposition. The cleanest LTL capture therefore needs a state variable tracking “has a login ever happened?” The lesson: temporal logic expresses what comes next, not what came before.

Exercise 3.

Proof. We unfold the semantics step by step.

$$\begin{aligned} s_i \models \neg \Diamond p &\iff s_i \not\models \Diamond p \\ &\iff \text{there is no } j \geq i \text{ with } s_j \models p \\ &\iff \text{for every } j \geq i, s_j \not\models p \\ &\iff \text{for every } j \geq i, s_j \models \neg p \\ &\iff s_i \models \Box \neg p. \end{aligned}$$

So $\neg \Diamond p \equiv \Box \neg p$. □

This is the same argument shape as the proof of $\neg(\exists x. P(x)) \equiv \forall x. \neg P(x)$ from Module 4: “there is no x with $P(x)$ ” is unfolded to “for every x , $\neg P(x)$ ” by straightforward negation of an existential. Temporal $\Diamond$ is an existential over future times and temporal $\Box$ is a universal over future times, so the duality is literally the first-order quantifier duality in temporal clothing.

Exercise 4. $\Diamond\Box p$: “it is eventually the case that p holds at every subsequent moment.” That is, after some finite delay, p holds and never turns off again.

$\Box\Diamond p$: “it is always the case that p will eventually hold.” That is, p is infinitely often true, but may alternate on and off forever.

Distinguishing example. Let p hold at every *even* state and fail at every odd state. Concretely, $s_0 \models p$, $s_1 \not\models p$, $s_2 \models p$, $s_3 \not\models p$, and so on.

- $\Box\Diamond p$ is true: from any state s_i , there is a later even state where p holds (take $j = 2\lceil i/2 \rceil$). So $\Diamond p$ holds at every s_i , hence $\Box\Diamond p$ holds at s_0 .
- $\Diamond\Box p$ is false: $\Box p$ at any state s_i would require p to hold at every s_j with $j \geq i$, but s_{i+1} (or s_i if i is odd) fails p . So $\Box p$ holds nowhere, hence $\Diamond\Box p$ is false at s_0 .

Operationally: $\Box\Diamond p$ means “ p keeps coming back,” and $\Diamond\Box p$ means “ p settles down to being true forever.” The settle-down version is strictly stronger.

Exercise 5. The LTL version of “at every state during the loop, the invariant P holds” is simply

$$\Box P.$$

(Implicitly restricted to states reachable during loop execution, but at the LTL level $\Box P$ is the natural translation.)

What the LTL formulation makes explicit. The Hoare rule

$$\{P \wedge B\} c \{P\} \implies \{P\} \text{while } B \text{ do } c \{P \wedge \neg B\}$$

proves that P is preserved by each iteration but only *states* the pre- and post-condition of the entire loop. The intermediate states—the moments in between iterations, during the iterations, at the start of each iteration—do not appear in the Hoare triple. The LTL statement $\Box P$ makes those intermediate states first-class: it says “at every state of the execution, P holds,” which is a stronger and more directly useful fact.

What temporal logic gives you that Hoare logic does not. Hoare logic describes the pre- and post-conditions of a program as if the program were a black box; temporal logic describes the whole *trajectory*, i.e., every intermediate state, so it can express properties of non-terminating systems (servers, operating systems, network protocols) where there is no “post-condition” because the program never finishes.

Exercise 6.

- $\Box \neg \text{deadlock}$: **safety**. Nothing bad (a deadlock) ever happens.
- $\Box (\text{request} \rightarrow \Diamond \text{response})$: **liveness**. Every request is eventually answered (something good always happens).
- $\Diamond \Box \text{stable}$: **liveness**. The system eventually reaches a stable state and stays there—this is “something good happens” (stability), but the condition involves a promise about the future, not the absence of a bad event. (Strictly, this is the kind of liveness property called an “eventually stable” or “persistence” property.)
- $\Box \Diamond \text{progress}$: **liveness**. Progress is made infinitely often. No specific bad event is forbidden; the property is about something good happening repeatedly.

Some authors classify (c) as neither pure safety nor pure liveness because it requires *both* a promise (reaching stability) and a guarantee (staying there). The general rule is that safety properties have the shape $\Box(\text{good now})$ and liveness properties have the shape $\Box\Diamond \dots$ or $\Diamond \dots$ —i.e., safety says “nothing bad has happened yet,” while liveness says “something good eventually happens.”

Module 8

Warm-up

Exercise 1. Recursive definition of $a_n = 3n$:

$$a_0 = 0, \quad a_n = a_{n-1} + 3 \quad \text{for } n \geq 1.$$

Verification: $a_0 = 0$, $a_1 = 0 + 3 = 3$, $a_2 = 3 + 3 = 6$, $a_3 = 6 + 3 = 9$. These match 0, 3, 6, 9. ✓

Exercise 2.

(a) $\sum_{i=1}^5 (2i + 1) = 3 + 5 + 7 + 9 + 11 = 35$.

(b) $\sum_{k=0}^4 2^k = 1 + 2 + 4 + 8 + 16 = 31$.

(c) $\sum_{j=1}^3 (-1)^{j+1} j^2 = 1 - 4 + 9 = 6$.

(d) $\prod_{i=1}^4 i = 1 \cdot 2 \cdot 3 \cdot 4 = 24 = 4!$.

Exercise 3.

(a) $a_0 = 2$, $a_n = a_{n-1} + 4$. Starting from 2 and adding 4 each step gives 2, 6, 10, 14, ..., so $a_n = 4n + 2$.

(b) $b_n = n^2 + 1$. Compute $b_{n-1} = (n-1)^2 + 1 = n^2 - 2n + 2$, so $b_n - b_{n-1} = 2n - 1$. A valid recursive definition is therefore $b_0 = 1$ and $b_n = b_{n-1} + 2n - 1$ for $n \geq 1$.

Exercise 4. Fibonacci: $\text{fib}(0) = 0$, $\text{fib}(1) = 1$, $\text{fib}(2) = 1$, $\text{fib}(3) = 2$, $\text{fib}(4) = 3$, $\text{fib}(5) = 5$.
Check: $\text{fib}(5) = \text{fib}(4) + \text{fib}(3) = 3 + 2 = 5$. ✓

Standard

Exercise 5.

Proof. Let $P(n)$ be the statement $\sum_{i=1}^n (2i - 1) = n^2$.

Base case ($n = 1$): $\sum_{i=1}^1 (2i - 1) = 2(1) - 1 = 1 = 1^2$. ✓

Inductive step. Assume $P(k)$: $\sum_{i=1}^k (2i - 1) = k^2$.

Then:

$$\begin{aligned} \sum_{i=1}^{k+1} (2i - 1) &= \sum_{i=1}^k (2i - 1) + (2(k+1) - 1) \\ &= k^2 + (2k + 1) \quad (\text{by the inductive hypothesis}) \\ &= k^2 + 2k + 1 \\ &= (k+1)^2. \end{aligned}$$

This establishes $P(k+1)$, completing the induction. □

Exercise 6.

Proof. Let $P(n)$ be $\sum_{i=1}^n i^2 = n(n+1)(2n+1)/6$.

Base case ($n = 1$): LHS = $1^2 = 1$. RHS = $1 \cdot 2 \cdot 3/6 = 1$. ✓

Inductive step. Assume $\sum_{i=1}^k i^2 = k(k+1)(2k+1)/6$. Then:

$$\begin{aligned}
 \sum_{i=1}^{k+1} i^2 &= \sum_{i=1}^k i^2 + (k+1)^2 \\
 &= \frac{k(k+1)(2k+1)}{6} + (k+1)^2 \\
 &= \frac{k(k+1)(2k+1) + 6(k+1)^2}{6} \\
 &= \frac{(k+1)[k(2k+1) + 6(k+1)]}{6} \\
 &= \frac{(k+1)(2k^2 + 7k + 6)}{6} \\
 &= \frac{(k+1)(k+2)(2k+3)}{6},
 \end{aligned} \tag{IH}$$

using the factorization $2k^2 + 7k + 6 = (k+2)(2k+3)$. This matches $(k+1)((k+1)+1)(2(k+1)+1)/6$, as required. $\square$

Exercise 7.

Proof. Let $P(n)$ be $\sum_{i=0}^n 2^i = 2^{n+1} - 1$.

Base case ($n = 0$): LHS = $2^0 = 1$. RHS = $2^1 - 1 = 1$. ✓

Inductive step. Assume $\sum_{i=0}^k 2^i = 2^{k+1} - 1$. Then

$$\sum_{i=0}^{k+1} 2^i = \sum_{i=0}^k 2^i + 2^{k+1} = (2^{k+1} - 1) + 2^{k+1} = 2 \cdot 2^{k+1} - 1 = 2^{k+2} - 1. \checkmark$$

$\square$

Exercise 8.

Proof. Let $T_n = n(n+1)/2$. Let $P(n)$ be $\sum_{i=1}^n i^3 = T_n^2$.

Base case ($n = 1$): LHS = 1, RHS = $T_1^2 = 1$. ✓

Inductive step. Assume $\sum_{i=1}^k i^3 = T_k^2$. Then

$$\begin{aligned}
 \sum_{i=1}^{k+1} i^3 &= T_k^2 + (k+1)^3 \\
 &= \frac{k^2(k+1)^2}{4} + (k+1)^3 \\
 &= (k+1)^2 \left[\frac{k^2}{4} + (k+1) \right] \\
 &= (k+1)^2 \cdot \frac{k^2 + 4k + 4}{4} \\
 &= (k+1)^2 \cdot \frac{(k+2)^2}{4} \\
 &= \left(\frac{(k+1)(k+2)}{2} \right)^2 = T_{k+1}^2. \checkmark
 \end{aligned}$$

□

The miracle: cubes sum to squares of triangles. There's also a pretty layered-L-shape geometric argument, but the algebraic induction works fine.

Challenge

Exercise 9.

Proof. Let $P(n)$ be $\sum_{i=1}^n \frac{1}{i(i+1)} = \frac{n}{n+1}$.

Base case ($n = 1$): LHS = $\frac{1}{1 \cdot 2} = \frac{1}{2}$. RHS = $\frac{1}{2}$. ✓

Inductive step. Assume $\sum_{i=1}^k \frac{1}{i(i+1)} = \frac{k}{k+1}$. Then

$$\begin{aligned}
 \sum_{i=1}^{k+1} \frac{1}{i(i+1)} &= \frac{k}{k+1} + \frac{1}{(k+1)(k+2)} \quad (\text{IH}) \\
 &= \frac{k(k+2) + 1}{(k+1)(k+2)} \\
 &= \frac{k^2 + 2k + 1}{(k+1)(k+2)} \\
 &= \frac{(k+1)^2}{(k+1)(k+2)} \\
 &= \frac{k+1}{k+2}. \checkmark
 \end{aligned}$$

□

Side note: each term $\frac{1}{i(i+1)}$ equals $\frac{1}{i} - \frac{1}{i+1}$ (partial fractions), so the sum telescopes to $1 - \frac{1}{n+1} = \frac{n}{n+1}$. The induction above is a drill; the telescoping observation is the insight.

Exercise 10.

Proof. Let $P(n)$ be $2^n < n!$, for $n \geq 4$.

Base case ($n = 4$): $2^4 = 16 < 24 = 4!$. ✓

Inductive step. Assume $2^k < k!$ for some $k \geq 4$. Show $2^{k+1} < (k+1)!$.

$$2^{k+1} = 2 \cdot 2^k \underset{\text{IH}}{<} 2 \cdot k! \leq (k+1) \cdot k! = (k+1)!,$$

where the second inequality uses $2 \leq k+1$ (which holds since $k \geq 4 > 1$). ✓ □

Morally, factorial grows much faster than 2^n because factorial multiplies by $k+1$ at step k while the exponential only multiplies by 2. Once $k+1 > 2$, factorial pulls ahead and stays ahead.

Exercise 11.

Proof. Let $P(n)$ be $T_n = n(n+1)/2$, where $T_1 = 1$ and $T_n = T_{n-1} + n$.

Base case ($n = 1$): $T_1 = 1$ and $1 \cdot 2/2 = 1$. ✓

Inductive step. Assume $T_k = k(k+1)/2$. Then

$$T_{k+1} = T_k + (k+1) = \frac{k(k+1)}{2} + (k+1) = \frac{k(k+1) + 2(k+1)}{2} = \frac{(k+1)(k+2)}{2}. \checkmark$$

□

This is the same proof as the sum-of-first- n -integers proof from the module; the recursive definition is literally encoding that sum.

Connection

Exercise 12.

```
def sum_of_squares(n):
    if n == 0:
        return 0
    else:
        return sum_of_squares(n - 1) + n * n
```

- The `if n == 0: return 0` branch corresponds to the **base case**.
- The recursive call `sum_of_squares(n - 1)` corresponds to the **inductive hypothesis**—we trust that the function already returns the right answer for $n - 1$.
- Adding `n * n` and returning mirrors the **inductive step**: the proof adds $(k+1)^2$ to the IH and simplifies; the code adds n^2 and returns.

The correspondence is exact: proof and recursive function have the same shape, because induction and recursion are two views of the same computation.

Exercise 13.

```
def fib_rec(n):
    if n < 2:
        return n
    return fib_rec(n - 1) + fib_rec(n - 2)

def fib_iter(n):
```

```

a, b = 0, 1
for _ in range(n):
    a, b = b, a + b
return a

from functools import lru_cache
@lru_cache(maxsize=None)
def fib_memo(n):
    if n < 2:
        return n
    return fib_memo(n - 1) + fib_memo(n - 2)

for i in range(11):
    assert fib_rec(i) == fib_iter(i) == fib_memo(i)

```

Timing on $n = 30$: the recursive version takes roughly $O(\phi^n)$ function calls (where $\phi \approx 1.618$ is the golden ratio) because it recomputes the same subproblems repeatedly; on $n = 30$ that’s already around a million calls, and it takes noticeable time. The iterative version is $O(n)$ in both time and space, essentially instant. The memoized recursive version is also $O(n)$ because each subproblem is computed once and cached.

Takeaway: a naive recursive definition matches the math exactly but throws away efficiency. Memoization recovers the efficiency while keeping the recursive shape—this is the core idea of dynamic programming.

Exercise 14. The “one induction principle per data definition” pattern works because each recursive definition is really specifying a *functor*: a description of how the data is built from subparts of itself. The initial-algebra construction takes that functor and hands back both the data type and its induction principle in lockstep. That’s why the base case and the recursive case of the definition line up exactly with the base case and the recursive case of the induction rule—they’re the same data structure read twice, once as “constructors” and once as “cases of induction.”

In practice: when you write `Tree := Leaf | Node Tree Tree`, you automatically know that induction on trees has two premises, one for `Leaf` (no IH) and one for `Node` (two IHs, one per subtree). The BNF writes the induction rule for you. Module 10 cashes this out for several concrete recursive types, and the type-algebra intermezzo explains the general mechanism.

Module 9

Warm-up

Exercise 1.

- (a) $a_0 = 1$, $a_1 = 2 \cdot 1 + 1 = 3$, $a_2 = 2 \cdot 3 + 1 = 7$, $a_3 = 2 \cdot 7 + 1 = 15$, $a_4 = 2 \cdot 15 + 1 = 31$. (Notice the pattern: $a_n = 2^{n+1} - 1$.)
- (b) $b_0 = 0$, $b_1 = 1$, $b_2 = 2 \cdot 1 - 0 = 2$, $b_3 = 2 \cdot 2 - 1 = 3$, $b_4 = 2 \cdot 3 - 2 = 4$. (This is $b_n = n$.)
- (c) $c_0 = 3$, $c_1 = 5$, $c_2 = 5 + 3 = 8$, $c_3 = 8 + 5 = 13$, $c_4 = 13 + 8 = 21$. These are the Lucas numbers—a close cousin of Fibonacci with different initial conditions.

Exercise 2.

Recurrence	2nd-order	linear	homogeneous	const. coeffs
(a) $a_k = 3a_{k-1} - 2a_{k-2}$	yes	yes	yes	yes
(b) $a_k = a_{k-1} + 2$	no (1st)	yes	no	yes
(c) $a_k = k \cdot a_{k-1}$	no (1st)	yes	yes	no
(d) $a_k = a_{k-1}^2 + a_{k-2}$	yes	no	yes	yes
(e) $a_k = 5a_{k-1} - 6a_{k-2} + 7$	yes	yes	no	yes

Only (a) satisfies all four conditions, so only (a) can be directly solved by the characteristic-equation method from this module.

Exercise 3.

- (a) $a_k = 5a_{k-1} - 6a_{k-2}$: characteristic equation $r^2 - 5r + 6 = 0$ (i.e., $r^2 = 5r - 6$).
(b) $a_k = a_{k-1} + a_{k-2}$: characteristic equation $r^2 - r - 1 = 0$ (Fibonacci).
(c) $a_k = 6a_{k-1} - 9a_{k-2}$: characteristic equation $r^2 - 6r + 9 = 0$.

Standard

Exercise 4.

Proof. We prove by strong induction that every integer amount of postage $n \geq 12$ can be made using 4-cent and 5-cent stamps.

Base cases:

$$\begin{aligned} 12 &= 3 \times 4, \\ 13 &= 4 + 4 + 5, \\ 14 &= 4 + 5 + 5, \\ 15 &= 3 \times 5. \end{aligned}$$

Inductive step. Let $n \geq 16$ and assume (strong IH) that every integer amount from 12 to $n - 1$ can be made with 4-cent and 5-cent stamps.

Since $n \geq 16$, we have $n - 4 \geq 12$. By the strong inductive hypothesis, $n - 4$ cents can be made with 4-cent and 5-cent stamps. Adding one more 4-cent stamp gives n cents. $\square$

Exercise 5. The recurrence is $T_1 = 1$, $T_k = T_{k-1} + 3$ for $k \geq 2$.

Iteration:

$$\begin{aligned} T_k &= T_{k-1} + 3 \\ &= (T_{k-2} + 3) + 3 = T_{k-2} + 2 \cdot 3 \\ &= (T_{k-3} + 3) + 2 \cdot 3 = T_{k-3} + 3 \cdot 3 \\ &\vdots \\ &= T_1 + 3(k-1) \\ &= 1 + 3(k-1) = 3k - 2. \end{aligned}$$

Verification by induction: $T_1 = 3(1) - 2 = 1$. $\checkmark$ $T_{k+1} = T_k + 3 = (3k - 2) + 3 = 3(k+1) - 2$. $\checkmark$
Closed form: $T_k = 3k - 2$.

Exercise 6. The recurrence is $a_0 = 1$, $a_1 = 3$, $a_k = 4a_{k-1} - 3a_{k-2}$ for $k \geq 2$.

Characteristic equation:

$$r^2 - 4r + 3 = 0 \implies (r-1)(r-3) = 0 \implies r_1 = 1, r_2 = 3.$$

General solution: $a_n = \alpha \cdot 1^n + \beta \cdot 3^n = \alpha + \beta \cdot 3^n$.

Solving for constants:

$$a_0 = 1 \implies \alpha + \beta = 1,$$

$$a_1 = 3 \implies \alpha + 3\beta = 3.$$

Subtracting gives $2\beta = 2$, so $\beta = 1$ and $\alpha = 0$.

Closed form: $a_n = 3^n$.

Exercise 7. The recurrence is $a_0 = 1$, $a_1 = 5$, $a_k = 5a_{k-1} - 6a_{k-2}$.

Characteristic equation: $r^2 - 5r + 6 = 0$, which factors as $(r-2)(r-3) = 0$. So $r_1 = 2$, $r_2 = 3$.

General solution: $a_n = \alpha \cdot 2^n + \beta \cdot 3^n$.

Solving for constants:

$$a_0 = 1 \implies \alpha + \beta = 1,$$

$$a_1 = 5 \implies 2\alpha + 3\beta = 5.$$

From the first equation $\alpha = 1 - \beta$, substitute: $2(1 - \beta) + 3\beta = 5$, so $2 + \beta = 5$, so $\beta = 3$ and $\alpha = -2$.

Closed form: $a_n = -2 \cdot 2^n + 3 \cdot 3^n = 3^{n+1} - 2^{n+1}$.

Sanity check: $a_2 = 3^3 - 2^3 = 27 - 8 = 19$, and $5a_1 - 6a_0 = 25 - 6 = 19$. ✓

Challenge

Exercise 8. The recurrence is $a_0 = 2$, $a_1 = 12$, $a_k = 6a_{k-1} - 9a_{k-2}$.

Characteristic equation: $r^2 - 6r + 9 = 0$, which factors as $(r-3)^2 = 0$. Repeated root $r = 3$.

General solution (repeated-root form): $a_n = \alpha \cdot 3^n + \beta \cdot n \cdot 3^n$.

Solving for constants:

$$a_0 = 2 \implies \alpha \cdot 3^0 + \beta \cdot 0 \cdot 3^0 = \alpha = 2,$$

$$a_1 = 12 \implies \alpha \cdot 3 + \beta \cdot 1 \cdot 3 = 6 + 3\beta = 12,$$

so $3\beta = 6$, i.e., $\beta = 2$.

Closed form: $a_n = 2 \cdot 3^n + 2n \cdot 3^n = 2(n+1) \cdot 3^n$.

Verification: a_2 from the formula is $2 \cdot 3 \cdot 9 = 54$; from the recurrence, $a_2 = 6a_1 - 9a_0 = 72 - 18 = 54$. ✓

If you had tried the distinct-root form $a_n = \alpha \cdot 3^n + \beta \cdot 3^n = (\alpha + \beta) \cdot 3^n$, you'd have only *one* free parameter and couldn't fit both initial conditions. The extra n factor in the repeated-root case is essential.

Exercise 9.

Proof sketch. We prove by strong induction that $T_n \leq 2n \log_2 n + n$ for all $n \geq 1$.

Base case ($n = 1$): $T_1 = 1$, and $2 \cdot 1 \cdot \log_2 1 + 1 = 0 + 1 = 1$, so $T_1 \leq 1$. ✓

Inductive step. Let $n \geq 2$ and assume $T_m \leq 2m \log_2 m + m$ for all $1 \leq m < n$. We show $T_n \leq 2n \log_2 n + n$.

Write $a = \lfloor n/2 \rfloor$ and $b = \lceil n/2 \rceil$, so $a + b = n$ and both are strictly less than n for $n \geq 2$. Also $a, b \leq n/2 + 1/2 \leq n$, and we have the bound $\log_2 a \leq \log_2(n/2) = \log_2 n - 1$ (and similarly for b , because $b \leq \lceil n/2 \rceil \leq (n+1)/2$, and for $n \geq 2$ this satisfies $\log_2 b \leq \log_2 n - c$ for some positive constant c ; for a cleaner bound, one typically assumes n is a power of 2 and does $a = b = n/2$).

Let's assume n is a power of 2 for simplicity, so $a = b = n/2$. Then

$$\begin{aligned}
 T_n &= T_a + T_b + n \\
 &= 2T_{n/2} + n \\
 &\leq 2(2 \cdot (n/2) \log_2(n/2) + n/2) + n & \text{(IH)} \\
 &= 2n \log_2(n/2) + n + n \\
 &= 2n(\log_2 n - 1) + 2n \\
 &= 2n \log_2 n - 2n + 2n \\
 &= 2n \log_2 n \\
 &\leq 2n \log_2 n + n. \checkmark
 \end{aligned}$$

For general n (not a power of 2), you bound $\lfloor n/2 \rfloor$ and $\lceil n/2 \rceil$ by $n/2 + 1$ and propagate additive constants through the same argument. The algebra is messier but the structure is identical. $\square$

The big-O version of this bound— $T_n = O(n \log n)$ —is exactly the time complexity of merge sort.

Exercise 10. Tower of Hanoi: $H_1 = 1$, $H_n = 2H_{n-1} + 1$.

Iteration:

$$\begin{aligned}
 H_n &= 2H_{n-1} + 1 \\
 &= 2(2H_{n-2} + 1) + 1 = 2^2 H_{n-2} + 2 + 1 \\
 &= 2^2(2H_{n-3} + 1) + 2 + 1 = 2^3 H_{n-3} + 2^2 + 2 + 1 \\
 &\vdots \\
 &= 2^{n-1} H_1 + 2^{n-2} + 2^{n-3} + \cdots + 2 + 1 \\
 &= 2^{n-1} + (2^{n-1} - 1) & \text{(geometric series)} \\
 &= 2^n - 1.
 \end{aligned}$$

Verification by induction: $H_1 = 2^1 - 1 = 1$. $\checkmark$ For the inductive step, $H_{k+1} = 2H_k + 1 = 2(2^k - 1) + 1 = 2^{k+1} - 1$. $\checkmark$

Closed form: $H_n = 2^n - 1$. This is why the Tower of Hanoi with n disks requires at least $2^n - 1$ moves—it grows exponentially in the number of disks.

Connection

Exercise 11.

```
import math

PHI = (1 + math.sqrt(5)) / 2
PSI = (1 - math.sqrt(5)) / 2

def fib_closed(n):
```

```

    return round((PHI**n - PSI**n) / math.sqrt(5))

def fib_recurrence(n):
    a, b = 0, 1
    for _ in range(n):
        a, b = b, a + b
    return a

def fib_matrix(n):
    if n == 0:
        return 0
    def mat_mul(A, B):
        return ((A[0][0]*B[0][0] + A[0][1]*B[1][0],
                 A[0][0]*B[0][1] + A[0][1]*B[1][1]),
                (A[1][0]*B[0][0] + A[1][1]*B[1][0],
                 A[1][0]*B[0][1] + A[1][1]*B[1][1]))
    def mat_pow(M, k):
        if k == 1:
            return M
        half = mat_pow(M, k // 2)
        full = mat_mul(half, half)
        return mat_mul(full, M) if k % 2 else full
    M = ((1, 1), (1, 0))
    return mat_pow(M, n)[0][1]

for i in range(31):
    assert fib_recurrence(i) == fib_matrix(i)

for i in range(31):
    if fib_closed(i) != fib_recurrence(i):
        print(f"Closed-form diverges at n={i}")
        break

```

The closed-form version starts disagreeing once the floating-point representation of φ^n loses precision in the trailing digits—typically around $n = 71$ for standard double-precision floats, depending on the rounding semantics of your platform. The recurrence and matrix versions are exact (they use Python’s arbitrary-precision integers) and always agree. The matrix version is asymptotically $O(\log n)$ while the recurrence is $O(n)$, so the matrix version wins for very large inputs, though for small n the recurrence is faster due to lower constant overhead.

Takeaway: the “elegant” closed form is great for intuition and for asymptotic analysis, but for exact computation of large Fibonacci numbers, the iterative or matrix versions are the right tools.

Exercise 12. Ordinary induction on $n = \text{height of the tree}$ *can* be made to work, but only by folding all the information about “trees of height $\leq n$ ” into a single predicate $Q(n) = \text{“for every tree } t \text{ of height } \leq n, P(t) \text{ holds.”}$ Then Q satisfies ordinary induction—the step from $Q(k)$ to $Q(k+1)$ internally invokes P on each subtree, which has height $\leq k$. So the proof goes through.

But doing it this way is ugly for two reasons. First, you’re proving something about heights when the claim is naturally about trees—you’ve introduced an extra level of indirection. Second, you’ve lost the one-to-one correspondence between the shape of the data definition and the shape of the proof: structural induction on trees has one case per constructor, but “induction on height” has one case for the base level and then a big all-trees-of-this-height step that internally re-derives the case analysis. The recipes are the same; the packaging is worse. Structural induction (Module 10)

gives you the clean version directly.

Module 10

Warm-up

Exercise 1. Recursive definition:

$$\begin{aligned}\text{reverse}(\text{Nil}) &= \text{Nil}, \\ \text{reverse}(\text{Cons}(x, xs)) &= \text{reverse}(xs) ++ \text{Cons}(x, \text{Nil}).\end{aligned}$$

Step-by-step computation of $\text{reverse}(\text{Cons}(1, \text{Cons}(2, \text{Cons}(3, \text{Nil}))))$:

$$\begin{aligned}\text{reverse}(\text{Cons}(1, \text{Cons}(2, \text{Cons}(3, \text{Nil})))) &= \text{reverse}(\text{Cons}(2, \text{Cons}(3, \text{Nil}))) ++ \text{Cons}(1, \text{Nil}) \\ &= (\text{reverse}(\text{Cons}(3, \text{Nil})) ++ \text{Cons}(2, \text{Nil})) ++ \text{Cons}(1, \text{Nil}) \\ &= ((\text{Nil} ++ \text{Cons}(3, \text{Nil})) ++ \text{Cons}(2, \text{Nil})) ++ \text{Cons}(1, \text{Nil}) \\ &= (\text{Cons}(3, \text{Nil}) ++ \text{Cons}(2, \text{Nil})) ++ \text{Cons}(1, \text{Nil}) \\ &= \text{Cons}(3, \text{Cons}(2, \text{Nil})) ++ \text{Cons}(1, \text{Nil}) \\ &= \text{Cons}(3, \text{Cons}(2, \text{Cons}(1, \text{Nil}))) \\ &= [3, 2, 1].\end{aligned}$$

Exercise 2.

- (a) $\text{length}(\text{Cons}(a, \text{Cons}(b, \text{Nil}))) = 1 + \text{length}(\text{Cons}(b, \text{Nil})) = 1 + 1 + \text{length}(\text{Nil}) = 1 + 1 + 0 = 2.$
- (b) $\text{Cons}(1, \text{Cons}(2, \text{Nil})) ++ \text{Cons}(3, \text{Nil}) = \text{Cons}(1, \text{Cons}(2, \text{Nil}) ++ \text{Cons}(3, \text{Nil})) = \text{Cons}(1, \text{Cons}(2, \text{Nil} ++ \text{Cons}(3, \text{Nil}))) = \text{Cons}(1, \text{Cons}(2, \text{Cons}(3, \text{Nil})))$.
- (c) $\text{size}(\text{Node}(\text{Leaf}, \text{Node}(\text{Leaf}, \text{Leaf}))) = 1 + \text{size}(\text{Leaf}) + \text{size}(\text{Node}(\text{Leaf}, \text{Leaf})) = 1 + 0 + (1 + 0 + 0) = 2.$
- (d) $\text{edges}(\text{Node}(\text{Leaf}, \text{Node}(\text{Leaf}, \text{Leaf}))) = 2 + \text{edges}(\text{Leaf}) + \text{edges}(\text{Node}(\text{Leaf}, \text{Leaf})) = 2 + 0 + (2 + 0 + 0) = 4.$ (Consistent with $2 \cdot \text{size} = 2 \cdot 2 = 4.$)

Exercise 3.

- (a) **Color**: three base cases, **zero** inductive steps. No recursion, so structural induction collapses to “check each case.”
- (b) **RoseTree**: one base case ($\text{Leaf}(A)$)—no recursion in that constructor) and one inductive step (**Branch**) with *one IH per subtree in the list*, i.e., an unbounded number. In practice this kind of induction is phrased as “assume P holds for every subtree t_i in the list,” and you get a *list of IHs*. The number of IHs is the length of the list you’re branching on.
- (c) **Expr**: two base cases (**Const** and **Var**), one inductive step with one IH (**Neg**), and two inductive steps with two IHs each (**Add** and **Mul**). Total: 2 base + 3 inductive.

Standard

Exercise 4. Define $\text{leaves}(t)$ by:

$$\begin{aligned}\text{leaves}(\text{Leaf}) &= 1, \\ \text{leaves}(\text{Node}(l, r)) &= \text{leaves}(l) + \text{leaves}(r).\end{aligned}$$

Proof. We prove $\text{leaves}(t) = \text{size}(t) + 1$ for all binary trees t by structural induction.

Base case ($t = \text{Leaf}$):

$$\text{leaves}(\text{Leaf}) = 1 = 0 + 1 = \text{size}(\text{Leaf}) + 1. \checkmark$$

Inductive step ($t = \text{Node}(l, r)$). Assume (IH) that $\text{leaves}(l) = \text{size}(l) + 1$ and $\text{leaves}(r) = \text{size}(r) + 1$. Then:

$$\begin{aligned}\text{leaves}(\text{Node}(l, r)) &= \text{leaves}(l) + \text{leaves}(r) \\ &= (\text{size}(l) + 1) + (\text{size}(r) + 1) && \text{(by IH)} \\ &= \text{size}(l) + \text{size}(r) + 2 \\ &= (1 + \text{size}(l) + \text{size}(r)) + 1 \\ &= \text{size}(\text{Node}(l, r)) + 1. \checkmark\end{aligned}$$

□

Exercise 5. Define vars on the BNF for propositional logic formulas:

$$\begin{aligned}\text{vars}(T) &= 0, \\ \text{vars}(F) &= 0, \\ \text{vars}(x) &= 1, \\ \text{vars}(\sim L) &= \text{vars}(L), \\ \text{vars}(L_1 \wedge L_2) &= \text{vars}(L_1) + \text{vars}(L_2), \\ \text{vars}(L_1 \vee L_2) &= \text{vars}(L_1) + \text{vars}(L_2), \\ \text{vars}(L_1 \rightarrow L_2) &= \text{vars}(L_1) + \text{vars}(L_2).\end{aligned}$$

Proof that $\text{vars}(f) \leq \text{size}(f) + 1$. By structural induction on f .

Base cases. $\text{vars}(T) = 0 \leq 0 + 1 = \text{size}(T) + 1$. Same for F . For a variable x : $\text{vars}(x) = 1 \leq 0 + 1 = \text{size}(x) + 1$. ✓

Negation ($\sim L$). Assume $\text{vars}(L) \leq \text{size}(L) + 1$ (IH). Then

$$\text{vars}(\sim L) = \text{vars}(L) \leq \text{size}(L) + 1 \leq 1 + \text{size}(L) + 1 = \text{size}(\sim L) + 1. \checkmark$$

Binary connectives ($L_1 \star L_2$, for $\star \in \{\wedge, \vee, \rightarrow\}$). Assume $\text{vars}(L_i) \leq \text{size}(L_i) + 1$ for $i = 1, 2$. Then

$$\begin{aligned}\text{vars}(L_1 \star L_2) &= \text{vars}(L_1) + \text{vars}(L_2) \\ &\leq (\text{size}(L_1) + 1) + (\text{size}(L_2) + 1) && \text{(both IHs)} \\ &= \text{size}(L_1) + \text{size}(L_2) + 2 \\ &\leq 1 + \text{size}(L_1) + \text{size}(L_2) + 1 \\ &= \text{size}(L_1 \star L_2) + 1. \checkmark\end{aligned}$$

□

(The bound is tight: a formula like $x_1 \wedge x_2$ has **vars** = 2 and **size** = 1, so **vars** = **size** + 1. A formula like $\sim\sim x$ has **vars** = 1 and **size** = 2, so the bound is not tight in general.)

Exercise 6.

Proof that $(xs ++ ys) ++ zs = xs ++ (ys ++ zs)$. By structural induction on xs , with ys and zs universally quantified inside the predicate.

Base case $(xs = \text{Nil})$:

$$\begin{aligned} (\text{Nil} ++ ys) ++ zs &= ys ++ zs && \text{(by def of ++)} \\ \text{Nil} ++ (ys ++ zs) &= ys ++ zs && \text{(by def of ++)} \end{aligned}$$

Both sides equal, so the base case holds.

Inductive step $(xs = \text{Cons}(x, xs'))$. IH: for all ys, zs , $(xs' ++ ys) ++ zs = xs' ++ (ys ++ zs)$. Then:

$$\begin{aligned} (\text{Cons}(x, xs') ++ ys) ++ zs &= \text{Cons}(x, xs' ++ ys) ++ zs && \text{(def of ++)} \\ &= \text{Cons}(x, (xs' ++ ys) ++ zs) && \text{(def of ++)} \\ &= \text{Cons}(x, xs' ++ (ys ++ zs)) && \text{(IH)} \\ &= \text{Cons}(x, xs') ++ (ys ++ zs) && \text{(def of ++, backward)} \end{aligned}$$

✓

□

Exercise 7.

Proof that $\text{length}(\text{map}(f, xs)) = \text{length}(xs)$. By structural induction on xs .

Base case $(xs = \text{Nil})$: $\text{length}(\text{map}(f, \text{Nil})) = \text{length}(\text{Nil}) = 0 = \text{length}(\text{Nil})$. ✓

Inductive step $(xs = \text{Cons}(x, xs'))$. IH: $\text{length}(\text{map}(f, xs')) = \text{length}(xs')$. Then:

$$\begin{aligned} \text{length}(\text{map}(f, \text{Cons}(x, xs'))) &= \text{length}(\text{Cons}(f(x), \text{map}(f, xs'))) && \text{(def of map)} \\ &= 1 + \text{length}(\text{map}(f, xs')) && \text{(def of length)} \\ &= 1 + \text{length}(xs') && \text{(IH)} \\ &= \text{length}(\text{Cons}(x, xs')). && \text{(def of length, backward)} \end{aligned}$$

✓

□

Map preserves length. This is the reason **map** is a “length-preserving” operation and why you can safely zip the result of a **map** with the original list.

Challenge

Exercise 8. *Helper 1:* $xs ++ \text{Nil} = xs$.

Base case: $\text{Nil} ++ \text{Nil} = \text{Nil}$ by definition of $++$. ✓

Inductive step: assume $xs' ++ \text{Nil} = xs'$. Then

$$\begin{aligned} \text{Cons}(x, xs') ++ \text{Nil} &= \text{Cons}(x, xs' ++ \text{Nil}) && \text{(def ++)} \\ &= \text{Cons}(x, xs'). && \text{(IH)} \end{aligned}$$

✓

Helper 2: $\text{reverse}(xs ++ ys) = \text{reverse}(ys) ++ \text{reverse}(xs)$.

Base case ($xs = \text{Nil}$):

$$\begin{aligned} \text{reverse}(\text{Nil} ++ ys) &= \text{reverse}(ys) && (\text{def } ++) \\ \text{reverse}(ys) ++ \text{reverse}(\text{Nil}) &= \text{reverse}(ys) ++ \text{Nil} && (\text{def reverse}) \\ &= \text{reverse}(ys). && (\text{Helper 1}) \end{aligned}$$

✓

Inductive step ($xs = \text{Cons}(x, xs')$). IH: for all ys , $\text{reverse}(xs' ++ ys) = \text{reverse}(ys) ++ \text{reverse}(xs')$.

$$\begin{aligned} \text{reverse}(\text{Cons}(x, xs') ++ ys) &= \text{reverse}(\text{Cons}(x, xs' ++ ys)) && (\text{def } ++) \\ &= \text{reverse}(xs' ++ ys) ++ \text{Cons}(x, \text{Nil}) && (\text{def reverse}) \\ &= (\text{reverse}(ys) ++ \text{reverse}(xs')) ++ \text{Cons}(x, \text{Nil}) && (\text{IH}) \\ &= \text{reverse}(ys) ++ (\text{reverse}(xs') ++ \text{Cons}(x, \text{Nil})) && (\text{Exercise 6: } ++ \text{ assoc}) \\ &= \text{reverse}(ys) ++ \text{reverse}(\text{Cons}(x, xs')) && (\text{def reverse, backward}) \end{aligned}$$

✓

Main theorem: $\text{reverse}(\text{reverse}(xs)) = xs$.

Base case ($xs = \text{Nil}$): $\text{reverse}(\text{reverse}(\text{Nil})) = \text{reverse}(\text{Nil}) = \text{Nil}$. ✓

Inductive step ($xs = \text{Cons}(x, xs')$). IH: $\text{reverse}(\text{reverse}(xs')) = xs'$.

$$\begin{aligned} \text{reverse}(\text{reverse}(\text{Cons}(x, xs'))) &= \text{reverse}(\text{reverse}(xs') ++ \text{Cons}(x, \text{Nil})) && (\text{def reverse}) \\ &= \text{reverse}(\text{Cons}(x, \text{Nil})) ++ \text{reverse}(\text{reverse}(xs')) && (\text{Helper 2}) \\ &= \text{Cons}(x, \text{Nil}) ++ xs' && (\text{by direct computation and IH}) \\ &= \text{Cons}(x, \text{Nil} ++ xs') && (\text{def } ++) \\ &= \text{Cons}(x, xs'). && (\text{def } ++) \end{aligned}$$

✓

Three lemmas plus one main theorem. This is a very typical shape for “serious” structural induction proofs: the main result depends on helpers, which depend on smaller helpers. The dependency chain is what makes proof assistants useful in practice—keeping track of three or four nested inductions by hand is error-prone.

Exercise 9.

Proof that every negation-free formula is monotone in every variable. We induct on the structure of negation-free formulas. The BNF of negation-free formulas is $L := T \mid F \mid x \mid L \wedge L \mid L \vee L$ (five cases, one recursive sub-part per case for the binary connectives).

Fix an arbitrary variable x_0 . We show: the function f represented by any negation-free formula is monotone in x_0 .

Base cases.

- $f = T$: the constant- T function is monotone (flipping any input doesn’t change the output). ✓
- $f = F$: similarly, the constant- F function is monotone. ✓
- $f = y$ where y is a variable. If $y = x_0$, then flipping x_0 from F to T flips the output from F to T , which is a non-decrease—monotone. If $y \neq x_0$, then flipping x_0 doesn’t change the output at all—also monotone.

Conjunction case. $f = L_1 \wedge L_2$. IHs: L_1 and L_2 are each monotone in x_0 . Suppose we flip x_0 from F to T . By IH, the values of L_1 and L_2 either stay the same or increase (from F to T). In a truth table: $\wedge$ is itself monotone—if you check the table, increasing either input can only keep $\wedge$ the same or flip it from F to T . So $L_1 \wedge L_2$ is monotone in x_0 . ✓

Disjunction case. $f = L_1 \vee L_2$. Same argument: both IHs say L_i can only go from F to T as x_0 flips, and $\vee$ is monotone in each argument (if either input goes up, the output goes up or stays the same). So $L_1 \vee L_2$ is monotone. ✓

All cases covered. By structural induction, every negation-free formula is monotone in every variable. □

This is the positive side of the Module 2 Exercise 10 result: that exercise used monotonicity to prove $\{\wedge, \vee\}$ is *not* functionally complete (because $\neg$ is not monotone), and this exercise proves *why* monotonicity is preserved by $\{\wedge, \vee\}$ —structural induction on the BNF that excludes $\neg$.

Exercise 10. First, define “perfect” inductively:

- Leaf is perfect.
- $\text{Node}(l, r)$ is perfect iff l and r are both perfect and $\text{height}(l) = \text{height}(r)$.

Proof that $\text{leaves}(t) = 2^{\text{height}(t)}$ for every perfect tree t . By structural induction on the perfect tree t .

Base case ($t = \text{Leaf}$): $\text{leaves}(\text{Leaf}) = 1 = 2^0 = 2^{\text{height}(\text{Leaf})}$. ✓

Inductive step ($t = \text{Node}(l, r)$ with l, r perfect and $\text{height}(l) = \text{height}(r) = h$). IHs: $\text{leaves}(l) = 2^h$ and $\text{leaves}(r) = 2^h$. Then

$$\begin{aligned} \text{leaves}(\text{Node}(l, r)) &= \text{leaves}(l) + \text{leaves}(r) \\ &= 2^h + 2^h \\ &= 2 \cdot 2^h = 2^{h+1} \\ &= 2^{1+\max(h, h)} \\ &= 2^{1+\max(\text{height}(l), \text{height}(r))} \\ &= 2^{\text{height}(\text{Node}(l, r))}. \checkmark \end{aligned}$$

□

The balancedness constraint is essential: without it, the two subtrees could have different heights, and $\text{leaves}(l) + \text{leaves}(r)$ wouldn’t simplify to 2^{h+1} . This is why perfectly balanced binary trees are exceptionally clean objects to reason about.

Connection

Exercise 11.

```
from dataclasses import dataclass
from typing import Union

# Lists
class Nil: pass
NIL = Nil()

@dataclass
```

```

class Cons:
    head: object
    tail: object

def length(xs):
    if isinstance(xs, Nil):
        return 0
    return 1 + length(xs.tail)

def append(xs, ys):
    if isinstance(xs, Nil):
        return ys
    return Cons(xs.head, append(xs.tail, ys))

def reverse(xs):
    if isinstance(xs, Nil):
        return NIL
    return append(reverse(xs.tail), Cons(xs.head, NIL))

# Binary trees
class Leaf: pass
LEAF = Leaf()

@dataclass
class Node:
    left: object
    right: object

def size(t):
    if isinstance(t, Leaf):
        return 0
    return 1 + size(t.left) + size(t.right)

def edges(t):
    if isinstance(t, Leaf):
        return 0
    return 2 + edges(t.left) + edges(t.right)

def leaves(t):
    if isinstance(t, Leaf):
        return 1
    return leaves(t.left) + leaves(t.right)

def height(t):
    if isinstance(t, Leaf):
        return 0
    return 1 + max(height(t.left), height(t.right))

# Property-based smoke tests
import random

def random_list(n):
    xs = NIL
    for _ in range(n):

```



```

        xs = Cons(random.randint(0, 99), xs)
    return xs

def random_tree(depth):
    if depth == 0 or random.random() < 0.3:
        return LEAF
    return Node(random_tree(depth - 1), random_tree(depth - 1))

# length(xs ++ ys) = length(xs) + length(ys)
for _ in range(100):
    xs = random_list(random.randint(0, 10))
    ys = random_list(random.randint(0, 10))
    assert length(append(xs, ys)) == length(xs) + length(ys)

# edges(t) = 2 * size(t)
for _ in range(100):
    t = random_tree(5)
    assert edges(t) == 2 * size(t)

# leaves(t) = size(t) + 1
for _ in range(100):
    t = random_tree(5)
    assert leaves(t) == size(t) + 1

print("all checks pass")

```

Random testing is no substitute for proof—it can only show the *absence of small counterexamples*—but it’s a cheap sanity check that your proof wasn’t vacuously true or based on a buggy definition.

Exercise 12.

- (a) $\text{Nat} := 0 \mid \text{succ Nat}$ corresponds to $N = 1 + N$ (one base case, one recursive case with one Nat sub-part).
- (b) $\text{BinTree} := \text{Leaf} \mid \text{Node}(\text{BinTree}, \text{BinTree})$ corresponds to $T = 1 + T \cdot T$ (one base case, one recursive case with two BinTree sub-parts, which multiply because they’re ordered and independent).
- (c) $\text{Maybe } A := \text{Nothing} \mid \text{Just}(A)$ corresponds to $M = 1 + A$ (one base case, one non-recursive case carrying an A).

They’re called “polynomial” functors because the expressions on the right are literally polynomials in A (and in the type being defined, if it’s recursive). The algebra of datatypes is a formalization of this intuition: “+” is the disjoint union (sum types), “ $\cdot$ ” is the Cartesian product (pair/tuple types), 1 is the one-element type (`unit`), 0 is the empty type (`void`), and exponents B^A correspond to function types $A \rightarrow B$. The intermezzo explains how this algebra lets you *compute* with types the way you compute with numbers—and why the resulting equations (like $A + A \cong 2 \cdot A$) really do correspond to type isomorphisms.

Intermezzo: The Algebra of Types

Exercise 1. Using the exponent-of-a-product law from the algebra of types:

$$(A \rightarrow B) \times (A \rightarrow C) \cong A \rightarrow (B \times C).$$

A pair of functions from A is the same as a single function from A that returns a pair. Given $f : A \rightarrow B$ and $g : A \rightarrow C$, we can build $h : A \rightarrow (B \times C)$ by $h(a) = (f(a), g(a))$. Conversely, given h , we recover $f = \text{fst} \circ h$ and $g = \text{snd} \circ h$.

Under the Curry–Howard correspondence, this is the logical equivalence:

$$(A \rightarrow B) \wedge (A \rightarrow C) \equiv A \rightarrow (B \wedge C).$$

In words: “if A implies B and A implies C , then A implies B and C ”—and vice versa.

Exercise 2. The BNF $\text{Nat} = \text{Zero} \mid \text{Succ}(\text{Nat})$ translates to $N = 1 + N$. “Solving” by repeated substitution:

$$\begin{aligned} N &= 1 + N \\ &= 1 + (1 + N) \\ &= 1 + 1 + (1 + N) \\ &= 1 + 1 + 1 + (1 + N) \\ &= \dots \\ &= \underbrace{1 + 1 + 1 + \dots}_{\text{infinitely many}} \end{aligned}$$

This is an infinite sum of 1’s. It makes perfect sense as a description of the natural numbers: each term “1” in the sum represents one specific natural number (the first 1 is 0, the second is 1, the third is 2, etc.). Since each 1 contributes exactly one inhabitant, the type N has inhabitants 0, 1, 2, 3, ...—exactly the natural numbers.

Exercise 3. Apply the distributive law of the type algebra:

$$(A + B) \times (C + D) \cong A \times C + A \times D + B \times C + B \times D.$$

The right side has four summands, so a Python realisation uses a tagged union with four cases:

```
class AC: # first constructor: A * C
    def __init__(self, a, c): self.a, self.c = a, c

class AD: # second constructor: A * D
    def __init__(self, a, d): self.a, self.d = a, d

class BC: # third constructor: B * C
    def __init__(self, b, c): self.b, self.c = b, c

class BD: # fourth constructor: B * D
    def __init__(self, b, d): self.b, self.d = b, d
```

A value of $(A + B) \times (C + D)$ —a pair where each component is a tagged union—can be converted to one of AC, AD, BC, BD by case analysis on both components.

Under Curry–Howard this corresponds to the logical tautology

$$(A \vee B) \wedge (C \vee D) \equiv (A \wedge C) \vee (A \wedge D) \vee (B \wedge C) \vee (B \wedge D),$$

i.e., the distributivity of $\wedge$ over $\vee$ applied twice.

Exercise 4. Write $L = \text{NList } A$. The BNF

$$\text{NList } A = A \mid A \times \text{NList } A$$

translates to the polynomial equation

$$L = A + A \cdot L.$$

Factor out A : $L = A \cdot (1 + L)$. Or, solving for L directly,

$$L - A \cdot L = A \implies L(1 - A) = A \implies L = \frac{A}{1 - A}.$$

Expanding the geometric series $\frac{1}{1-A} = 1 + A + A^2 + A^3 + \dots$ and multiplying through by A :

$$L = A \cdot (1 + A + A^2 + A^3 + \dots) = A + A^2 + A^3 + A^4 + \dots.$$

The n th term, A^n , corresponds to a non-empty list of exactly n elements. Unlike the ordinary **List** (whose expansion included a 1 for the empty list), **NEList** has no 1 term—the constant part is gone, reflecting the fact that non-empty lists cannot have zero elements. The series starts at A^1 and goes on forever.

Exercise 5.

- (a) $|\text{Maybe } A| = |A| + 1$: it's the A values tagged **Just**, plus the single **Nothing**.
- (b) $|\text{Maybe } (\text{Maybe } A)| = |A| + 2$: the outer **Maybe** adds 1 to the cardinality of **Maybe** A , which itself has cardinality $|A| + 1$.
- (c) In algebraic notation,

$$\text{Maybe } (\text{Maybe } A) = \text{Maybe } A + 1 = (A + 1) + 1 = A + 2.$$

The right side has $|A| + 2$ inhabitants: $|A|$ of them tagged **Just**(**Just**($\cdot$)), one **Just Nothing**, and one **Nothing**. The three distinct outer-tag/inner-tag combinations are exactly the three ways to sit in the cardinality- $|A| + 2$ result, matching part (b).

Exercise 5b.

```
def list_map(f, xs):
    # Recursive version
    if not xs:
        return []
    return [f(xs[0])] + list_map(f, xs[1:])

# Or, equivalently, as a comprehension:
# def list_map(f, xs): return [f(x) for x in xs]
```

(a) *Identity law.*

```
>>> list_map(lambda x: x, [1, 2, 3])
[1, 2, 3]
```

Matches the original list. ✓

(b) *Composition law.* Let $f(x) = x + 1$ and $g(x) = 2x$.

```
>>> list_map(lambda x: g(f(x)), [1, 2, 3])
[4, 6, 8]
>>> list_map(g, list_map(f, [1, 2, 3]))
[4, 6, 8]
```

The two sides agree. ✓

(c) *Why the composition law matters.* Without the composition law, the two-pass computation “first `list_map(f, xs)`, then `list_map(g, ...)`” could silently disagree with the one-pass “`list_map(lambda x: g(f(x)), xs)`.” That would break the optimisation of fusing multiple map passes into one (the basis of stream-fusion and map-fusion rewrites used in compilers like GHC)—and it would also break the conceptual identity between “list of transformed things” and “list with transformations composed,” making the functor abstraction meaningless.

Exercise 6.

```
# Left-to-right: A -> (B * C) becomes (A -> B) * (A -> C)
def split(h):
    return (lambda a: h(a)[0],
            lambda a: h(a)[1])

# Right-to-left: (A -> B) * (A -> C) becomes A -> (B * C)
def merge(fg):
    f, g = fg
    return lambda a: (f(a), g(a))
```

Inverses. Start with $h : A \rightarrow (B \times C)$. Then $\text{merge}(\text{split}(h))(a) = ((\lambda a'. h(a')[0])(a), (\lambda a'. h(a')[1])(a)) = (h(a)[0], h(a)[1]) = h(a)$, so $\text{merge}(\text{split}(h)) = h$. In the other direction, start with (f, g) where $f : A \rightarrow B$ and $g : A \rightarrow C$. Then $\text{split}(\text{merge}((f, g)))$: the first component is $\lambda a. \text{merge}((f, g))(a)[0] = \lambda a. (f(a), g(a))[0] = \lambda a. f(a) = f$ (up to eta-equivalence), and similarly the second component is g . So `split` and `merge` are mutual inverses, witnessing the isomorphism.

In pure-arithmetic terms, this is the exponent-of-a-product law

$$(b \cdot c)^a = b^a \cdot c^a.$$

On the type side: the number of functions from A to $B \times C$ is the product of the number of functions $A \rightarrow B$ and the number of functions $A \rightarrow C$. On the arithmetic side: raising a product to a power distributes over the factors. The two statements are the same fact seen through cardinality.

Intermezzo: The Lambda Calculus

Exercise 1. Application is left-associative, so $(\lambda x. \lambda y. x) 3 7 = ((\lambda x. \lambda y. x) 3) 7$.

Step 1. Apply $(\lambda x. \lambda y. x)$ to 3: substitute 3 for x in the body $\lambda y. x$:

$$(\lambda x. \lambda y. x) 3 \rightarrow \lambda y. 3.$$

Step 2. Apply $(\lambda y. 3)$ to 7: substitute 7 for y in the body 3. Since y does not appear in 3, nothing changes:

$$(\lambda y. 3) 7 \rightarrow 3.$$

Final result: 3.

This function takes two arguments and returns the first, ignoring the second—it is the “constant” function (or, in Church encoding terms, it is true: given two options, pick the first).

Exercise 2. Define $\text{and} = \lambda a. \lambda b. a \ b$ false.

The idea: if a is true, the result depends on b ; if a is false, the result is false regardless. Verification on all four cases:

Case 1: and true true:

$$\begin{aligned}
\text{and true true} &= (\lambda a. \lambda b. a \ b \ \text{false}) \ \text{true} \ \text{true} \\
&\longrightarrow (\lambda b. \text{true} \ b \ \text{false}) \ \text{true} \\
&\longrightarrow \text{true true false} \\
&= (\lambda t. \lambda f. t) \ \text{true} \ \text{false} \\
&\longrightarrow (\lambda f. \text{true}) \ \text{false} \\
&\longrightarrow \text{true}. \quad \checkmark
\end{aligned}$$

Case 2: and true false:

$$\longrightarrow \text{true false false} = (\lambda t. \lambda f. t) \ \text{false} \ \text{false} \longrightarrow (\lambda f. \text{false}) \ \text{false} \longrightarrow \text{false}. \quad \checkmark$$

Case 3: and false true:

$$\longrightarrow \text{false true false} = (\lambda t. \lambda f. f) \ \text{true} \ \text{false} \longrightarrow (\lambda f. f) \ \text{false} \longrightarrow \text{false}. \quad \checkmark$$

Case 4: and false false:

$$\longrightarrow \text{false false false} = (\lambda t. \lambda f. f) \ \text{false} \ \text{false} \longrightarrow (\lambda f. f) \ \text{false} \longrightarrow \text{false}. \quad \checkmark$$

All four cases match the expected truth table for AND.

Exercise 3. Again application is left-associative, so $(\lambda x. \lambda y. y) \ 3 \ 7 = ((\lambda x. \lambda y. y) \ 3) \ 7$.

Step 1. Apply $(\lambda x. \lambda y. y)$ to 3: substitute 3 for x in the body $\lambda y. y$. Since x does not occur in $\lambda y. y$, nothing changes:

$$(\lambda x. \lambda y. y) \ 3 \longrightarrow \lambda y. y.$$

Step 2. Apply $(\lambda y. y)$ to 7: substitute 7 for y in the body y :

$$(\lambda y. y) \ 7 \longrightarrow 7.$$

Final result: 7.

This function takes two arguments and returns the *second*, ignoring the first—the mirror of Exercise 1. Together, these two functions encode the Church booleans from Section 3:

$$\text{true} = \lambda x. \lambda y. x, \quad \text{false} = \lambda x. \lambda y. y.$$

Exercise 1 is true applied to two arguments; Exercise 3 is false applied to two arguments. A Church boolean is precisely a two-argument selector: given two options, pick one of them.

Exercise 4. Apply succ to 2:

$$\begin{aligned}
\text{succ } \mathbf{2} &= (\lambda n. \lambda f. \lambda x. f \ (n \ f \ x)) \ \mathbf{2} \\
&\longrightarrow \lambda f. \lambda x. f \ (\mathbf{2} \ f \ x) \\
&= \lambda f. \lambda x. f \ ((\lambda f. \lambda x. f \ (f \ x)) \ f \ x).
\end{aligned}$$

Reduce the inner $\mathbf{2} \ f \ x$ step by step. First apply $\mathbf{2}$ to f :

$$(\lambda f. \lambda x. f \ (f \ x)) \ f \longrightarrow \lambda x. f \ (f \ x).$$

Then apply the result to x :

$$(\lambda x. f \ (f \ x)) \ x \longrightarrow f \ (f \ x).$$

Plugging back in:

$$\text{succ } \mathbf{2} \longrightarrow \lambda f. \lambda x. f (f (f x)).$$

This is exactly the definition of **3**: apply f three times to x . So $\text{succ } \mathbf{2} = \mathbf{3}$, as expected.

Exercise 5. Define $\text{or} = \lambda a. \lambda b. a \text{ true } b$.

The idea: if a is true, the result is true regardless of b ; if a is false, the result depends on b .
Verification on all four cases:

Case 1: or true true:

$$\begin{aligned} \text{or true true} &= (\lambda a. \lambda b. a \text{ true } b) \text{ true true} \\ &\longrightarrow (\lambda b. \text{true true } b) \text{ true} \\ &\longrightarrow \text{true true true} \\ &= (\lambda t. \lambda f. t) \text{ true true} \\ &\longrightarrow (\lambda f. \text{true}) \text{ true} \\ &\longrightarrow \text{true. } \checkmark \end{aligned}$$

Case 2: or true false:

$$\longrightarrow \text{true true false} = (\lambda t. \lambda f. t) \text{ true false} \longrightarrow (\lambda f. \text{true}) \text{ false} \longrightarrow \text{true. } \checkmark$$

Case 3: or false true:

$$\longrightarrow \text{false true true} = (\lambda t. \lambda f. f) \text{ true true} \longrightarrow (\lambda f. f) \text{ true} \longrightarrow \text{true. } \checkmark$$

Case 4: or false false:

$$\longrightarrow \text{false true false} = (\lambda t. \lambda f. f) \text{ true false} \longrightarrow (\lambda f. f) \text{ false} \longrightarrow \text{false. } \checkmark$$

All four cases match the expected truth table for OR.

Exercise 6. Apply mult to **2** and **3**:

$$\begin{aligned} \text{mult } \mathbf{2} \mathbf{3} &= (\lambda m. \lambda n. \lambda f. m (n f)) \mathbf{2} \mathbf{3} \\ &\longrightarrow (\lambda n. \lambda f. \mathbf{2} (n f)) \mathbf{3} \\ &\longrightarrow \lambda f. \mathbf{2} (\mathbf{3} f). \end{aligned}$$

The inner term $\mathbf{3} f$ reduces to “apply f three times”:

$$\mathbf{3} f = (\lambda f. \lambda x. f (f (f x))) f \longrightarrow \lambda x. f (f (f x)).$$

Call this function $g = \lambda x. f (f (f x))$. Then $\mathbf{2} g$ reduces to “apply g twice”:

$$\begin{aligned} \mathbf{2} g &= (\lambda f. \lambda x. f (f x)) g \\ &\longrightarrow \lambda x. g (g x). \end{aligned}$$

And $g (g x)$ applies f three times to x , then three more times, for six total:

$$g (g x) = g (f (f (f x))) = f (f (f (f (f (f x)))).$$

Pulling the outer $\lambda f. \lambda x.$ back on, we get

$$\text{mult } \mathbf{2\ 3} \longrightarrow \lambda f. \lambda x. f (f (f (f (f (f x))))),$$

which is exactly **6**—apply f six times to x .

Why $m (n f)$ encodes multiplication. The term $n f$ is “apply f n times”—it is a new function, call it g , that applies f exactly n times. Then $m g$ is “apply g m times”—which applies f n times, then another n times, m rounds in total, for $m \cdot n$ applications of f . The body $m (n f)$ is literally the identity “iterate ($\text{iterate}_n f$) m times,” which is iteration of f $m \cdot n$ times.

Exercise 7. *Python to lambda.* The Python code

```
lambda x: lambda y: x + y
```

translates to the lambda term

$$\lambda x. \lambda y. x + y$$

(treating $+$ as a primitive—it is not itself a pure lambda term). This is the curried addition function: it takes x , returns a function that takes y , and returns $x + y$.

Lambda to Python. The term $\lambda f. \lambda g. \lambda x. f (g x)$ translates to

```
lambda f: lambda g: lambda x: f(g(x))
```

This is **function composition** (usually written $f \circ g$ in mathematics). Given f and g , it returns a new function that first applies g to its input and then applies f to the result. It is the same gadget as Exercise 4 of the Curry–Howard intermezzo—the computational content of transitivity of implication—expressed in pure lambda-calculus form.

Index

- argument form, 48
- assignment rule, 120
- associativity, 80

- beta reduction, 182
- Bézout’s identity, 101
- BHK interpretation, 129
- $\leftrightarrow$ (biconditional), 34
- bijection, 23
- binary tree, 163
- BNF (Backus-Naur form), 31
- bound variable, 68

- Cantor’s diagonal argument, 25
- cardinality, 14
- Cartesian product, 14
- category theory, 178
- ceiling, 100
- characteristic equation, 154
- characteristic function, 27
- Chinese remainder theorem, 101
- Church, Alonzo, 181
- Church–Turing thesis, 184
- circuit, 51
- commutativity, 81
- composition of functions, 15
- $\wedge$ (conjunction), 31
- conjunctive normal form (CNF), 51
- Cons, 161
- constructive proof, 90
- contradiction, 33
- converse error, 49
- countable, 25
- countably infinite, 25
- counterexample, 90
- Curry–Howard correspondence, 130

- De Morgan’s laws, 35
- discharged assumption, 59
- disjoint, 13
- $\vee$ (disjunction), 31

- disjunctive normal form (DNF), 38, 51
- disjunctive syllogism, 49
- divisibility, 89

- element, 11
- elimination rule, 45
- empty set, 14
- equivalence relation, 16
- Euclidean algorithm, 118
- even, 89
- existence proof, 90
- explicit formula, 141
- extended Euclidean algorithm, 118
- extensional equality, 13

- fallacy, 49
- Fibonacci sequence, 155
- finite field, 101
- first-order logic, 67
- floor, 100
- free variable, 68
- function, 15
- function space, 26
- functional completeness, 35
- functor, 176

- Gentzen style, 58
- geometric series, 153
- golden ratio, 155
- greatest common divisor, 118
- group, 92

- Hoare logic, 119, 135
- Hoare triple, 119
- homomorphism, 93

- identity element, 92
- identity function, 15
- $\rightarrow$ (implication), 31
- indicator function, 27
- induction, 73

- induction on sequences, 143
- injection, 23
- intensional equality, 13
- $\cap$ (intersection), 12
- introduction rule, 45
- inverse, 92
- inverse error, 49
- irrational number, 116
- isomorphism
 - of groups, 94
- Kripke structure, 138
- lambda calculus, 181
- Leaf, 163
- linear logic, 135
- list, 161
- liveness, 137
- logic gate, 51
- logical equivalence, 34
- loop invariant, 120
- LTL, 137
- membership ($\in$), 11
- metalanguage, 43
- modal logic, 135
- model checking, 137
- modulus, 99
- modus ponens, 43
- modus tollens, 44
- NAND, 36
- natural deduction, 58
- natural numbers, 73
- necessary condition, 71
- $\neg$ (negation), 31
- Nil, 161
- Node, 163
- odd, 89
- partial function, 15
- postcondition, 119
- precondition, 119
- predicate, 67
- proof
 - by contradiction, 44, 115
 - by contrapositive, 116
 - by exhaustion, 94
 - direct, 88
 - proof tree, 60
- propositional logic, 31
- quantifier
 - existential, 68
 - universal, 68
- quotient set, 107
- quotient-remainder theorem, 99
- rational number, 90
- recurrence relation, 152
- recursive definition, 142
- reflexive, 16
- relation, 16
- rig, 177
- safety property, 137
- semantics, 30
- sequence, 141
- set, 11
 - complement, 13
 - difference, 12
 - power, 14
- set-builder notation, 12
- strengthening, 121
- strong induction, 150
- structural induction, 160
 - on lambda terms, 182
- subset, 12
- sufficient condition, 71
- summation notation, 142
- surjection, 23
- symmetric, 16
- syntax, 30
- tautology, 33
- temporal logic, 135
- transitive, 16
- truth set, 69
- truth table, 32
- tuple, 14
- uncountable, 25
- $\cup$ (union), 12
- universal property, 178
- valid argument, 48
- Venn diagram, 12

weakening, [121](#)

XOR, [36](#)